

El presente documento es el reporte técnico final del Trabajo Terminal con número de registro 2025-A138, titulado:

"Prototipo de sistema de identificación de estudiantes basado en reconocimiento facial dentro de la Escuela Superior de Cómputo para evitar la suplantación de identidad durante la aplicación de exámenes a título de suficiencia mediante el uso de credenciales y dispositivos móviles", elaborado por Alejandra De la Cruz De la Cruz, Luis Antonio Flores Esquivel, José Alfredo Jiménez Rodríguez y Daniel Martín Huertas Ramírez para obtener el grado de Ingeniero en Inteligencia Artificial.

Este trabajo fue dirigido por los profesores José Félix Serrano Talamantes y Ulises Vélez Saldaña, quienes fungieron como directores del proyecto.

El presente documento va dirigido a los sinodales, Andrés García Floriano, Chadwick Carreto Arellano y Saúl de la O Torres, con la finalidad de exponer de manera detallada los objetivos, el desarrollo, los resultados y conclusiones del trabajo realizado, así como sustentar su validez técnica y académica para la evaluación correspondiente.

En este documento se describe el diseño, implementación, pruebas y resultados de un sistema de identificación mediante reconocimiento facial, enfocado en asegurar la identidad de los estudiantes que presentan Exámenes a Título de Suficiencia en la Escuela Superior de Cómputo perteneciente al Instituto Politécnico Nacional.

Organización del documento

El presente documento se encuentra estructurado por los siguientes capítulos y anexos:

En el capítulo 1, se encuentran los antecedentes que permitieron identificar la problemática que rige este proyecto, lo que a su vez nos guió al análisis de esta misma para sustentar la propuesta de solución planteada a lo largo de este documento

En el capítulo 2, se presenta la definición del proyecto, donde se explica la justificación desde un contexto social, personal y académico del proyecto y además, se da una explicación breve y concisa sobre el alcance, los requerimientos, la arquitectura, las propiedades de software, el modelo de información y la metodología de desarrollo.

En el capítulo 3, se encuentra el estado del arte, en donde se revisaron y analizaron trabajos anteriores con similitudes al proyecto a realizar, esto con el fin de realizar una comparación de las principales características e identificar posibles áreas de mejora.

En el capítulo 4, se presenta el marco teórico donde se explican los términos, descripciones y demás elementos importantes que se relacionan en menor o mayor medida con este trabajo a fin de tener un panorama general sobre la terminología utilizada.

En el capítulo 5, se presenta los detalles del trabajo realizado, es decir se explica cómo y con que tecnologías y métodos se implementó cada una de las secciones del sistema.

En el capítulo 6, se exponen las pruebas realizadas en la aplicación móvil a los casos de uso más importantes, para buscar errores para su posterior corrección y/o determinar si la funcionalidad y/o proceso se está realizando de manera satisfactoria o cumple con lo esperado, además, se muestran las pruebas hechas al sistema de reconocimiento facial.

En el capítulo 7, se presentan los resultados obtenidos de las encuestas hechas a los alumnos, docentes y personal de seguridad de la escuela superior de cómputo (ESCOM), sobre el funcionamiento de la aplicación, las opiniones sobre esta y si les parece útil o si creen que podría ayudar reducir el problema de la suplantación de identidad durante los ETS. También, se presentan los resultados de las pruebas de la red neuronal y sus matrices de confusión.

Finalmente, en el capítulo 8, se presentan las direcciones futuras y consideraciones identificadas para este proyecto, una vez culminada la fase de Trabajo Terminal actual. Si bien se ha logrado la implementación de la aplicación móvil y el sistema de gestión asociado, incluyendo el reconocimiento facial en tiempo real, este capítulo delinea las potenciales expansiones y mejoras que podrían llevarse a cabo en una eventual continuación del proyecto, más allá de los objetivos académicos de este Trabajo Terminal.

En el anexo A, se sustenta la metodología de trabajo empleada, se presentan los resultados del estudio de factibilidad realizado y el análisis del sistema que compone la propuesta de solución a fin de delimitar el alcance de este y los elementos a desarrollar, asimismo, se presenta el análisis de riesgos correspondiente.

En el Anexo B, se muestran las decisiones de diseño tomadas para este proyecto, esto se encuentra representado a través de diagramas para ofrecer una visualización y explicación del funcionamiento del sistema y las interacciones que existen entre los componentes que lo conforman. Estos incluyen el diagrama de estructura general del sistema, la estructura interna de los módulos, diagramas de clases, diagramas de componentes y diagramas de secuencia.

Planteamiento del problema

En este capítulo se describen los antecedentes y se delimita la problemática principal que guía el proyecto a realizar, así como los objetivos propuestos y la justificación de su desarrollo.

1.1. Antecedentes

En la actualidad, la Escuela Superior de Cómputo (ESCOM) cuenta con diversos mecanismos que permiten evaluar los conocimientos del alumnado, siguiendo un esquema de educación tradicional en el que los alumnos tienen que obtener un mínimo de calificación para acreditar una materia y, donde además, es necesario acreditar todas las materias que componen el plan de estudios de la carrera para poder egresar de esta [?].

Estos mecanismos de evaluación difieren unos de otros dependiendo del momento en el que se aplican, del valor sobre la calificación final del alumno y de la situación académica de este. Algunos ejemplos incluyen las evaluaciones ordinarias parciales, la entrega de proyectos, evaluaciones extraordinarias, etc.

Sin embargo, la *evaluación a título de suficiencia* también conocida como ETS, es un tipo de evaluación especial que se aplica en los planteles del Instituto Politécnico Nacional (IPN) y que permite a los alumnos acreditar materias que no hayan podido acreditar durante los periodos ordinarios y extraordinarios [?].

A pesar de ser una evaluación que se realiza en todos los planteles del IPN, no existe medio alguno que permita obtener información clara y precisa sobre esta, ya que la poca información que se puede encontrar en la red se limita a instructivos de su inscripción de distintos planteles.

Es debido a esto, y a su importancia en el proceso académico de cada una de las instituciones que conforman al IPN que se propuso la realización de este proyecto.

1.2. Planteamiento del problema

Actualmente, la inseguridad es una de las principales problemáticas de nuestro país, lo que obliga a las instituciones educativas a destinar recursos significativos para reforzar los protocolos de seguridad, tanto dentro como fuera de sus instalaciones. Estos esfuerzos buscan garantizar la integridad física y emocional de la comunidad educativa y propiciar un entorno seguro donde los estudiantes puedan desarrollar plenamente sus capacidades creativas e intelectuales [?].

Una de las principales preocupaciones de las instituciones educativas es el acceso no autorizado a sus instalaciones, ya que las brechas en los protocolos de control de acceso pueden derivar en situaciones que comprometen la seguridad de la comunidad escolar [?]. Según un estudio realizado en los planteles de Iztapalapa y Xochimilco de la Universidad Autónoma Metropolitana, los alumnos han sido víctimas de robos, tanto con violencia como sin ella, frecuentemente atribuidos al ingreso de personas ajenas a la institución. Por ejemplo, en el plantel de Iztapalapa, el 79.3 por ciento del alumnado considera que la inseguridad está relacionada con el acceso de personas externas [?].

En el caso de la Escuela Superior de Cómputo (ESCOM), la creciente matrícula estudiantil ha presentado desafíos adicionales en materia de seguridad. Entre las problemáticas más frecuentes se encuentra la suplantación de identidad durante los Exámenes a Título de Suficiencia (ETS) y los Exámenes a Título de Suficiencia Especiales. Este problema ocurre cuando personas externas o incluso estudiantes de la institución presentan exámenes en lugar de los alumnos registrados, ya sea mediante credenciales falsificadas o acuerdos entre estudiantes. Estas prácticas no solo afectan la transparencia del proceso educativo, sino que también ponen en riesgo la seguridad de la comunidad escolar al permitir el ingreso de personas cuya intención puede ser desconocida.

La falta de un control de acceso eficiente contribuye a este problema, ya que protocolos débiles permiten que las personas ingresen sin una verificación adecuada de su identidad. Actualmente, el ingreso a las instalaciones y a los exámenes en la ESCOM depende de métodos tradicionales que pueden ser fácilmente manipulados.

1.3. Propuesta de solución

Tomando como punto de partida las tecnologías que se describen en capítulo 3 la propuesta de solución a la problemática de la suplantación de la identidad durante las evaluaciones ETS en la ESCOM consta de dos partes.

En primer lugar se desarrollara una aplicación móvil con soporte para dispositivos Android que servirá como una herramienta tanto para el personal de seguridad como para los docentes para reforzar los protocolos de control de acceso a las instalaciones y verificación de la identidad de los alumnos respectivamente.

Al funcionar como una herramienta se pretende brindar la funcionalidad necesaria para identificar situaciones en las que los alumnos quieran acceder a las instalaciones aún cuando estos no tienen inscrito algún ETS, en las fechas de aplicación de este. Por otro lado, también se busca que los docentes puedan tener los elementos necesarios para verificar la identidad de los estudiantes. Estos dos enfoques de la aplicación móvil serán abordados mediante la implementación de un sistema de consulta de alumnos que se conectara a la base de datos escolar de la institución para verificar los accesos y asistencia a los ETS según sea el caso, en este se mostrara la información necesaria para que tanto el personal de seguridad como los docentes puedan

tomar la decisión de permitir el acceso o presentar el examen a los alumnos, esto último también permitirá registrar la asistencia de los alumnos a las evaluaciones para su posterior consulta.

Además de esto, el módulo de reconocimiento facial se encuentra pensando para servir como una capa de seguridad adicional que permita a los docentes verificar la identidad de los estudiantes cuando la información que estos obtienen de las consultas al sistema no son suficientes para tomar una decisión.

De igual forma la propuesta de solución contará con un módulo de notificaciones e informativa relacionada a los ETS inscritos, asignados y horarios de aplicación.

Finalmente, para que la aplicación pueda simular un escenario real se pretende realizar un sistema web para la gestión de los usuarios que componen al sistema es decir, alumnos, docentes, personal de seguridad y de exámenes ETS.

Definición del proyecto

Retomando lo dicho en la propuesta de solución, se llevó a cabo una aplicación móvil que sirve como herramienta para el personal de seguridad y el personal académico en temporadas de ETS para disminuir la suplantación de identidad dentro de la ESCOM.

La aplicación móvil cuenta con varios apartados dependiendo del usuario que la esté usando, ya sea un alumno, un personal académico o un personal de seguridad, sin embargo, la parte fundamental de esta aplicación es la del docente y la del alumno en la que pueden realizar el reconocimiento facial. El alumno puede hacer el reconocimiento facial a su persona para rectificar que no haya ningún problema con su pase de asistencia al momento de llegar a realizar su ETS. El personal académico cuenta con la parte de reconocimiento facial como una forma de pasarle la asistencia al ETS al alumno.

Es importante mencionar que el pase de asistencia no se realiza únicamente con el reconocimiento facial, se puede realizar mediante otras formas, como con un escaneo a la credencial del alumno o más fácil, si el personal académico ya conoce al alumno de antemano, lo puede especificar así al momento de registrar la asistencia al alumno.

Por parte del personal de seguridad, puede registrar el ingreso a la unidad académica (ESCOM), por medio del escaneo de la credencial del alumno.

La aplicación móvil cuenta con más secciones, tales como la visualización de los datos de los ETS, el calendario escolar del IPN, una sección de mensajes para que se los alumnos se puedan comunicar con el personal académico y para que el personal académico se pueda comunicar tanto con los alumnos como con otro personal académico.

2.1. Objetivo general

Desarrollar un sistema de autenticación y control de acceso de alumnos con el motivo de evitar posibles casos de suplantación de identidad durante la aplicación de Exámenes a Título de Suficiencia (ETS) en la Escuela Superior de Cómputo (ESCOM), mediante el uso de tecnologías de reconocimiento facial, dispositivos móviles y credenciales escolares.

2.1.1. Objetivos específicos

- Explorar el estado del arte en cuanto a técnicas para el reconocimiento facial y recuperación de datos (rostros).
- Evaluar distintos algoritmos y técnicas de reconocimiento facial para determinar aquellas que se ajusten mejor a las necesidades y recursos del proyecto.
- Desarrollar una aplicación móvil para estudiantes y docentes que integre la tecnología de reconocimiento facial y proporcione un acceso seguro a los servicios relacionados con los exámenes (consulta de horarios), incluida la verificación de la identidad.
- Diseñar e implementar un módulo de reconocimiento facial capaz de verificar la identidad de los estudiantes durante las fechas de aplicación de los ETS, garantizando que solo los estudiantes inscritos puedan presentar el examen.
- Implementar una sección dentro de la aplicación móvil que permita reforzar el control de acceso a las instalaciones durante los días de aplicación de ETS con la ayuda de códigos QR.
- Desarrollar un sistema web básico para la gestión de ETS y que permita la integración de una API para simular escenarios reales con el fin de demostrar y evaluar la funcionalidad del sistema.
- Evaluar el desempeño del prototipo de sistema de control de acceso en cuanto a precisión, confiabilidad y seguridad.

2.2. Justificación

La implementación de un sistema de control de acceso basado en técnicas biométricas, como el reconocimiento facial, en conjunto con el uso de métodos de autenticación convencionales y ya establecidos como lo es el uso de las credenciales escolares, representa una innovación significativa en el ámbito educativo, particularmente en el contexto de los Exámenes a Título de Suficiencia (ETS). Este proyecto se justifica no solo por su capacidad de abordar retos específicos, sino también por la oportunidad que ofrece para explorar y aplicar tecnologías emergentes en el campo de la Inteligencia Artificial (IA) y la Visión por Computadora.

El uso de reconocimiento facial como método biométrico es especialmente relevante en el contexto educativo porque aporta precisión y fiabilidad en la verificación de identidad. Además, es un método de autenticación no invasivo y difícil de vulnerar en tiempo real ante la presencia del personal humano. Según estudios recientes, las tecnologías de autenticación biométrica presentan tasas de error mucho menores comparadas con métodos tradicionales, como contraseñas o credenciales físicas, las cuales pueden ser manipuladas fácilmente [?].

Desde una perspectiva educativa y tecnológica, este proyecto también tiene un valor en la formación de profesionales en el área de la inteligencia artificial. Implementar un sistema basado en Visión por Computadora y aprendizaje profundo permite a los desarrolladores ampliar sus conocimientos en tecnologías avanzadas. Estas habilidades son esenciales en un entorno global donde la IA está transformando rápidamente múltiples sectores, incluyendo el educativo. Incluso en algunos países como Estados Unidos y Australia este tipo de tecnologías ya han sido implementadas para cuestiones relacionadas con la seguridad dentro de los planteles [?].

En términos de beneficios para la comunidad que integra a la ESCOM, este proyecto no solo refuerza la seguridad y transparencia de los procesos evaluativos, sino que también genera confianza en la comunidad educativa. Profesores, estudiantes y personal administrativo podrán realizar sus actividades en un entorno más seguro, minimizando riesgos asociados con el acceso no autorizado [?].

Finalmente, la integración de estas tecnologías en dispositivos tan accesibles como los son los teléfonos celulares permite su escalabilidad a otros procesos académicos que requieran de la validación de la identidad de los alumnos o de llevar un control sobre las personas que entran y salen de las instalaciones, lo que a su vez puede influir en la adopción de este tipo de tecnologías en otros sectores públicos.

2.3. Alcance

Este proyecto tiene como objetivo realizarse dentro de la Ciudad de México, específicamente dentro de la ESCOM del Instituto Politécnico Nacional.

Específicamente, está pensado para que tanto el personal académico (específicamente para los docentes), el personal de seguridad y los alumnos, puedan usar la aplicación móvil planteada dentro de los periodos de ETS, tanto ETS ordinario como especial.

2.4. Requerimientos

El análisis de requerimientos arrojó un total de 26 requisitos de usuario (RU), distribuidos entre ambas plataformas: 17 para la aplicación móvil y 9 para el sistema web. Estos requerimientos fundamentales sirvieron como base para el diseño del sistema, generando 17 casos de uso para la plataforma móvil y 23 para la versión web.

Para consultar la documentación completa relacionada con este proceso, se recomienda revisar los siguientes apartados del Apéndice A:

- **A.1 Modelo del Alcance:** Contiene el detalle completo de los requerimientos de usuario identificados.
- **A.4 Modelo Dinámico:** Incluye la especificación completa de casos de uso para ambas plataformas.

2.5. Arquitectura

El desarrollo de la arquitectura general de todo el proyecto presenta una arquitectura de servicios con una comunicación REST. Apesar de dividir ciertas tareas en diferentes servicios, se presenta una arquitectura monolítica de servicio, puesto que mayor parte de la lógica de negocio se encuentra dentro del servidor Java Spring Boot.

Por otra parte, el servidor que proporciona los servicios de reconocimiento facial se encuentra funcionando como API de forma aislada usando una arquitectura en capas compuesta por una capa de presentación que contiene los puntos de entrada de la API, una capa de aplicación o dominio que contiene las reglas de negocio de la aplicación y la funcionalidad del flujo de trabajo del sistema de reconocimiento facial y finalmente, una capa de persistencia que contiene la lógica para consultar y modificar las fuentes de datos necesarias.

Para una descripción más detallada sobre las estructuras internas de cada parte del sistema, así como la organización de cada sección, se puede consultar la anexo ??

2.6. Modelo de información

Para realizar la parte del modelo de información se realizó un análisis que iba desde cubrir los datos de los alumnos, del personal académico y del personal de seguridad, así como las unidades académicas en las que estos trabajaban. Dentro de las unidades académicas también se vio que tuvieran programas académicos y dentro de estos, unidades de aprendizaje.

Todo el análisis que se llevó a cabo se pensó para cubrir todo el proceso realizado desde que se inscribe un alumno a alguna unidad académica dentro del Instituto Politécnico Nacional, hasta que un alumno reprueba alguna unidad de aprendizaje.

En el anexo ?? se detallan todas las entidades que se abstraieron del análisis de todo este proceso, a su vez, se detalla cada una de las variables que estas entidades contienen.

2.7. Metodología de desarrollo

Se adoptó el modelo en espiral como metodología principal, dado su enfoque iterativo y adaptable que permite incorporar mejoras progresivas, gestionar riesgos y ajustar los requerimientos durante cada fase. Este modelo combina la estructuración del enfoque en cascada con la flexibilidad para evolucionar conforme a las necesidades del proyecto.

El proceso se organizó en seis iteraciones clave, desde la investigación inicial hasta la implementación de prototipos funcionales. En cada ciclo se definieron objetivos específicos, se evaluaron riesgos y se generaron productos intermedios para garantizar el avance controlado del sistema. Para una descripción detallada de la metodología aplicada, consúltase el **Anexo A.3 Metodología**.

Actualmente, las instituciones educativas enfrentan desafíos significativos en cuanto a la protección de la identidad estudiantil y la seguridad académica, especialmente durante evaluaciones de alta importancia académica. En el Instituto Politécnico Nacional (IPN), particularmente en la Escuela Superior de Cómputo (ESCOM), los Exámenes a Título de Suficiencia (ETS) constituyen una oportunidad para que los estudiantes acrediten asignaturas pendientes. No obstante, la ausencia de mecanismos de autenticación robustos ha facilitado casos de suplantación de identidad, comprometiendo así la integridad académica y la seguridad dentro del plantel.

Este proyecto propone el desarrollo de un sistema de identificación y control de acceso basado en reconocimiento facial y códigos QR, integrado en una aplicación móvil para dispositivos Android. La solución busca no solo prevenir la suplantación de identidad, sino también facilitar y agilizar los procesos de verificación por parte de docentes y personal de seguridad.

Para sustentar técnicamente esta propuesta, el marco teórico explora conceptos clave como los ETS, tecnologías biométricas, desarrollo de software móvil y arquitecturas de sistemas, proporcionando así una base sólida para el diseño e implementación del prototipo.

3.1. Evaluaciones a Título de Suficiencia (ETS)

3.1.1. Definición y contexto

En el Instituto Politécnico Nacional (IPN), incluyendo unidades académicas como la Escuela Superior de Cómputo (ESCOM), la acreditación de cada unidad de aprendizaje se realiza semestralmente a través de 3 evaluaciones ordinarias. Si un alumno no acredita alguna unidad de aprendizaje, tendrá la oportunidad de presentar una evaluación extraordinaria. Estos procedimientos están detallados en el programa de estudios y se encuentran especificados en el calendario académico.

El alumno que no logre acreditar una o más de las unidades de aprendizaje en la que se haya inscrito podrá optar por acreditarlas mediante Evaluación a Título de Suficiencia, de acuerdo con lo establecido en el

Artículo 39 del Reglamento Interno del IPN, que señala:

“La evaluación del aprendizaje se llevará a cabo a través de exámenes ordinarios, extraordinarios y a título de suficiencia, cuyos requisitos y procedimientos de elaboración, presentación y exención, así como de otros mecanismos de evaluación continua, se realizarán en los términos que fijen los planes y programas de estudio, el presente Reglamento y los reglamentos respectivos.”

Existen dos rondas de ETS:

- ETS Ordinario: Esta es la primera oportunidad que tiene el alumno para acreditar la materia en la que no obtuvo una calificación aprobatoria. Los ETS ordinarios generalmente se aplican al finalizar el semestre, permitiendo al estudiante demostrar sus conocimientos sin necesidad de repetir el curso completo.
- ETS Especiales: Si el alumno no acredita la materia en el ETS ordinario, puede optar por presentar un ETS Especial. Esta es la segunda oportunidad que tiene el alumno para pasar la materia. Esta evaluación adicional suele programarse el primer viernes del nuevo semestre, brindando al estudiante una opción rápida para regularizar sus situación académica y continuar avanzando en su plan de estudios.

3.1.2. Procedimiento para realizar un ETS

- Pagar en caja, verificar que estén correctos los siguientes datos: Nombre, Boleta, Carrera y Número de unidades de aprendizaje.
- Acudir a ventanilla de gestión escolar para generar créditos en el “SAES”.
- Una vez generados los créditos, inscribe las unidades de aprendizaje en la página del “SAES”.
- Entregar en ventanilla de gestión escolar, el comprobante de inscripción de ES generador por SAES, y el recibo de pago para dar fin a la inscripción al ETS.
- Acudir el día y la hora establecida en el calendario.

3.1.3. Problemáticas identificadas

- Suplantación de identidad: La verificación manual de credenciales es vulnerable a fraudes, ya que no existe un sistema automatizado que valide la autenticidad del estudiante.
- El personal de seguridad no cuenta con herramientas para verificar si un alumno está inscrito en un ETS el día de su aplicación.

Estas limitaciones reflejan la importancia de implementar un sistema integral que incorpore tecnologías como el reconocimiento facial y los códigos QR, con el propósito de asegurar procesos más seguros, eficientes y transparentes durante la aplicación de los ETS.

3.2. Tecnologías de Identificación y Control de Acceso

3.2.1. Códigos QR

Actualmente, las credenciales del Instituto Politécnico Nacional (IPN) incluyen un código QR que, al ser escaneado, proporciona acceso a la información del estudiante. Este código QR contiene datos esenciales como el nombre completo, número de boleta, carrera en la que está inscrito y su fotografía. Además, en algunas unidades académicas del IPN, los códigos QR se utilizan como medio de verificación de identidad para el ingreso a las instalaciones mediante torniquetes electrónicos.

En la elaboración de nuestro trabajo terminal, el código QR desempeña un papel fundamental en la corroboración de la identidad de los estudiantes. Dado que las credenciales escolares contienen un código QR, se puede aprovechar su capacidad para almacenar y transmitir información de manera segura y rápida. Este enfoque permite verificar los datos del estudiante mediante un escaneo simple, reduciendo el tiempo necesario para validar su identidad. Al escanear el código QR de la credencial, el sistema accede a los datos del alumno, lo cual facilita confirmar si coinciden con la persona que se presenta al Examen a Título de Suficiencia (ETS).

Un código QR es un tipo de código de barras bidimensional que puede ser leído por teléfonos inteligentes o dispositivos especializados en su lectura. Al escanear un código QR, los dispositivos pueden acceder directamente a mensajes de texto, correos electrónicos, sitios web, números de teléfono, entre otros [?].

Anatomía y funcionamiento

A continuación, se describen los principales componentes estructurales de un código QR:

Patrones de detección de posición Estos patrones se encuentran en tres de las esquinas del código QR. Gracias a ellos, el escáner puede reconocer y leer el código rápidamente, ya que indican la orientación del mismo y facilitan su detección.



Figura 3.1: Patrones de detección de posición.

Patrones de alineación Estos patrones permiten corregir distorsiones cuando el código QR se imprime sobre superficies curvas. Su tamaño y cantidad varían dependiendo de la cantidad de información almacenada.



Figura 3.2: Patrones de alineación.

Patrones de temporización La alternancia de módulos negros y blancos determina el sistema de información, también conocido como cuadrícula de datos. Estas líneas permiten al escáner identificar la estructura del código.



Figura 3.3: Patrones de temporización.

Información sobre la versión Estos marcadores indican cuál de las 40 versiones del código QR se está utilizando. Generalmente, las versiones utilizadas van de la 1 a la 7.



Figura 3.4: Información sobre la versión.

Información del formato Este componente contiene datos sobre la tolerancia a errores y el patrón de enmascaramiento. Su función es facilitar el proceso de escaneo.



Figura 3.5: Información del formato.

Código de corrección de datos y errores El sistema de corrección de errores comparte espacio con los datos almacenados. Esto permite recuperar información incluso si una parte del código está dañada o ilegible [?].



Figura 3.6: Código de corrección de datos y errores.

Márgenes Los márgenes o zonas quietas rodean el código QR y permiten que el lector distinga claramente sus límites. Son fundamentales para que el escáner identifique correctamente el código.



Figura 3.7: Márgenes o zonas quietas.

Fiabilidad y aplicaciones en seguridad

Los códigos QR están diseñados para mantener su legibilidad incluso si están parcialmente dañados u oscurecidos. Esto es posible gracias a su capacidad de corrección de errores, que inserta la información varias veces en el patrón del código. Dependiendo del nivel de seguridad utilizado, un código QR puede seguir siendo legible incluso si se ha perdido hasta un tercio de su información original. Esta característica los convierte en una herramienta muy confiable para almacenar datos, especialmente en aplicaciones donde la seguridad y la verificación rápida de identidad son críticas. Por ello, su uso en credenciales escolares representa una solución eficaz para garantizar el acceso controlado a instalaciones o servicios institucionales.

3.3. Desarrollo de la Aplicación Móvil

Una aplicación móvil es un tipo de software diseñado para ejecutarse en dispositivos móviles, como teléfonos inteligentes o tabletas. A diferencia de las aplicaciones tradicionales para computadoras de escritorio, las aplicaciones móviles están optimizadas para operar con los recursos limitados de hardware de los dispositivos móviles, brindando funcionalidades específicas de manera eficiente. Aunque los dispositivos actuales son mucho más sofisticados que en sus primeras generaciones, las aplicaciones móviles siguen enfocándose en funciones concretas, permitiendo a los usuarios seleccionar solo las herramientas que necesitan en sus dispositivos [?].

Esta sección está enfocado en detallar el tipo de aplicación desarrollada para el proyecto, justificando las decisiones tecnológicas tomadas en cuanto a lenguajes, sistemas operativos y herramientas de desarrollo. Se explican los distintos tipos de aplicaciones móviles existentes, las razones detrás del uso de Kotlin y Android, así como las herramientas empleadas para su implementación.

3.3.1. Tipos de aplicaciones móviles

Existen diferentes tipos de aplicaciones móviles que responden a las necesidades y preferencias de los usuarios, así como a las capacidades técnicas de los dispositivos. A continuación, se detallan cada uno:

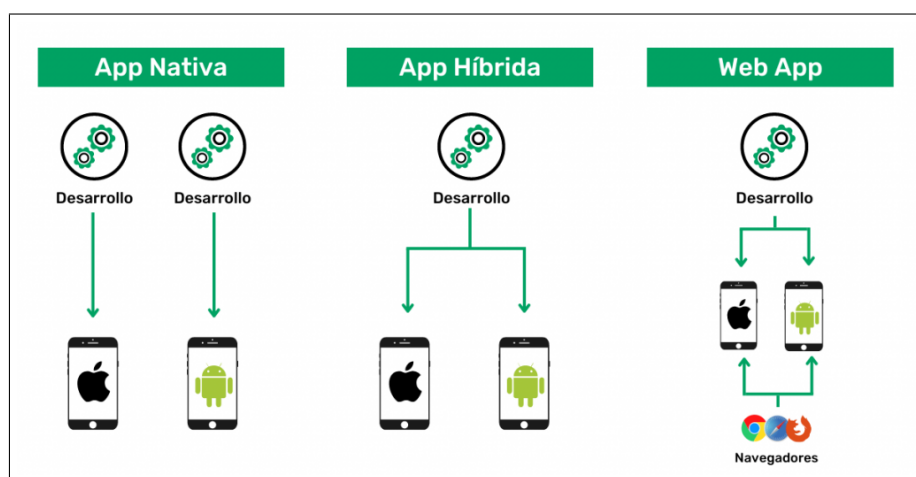


Figura 3.8: Tipo de aplicaciones móviles.

3.3.2. Aplicaciones nativas

Las aplicaciones nativas son apps desarrolladas para un sistema operativo móvil concreto (iOS o Android normalmente), en el lenguaje de programación específico de cada plataforma. Esto quiere decir que una app nativa creada para Android no puede ser utilizada en un dispositivo iOS y viceversa.

Es el tipo de aplicación móvil más conocida. Para que funcione, debemos descargarla desde los markets de apps, como App Store o Google Play e instalarla en nuestro teléfono [?].

Ventajas

- **Tienen el mejor rendimiento.** Las aplicaciones nativas son las más rápidas y tienen un rendimiento superior a otros tipos de apps, ya que han sido optimizadas específicamente para el hardware y el sistema operativo del dispositivo.
- **Acceso completo e integración con las funciones hardware del dispositivo.** Las apps nativas permiten aprovechar al máximo las funcionalidades móviles: cámara, micrófono, lector biométrico de huella, sensores y redes inalámbricas.
- **Pueden funcionar sin acceso a internet (funcionamiento offline)** si han sido diseñadas para ello.

Desventajas

- **Costes de desarrollo altos.** Si queremos tener nuestra app disponible para los dos sistemas, necesitaremos dos líneas de desarrollo diferentes, ya que el código utilizado para un sistema no es reutilizable para otro.
- **Complejidad de desarrollo.** Necesitamos equipos expertos en el lenguaje específico de cada sistema. Por ejemplo, en Kotlin para Android y en Swift para iOS.
- **Tiempo de desarrollo superior.** El desarrollo puede tomar entre 4 a 6 meses.

3.3.3. Aplicaciones Web

Las aplicaciones web realmente son webs especiales diseñadas para navegadores móviles. A diferencia de las apps nativas o híbridas, no necesitan ser descargadas, ya que se accede a ellas desde un navegador web.

Emplean las mismas tecnologías de desarrollo que una web, como HTML, CSS o JavaScript. Así, estaríamos hablando de una web con apariencia de app, por lo que presentaría sus mismas limitaciones. Sin embargo, con la llegada del HTML5, se han conseguido salvar algunas limitaciones, como el acceso a algunas funciones del móvil (geolocalización, cámaras) [?].

Ventajas

- **Carácter multiplataforma.** Con una sola línea de desarrollo.
- **Fácil desarrollo.** Se emplean tecnologías ampliamente conocidas.
- **Tiempo y coste de desarrollo bajo.**

Desventajas

- **Acceso limitado a las funciones del dispositivo.**
- **No se pueden subir a las tiendas de aplicaciones.**
- **Diferentes experiencias de usuario.** Estas dependen del navegador utilizado.
- **Necesidad de conexión a Internet.** Incluso si se cuenta con un modo pensado para ello, es necesario para acceder a las posibles actualizaciones o para entrar por primera vez.

3.3.4. Aplicaciones Híbridas

Las aplicaciones híbridas o multiplataforma combinan elementos de las aplicaciones nativas y las aplicaciones web. Estas aplicaciones se desarrollan utilizando tecnologías web como HTML, CSS y JavaScript, pero se empaquetan en un formato que puede ser instalado en un dispositivo móvil como cualquier otra aplicación nativa. Por tanto, podemos obtener una aplicación para varias plataformas con un único desarrollo [?].

Ventajas

- **Menor coste.** Gracias al uso de lenguajes de programación más conocidos, con una mayor disponibilidad de profesionales en el mercado.
- **Carácter multiplataforma.** Con una sola línea de desarrollo.
- **Acceso a algunas funcionalidades del móvil.**
- **Reducción de los tiempos de desarrollo.** Generalmente, el tiempo de desarrollo se reduce a 3 meses.
- **Disponibilidad en markets.** Se pueden subir a los markets de aplicaciones, como App Store y Google Play.

Desventajas

- **Rendimiento inferior.** Su rendimiento es inferior al de una app nativa, suelen tener un tamaño considerable y, además, ser más lentas.
- **Acceso limitado a las funciones del dispositivo.**

3.3.5. Aplicaciones Progresivas Web Apps (PWA)

Las aplicaciones progresivas son un reciente avance de las Web Apps. Al igual que las Web Apps, son webs diseñadas para móviles, pero esta vez, sí pueden ser descargadas en el móvil como una aplicación más, aunque no es necesario para que ofrezcan un comportamiento similar al de una app nativa a través del navegador.

Las PWA adoptan un comportamiento más propio de aplicaciones nativas que de web, como el funcionamiento sin Internet, un mayor rendimiento o su funcionamiento en segundo plano. Sin embargo, como desventaja, seguimos contando con la imposibilidad de subirlas a los markets de aplicaciones [?].

Para comprender mejor las diferencias entre los tipos de aplicaciones móviles, a continuación se presenta una tabla comparativa que destaca sus características clave:

Tipos de app	Nativa	Híbrida	Web
Interfaz	Basada en web	Específica de la plataforma (iOS, Android)	Basada en web
Tiempo de desarrollo	Alto	Medio	Bajo
Coste de desarrollo	Alto	Medio	Bajo
Multiplataforma	No	Sí	Sí
Rendimiento	Alto	Medio	Bajo
Acceso a los sensores del dispositivo	Completo	Alto o Completo	Limitado
Tiendas de aplicaciones	Sí	Sí	No

Tabla 3.1: Comparación tipos de aplicaciones móviles. Elaboración propia

Para el desarrollo de nuestro trabajo terminal, hemos decidido optar por una aplicación híbrida con el uso de Kotlin. Esta decisión se basa en varios factores relacionados con los recursos disponibles, las características de nuestro público objetivo y los plazos establecidos.

La aplicación móvil está dirigida para estudiantes, alumnos y personal de seguridad de la Escuela Superior de Cómputo, donde la mayoría utiliza dispositivos con sistema operativo Android. La elección de una aplicación híbrida nos permite optimizar la experiencia en Android, que es la plataforma que predomina entre nuestros usuarios.

Aunque las aplicaciones híbridas suelen desarrollarse con tecnologías web (como React Native o Flutter), para nuestro proyecto hemos decidido incorporar Kotlin para desarrollar modelos donde se requiera un rendimiento nativo o un acceso más profundo a las funciones del sistema operativo Android. Además, las aplicaciones híbridas permiten un desarrollo más rápido en comparación con las aplicaciones completamente nativas, ya que gran parte del código puede compartirse entre plataformas, así mismo, es una buena opción económica que se adapta a nuestro presupuesto [?].

3.4. Elección de tecnología

La elección de Kotlin como lenguaje principal y Android como plataforma se basa en criterios técnicos, prácticos y económicos que se alinean directamente con los requisitos del proyecto.

3.4.1. Kotlin

Kotlin es un lenguaje de alto nivel debido a su abstracción respecto al hardware y la plataforma subyacente. Esto significa que permite a los desarrolladores escribir código sin preocuparse por la gestión de memoria o las interacciones directas con el sistema operativo o el hardware, lo que mejora la productividad y la legibilidad del código, además, este es estáticamente tipado, lo que implica que los tipos de las variables se definen en tiempo de compilación, lo que permite detectar errores antes de ejecutar el programa. Este se usa principalmente en el desarrollo de aplicaciones Android, donde es el lenguaje recomendado por Google [?].

Además, Kotlin permite el uso de Jetpack Compose, un framework declarativo que moderniza la creación de interfaces de usuario en Android. También es compatible con bibliotecas clave para este proyecto como OpenCV, empleada en el módulo de reconocimiento facial.

3.4.2. Android

La decisión de desarrollar exclusivamente para Android se basa en:

- La mayoría de los usuarios objetivo (alumnos, docentes y personal de seguridad) utilizan dispositivos Android.
- Android permite mayor flexibilidad y personalización, crucial para funciones como escaneo de códigos QR, acceso a la cámara, integración con Python y OpenCV.
- El entorno Android es ideal para pruebas, ya que se puede emular fácilmente en computadoras con Windows.

La compatibilidad de Android con Python facilita la integración del frontend (Android) con el backend, donde Python se encarga del procesamiento de imágenes y entrenamiento de modelos de reconocimiento facial, mientras que Spring Boot gestiona la lógica del sistema. Esta arquitectura garantiza un funcionamiento sincronizado, eficiente y seguro.

3.5. Herramientas de Desarrollo

El desarrollo de la aplicación se realizó con herramientas modernas que forman parte del ecosistema oficial de Android, permitiendo una implementación robusta y escalable.

3.5.1. Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial para el desarrollo de aplicaciones del mismo OS. Se basa en IntelliJ IDEA, un entorno de desarrollo integrado de Java para software, e incorpora sus herramientas de desarrollo y edición de código [?].

Para respaldar el desarrollo de aplicaciones dentro del sistema operativo, Android Studio utiliza un sistema de compilación basado en Gradle, un emulador de Android, plantillas de código e integración de GitHub. Cada proyecto en Android Studio tiene una o más modalidades con código fuente y archivos de recursos [?].

3.5.2. Jetpack Compose

Para la implementación de la aplicación móvil que forma parte de nuestro sistema de identificación y control de acceso, hemos decidido utilizar Jetpack Compose. La elección de la tecnología adecuada es importante para ofrecer una mejor experiencia de usuario y cumplir nuestros objetivos de diseño y funcional.

¿Por que Jetpack compose?

Jetpack compose es un framework (estructura o marco de trabajo que, bajo parámetros estandarizados, ejecutan tareas específicas en el desarrollo de un software) con la particularidad de ejecutar prácticas modernas en los desarrolladores de software a partir de la reutilización de componentes, así como también contando con la oportunidad de crear animaciones y temas oscuros. En este sentido, Jetpack Compose es el conjunto de herramientas ofrecidas por Android para el desarrollo de aplicaciones con un objetivo específico: simplificar y optimizar los códigos en la IU nativas [?].

Ventajas

- **Menos código:** Simplifica el proceso de desarrollo haciendo menos código, todo se basa en funciones de modo que el código será simple y fácil de mantener.
- **Intuitiva:** Tan solo describe tu IU con un enfoque declarativo haciendo “qué hay que hacer” en vez de “cómo se debe hacer”.
- **Potente:** Tiene integrado Material Design con el cual puede crear apps atractivas al usuario con animaciones y mucho más.
- **Acelera el desarrollo:** Es compatible con proyectos existentes, puedes empezar a integrarlo por partes cuando quieras y donde quieras.
- **Kotlin:** Está escrito 100% en Kotlin, lo cual nos permitirá usar sus herramientas potentes y API's intuitivas.

Arquitectura Jetpack

La arquitectura o estructura sobre la que se basa Jetpack Compose es una estructura Jetpack que se encarga principalmente de seguir ejecutando y beneficiándose de aquellos componentes de Android según

la funcionalidad disponible. Por lo tanto, Jetpack Compose desarrolla herramientas denominadas «composables» a partir de elementos como botones o listados de objetos. A partir de fuentes de datos, pueden ser reutilizadas y ejecutadas en distintas fuentes sin la necesidad de programar varios códigos repetitivos.

Desde un aspecto comparativo, la creación de UI con Compose se realiza de forma similar a los de React Native. Es decir, mediante componentes reutilizables evitan maximizar la cantidad de códigos necesarios y repetitivos, tal como ocurre con HTML en la forma en la que se conjugan uno con otro. Sin embargo, es importante destacar que dicha compilación de componentes que logran la disminución en cantidad de códigos se realiza gracias a los plugins de compiladores de Kotlin, ejecutando así los componentes mediante la estructura de archivos de Kotlin [?].

Estructura de Jetpack Compose

Para entender de forma más sencilla la estructura de esta herramienta, conviene tener en cuenta componentes como:

- **Compiler:** Se encarga mediante una estructura de plugins en Gradle, donde logra la interpretación y simplificación de códigos.
- **El entorno de ejecución:** Es el ámbito “depurador”, si se puede establecer de este modo, en el que se establecen y diferencian los componentes que necesitan actualización y cuáles no, así como también se crean listados de mantenimiento y composición.
- **UI:** Es el componente que interpreta el lenguaje establecido en el entorno de ejecución y lo refleja en pantalla.

3.5.3. Gradle

Citando a [?], Gradle es el sistema de automatización de compilación utilizado por Android Studio, y fue esencial para el desarrollo del presente proyecto, permitiendo una integración fluida entre los módulos móviles y el backend. Esta herramienta de código abierto destaca por su rendimiento y flexibilidad, utilizando Kotlin DSL (Domain Specific Language) para definir scripts de compilación.

Gradle facilita la gestión eficiente de dependencias externas como Jetpack Compose, Retrofit y OpenCV, utilizadas en el desarrollo de la aplicación. Además, su capacidad para realizar compilaciones optimizadas en paralelo y reutilizar salidas anteriores permite reducir considerablemente los tiempos de construcción. También permite configurar distintos entornos de ejecución (debug y release), lo que fue útil para probar características específicas antes de su despliegue oficial.

En este proyecto, Gradle también permitió la integración con servicios backend mediante llamadas a APIs REST, conectando la aplicación con el servidor desarrollado en Spring Boot, fortaleciendo así la arquitectura cliente-servidor de la solución.

3.5.4. Git y GitHub

Citando a [?], durante el desarrollo del proyecto, se utilizó Git como sistema de control de versiones y GitHub como plataforma de colaboración y almacenamiento en la nube. Git, diseñado por Linus Torvalds,

permite llevar un control detallado del historial de cambios en el código, lo cual es esencial en proyectos donde trabajan varios desarrolladores de forma simultánea .

Cada miembro del equipo pudo trabajar en ramas independientes, realizar modificaciones, proponer mejoras y fusionar los cambios mediante revisiones de código, garantizando la integridad del repositorio principal. Esta metodología permitió detectar errores, revertir versiones cuando fue necesario, y mantener una trazabilidad clara del desarrollo de cada módulo.

Citando a [?], GitHub, además de servir como repositorio central, funcionó como herramienta de comunicación y documentación del avance del proyecto. Las funcionalidades de issues, pull requests y proyectos ayudaron a distribuir tareas, dar seguimiento a problemas y mantener una organización clara durante todo el ciclo de desarrollo.

3.6. Bases de datos (BD)

Para que todo sistema pueda funcionar, debe de tener una forma de almacenar toda su información, ya sea información de los usuarios o para almacenar imágenes, cualquier tipo de información que se requiera almacenar, es necesario usar una base de datos. Pero en sí, ¿qué es una base de datos?. Una base de datos, según Microsoft [?] , es una herramienta para recopilar y organizar información. Estas pueden almacenar información sobre personas, productos, pedidos u otras cosas. Las bases de datos pueden empezar desde un simple documento de textos o archivos Excel, pero conforme más va creciendo esta base de datos empiezan a aparecer redundancias, por lo que es mejor optar por usar una base de datos creada por un sistema gestor de bases de datos. Un sistema gestor de bases de datos es un software constituido por una serie de programas dirigidos a crear, gestionar y administrar la información que se encuentra en una base de datos. El principal objetivo de estos sistemas es servir de interfaz entre los usuarios y las aplicaciones para facilitar la organización de los datos [?]. Por otro lado, dentro del mundo de las bases de datos, existen diferentes tipos de bases de datos, y cada tipo de base de datos tiene una organización diferente y propósito diferente. A continuación, les hablaremos de ciertos tipos de bases de datos:

3.6.1. Bases de datos relacionales:

Este tipo de bases de datos es el más usado en sistemas que necesitan almacenar una gran cantidad de información que está relacionada entre sí. Ahora bien, una base de datos relacional, según Google [?], es una forma de estructurar información en tablas, filas y columnas. La ventaja de este tipo de bases de datos es que tiene la capacidad de establecer relaciones entre la información mediante tablas, esto nos ayuda a visualizar mejor la información sobre la relación entre los datos. Las bases de datos relacionales se componen de una serie de conceptos clave que necesitamos desarrollar para mejorar la comprensibilidad acerca de este tipo de bases de datos:

- **Tablas:** Las tablas, en una base de datos relacional, son objetos que contienen todos los datos de dicho objeto. Como se mencionó anteriormente, estas tablas se organizan en filas y columnas. Aquí, cada fila representa un registro único y cada columna un campo dentro del registro. La manera en que este tipo de tablas guardan datos únicos es por medio de una clave principal, la cuál se va a explicar a continuación. [?]

- **Clave principal:** Como ya se mencionó, las tablas tienen una clave principal, la cuál suele ser una columna o un conjunto de columnas cuyos valores identifican la forma única de cada fila de la tabla. Estas columnas se denominan claves principales de la tabla y aunque la clave principal sea una combinación de columnas, esta combinación es única dentro de la tabla. [?]
- **Clave externa:** Esta es una columna o combinación de columnas que se usa para establecer un vínculo entre los datos de dos tablas. Cuando una tabla tiene dentro de sus columnas la clave principal de otra tabla, se dice que esa primera tabla está referenciando a la otra por medio de una clave externa. [?]
- **Sistema de gestión de bases de datos (SGBD):** Son sistemas que ayudan a controlar las bases de datos. Estos sistemas actúan como interfaz entre los usuarios y las bases de datos, y se encargan justamente de gestionar los datos y las bases de datos como tal. En otras palabras, un sistema de gestión de bases de datos es un software utilizado para gestionar, almacenar y recuperar bases de datos, a su vez, proporciona una interfaz que permite a los usuarios leer, crear, borrar y actualizar datos. [?]

PostgreSQL:

PostgreSQL es una SGBD relacional, el cual, a diferencia de otros este soporta tipos de datos relacionales y no relacionales. Este fue creado con el propósito de soportar cargas de trabajo desde pequeñas aplicaciones hasta sistemas complejos de procesamiento de datos a gran escala. Para este proyecto, se hará uso de este SGBD, ya que tiene compatibilidad con los diferentes lenguajes que vamos a usar, como lo es Java, y además, es libre de restricciones de licencia. [?]

3.6.2. Bases de datos no relacionales:

Este tipo de bases de datos no siguen el esquema de filas y columnas como las bases de datos relacionales, en su lugar, este tipo de bases de datos usan un modelo de almacenamiento que está optimizado para los requisitos del tipo de dato que van a guardar. Dentro de este tipo de bases de datos existe una gran variedad, hay bases de datos que almacenan su información como pares clave/valor simple, como formatos JSON, o como un grafo que consta de bordes y vértices. Lo que caracteriza a este tipo de bases de datos es que no usan el modelo relacional. [?]

3.6.3. Bases de datos vectoriales

Dado la reciente popularidad de los sistemas potenciados por inteligencia artificial cuyas aplicaciones van desde el procesamiento inteligente de los textos, la detección de anomalías o, para fines de este trabajo, la identificación y verificación de las características faciales de las personas es que ha surgido la necesidad de contar con soluciones para almacenar y procesar de forma rápida grandes volúmenes de información y, aunque existen herramientas especializadas para estas tareas es importante saber distinguir la solución que mejor se adapte a las necesidades del proyecto.

Por lo tanto, algunos proyectos trabajan con un tipo de dato conocido como *embedding* que no es más que una representación numérica (vector) de objetos del mundo real como texto, audio o imágenes. Se

encuentran optimizados para funcionar como entradas a modelos de aprendizaje automático, aprendizaje profundo y, especialmente, para llevar a cabo búsquedas semánticas.

Los embeddings son representados en un espacio vectorial multidimensional en donde los objetos con características similares se encuentran más cerca entre sí. Además, otra característica distintiva de este tipo de representación es que, a diferencia de otras técnicas de aprendizaje automático, se aprende directamente de los datos mediante el uso de algoritmos de aprendizaje profundo teniendo la ventaja de prescindir de una definición explícita por parte de expertos humanos.

Con esto en mente, una base de datos vectorial es un tipo de base de datos especializada y optimizada para almacenar, indexar y recuperar datos vectoriales de alta dimensionalidad (*embeddings*). A diferencia de las bases de datos relacionales tradicionales, que se basan en tablas estructuradas y coincidencias exactas, las bases de datos vectoriales se centran en las búsquedas de similitud. Esto les permite recuperar puntos de datos que están semánticamente o contextualmente relacionados, en lugar de solo coincidencias exactas [?].

Debido a que este tipo de bases de datos se encuentran diseñadas para trabajar con conjuntos de datos complejos y multidimensionales, el uso de bases de datos tradicionales como las relacionales para la integración de sistemas de inteligencia artificial es difícil, por no decir imposible.

A modo de resumen, se presenta la siguiente tabla comparativa para identificar las principales diferencias entre este mecanismo de almacenamiento con respecto a las bases de datos tradicionales.

Tabla 3.2: Comparación entre bases de datos vectoriales y tradicionales.

Característica	Bases de datos vectoriales	Bases de datos tradicionales (Ej. Relacionales)
Estructura de datos	Vectores de alta dimensionalidad (embeddings)	Tablas estructuradas con esquemas predefinidos
Método de consulta	Búsquedas de similitud (distancia euclidiana, coseno)	SQL (operaciones relacionales, filtros)
Caso de uso principal	Búsqueda semántica, sistemas de recomendación, RAG, reconocimiento facial	Gestión de transacciones, informes estructurados
Modelo de escalabilidad	Horizontal (añadir nodos para capacidad y rendimiento)	Vertical (mejorar hardware) y Horizontal (sharding)
Enfoque de rendimiento	Búsquedas de vecinos más cercanos, indexación ANN	Consultas SQL rápidas, integridad transaccional

3.7. Spring Boot

Spring Boot es una herramienta que sirve para desarrollar tanto aplicaciones web, como microservicios o aplicaciones web en Java. Este se basa en Spring Framework, el cual es un framework para el desarrollo de aplicaciones y contenedores de inversión de control de código abierto, igualmente para Java. Spring Boot permite el desarrollo de aplicaciones web y microservicios de manera rápida y fácil gracias a 3 características que tiene, las cuales son:

- **Configuración automática:** Con esto se refiere a que Spring Boot lo que hace es analizar las dependencias incluidas en un proyecto, como lo son bases de datos, servidores, entre otras cosas y decide qué configuraciones aplicar. Esto se basa en anotaciones que usualmente empiezan con un "@", gracias a esto, podemos desarrollar aplicaciones de manera más rápida y eficiente, pues nos ahorra todo el trabajo de realizar método por método. [?]
- **Enfoque obstinado de la configuración:** Con base en las mejores prácticas de la programación, Spring Boot toma ciertas decisiones de configuración que funcionan bien para la mayoría de casos, esto se refiere a configuraciones de puertos, rutas, entre otras cosas. Además, en caso de que nosotros queramos agregar configuraciones adicionales, o bien, cambiar la configuración de algo de nuestro proyecto, nos va a crear un archivo de configuración en donde podremos especificarle de qué manera lo queremos. [?]
- **Capacidad de crear aplicaciones independientes:** Nos permite crear aplicaciones que se ejecutan de forma independiente, es decir, sin necesidad de un servidor web externo. Esto lo logra gracias a que en el proceso de inicialización, se integra un servidor web en la aplicación, o que le permite funcionar de manera autónoma. [?]

En este caso en particular, se utilizó de Java Spring Boot para hacer un servidor que funcionara como una API REST para que los diferentes sistemas involucrados se pudieran comunicar de manera más sencilla.

3.8. Java Persistence API (JPA)

Antes de irnos a la definición de JPA, hay que entender qué es la persistencia de los datos. La persistencia de los datos es un medio mediante el cual una aplicación puede recuperar información desde un sistema de almacenamiento y hacer que esta persista. Ahora bien, JPA lo que hace es proporcionarnos varias funciones para poder gestionar la persistencia y la correlación de objetos, es decir, lo que hace es proveernos con una serie de interfaces que podemos utilizar para implementar la capa de persistencia de nuestra aplicación. [?]

Algunas de las ventajas que nos presenta JPA es que nos permite hacer el mapeo de entidades, es decir, el definir cómo se relacionan las clases de nuestra aplicación con los elementos de nuestra base de [?]. Esto abarca temas como las relaciones entre clases y tablas en nuestra abse de datos, propiedades de las clases y los campos de las tablas e incluso la relación entre diferentes clases y las claves exrternas de nuestras tablas.

Además, JPA ofrece la capacidad de interactuar con diferentes sistemas de gestión de bases de datos, ya que no usa sentencias SQL o de algún tipo de base de datos en específico, por lo que mejora la portabilidad y escalabilidad de las aplicaciones. [?]

Spring Boot cuenta con una serie de anotaciones que ayudan a simplificar el desarrollo de las aplicaciones. Esta anotaciones son metadatos que se adjuntan a elementos del código para proporcionar información adicional a la máquina virtual de Java o al contenedor IoC de Spring, estas actúan como invariantes de compilación, garantizando que una condición específica (como la sobreescritura correcta de un método) se cumpla en tiempo de compilación.

Las anotaciones usadas en este proyecto fueron las siguientes:

- **@Configuration:** Esta anotación se usa en clases que definen beans. Esta anotación es un análogo para un archivo de configuración XML, solo que la configuración se hace mediante clases Java.

- **@Value:** Indica una expresión de valor predeterminado para un campo o parámetro al inicializar una propiedad.
- **@Bean:** Se usa a la par de @Configuration para crear beans Spring, es decir, para crear objetos gestionados por Spring.
- **@Override:** Se usa para sobrescribir un método. Usar @Override garantiza en tiempo de compilación que estás sobrescribiendo correctamente un método.
- **@Service:** Marca una clase Java que realizará algún servicio, como ejecutar la lógica de negocio, realizar cálculos y llamar a API's externas.
- **@PostConstruct:** Define un método que debe ejecutarse después de que el bean haya sido inicializado y todas sus dependencias hayan sido inyectadas.
- **@Embeddable:** Esta anotación quiere decir que las propiedades de una clase pueden ser incluidas en otra como un valor
- **@Column:** Define variables de una clase como columnas en una base de datos.
- **@ManyToOne:** Se encarga de generar una relación de muchos a uno.
- **@JoinColumn:** Sirve para hacer referencia a la columna que es llave foránea en una tabla, y la cual se encarga de definir la relación.
- **@Entity:** Registra una clase como una entidad.
- **@Table:** Se encarga de mapear una entidad contra una tabla que tenga un nombre distinto.
- **@Id:** Indica que una variable será registrada como una llave primaria de una entidad.
- **@JsonProperty:** Sirve para indicar el nombre de una variable en un JSON.
- **@OneToOne:** Se encarga de generar una relación uno a uno en una base de datos.
- **@OneToMany:** Se encarga de generar una relación uno a muchos en una base de datos.
- **@EmbeddedId:** Se utiliza para definir una clave primaria compuesta embebida en una entidad.
- **@MapsId:** Indica que un atributo de una entidad comparte su identificador con otra entidad, esto llega a ser muy útil en relaciones donde se comparten claves primarias.
- **@GeneratedValue:** Esta anotación se usa muy seguido, sobre todo porque hay veces en las que las claves primarias que tenemos deben de ser valores autoincrementables o definidos. Esta anotación es justo lo que hace, definir un valor a una llave primaria.
- **@UniqueConstraint:** Define restricciones de unicidad en columnas de una tabla, para asegurar que los valores que se inserten sean únicos.
- **@Temporal:** Se usa para definir tipos de variables Date, Time.

- **@JoinColumns:** Define múltiples columnas de unión en relaciones compuestas entre entidades.
- **@PrePersist:** Sirve para ejecutar alguna acción antes de que se guarde una nueva entidad.
- **@PreUpdate:** Indica que un método debe ejecutarse antes de que una entidad sea actualizada.
- **@RestController:** Con esta anotación se construyen servicios REST de manera más fácil y efectiva. Indica que una clase manejará las solicitudes HTTP y que devolverá datos JSON.
- **@RequestMapping:** Se usa para mapear solicitudes HTTP a métodos Request.
- **@Autowired:** Inyecta dependencias de una clase automáticamente.
- **@GetMapping:** Se usa para mapear solicitudes HTTP a métodos GET.
- **@PostMapping:** Se usa para mapear solicitudes HTTP a métodos POST.
- **@RequestPart:** Se usa para vincular una parte de una solicitud a un parámetro de un método.
- **@RequestParam:** Indica que un parámetro de un método se debe vincular a un parámetro de una solicitud. Un ejemplo de esto es para extraer datos de una URL.
- **@DeleteMapping:** Se usa para mapear solicitudes HTTP a métodos DELETE.
- **@CrossOrigin:** Permite solicitudes desde diferentes dominios.
- **@ExceptionHandler:** Define un método para manejar excepciones específicas lanzadas por el controlador.
- **@Repository:** Registra una clase como un bean, en este caso, define clases que pueden acceder a la base de datos.
- **@Query:** Define consultas personalizadas usando JPQL o SQL nativo.
- **@Modifying:** Se usa junto con @Query, y es para indicar que la consulta realiza operaciones de modificación, como insert, update o delete.
- **@Transactional:** Permite a una clase ejecutarse dentro de una transacción de una base de datos, es decir que si alguna parte de un método llega a fallar, todas las operaciones realizadas en dicho método se cancelan.
- **@Param:** Se usa para nombrar parámetros en consultas definidas con @Query.
- **@Service:** Define a una clase como un componente de servicio. Estas clases son las que tienen toda la lógica de negocio.
- **@Async:** Permite ejecutar métodos de manera asíncrona.
- **@Scheduled:** Programa la ejecución de un método en intervalos específicos.
- **@SpringBootApplication:** Marca la clase principal de una aplicación Spring Boot.

3.9. Patrones de arquitectura de software

Al momento de desarrollar software, es común toparnos con problemáticas que requieren de la toma de decisiones especialmente cuando hablamos sobre cuestiones relacionadas con el diseño de un sistema de software. Un patrón de arquitectura de software es un conjunto de decisiones tomadas para atacar problemáticas relacionadas con el diseño de un software. Estos incluyen reglas y principios para organizar las interacciones entre subsistemas predefinidos y los roles que estos desempeñan [?].

A menudo pueden ser descritos como los "*Planos*" de un sistema, sin embargo, esto no quiere decir que sea la arquitectura final, sino que funcionan como una guía que describe los elementos necesarios para diseñar la arquitectura de la solución a desarrollar, la selección de una arquitectura sobre otra dependerá completamente de los objetivos a alcanzar, los recursos disponibles y la experiencia del equipo de desarrollo [?].

Es necesario aclarar que, los patrones de arquitectura no deben confundirse con los patrones de diseño, ya que ambos responden a problemas diferentes durante el desarrollo de un sistema de software, de forma muy breve un patrón de arquitectura describe como crear la lógica de negocio, acceso a los datos, etc. Mientras que los patrones de diseño se usan al implementar estos elementos [?].

A continuación, se describen algunos patrones de arquitectura que pueden ser de utilidad al implementar la propuesta de solución para este trabajo terminal.

3.9.1. Arquitectura de capas

También conocida como *arquitectura de N-capas*, estructura una aplicación en múltiples capas distintas, donde cada una esta encargada de ciertas tareas en específico, lo que permite dividir un sistema en componentes aislados, lo que facilita el desarrollo rápido de aplicaciones ya que los cambios realizados en una capa no deberían afectar la lógica de las demás [?].

Las arquitecturas basadas en este patrón suelen implementar cuatro capas distintas, la capa de presentación, la capa de negocio, de persistencia y de base de datos, sin embargo, y como es de esperarse, la arquitectura de un sistema basado en este patrón puede tener algunas diferencias en el número y tipo de capas que se implementan, por ejemplo, algunas pueden implementar capas de aplicación, servicio o acceso a datos [?].

3.9.2. Arquitectura orientada a servicios

Las aplicaciones diseñadas siguiendo este patrón de arquitectura implementan una colección de servicios poco acoplados que se comunican entre sí a través de una red. Cada uno de los servicios que conforman al sistema se encarga de llevar acabo una función del negocio en específico que después pueden ser requeridos por otro servicio o cliente [?].

3.9.3. Arquitectura de micro servicios

Es una arquitectura que combina patrones de diseño para crear múltiples servicios que trabajan de forma independiente y que en conjunto forman la lógica de una aplicación. Es una alternativa a las aplicaciones monolíticas y de la arquitectura basada en servicios, en donde cada servicio esta orientado a implementar partes muy puntuales de la lógica de negocio [?].

Su principal ventaja radica en que debido a que sus componentes se encuentran poco acoplados pueden ser desarrollados, desplegados y probados de forma independiente [?].

La principal desventaja de este tipo de arquitectura es que puede llegar a ser compleja de implementar ya que requiere de definir la granularidad correcta de los servicios y establecer una comunicación efectiva entre estos [?].

3.9.4. El patrón modelo vista controlador

Este patrón divide una aplicación en tres componentes interconectados. El modelo, la vista y el controlador. Esta separación permite organizar el código al desacoplar la lógica del negocio, interfaz de usuario y el manejo de las entradas de los usuarios, lo que a su vez promueve la modularidad, mantenibilidad y escalabilidad [?].

El modelo contiene los datos y lógica de negocio de la aplicación. Se encarga de regresar, almacenar y procesar la información.

La vista, también conocida como interfaz de usuario (UI) despliega la información al usuario y responde a las interacciones del usuario.

El controlador funciona como un intermediario entre el modelo y la vista. Se encarga de gestionar las entradas del usuario, actualizar el modelo y la vista para reflejar los cambios realizados en el modelo [?].

3.10. Redes neuronales artificiales

Una red neuronal artificial es una técnica para la creación de programas de computación que son capaces de aprender de los datos. Se basa en el conocimiento actual sobre como funciona el cerebro humano. Consisten en un conjunto de nodos o neuronas interconectados entre sí que procesan y aprenden de los datos con los que cuentan, lo que permite llevar a cabo tareas como el reconocimiento de patrones y la toma de decisiones en el aprendizaje de máquina [?].

3.10.1. Estructura de una red neuronal artificial

Toda red neuronal esta formada por capas de nodos, o neuronas artificiales, una capa de entrada, una o más capas ocultas y una capa de salida. Cada nodo se conecta a los demás y tiene un peso y un umbral determinado. A continuación se describe el papel que cada una de estas capas en la arquitectura de una red neuronal común [?].

Capa de entrada

La capa de entrada es responsable de recibir los datos iniciales en forma de vectores. Cada neurona en esta capa corresponde a una característica del conjunto de datos [?].

Capas ocultas

Las capas ocultas son el núcleo de las RNA y donde ocurre el procesamiento de los datos. Estas capas aplican transformaciones no lineales mediante funciones de activación, como *ReLU*, *sigmoide*, o *tangente hi-*

perbólica. El número y tamaño de las capas ocultas determinan la capacidad de la red para modelar patrones complejos [?].

Capa de salida

La capa de salida genera los resultados finales del modelo. Para tareas de clasificación, por ejemplo, esta capa utiliza funciones como *softmax* para proporcionar probabilidades asociadas a cada clase [?].

3.11. Proceso de aprendizaje

Como ya se mencionó con anterioridad, una red neuronal es un sistema complejo que busca imitar la forma en la que el cerebro humano aprende. Este proceso de aprendizaje se realiza en dos fases conocidas como *backpropagation* y *forward propagation* [?]:

3.11.1. Forward propagation

- **Capa de entrada:** La capa de entrada contiene nodos que representan cada característica del conjunto de datos inicial. Estos nodos reciben y procesan los datos de entrada.
- **Pesos y conexiones:** Los pesos asignados a cada conexión entre neuronas determinan la influencia de una neurona sobre otra. Durante el entrenamiento, estos valores se actualizan constantemente para optimizar el rendimiento de la red.
- **Capas ocultas:** Las neuronas en las capas ocultas combinan las entradas recibidas al multiplicarlas por sus respectivos pesos y sumarlas. Posteriormente, aplican una función de activación que introduce no linealidad, lo que permite identificar relaciones complejas en los datos.
- **Capa de salida:** Este proceso se repite hasta llegar a la capa de salida, donde se genera el resultado final de la red neuronal.

3.11.2. Backpropagation

- **Cálculo del error:** La salida generada por la red se compara con los valores esperados utilizando una función de pérdida. Para problemas de regresión, se emplea comúnmente el *Error Cuadrático Medio* (*Mean Squared Error, MSE*), que calcula la diferencia entre las predicciones y los valores reales:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.1)$$

Donde n es el número de muestras, y_i son los valores reales y $\hat{y}_i$ son las predicciones.

- **Optimización por descenso de gradiente:** Para reducir el error, la red emplea el algoritmo de descenso de gradiente. Este ajusta los pesos calculando la derivada de la función de pérdida con respecto a cada peso, guiando los ajustes necesarios para minimizar el error.

- **Ajuste de pesos:** Este proceso de actualización de pesos se realiza en sentido inverso a través de toda la red, lo que permite optimizar las conexiones entre neuronas.
- **Proceso iterativo de entrenamiento:** Durante el entrenamiento, los pasos de propagación hacia adelante, cálculo del error y retropropagación se repiten varias veces con diferentes muestras de datos, permitiendo que la red refine sus parámetros y aprenda patrones específicos.

3.11.3. Funciones de activación

Las funciones de activación son fundamentales para introducir no linealidad en el modelo. Ejemplos comunes incluyen la función *ReLU* (Rectified Linear Unit) y la sigmoide. Estas funciones determinan si una neurona se activa, lo que depende del valor ponderado de sus entradas.

3.12. Tipos de redes neuronales

Las redes neuronales pueden clasificarse en distintos tipos, cada uno diseñado para cumplir propósitos específicos. Aunque no se trata de una lista exhaustiva, a continuación se describen algunas de las variantes más comunes y sus casos de uso principales [?]:

3.12.1. Perceptrón

El perceptrón es la red neuronal más antigua, desarrollada por Frank Rosenblatt en 1958. Es el precursor de las redes neuronales modernas.

3.12.2. Perceptrón multicapa

También conocidas como *feedforward neural networks*, estas redes están formadas por una capa de entrada, una o más capas ocultas y una capa de salida. Aunque frecuentemente se les llama MLPs, técnicamente están compuestas por neuronas sigmoides y no por perceptrones simples, ya que los problemas del mundo real suelen ser no lineales. Estas redes se entrenan alimentándolas con datos y son la base de aplicaciones como visión por computadora, procesamiento de lenguaje natural y otros modelos avanzados.

3.12.3. Redes neuronales convolucionales

Las redes neuronales convolucionales (*Convolutional Neural Networks*, CNNs) son una variante de las redes de avance directo que se utilizan principalmente en tareas como el reconocimiento de imágenes, el análisis de patrones y la visión por computadora. Estas redes aplican principios del álgebra lineal, en particular la multiplicación de matrices, para identificar patrones dentro de las imágenes.

3.12.4. Redes neuronales recurrentes

Las redes neuronales recurrentes (*Recurrent Neural Networks*, RNNs) se caracterizan por tener lazos de retroalimentación en su estructura. Estas redes son especialmente útiles para trabajar con datos tempora-

les, donde las predicciones dependen de la información previa. Por ejemplo, se aplican en la predicción de mercados financieros y en la proyección de ventas.

3.13. Limitaciones

Las RNA tienen la capacidad de capturar patrones complejos en grandes volúmenes de datos. Sin embargo, también presentan desafíos como [?]:

- **Necesidad de datos:** Requieren grandes cantidades de datos etiquetados para entrenarse adecuadamente.
- **Costo computacional:** Su entrenamiento puede ser intensivo en términos de tiempo y recursos computacionales.
- **Interpretabilidad:** Las predicciones de las RNA suelen ser difíciles de interpretar, lo que plantea retos en aplicaciones críticas.

3.14. Reconocimiento facial

El reconocimiento facial es una técnica que permite a las computadoras predecir la identidad de una persona desde una imagen [?].

Tiene múltiples aplicaciones en la industria como el desarrollo de sistemas de asistencia, en la salud, sistemas de seguridad, etc [?].

3.14.1. Pipeline de un sistema moderno de reconocimiento facial

A menudo, los términos verificación de rostros e identificación de rostros se usan de forma indistinta, sin embargo, aunque ambos comparten el mismo dominio del problema, lo abordan de forma distinta [?].

Para entender mejor la diferencia entre ambas tareas y su papel en el desarrollo de un sistema de reconocimiento facial, explicaremos como los sistemas modernos de reconocimiento facial operan a través de una secuencia de etapas bien definidas, donde cada una es indispensable para obtener resultados precisos y fiables [?].

En cualquier pipeline, la interdependencia entre sus etapas es crucial, ya que la calidad y el éxito de cada fase dependen directamente del desempeño de la anterior. Un ejemplo claro es la etapa de detección de rostros: los modelos de extracción de características esperan recibir una imagen de un rostro correctamente identificada. Si se emplean técnicas de detección poco robustas, el módulo siguiente puede fallar en la extracción de características, lo que resulta en una menor precisión en los resultados finales [?].

Por lo tanto, a continuación se presentan las etapas fundamentales de un sistema de reconocimiento facial actual:

La figura (figura ?? fue obtenida de [?]).

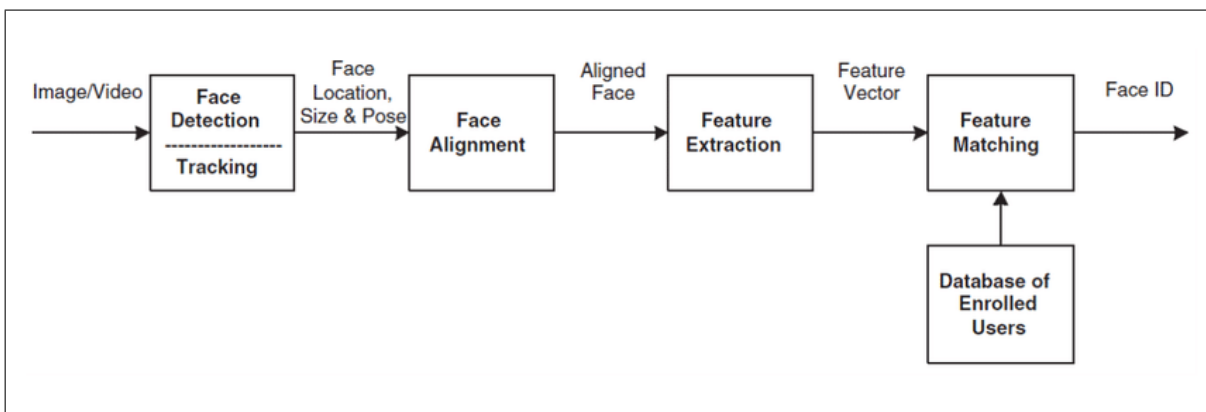


Figura 3.9: Pipeline moderno de un sistema de reconocimiento facial.

Detección de rostros

Esta es la etapa inicial, donde el sistema identifica la presencia de rostros en una imagen o video y determina su ubicación, generalmente mediante el cálculo de cuadros delimitadores. Se emplean modelos especializados que no solo identifican los cuadros delimitadores, sino que también detectan puntos clave faciales (como ojos, nariz y boca), los cuales son esenciales para la alineación posterior [?].

Algunos algoritmos y técnicas empleadas en este punto incluyen métodos tradicionales como el algoritmo Viola Jones o implementaciones más recientes basadas en aprendizaje profundo como MTCNN o RetinaFace [?].

Como se mencionó, este paso es la base del sistema de reconocimiento facial, por lo que su correcta implementación puede repercutir drásticamente en los resultados obtenidos.

Alineación de rostros

Una vez detectados, los rostros se estandarizan en orientación y tamaño utilizando los puntos clave extraídos. Este proceso implica rotar, escalar y recortar los rostros para asegurar que características como los ojos y la boca estén posicionadas de manera consistente [?].

El propósito de la alineación es reducir las variaciones causadas por la pose o la inclinación, haciendo que las entradas sean más uniformes para las etapas posteriores y asegurando que el modelo de embedding se enfoque en las características relevantes, lo que mejora la calidad y fiabilidad de los embeddings [?].

Se ha demostrado que, la alineación de rostros mejora la precisión obtenida por este tipo de sistemas [?].

Extracción de características o representación en forma de embeddings

Una vez que las imágenes de los rostros se encuentran correctamente procesadas, es decir, se identificaron y recortaron los rostros, seguidos de una alineación, se procede a usar algún modelo para la generación de embeddings como *FaceNet* o *ArcFace* los cuales generan vectores de alta dimensionalidad [?].

El propósito de esta etapa es transformar los datos complejos de una imagen en un formato numérico, compacto y sobre todo, comparable para lograr una eficiente comparación en el siguiente punto [?].

Identificación o verificación de rostro

Los embeddings faciales obtenidos se comparan con una base de datos utilizando medidas de similitud como la distancia coseno o euclidiana. Este paso permite determinar si dos rostros corresponden a la misma persona (verificación) o identificar a una persona entre múltiples registros (identificación) [?].

3.14.2. Identificación de rostros

Como ya se mencionó, la identificación de rostros y la verificación de rostros son dos tareas distintas involucradas al momento de implementar un sistema de reconocimiento facial [?].

La identificación facial se refiere a la tarea de identificar la identidad de una persona dada una imagen. La imagen se ingresa por un extractor de características para obtener una representación f . Luego, esta representación, entra a una red de clasificación para asignar la identidad asociada a la imagen de entrada [?].

La identificación de rostros es una tarea de clasificación, por lo que los sistemas de reconocimiento facial basados en esta tarea son entrenados usando la función *softmax* y la pérdida de entropía cruzada [4].

3.14.3. Verificación de rostros

Contrario a la identificación de rostros, la verificación de rostros busca verificar que dos imágenes pertenecen a la misma entidad [?]. Por lo tanto, en un sistema de reconocimiento facial basado en la verificación de rostros recibe como entrada una imagen x y el identificador del rostro a ser identificado y la salida es una decisión binaria sobre si la imagen x pertenece a la persona con el identificador dado [?].

Este enfoque requiere de un proceso de extracción de características de los rostros de las personas que queremos que el sistema verifique y, por lo tanto, almacenar dichas representaciones en una base de datos [?].

3.14.4. Reconocimiento facial mediante verificación

En los sistemas tradicionales de reconocimiento facial, se entrena un clasificador con K clases para asignar a cada rostro de entrada una identidad específica de entre K personas en la base de datos. Sin embargo, este enfoque presenta limitaciones de escalabilidad, ya que al agregar una nueva persona, es necesario reentrenar completamente el sistema con $K + 1$ clases. Esto ocurre porque el clasificador inicial tiene K neuronas, pero para incluir una nueva identidad, se requiere un clasificador con $K + 1$ neuronas [?].

Para abordar este problema, un enfoque más eficiente y escalable es utilizar la comparación basada en similitud, propia de los algoritmos de verificación. En este caso, dado un rostro de entrada x , se ejecuta el algoritmo de verificación K veces, una para cada rostro almacenado en la base de datos. La identidad del rostro de entrada se asigna a la correspondiente al ID para el cual el algoritmo de verificación indica una coincidencia [?].

Por lo tanto, la ventaja de este tipo de enfoque híbrido radica en el hecho de que es posible agregar nuevas entidades al sistema sin la necesidad de re-entrenar los modelos de deep learning utilizados para la generación de los vectores de características de los rostros, más aún si se combina con algún tipo de mecanismo de almacenamiento de estos como las bases de datos vectoriales para acelerar el proceso de inferencia [?].

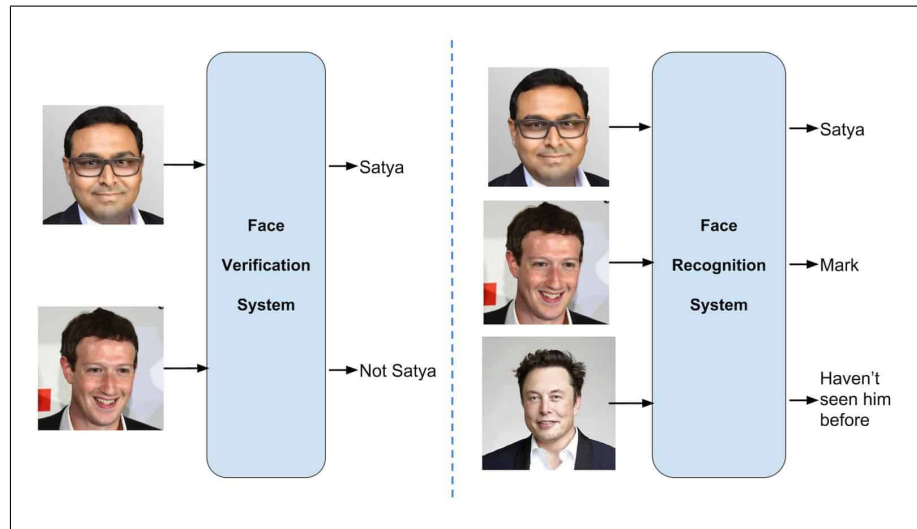


Figura 3.10: Diferencia entre verificación e identificación de rostros.

La figura (figura ?? fue obtenida de [?])

3.15. Algoritmos para la detección de rostros y extracción de características

Dentro del pipeline de reconocimiento facial, la detección y la extracción de embeddings son etapas críticas que se benefician de algoritmos especializados.

En este trabajo se prestara especial interés en el algoritmo MTCNN para la detección de rostros y FaceNet para la obtención de embeddings.

3.15.1. MTCNN (Multitask Cascaded Convolutional Networks)

La figura (figura ?? fue obtenida de [?])

Antecedentes

Cuando MTCNN (Multitask Cascaded Convolutional Networks) apareció en escena en 2016, con una presentación más formal en la conferencia ICCV de 2017, rápidamente se convirtió en un referente para la detección facial. ¿Por qué? Porque logra algo que, en ese momento, era un gran avance: combinar la detección de rostros y la alineación de puntos clave faciales en un solo proceso eficiente. En lugar de depender de modelos separados para cada tarea, MTCNN las integra en un pipeline único, lo que reduce significativamente el costo computacional [?].

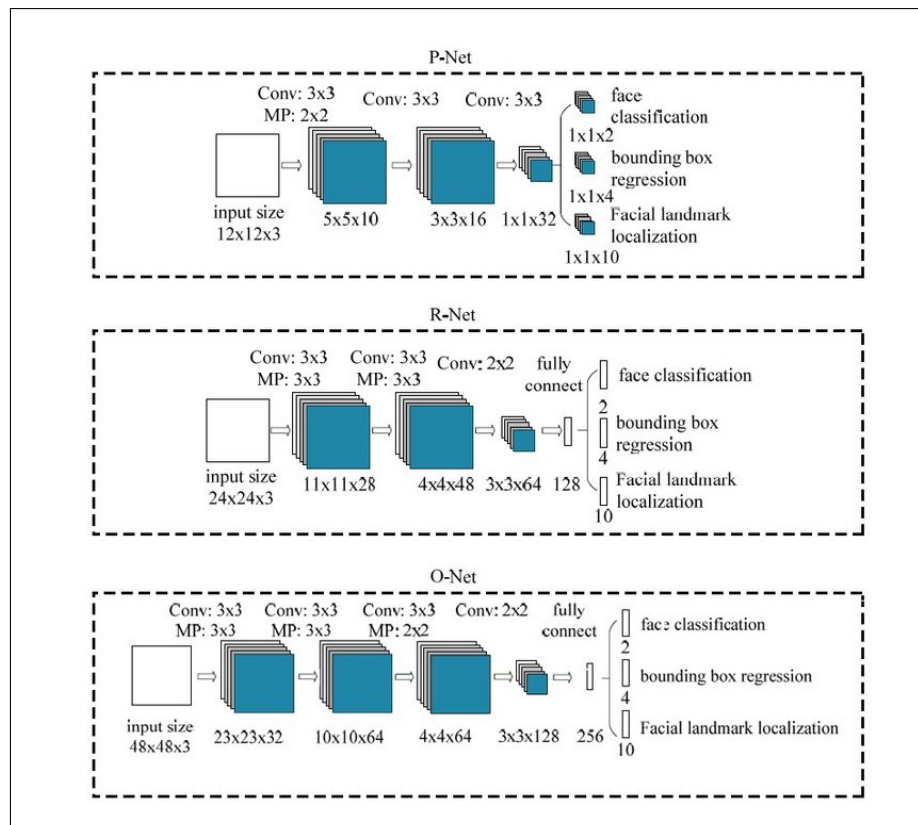


Figura 3.11: Arquitectura de MTCNN.

Arquitectura en Cascada: PNet, RNet, ONet

MTCNN emplea una estructura en cascada compuesta por tres redes neuronales convolucionales (CNNs) que colaboran para refinar progresivamente las propuestas de rostros y detectar los puntos clave [?].

Dada esta naturaleza en forma de cascada es que se da lugar a un refinamiento progresivo, iniciando con una detección general y avanzando hacia una alineación más precisa.

A continuación, se presenta una breve descripción sobre las redes neuronales que conforman a MTCNN [?]:

1. **PNet (Proposal Network)**: Esta es la red inicial que escanea la imagen en diferentes escalas para generar regiones candidatas de rostros, representadas por cuadros delimitadores. Su función es identificar posibles ubicaciones de rostros de manera eficiente.
2. **RNet (Refinement Network)**: RNet toma las regiones candidatas de rostros generadas por PNet, las refina filtrando los falsos positivos y ajusta aún más los cuadros delimitadores. Esta etapa mejora la precisión de las detecciones iniciales.
3. **ONet (Output Network)**: Como etapa final, ONet refina aún más los cuadros delimitadores y detecta cinco puntos clave faciales: los ojos, la nariz y las comisuras de la boca. Esta red proporciona la salida final de la detección y alineación.

3.15.2. FaceNet

Visión general

FaceNet es un sistema de reconocimiento facial desarrollado por investigadores de Google, presentado por primera vez en la Conferencia IEEE sobre Visión por Computadora y Reconocimiento de Patrones de 2015. Su concepto central es aprender un mapeo, o embedding, directamente desde un conjunto de imágenes faciales a un espacio euclidiano de 128 dimensiones [?].

Una vez que se encuentran los embeddings en este espacio vectorial, su similitud se determina calculando la distancia euclidiana entre los embeddings que se estén comparando.

Además de proporcionar un marco de trabajo para la generación de embeddings de calidad, FaceNet también puede ser usado para crear clusters de personas, lo que lo convierte en un modelo sencillo de entender y robusto hasta la fecha [?].

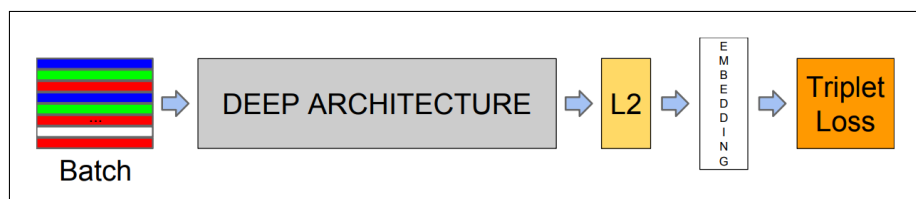


Figura 3.12: Arquitectura general de FaceNet.

La figura (figura ?? fue obtenida de [?])

Función de pérdida por tripletas (Triplet Loss)

La principal aportación de este modelo con respecto a otros es la implementación de un proceso de aprendizaje basado en Tripletas. Una tripleta hace referencia a un conjunto de tres imágenes, una imagen base, una positiva y una negativa en donde la imagen base es una imagen del rostro de una persona cualquiera, la imagen positiva es una imagen distinta de la misma persona base y, la imagen negativa, es una imagen de una persona distinta a la persona base [?].

Esta función está diseñada para asegurar que los embeddings de la misma persona estén muy cerca entre sí en el espacio vectorial, mientras que los embeddings de personas diferentes estén lo suficientemente lejos [?].

Esta optimización directa del espacio de embedding para el cálculo de similitud es lo que permitió la alta precisión de FaceNet y su influencia en modelos posteriores basados en embeddings [?].

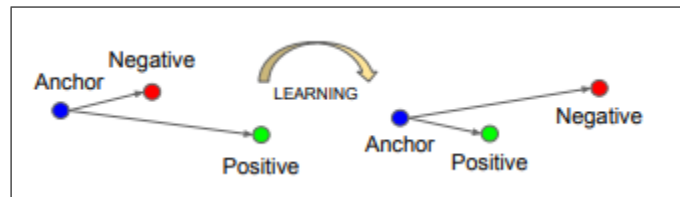


Figura 3.13: Aprendizaje basado en tripletas.

La figura (figura ?? fue obtenida de [?])

CASIA-WebFace

El conjunto de datos CASIA WebFace fue introducido en el trabajo "Learning Face Representation from Scratch" surgió como una necesidad de contar con conjuntos de datos a gran escala para el entrenamiento de modelos de aprendizaje profundo ya que, este tipo de sistemas requieren de una gran cantidad de datos etiquetados [?].

La principal ventaja de contar con conjuntos de datos grandes como este es la oportunidad que brindan para que nuevas arquitecturas aprendan a identificar y representar los rostros directamente desde cero, eliminando la dependencia de características manuales o los sesgos que pueden acarrear los modelos pre-entrenados [?].

Algunas de las características importantes de este conjunto de datos son [?].:

- **Tamaño:**

Es un conjunto de datos de gran tamaño en comparación con otros como LFW ya que su principal propósito es ser utilizado para el entrenamiento de modelos robustos. Por lo tanto, contiene aproximadamente 500 mil imágenes de rostros de un total de 10,575 personas extraídas directamente de la web.

- **Adquisición de imágenes**

Como su propio nombre lo sugiere, las imágenes que conforman a este conjunto de datos fueron obtenidas a través de la web, por lo que presentan una gran variabilidad en las condiciones en las que fueron tomadas, por ejemplo:

- **Iluminación:** Diferentes condiciones de luz (día, noche, sombras, reflejos).
- **Pose:** Varias poses de la cabeza (frontal, perfil, semi-perfil, inclinaciones).
- **Expresión:** Diversas expresiones faciales (neutra, feliz, triste, enojada, etc.).
- **Oclusión:** Posibles oclusiones parciales del rostro (gafas, cabello, objetos).
- **Fondo Fondos no controlados y variables, lo que añade complejidad.**
- **Calidad de imagen:** Las imágenes varían en resolución, nitidez y compresión.
- **Uso:** Dado el tamaño de este conjunto de datos, es posible usarlo para una variedad de tareas dentro del flujo de trabajo de un sistema de reconocimiento facial, por ejemplo, puede usarse para entrenar modelos basados en arquitecturas de redes neuronales profundas y, por otro lado, es posible usar una parte de este para evaluar el rendimiento de modelos que hayan sido entrenados con conjuntos de datos diferentes.



Figura 3.14: Algunas muestras del conjunto de datos CASIA WebFace.

La figura (figura ?? fue obtenida de [?])

Labeled Face in the Wild

Este conjunto de datos fue creado y mantenido por investigadores de la Universidad de Massachusetts, Amherst y se introdujo en el año 2007. Actualmente, su importancia radica en su papel como punto de

referencia para evaluar el rendimiento de algoritmos de reconocimiento facial en condiciones no controladas o como sus propios autores mencionan, en la naturaleza [?].

Sin embargo, a pesar de estar intrínsecamente relacionado con el reconocimiento facial, este conjunto de datos suele usarse especialmente al trabajar el problema de la verificación facial que como se explico anteriormente, consiste en decidir si dos imágenes pertenecen a la misma persona [?].

Dentro de los puntos que diferencian a este conjunto de datos con respecto a otros se destacan los siguientes [?]:

- **Tamaño** Contiene aproximadamente 13,233 imágenes de rostros de 5,749 personas únicas. A diferencia de CASIA WebFace, LFW es significativamente más pequeño en términos de número de imágenes y, sobre todo, en el número de identidades.
- **Adquisición de imágenes** El origen de las imágenes que conforman al conjunto de datos es la web, por lo que trae consigo una gran variabilidad y desafíos para el reconocimiento facial como variación intra-clase alta, casos de variación de pose, iluminación y oclusión y la presencia de fondos complejos.
- **Estructura para evaluación** LFW es conocido por su protocolo de evaluación estándar para la verificación facial. Este protocolo define 6,000 pares de imágenes, de los cuales 3000 pares son de la misma persona, 3000 pares son de personas distintas.
- **Uso** Se utiliza principalmente como un marco de referencia para la evaluación de tareas de verificación facial y, comparación de algoritmos de reconocimiento facial.

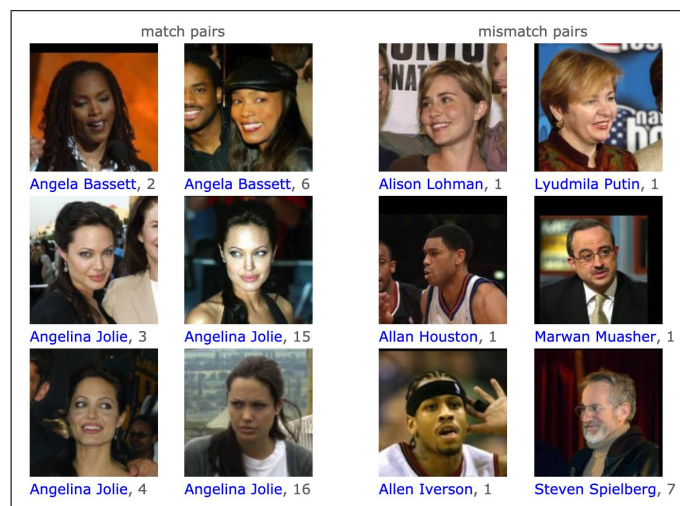


Figura 3.15: Algunas muestras del conjunto de datos LFW.

La figura (figura ?? fue obtenida de [?])

3.16. Herramientas para el desarrollo de modelos de Inteligencia Artificial (Reconocimiento facial)

El desarrollo de sistemas de reconocimiento facial se fundamenta en la integración de tecnologías modernas de aprendizaje profundo y servicios web. A continuación, se explica de forma breve algunas de las herramientas que mayor impacto tienen en el desarrollo de soluciones que integran este tipo de tecnologías de reconocimiento facial.

3.16.1. Tensorflow

TensorFlow, desarrollado por Google, es una biblioteca de código abierto que se ha consolidado como una de las herramientas más poderosas para el aprendizaje automático y profundo. Su diseño flexible permite construir y entrenar redes neuronales de diversa complejidad, desde modelos básicos hasta redes neuronales convolucionales (CNNs), ampliamente utilizadas en el procesamiento de imágenes y el reconocimiento facial [?].



Figura 3.16: Logo de Tensorflow.

TensorFlow ofrece un conjunto robusto de herramientas que son esenciales para el desarrollo de sistemas de reconocimiento facial [?]:

- **Definición de arquitecturas:** Permite construir modelos personalizados mediante la definición de capas neuronales, lo que facilita la creación de redes capaces de identificar patrones complejos en imágenes de rostros.
- **Entrenamiento escalable:** Soporta el entrenamiento de modelos en grandes conjuntos de datos, aprovechando la aceleración por GPU para reducir significativamente los tiempos de procesamiento.
- **Optimización de modelos:** Incluye algoritmos avanzados de optimización, como el descenso de gradiente estocástico, que ajustan los parámetros del modelo para maximizar la precisión en tareas como la clasificación y verificación facial.
- **Despliegue:** Facilita la exportación de modelos entrenados en formatos compatibles con entornos de producción, como aplicaciones móviles o servidores web.

La capacidad de TensorFlow para manejar tareas de aprendizaje profundo a gran escala lo convierte en una herramienta fundamental para el desarrollo de modelos de reconocimiento facial. Permite experimentar con diferentes arquitecturas, optimizar hiperparámetros y garantizar que los modelos sean lo suficientemente robustos para aplicaciones en el mundo real.

Algunos modelos de reconocimiento facial como FaceNet fueron desarrollados usando este framework [?].

3.16.2. DeepFace

DeepFace es una biblioteca de código abierto diseñada específicamente para tareas de reconocimiento facial, lo que reduce la necesidad de implementar algoritmos complejos desde cero. Al aprovechar modelos pre-entrenados y algoritmos optimizados, DeepFace permite a los desarrolladores centrarse en la integración y personalización de soluciones, en lugar de la construcción de modelos desde sus fundamentos [?].



Figura 3.17: Logo de Deepface.

Entre las funcionalidades clave de DeepFace se encuentran [?]:

- **Detección de rostros:** Identifica y localiza rostros en imágenes, aislando las regiones relevantes para su posterior análisis.
- **Alineación facial:** Normaliza la orientación y posición de los rostros detectados, minimizando la variabilidad causada por diferentes ángulos o expresiones faciales.
- **Extracción de embeddings:** Genera representaciones numéricas (vectores o embeddings) que capturen las características únicas de un rostro. Estos embeddings permiten comparar rostros de manera eficiente, ya que rostros similares generan vectores cercanos en un espacio de alta dimensionalidad, mientras que rostros diferentes producen vectores distantes.
- **Verificación facial:** Compara dos embeddings para determinar si corresponden a la misma persona, una tarea esencial en aplicaciones de autenticación.

- **Identificación facial:** Busca coincidencias de un rostro en una base de datos de rostros conocidos, permitiendo identificar a una persona en contextos como seguridad o control de acceso.

La integración de DeepFace en proyectos de reconocimiento facial agiliza el desarrollo al proporcionar herramientas preconfiguradas que encapsulan algoritmos complejos. Esto permite a los desarrolladores implementar soluciones robustas sin necesidad de un conocimiento exhaustivo de las arquitecturas subyacentes, haciendo que el reconocimiento facial sea más accesible y eficiente [?].

En pocas palabras, la importancia de DeepFace es que cuenta con herramientas fuertemente documentadas y probadas para llevar a cabo cada una de las etapas de un sistema de reconocimiento facial pero además, tiene una comunidad bastante activa y se encuentra en constante evolución [?]

3.16.3. FastAPI

Para que los modelos de reconocimiento facial sean accesibles en aplicaciones prácticas, es necesario exponer sus funcionalidades a través de interfaces de programación de aplicaciones (APIs). FastAPI, un framework web moderno basado en Python, se destaca por su alto rendimiento y facilidad de uso, siendo ideal para conectar modelos de aprendizaje profundo con aplicaciones externas [?].



Figura 3.18: Logo de FastAPI.

FastAPI ofrece varias ventajas que lo hacen adecuado para este propósito [?]:

- **Alto rendimiento:** Construido sobre Starlette y Pydantic, FastAPI es uno de los frameworks más rápidos de Python, lo que permite manejar solicitudes intensivas de aplicaciones de inteligencia artificial.
- **Documentación automática:** Genera documentación interactiva (Swagger UI y ReDoc) que facilita la prueba y adopción de la API por parte de otros desarrolladores.
- **Validación de datos:** Utiliza Pydantic para garantizar que las entradas y salidas de la API cumplan con los formatos esperados, reduciendo errores y mejorando la robustez.
- **Soporte asíncrono:** Permite manejar múltiples solicitudes simultáneamente sin bloquear el servidor, lo que es crucial para servicios de IA con tiempos de respuesta variables.

En el contexto del reconocimiento facial, FastAPI permite exponer funcionalidades como la detección de rostros, la extracción de embeddings y la verificación facial como servicios web. Esto significa que aplicaciones móviles, de escritorio o web pueden enviar imágenes al servidor y recibir respuestas estructuradas, como

la identidad de una persona o la confirmación de una verificación facial. FastAPI actúa como el puente que conecta los modelos de aprendizaje profundo con los usuarios finales, permitiendo la creación de sistemas integrales y escalables [?].

Finalmente, al ser un framework de python, facilita la integración con otras herramientas fundamentales en el campo del reconocimiento facial como las discutidas con anterioridad [?].

Dentro del presente proyecto, se escogió utilizar Java Spring Boot para la parte del backend debido a su facilidad para la organización de los archivos y debido a la rapidez con la que este realiza las tareas. Por parte del reconocimiento facial se usó Python, en el que se programó el reconocimiento facial con Facenet. Se hizo uso de este modelo puesto que con este, nos daba la facilidad de contar con un modelo ya entrenado y no hacer un dataset desde cero.

Por otra parte, para la aplicación, usamos Android, específicamente Jetpack Compose, debido a la accesibilidad y flexibilidad para modificar la interfaz de usuario en comparación con el enfoque tradicional de Android, el cual está basado en XML.

Para la parte de la página web de la simulación del SAES se utilizó Django por su forma rápida de desarrollar aplicaciones web, puesto que cuenta con muchas herramientas como la creación y manejo de formularios en un archivo específico. Así también, el manejo de rutas bien organizadas y su fácil consumo de APIs externas.

Este capítulo profundiza en la estructura y construcción del sistema, detallando los componentes clave y las tecnologías empleadas en su desarrollo. Se abordará la implementación de la aplicación móvil, el servidor de Spring Boot, y el servidor de la red neuronal.

4.1. Aplicación Móvil

La aplicación móvil está diseñada para ofrecer una interfaz de usuario intuitiva y reactiva. Su desarrollo se realizó íntegramente utilizando el framework moderno de Android, Jetpack Compose, el cual permite construir interfaces declarativas de manera eficiente. La gestión de dependencias y la configuración del proyecto se realizan a través de Gradle, donde se configuran los módulos necesarios para el desarrollo de la interfaz de usuario y la navegación.

4.1.1. Tecnologías y herramientas utilizadas

- **Lenguaje de programación:** Kotlin (versión 2.1.0)
- **Entorno de desarrollo:** Android Studio (versión 2024.2.1 Patch 3)
- **SDK de Android:** API nivel 35 (versión 15.0)

4.1.2. Arquitectura MVVM (Model-View-ViewModel)

La aplicación sigue el patrón MVVM para separar claramente la lógica del negocio, la UI y los datos. A continuación, se detalla cada componente:

Model (Modelo)

Esta capa es la responsable de la gestión de los datos de la aplicación mediante el acceso y la comunicación con el backend (Java Spring Boot).

ViewModel

Esta capa conecta la lógica de datos con la interfaz de usuario. Se encarga de preparar los datos, aplicar reglas de negocio y actualizar la vista cuando la información cambia.

View (Vista)

Esta capa es la encargada de definir y manejar los elementos visuales de la aplicación. Su función principal es presentar la información al usuario y registrar sus interacciones.

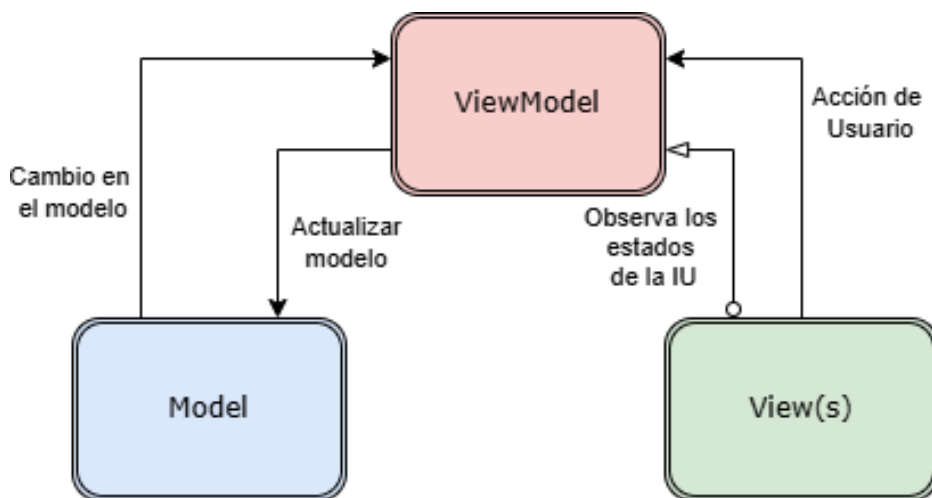


Figura 4.1: Arquitectura MVVM.

4.1.3. Componentes de las pantallas (Jetpack Compose y LaunchEffect)

Las pantallas de la aplicación móvil están compuestas por Composables de Jetpack Compose, que son funciones programables que describen cómo debe ser una parte de la interfaz de usuario. Estos Composables son los bloques fundamentales que construyen la interfaz de usuario, permitiendo un desarrollo modular y reutilizable. Para su funcionamiento, el proyecto utiliza dependencias clave de Jetpack Compose gestionadas por Gradle, tales como:

- **‘androidx.compose.foundation’:** Para componentes de interfaz de usuario de bajo nivel y funcionalidades de diseño como fondos, bordes y gestos.
- **‘androidx.compose.material3’:** Proporciona componentes de Material Design 3, facilitando la creación de interfaces modernas y consistentes.

- **‘androidx.compose.runtime’:** Contiene las APIs principales del tiempo de ejecución de Compose, fundamentales para la composición y recomposición de la interfaz de usuario.
- **‘androidx.compose.ui’:** Incluye las APIs básicas de UI de Compose, como elementos gráficos, eventos de entrada y diseños.
- **‘androidx.navigation.NavController’:** Es crucial para la gestión de la navegación entre las diferentes pantallas de la aplicación, permitiendo una experiencia de usuario fluida al moverse entre los distintos destinos Composables.

Para la gestión del ciclo de vida de la interfaz de usuario y la ejecución de operaciones con efectos secundarios (como la carga de datos o la inicialización de estados), se utiliza ‘LaunchEffect’. ‘LaunchEffect’ es un Composable de Jetpack Compose que se ejecuta cuando su clave cambia y se cancela cuando el Composable abandona la composición o la clave cambia nuevamente. Esto es fundamental para:

- **Carga inicial de datos:** Iniciar la obtención de información desde una fuente externa al entrar en una pantalla específica.
- **Manejo de eventos de ciclo de vida:** Realizar acciones cuando la pantalla se vuelve visible o deja de serlo.
- **Conexión con ViewModels:** ‘LaunchEffect’ es el punto donde las pantallas suelen invocar funciones de sus ViewModels asociadas para iniciar procesos asíncronos, como la recuperación de datos de la API.

Además de los componentes visuales principales y la gestión del ciclo de vida, la aplicación hace uso extensivo de los ‘Dialog’ (o ‘AlertDialog’) de Jetpack Compose. Estos elementos de interfaz son cruciales para:

- **Mostrar avisos al usuario:** Informar sobre operaciones completadas, errores, confirmaciones o cualquier mensaje importante que requiera la atención del usuario sin interrumpir drásticamente el flujo de la aplicación.
- **Solicitar confirmaciones:** Pedir al usuario que confirme una acción antes de proceder.
- **Presentar entradas cortas:** Recopilar información mínima en un contexto específico.

Estos diálogos mejoran la interacción del usuario al proporcionar retroalimentación contextual y opciones de control claras.

En resumen, la combinación de los Composables de Jetpack Compose para la construcción visual, el sistema de dependencias de Gradle para integrar las librerías esenciales (‘androidx.compose.foundation’, ‘androidx.compose.material3’, ‘androidx.compose.runtime’, ‘androidx.compose.ui’, ‘androidx.navigation.NavController’), ‘LaunchEffect’ para la orquestación de efectos secundarios en el ciclo de vida de la interfaz de usuario, y la utilización estratégica de ‘Dialog’ para los avisos al usuario, asegura que las pantallas de la aplicación móvil sean dinámicas, eficientes, navegables y respondan adecuadamente a las interacciones del usuario y a los cambios de estado del sistema.

4.2. Lógica de los ViewModels

La aplicación móvil se complementa y es orquestada por los ViewModels, que residen en el paquete `'com.example.prueba3.Views'`. Estos componentes son fundamentales en una arquitectura moderna de Android, ya que actúan como un puente robusto entre la interfaz de usuario (las pantallas de Jetpack Compose) y la lógica de negocio o las fuentes de datos. Su principal ventaja es que están diseñados para sobrevivir a los cambios de configuración del dispositivo (como rotaciones de pantalla), manteniendo el estado de la interfaz de usuario y los datos sin interrupciones.

Los ViewModels extienden la clase `'androidx.lifecycle.ViewModel'` y hacen un uso extensivo de las capacidades de concurrencia y gestión de estados de Kotlin Coroutines y Kotlin Flow para operaciones asíncronas y reactividad de datos.

4.2.1. Tecnologías y Componentes Clave en ViewModels

Para la gestión eficiente del estado de la interfaz de usuario y la ejecución de operaciones asíncronas, los ViewModels utilizan las siguientes tecnologías y componentes:

- **'androidx.lifecycle.ViewModel'**: Es la clase base principal de la que heredan todos los ViewModels de la aplicación. Proporciona un ámbito de ciclo de vida ('viewModelScope') que permite ejecutar operaciones asíncronas que se cancelan automáticamente cuando el ViewModel es destruido, evitando fugas de memoria.
- **'androidx.lifecycle.viewModelScope'**: Es un 'CoroutineScope' predefinido que pertenece al ViewModel. Esto significa que cualquier coroutine lanzada dentro de 'viewModelScope' se ejecutará en un hilo en segundo plano y se cancelará automáticamente si el ViewModel se destruye. Esto simplifica enormemente la gestión de operaciones de larga duración, como llamadas a la red o consultas a la base de datos.
- **'kotlinx.coroutines' (Coroutines)**: Los ViewModels emplean Coroutines para manejar la asincronía de manera estructurada y concisa. Permiten escribir código asíncrono que parece síncrono, mejorando la legibilidad y la depuración. Las coroutines son esenciales para:
 - **'kotlinx.coroutines.launch'**: Para iniciar una nueva coroutine en un 'CoroutineScope' específico, como 'viewModelScope'.
 - **'kotlinx.coroutines.Dispatchers'**: Para especificar el hilo en el que se ejecutará una coroutine (e.g., 'Dispatchers.IO' para operaciones de red o base de datos, 'Dispatchers.Main' para actualizaciones de interfaz de usuario).
 - **'kotlinx.coroutines.withContext'**: Para cambiar el contexto de ejecución de una coroutine a un 'Dispatcher' diferente de manera segura, útil para alternar entre hilos de secundarios y el hilo principal.
- **'kotlinx.coroutines.flow' (StateFlow y MutableStateFlow)**: Los ViewModels gestionan el estado de la interfaz de usuario y exponen los datos de manera reactiva utilizando Kotlin Flow, específicamente:

- **'kotlinx.coroutines.flow.MutableStateFlow<T>'**: Es un tipo de 'Flow' que puede emitir actualizaciones de estado mutables. Los ViewModels lo utilizan internamente para mantener el estado actual de la UI (ej., datos a mostrar, estado de carga, mensajes de error).
- **'kotlinx.coroutines.flow.StateFlow<T>'**: Es una versión de solo lectura de 'MutableStateFlow'. Los ViewModels exponen sus 'MutableStateFlow' internos como 'StateFlow' a las Composables de la UI ('.asStateFlow()'). Esto garantiza que la interfaz de usuario solo pueda observar los cambios de estado, sin poder modificarlos directamente, promoviendo la inmutabilidad y una gestión de estado más segura.
- **Manejo de Respuestas de API y Errores**: Para la interacción con las APIs y el procesamiento de respuestas, los ViewModels utilizan:
 - **'retrofit2.HttpException'**: Esta excepción se utiliza para capturar y manejar errores específicos que ocurren durante las llamadas HTTP realizadas por Retrofit (por ejemplo, errores de servidor como 404, 500, etc.). Permite a los ViewModels reaccionar a problemas de red o de API de forma controlada.
 - **'org.json.JSONObject'**: Se emplea para el análisis y la manipulación de datos en formato JSON, especialmente útil para extraer información de cuerpos de respuesta complejos o para construir cuerpos de solicitud.

En resumen, los ViewModels son el corazón de la lógica de presentación, utilizando un conjunto robusto de herramientas de Jetpack y Kotlin Coroutines/Flow para gestionar el estado de la interfaz de usuario de forma eficiente, reactiva y segura, facilitando la comunicación con las capas de datos y la lógica de negocio del sistema.

4.3. Lógica de las Instancias de Retrofit

La aplicación móvil se comunica con los servicios backend a través de Retrofit, una biblioteca de cliente HTTP de tipo seguro para Android y Java. Retrofit simplifica enormemente el proceso de realizar solicitudes de red, mapeando APIs RESTful a interfaces Kotlin con anotaciones. Para gestionar la interacción con los distintos servidores, el sistema utiliza dos instancias de Retrofit, cada una configurada específicamente para su respectivo servicio.

4.3.1. Configuración de Instancias de Retrofit

Ambas instancias de Retrofit utilizan `'GsonConverterFactory'` para la serialización y deserialización de objetos JSON, y `'OkHttpClient'` para la gestión de las operaciones HTTP subyacentes, incluyendo la configuración de timeouts.

Instancia para el Servidor Java Spring-Boot

Esta instancia de Retrofit está dedicada a la comunicación con el servidor principal de lógica de negocio, el Servidor Java Spring-Boot, desplegado en Azure. Su configuración prioriza la fiabilidad y un balance adecuado en los tiempos de respuesta para las operaciones transaccionales.

- **'RetrofitInstance' (Objeto Singleton):** Implementado como un objeto singleton para asegurar que solo exista una única instancia de Retrofit para este servidor en toda la aplicación, optimizando recursos.
- **'OkHttpClient':** Se utiliza un cliente HTTP personalizado con los siguientes timeouts:
 - **'connectTimeout(10, TimeUnit.SECONDS)'**: Establece un tiempo máximo de 10 segundos para establecer una conexión con el servidor.
 - **'writeTimeout(120, TimeUnit.SECONDS)'**: Permite hasta 120 segundos para enviar un cuerpo de solicitud al servidor.
 - **'readTimeout(120, TimeUnit.SECONDS)'**: Define un límite de 120 segundos para recibir una respuesta completa del servidor.

Estos tiempos están ajustados para manejar operaciones que podrían requerir un procesamiento moderado en el servidor, sin ser excesivamente laxos.

- **'GsonConverterFactory.create()'**: Utilizado para convertir automáticamente objetos Kotlin en JSON para las solicitudes y viceversa para las respuestas.

Esta configuración asegura una comunicación eficiente y tolerante a cierto nivel de latencia con el servidor principal de la aplicación.

Instancia para el Servidor de la Red Neuronal

Esta instancia de Retrofit está optimizada para interactuar con el Servidor Red Neuronal, también alojado en Azure, que es responsable específicamente de las funcionalidades de reconocimiento facial. Dada la naturaleza de estas operaciones (envío de imágenes y procesamiento intensivo), se requieren timeouts más generosos.

- **'RetrofitCamaraInstance' (Objeto Singleton):** Similar al anterior, es un singleton para una gestión eficiente de la conexión con el servidor de la red neuronal.
- **'OkHttpClient' con 'HttpLoggingInterceptor':**
 - **'HttpLoggingInterceptor().apply level = HttpLoggingInterceptor.Level.BODY':** Se incluye un interceptor de logging para fines de depuración y monitoreo. Este interceptor permite registrar en detalle (cuerpo incluido) las solicitudes y respuestas HTTP, lo cual es invaluable durante el desarrollo y la fase de pruebas para entender el flujo de datos.
 - **'connectTimeout(300, TimeUnit.SECONDS)':** Configurado a 300 segundos (5 minutos) para permitir conexiones en entornos con latencia o carga.
 - **'writeTimeout(300, TimeUnit.SECONDS)':** Permite hasta 300 segundos para la escritura de datos, esencial para el envío de imágenes que pueden ser de tamaño considerable.
 - **'readTimeout(300, TimeUnit.SECONDS)':** Un timeout de lectura de 300 segundos para esperar la respuesta del servidor, anticipando que el procesamiento de la red neuronal puede ser intensivo y tomar tiempo.
- **'GsonConverterFactory.create()':** También utilizado para la serialización/deserialización JSON.

La configuración de esta instancia subraya la necesidad de manejar operaciones que implican el envío de datos más pesados (imágenes) y un procesamiento que puede ser más lento, asegurando que la conexión no se cierre prematuramente.

4.4. Lógica de las Data Classes

Para la correcta interpretación y manipulación de la información intercambiada entre la aplicación móvil y los servidores (tanto el de Spring Boot como el de la Red Neuronal), el sistema hace uso de Data Classes de Kotlin. Estas clases son un componente esencial en la arquitectura de datos del proyecto, ubicadas principalmente en el paquete `'com.example.prueba3.DataClasses'`.

4.4.1. Propósito y Características de las Data Classes

Las Data Classes en Kotlin están específicamente diseñadas para contener datos. El compilador de Kotlin genera automáticamente funciones útiles para ellas, como `'equals()'`, `'hashCode()'`, `'toString()'`, `'copy()'`, y `'componentN()'`, lo que las hace ideales para modelar la estructura de los datos que se reciben de las APIs o se envían a ellas.

Su rol principal en el sistema es:

- **Modelado de Estructuras de Datos:** Definen de forma clara y concisa la estructura de los objetos JSON que se serializan o deserializan a través de Retrofit y Gson. Cada propiedad de una Data Class se mapea a un campo correspondiente en el JSON.
- **Inmutabilidad y Seguridad:** Por convención, las propiedades de las Data Classes se declaran como `'val'` (valores inmutables), lo que promueve una gestión de datos más segura y predecible a lo largo de la aplicación.
- **Facilidad de Uso con Retrofit y Gson:** Son perfectamente compatibles con `'GsonConverterFactory'` de Retrofit, lo que permite que el proceso de conversión de JSON a objetos Kotlin y viceversa sea automático y eficiente.

4.4.2. Ejemplos de Data Classes Implementadas

A continuación, se presentan ejemplos de Data Classes utilizadas en el proyecto para ilustrar su función en el modelado de la información:

Estas Data Classes, junto con otras implementadas en el proyecto, aseguran que los datos sean manejados de manera estructurada y segura a lo largo de la aplicación móvil, facilitando la comunicación con los servicios backend y la manipulación de la lógica de negocio.

4.5. Sistema Web Django

El sistema web fue implementado para representar el proceso de un escenario real completo que va desde la inscripción de un alumno a una escuela, la creación de su credencial, hasta su inscripción a un ETS en caso de reprobar alguna asignatura.

Este sistema no se desplegó en ningún servicio en la nube puesto que para los fines del proyecto no fue necesario. Sin embargo, este sistema se comunica con el servidor Java, el cual contiene todo el backend, del cual hablaremos más adelante.

Este sistema web fue implementado con el framework Django, el cual tradicionalmente se basa en el patrón de arquitectura Modelo-Vista-Plantilla (MTV por sus siglas en inglés). Sin embargo, para este caso en particular, el modelo no se encuentra implementado dentro de Django, ya que, tanto la lógica como la persistencia están completamente administradas por el servidor Java, tal y como se mencionó anteriormente.

Apesar de no seguir fielmente este patrón de arquitectura, en este sistema se adaptó de tal manera en la que la vista (archivos Views) actúa como el controlador, es decir, recibe las peticiones del usuario, consume la API externa y procesa los datos recibidos. Por otro lado, las plantillas (archivos template) generan las páginas HTML que se le mostrarán al usuario, y a su vez, recibe los datos procesados desde la vista.

El tipo de comunicación que se usa en este sistema junto con el servidor Java, es como el de una arquitectura cliente servidor descoplada, en el que el sistema web Django actúa como el cliente, consumiendo servicios por una API REST desarrollada en el servidor mediante peticiones HTTP y con un intercambio de datos en formato JSON.

4.6. Servidor Java Spring Boot

El servidor principal está diseñado con Spring Boot y desplegado en Azure como un App Service. Su despliegue en la nube permite la disponibilidad en línea para poder ser consumido por otras aplicaciones, como el sistema web desarrollado en Django y la aplicación móvil desarrollada en Kotlin.

El sistema fue implementado con una arquitectura en capas, es decir, cada capa tiene una responsabilidad específica. Las capas en las que está dividido el servidor son:

- Capa de presentación: Se encarga de manejar las solicitudes HTTP, la interacción con el usuario y la presentación de los datos. Aquí entran en juego los archivos controlador, es decir, los que tienen la anotación `@Controller` y `@RestController`.
- Capa de negocio o servicio: Contiene la lógica de negocio central de la aplicación. Los archivos de servicio son los que se encargan de manejar todas las operaciones y las que interactúan con la capa de persistencia. Estos archivos presentan la anotación `@Service`.
- Capa de persistencia: Maneja la interacción con la base de datos. Las clases encargadas de esto son los archivos repositorio, es decir, los que tienen la anotación `@Repository`, o, en su defecto, las que extienden interfaces de Spring Data JPA.
- Capa de Base de Datos: Es decir, la capa en donde se almacenan todos los datos.

La comunicación entre cada una de estas capas se realiza mediante objetos DTO, mientras que se comunica con otros servicios mediante servicios REST. Por otra parte, es importante mencionar que para la implementar la base de datos, se hizo uso de JPA para el mapeo de las clases a las tablas dentro de la base de datos.

Se definieron entidades como 'Alumno', 'UnidadAcademica' y 'UnidadAprendizaje', las cuales tienen la anotación @Entity y son mapeadas a tablas PostgreSQL. La relación entre tablas se configuró con anotaciones @OneToMany, @ManyToOne, entre otras.

Por otro lado, para la capa de persistencia, como se mencionó anteriormente, se usaron archivos repositorio, los cuales son interfaces que extienden de JpaRepository, los cuales permiten el acceso a datos sin necesidad de usar SQL manual.

En los servicios se crearon archivos tales como AlumnoServicio, DocenteServicio, entre otros, en los cuales se encargan de guardar nuevos registros, realizar consultas o incluso actualizaciones.

Por último, los controladores, que como se mencionó, son los que se encargan de recibir peticiones HTTP y de devolver respuestas JSON al usuario.

Adicional a esto, se utilizaron servicios externos como 'Cloudinary' y 'Firebase'. El primero se usó para guardar las imágenes que se toman al momento del registro de los alumnos desde la simulación del SAES, mientras que el Firebase se usó para manejar los mensajes dentro de la aplicación móvil.

4.7. Servidor Python Fast API

La API de reconocimiento facial es una aplicación REST diseñada para registrar y verificar estudiantes mediante imágenes faciales. Desarrollada con FastAPI, ofrece endpoints para procesar imágenes, extraer embeddings faciales y gestionar datos en Pinecone. Su estructura modular permite una fácil integración con clientes móviles (Kotlin) y web (Spring Boot, Django), proporcionando respuestas rápidas y consistentes.

4.7.1. Tecnologías y herramientas utilizadas

- **Lenguaje de programación:** Python
- **Framework:** FastAPI
- **Reconocimiento facial:** DeepFace (Modelo FastMtcnn para detección de rostros, FaceNet para extracción de embeddings)
- **Base de datos vectorial:** Pinecone
- **Entorno de desarrollo:** Visual Studio Code
- **Pruebas:** Pytest
- **Despliegue:** Docker, Azure

4.7.2. Arquitectura en capas

La API sigue una arquitectura en capas para separar responsabilidades:

Configuración

Definida en archivos de configuración, establece constantes como el host, puerto, claves de Pinecone y parámetros de DeepFace (e.g., MODEL_NAME). Estas constantes son luego utilizadas a lo largo de la aplicación y permiten una rápida configuración y la posibilidad de experimentar con otros modelos de reconocimiento facial, por ejemplo.

Rutas (Controladores)

Implementadas en `app/routes/`, manejan solicitudes HTTP (POST `/api/register_student`, POST `/api/search_verify_student`) desde clientes (Kotlin, Django) y delegan la lógica del negocio a la siguiente capa (servicios).

Servicios

Ubicados en `app/services/`, contienen la lógica de negocio, generando embeddings con DeepFace, interactuando con el repositorio y retornando respuestas estructuradas. Para el caso del servicio asociado al endpoint de registrar un estudiante se genera el embedding del estudiante y luego se hace una consulta a la base de datos vectorial para determinar si se hará una inserción o una actualización. Por otra parte, para la verificación de alumnos primero se realizará la búsqueda del alumno por boleta y se extraerá el embedding con el que se compara el embedding de entrada, en caso de no encontrarse notificará al controlador y este a su vez al cliente.

Repositorio

Definido en `app/repositories`, abstrae el acceso a Pinecone para operaciones de inserción y consulta de embeddings faciales. Contiene varios métodos según las necesidades del servicio con el que interactúa, para más detalles véase el apéndice ??.

Base de datos vectorial

La ultima capa corresponde a la capa de la base de datos, para esta aplicación se usó una base de datos vectorial debido a la naturaleza de los datos con los que se trabajaron (embeddings faciales). Esta decisión se debe a las facilidades de esta herramienta para realizar operaciones de búsqueda y registro sobre datos de alta dimensionalidad y la posibilidad de integrarse de forma nativa en la nube.

Esquemas

En `app/schemas`, validan solicitudes y respuestas (e.g., `RegisterResponse`) usando Pydantic. Esta es una parte importante de la API ya que garantiza que las respuestas que reciban los clientes se encuentren en el formato correcto y por ende, puedan funcionar correctamente.

Utilidades

Implementadas en `app/utills`, encapsulan la lógica de DeepFace para detección, alineación y generación de embeddings. También contiene otros archivos de utilidad que permiten leer y procesar imágenes y calcular la similitud entre embeddings faciales para determinar la identidad de los estudiantes siguiendo un valor de umbral establecido en 0.4 que surge de la experimentación realizada sobre el conjunto de datos LFW y los resultados presentados en ??.



Layered Architecture Pattern

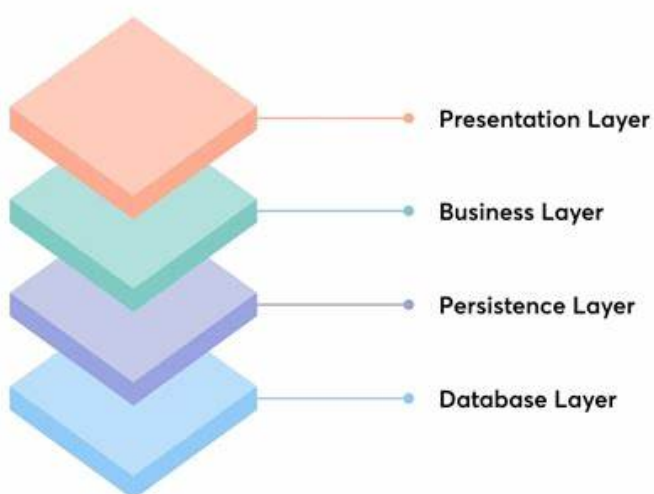


Figura 4.2: Arquitectura en capas.

4.7.3. Despliegue - Azure App Service

Al igual que el servidor principal de Java - Spring boot, la API de reconocimiento facial se encuentra desplegada en Azure a través de Azure App Services, sin embargo, a diferencia de dicho servidor, se considero un App Service Plan más robusto de cara a las operaciones intensivas involucradas en el proceso de reconocimiento facial que demandan una cantidad de recursos mayor, en este caso se optó por el plan Basic B2 que cuenta con 3.5 GB de memoria RAM y 2 vCPU, el doble con respecto al plan Basic B1 que se contrato para el servidor Java [?].

Al desplegar la API en este servicio, fue posible que clientes como la aplicación móvil o cualquiera de los dos backends, Django o Spring Boot hicieran uso de esta.

El aseguramiento de la calidad es un pilar fundamental en el desarrollo de cualquier sistema de software. En el presente capítulo, se detalla el proceso de validación del sistema propuesto a través de la ejecución de pruebas enfocadas en sus principales casos de uso. El objetivo es verificar que cada funcionalidad crucial del sistema se comporta de acuerdo con las especificaciones y requisitos definidos.

5.1. Pruebas de humo y detección temprana de errores

Durante la fase de codificación y desarrollo del sistema, se llevó a cabo varias pruebas de humo de manera continua. Estas pruebas informales, realizadas al finalizar cada sección o módulo de código, fueron cruciales para detectar defectos críticos en las etapas más tempranas del ciclo de desarrollo. Esta práctica permitió identificar y corregir errores conforme se presentaban, asegurando la funcionalidad básica de cada componente antes de la integración completa.

Entre los errores más destacables que se identificaron y subsanaron durante estas pruebas iniciales, se encuentran:

- **Errores de formato en la comunicación con el servidor:** Se observó que, en cualquier interacción con el servidor (tanto el de Spring Boot como el de la Red Neuronal), se producían errores de servidor ('Server Error') si el formato de la información enviada al servidor no coincidía con el esperado o, a la inversa, si la respuesta del servidor no se ajustaba a la estructura de datos que la aplicación móvil estaba preparada para recibir. Esto obligó a una revisión exhaustiva de la serialización y deserialización de datos para garantizar la coherencia.
- **Limitación de tamaño de imagen en el módulo de Reconocimiento Facial:** Se identificó un error crítico en el módulo de reconocimiento facial cuando un dispositivo móvil intentaba enviar una fotografía de muy alta resolución o de gran tamaño. El servidor de Azure, donde reside el modelo de red neuronal,

cancelaba la petición al exceder los límites de tamaño permitidos, lo que resultaba en fallos de comunicación. La solución implicó la implementación de un proceso de compresión de imágenes en el lado del cliente (la aplicación móvil) antes de su envío.

- **Manejo de errores en detección de rostros del módulo de reconocimiento facial:** Otro error significativo en el módulo de reconocimiento facial era que si la imagen enviada al servidor de la red neuronal no contenía un rostro detectable, el servidor fallaba inesperadamente en lugar de retornar un error manejable. Esto causaba un comportamiento no deseado en la aplicación y se requirió una modificación para que el servidor, al no detectar un rostro, respondiera con un error específico que pudiera ser capturado y gestionado adecuadamente por la aplicación móvil, informando al usuario de la situación de manera clara.

La detección temprana y corrección de estos y otros errores mediante las pruebas de humo fue vital para el correcto funcionamiento del sistema final, permitiendo avanzar a fases de prueba más formales con una base más sólida.

5.2. Pruebas de integración

Además de las pruebas de humo realizadas durante el desarrollo, se llevaron a cabo pruebas de integración para asegurar que los distintos componentes del sistema interactuaran correctamente entre sí. Estas pruebas se centraron específicamente en la conexión y el flujo de datos entre la aplicación móvil, el servidor de Spring Boot, el servidor de la red neuronal y la simulación del SAES.

El objetivo principal de estas pruebas fue verificar que:

- La aplicación móvil pudiera establecer conexiones exitosas con ambos servidores.
- Los datos se enviaran y recibieran correctamente entre la aplicación móvil y el servidor de Spring Boot (ej. inicio de sesión, gestión de datos de alumnos/ETS).
- La comunicación para el reconocimiento facial funcionara fluidamente entre la aplicación móvil y el servidor de la red neuronal, incluyendo el envío de imágenes y la recepción de resultados de verificación.
- La **simulación del SAES** se conectara adecuadamente con el servidor de la red neuronal para llevar a cabo los procesos de registro de usuarios y la generación de embeddings faciales.

Estas pruebas de integración fueron fundamentales para validar la arquitectura de comunicación de los servicios desplegados.

5.3. Pruebas de subsistema

Adicionalmente a las pruebas de humo e integración, se realizaron pruebas de subsistema exhaustivas para verificar el correcto funcionamiento y la interacción de conjuntos específicos de componentes que conforman funcionalidades clave del sistema. Estas pruebas se diseñaron para asegurar que cada módulo principal operara de manera coherente y eficiente, tanto de forma independiente como en conjunto con sus dependencias directas. Los subsistemas clave evaluados incluyen:

- **Gestión y visualización de reportes por docentes:** Se comprobó la funcionalidad completa de la creación y visualización de reportes por parte del personal docente. Esto incluyó la verificación de la generación de reportes de asistencia (con o sin el uso de reconocimiento facial), la correcta agregación de datos y la presentación adecuada de la información en la interfaz de usuario del docente.
- **Registro de asistencia escolar por personal de seguridad:** Se validó el flujo completo de registro de asistencia de alumnos a la escuela a través del personal de seguridad. Esta prueba abarcó desde la captura de la identificación del alumno hasta el registro exitoso de su entrada en la base de datos, asegurando la fiabilidad del proceso en los puntos de acceso.
- **Verificación de reconocimiento facial por alumnos:** Se realizaron pruebas detalladas sobre la funcionalidad de reconocimiento facial utilizada por los alumnos para verificar su identidad. Esto implicó la verificación de la captura de la imagen, el envío al servidor de la red neuronal y la correcta interpretación de la respuesta para determinar la identidad del alumno.
- **Funcionamiento del servidor Spring Boot:** Se sometió el servidor de Spring Boot a pruebas de subsistema intensivas para asegurar el correcto procesamiento de la lógica de negocio, la gestión de la base de datos (PostgreSQL) y la orquestación de las peticiones recibidas desde la aplicación móvil. Esto incluyó la verificación de la autenticación de usuarios, la gestión de datos de alumnos y ETS, y la integridad de las transacciones.
- **Funcionamiento de las funciones de creación de Embeddings y reconocimiento facial en el Servidor de la Red Neuronal:** El servidor de la red neuronal fue específicamente probado para validar la exactitud y eficiencia de sus funciones principales. Se verificó el correcto funcionamiento de:
 - La **creación de embeddings:** La capacidad de transformar una imagen facial en un vector numérico único que representa las características faciales.
 - El **reconocimiento facial:** La comparación de embeddings para determinar la similitud entre rostros y, por ende, la identidad o no identidad de un individuo.

Estas pruebas fueron cruciales para asegurar la precisión y el rendimiento del componente de inteligencia artificial.

Las pruebas de subsistema permitieron identificar y resolver interacciones complejas entre módulos, garantizando que cada pieza fundamental del sistema funcionara de manera óptima y contribuyera a la robustez general de la solución.

5.4. Pruebas de aceptación (encuestas de usuario)

La fase final de validación del sistema se llevó a cabo mediante pruebas de aceptación de usuario, específicamente a través de la aplicación de encuestas. Estas encuestas fueron diseñadas para recopilar la percepción directa de los usuarios finales (docentes, personal de seguridad y alumnos) sobre la usabilidad, la eficiencia, la satisfacción general y la efectividad del sistema en un entorno cercano al real.

El objetivo principal de estas encuestas fue:

- **Validar la satisfacción del usuario:** Medir el grado en que el sistema cumple con las expectativas y necesidades de los usuarios, más allá de la mera funcionalidad técnica.
- **Identificar oportunidades de mejora:** Recopilar retroalimentación cualitativa y cuantitativa que permita identificar áreas de mejora en la interfaz de usuario, la experiencia de usuario y las funcionalidades futuras del sistema.
- **Confirmar la aceptación del sistema:** Determinar si el sistema es adecuado para su implementación y uso en el entorno operativo previsto, asegurando que los usuarios se sientan cómodos y productivos con la herramienta.
- **Evaluar la efectividad percibida:** Cuantificar la percepción de los usuarios sobre cómo el sistema resuelve los problemas planteados, como la agilización del registro o la prevención de la suplantación de identidad.

Los resultados de estas encuestas, analizados detalladamente en el Capítulo de Resultados, fueron cruciales para obtener una perspectiva valiosa sobre la experiencia de usuario y confirmaron la viabilidad y aceptación del sistema en el contexto académico. La información recolectada no solo sirvió para validar el trabajo realizado, sino que también guió las recomendaciones para el trabajo futuro del proyecto.

5.5. Pruebas funcionales

Para cada caso de uso evaluado, se presentará un informe detallado que incluye la descripción del caso de prueba (CU), los supuestos y condiciones previas necesarias para su ejecución, los datos de prueba utilizados, los pasos específicos a ejecutar, el resultado esperado y el resultado real obtenido. Asimismo, se documentarán los errores detectados, si los hubo, y el estado final de la prueba. Esta metodología permite una revisión exhaustiva del comportamiento del sistema frente a las interacciones clave de los usuarios.

5.5.1. Inicio de sesión de la app móvil

Caso de Prueba	
	CP01 Inicio de sesión de la app móvil
Caso de uso relacionado	CU-01 Inicio de sesión de la app móvil
Cantidad de ejecuciones	100
Realizado por	Daniel Martín Huertas Ramírez
Supuestos y Condiciones Previas	• El usuario debe de haber instalado la aplicación en su celular. • El usuario debe de estar registrado en la base de datos.
Datos de Prueba	• Boleta en caso de ser alumno • RFC en caso de ser personal
Pasos a Ejecutar	1. Ingresar credenciales en campo usuario 2. Ingresar credenciales en campo contraseña 3. Presionar el botón 
Resultado Esperado	• Redirige al usuario a su pantalla principal • Muestra menú según tipo de usuario
Errores Detectados	Ninguno
Estado	Aprobado

5.5.2. Consultar calendario escolar

Caso de Prueba	
	CP02 Consultar calendario escolar
Caso de uso relacionado	CU-02 Consultar calendario escolar
Descripción	Validar la funcionalidad de obtención y visualización del calendario escolar del IPN, así como el cálculo de días faltantes para periodos de ETS.
Cantidad de ejecuciones	100
Realizado por	Daniel Martín Huertas Ramírez
Supuestos y Condiciones Previas	• El usuario inició sesión en el sistema. • El usuario ingresó a la pantalla  IU02 Pantalla Consultar calendario escolar. • Conexión a internet estable.
Datos de Prueba	• Ninguno requerido.
Pasos a Ejecutar	1. Presionar el ícono de calendario en el menú inferior. 2. Seleccionar 'Calcular días faltantes para ETS'.
Resultado Esperado	• Muestra correctamente la imagen del calendario escolar actual. • Proporciona mensaje preciso sobre días faltantes/transcurridos para ETS.
Errores Detectados	• El sistema no identificaba cuando el usuario estaba en periodo ETS activo. • No reconocía múltiples periodos ETS en un ciclo escolar.
Correcciones Implementadas	• Ajuste en la lógica para detectar periodos ETS activos. • Modificación en consultas para reconocer múltiples periodos.
Estado	Aprobado

5.5.3. Consultar ETS asignados

Caso de Prueba	
	CP05 Consultar ETS asignados
Caso de uso relacionado	CU-05 Consultar ETS asignados
Descripción	Validar el sistema de consulta y filtrado de ETS para docentes, incluyendo vista completa y asignaciones personales.
Cantidad de ejecuciones	100
Realizado por	Luis Antonio Flores Esquivel
Precondiciones	• Docente autenticado • Mínimo 5 ETS en sistema • Al menos 2 ETS asignados al docente • Conexión estable a BD
Conjuntos de Datos	• 3 perfiles de prueba: • Docente con 2 ETS asignados • Docente con 5+ ETS asignados • Docente sin asignaciones
Flujo de Prueba	1. Autenticarse como docente 2. Acceder a módulo ETS 3. Verificar: a) Vista completa (todos ETS) b) Filtro "Mis ETS" c) Ordenamiento por fecha
Criterios de Validación	• Vista completa muestra todos ETS activos • Filtro "Mis ETS" muestra solo asignaciones • Campos obligatorios por ETS: • Nombre • Fecha/hora • Estado • Salón • Tiempo respuesta <2s
Incidente Crítico	• IS-505: Filtro "Mis ETS" vacío • Causa: Variable de asignación no incluida en API • Solución: Modificación endpoint /ets/asignados • Verificación: 100% precisión en pruebas
Pruebas de Rendimiento	• 50 usuarios concurrentes • Tiempo promedio: 1.8s • Memoria máxima: 45MB

Validaciones Adicionales	• Paginación (>15 items) • Sincronización BD-UI • Offline mode (caché)
Estado	Aprobado

5.5.4. Mostrar información de los ETS asignados

Caso de Prueba	
	CP06 Mostrar información de los ETS asignados
Caso de uso relacionado	CU-06 Mostrar información de los ETS asignados
Descripción	Validar la visualización estable de datos completos de ETS, incluyendo manejo de concurrencia y validación de datos.
Cantidad de ejecuciones	100
Realizado por	Luis Antonio Flores Esquivel
Precondiciones	• Docente autenticado • Mínimo 3 ETS asignados • Datos completos en BD: • Tipo ETS • Unidad de aprendizaje • Detalles de horario • Asignación de salones
Conjunto de Datos	• 5 perfiles de ETS diferentes: • Regular (teórico) • Laboratorio • Examen • Extraordinario • Especial
Procedimiento de Prueba	1. Acceder como docente 2. Navegar a módulo ETS 3. Realizar 3 operaciones: a) Visualización normal (1 ETS) b) Cambio rápido entre ETS (stress test) c) Consulta en baja conexión (<1Mbps)
Requisitos de Visualización	• Campos obligatorios: • Tipo ETS • Unidad aprendizaje • Periodo/fecha • Detalles de salón • Cupo/disponibilidad • Tiempo carga <1.5s
Incidentes Críticos	• IS-506: Crash en cambios rápidos • Causa: Race condition en carga de datos • Solución: Implementación de patrón Promise.allSettled() • Verificación: 1000 cambios consecutivos exitosos

Métricas de Rendimiento	• Tiempo promedio carga: 1.2s • Uso memoria: <50MB • Estabilidad: 100 % en 100 ejecuciones
Validaciones Adicionales	• Consistencia BD-UI: 100 % • Prueba offline (caché) • Responsive (mobile/desktop)
Estado	Aprobado

5.5.5. Solicitar reemplazo de ETS

Caso de Prueba	
	CP07 Solicitar reemplazo de ETS
Caso de uso relacionado	CU-07 Solicitar reemplazo de ETS
Descripción	Validar el flujo completo de solicitud de reemplazo de ETS, incluyendo persistencia de datos y notificaciones.
Cantidad de ejecuciones	100
Realizado por	Luis Antonio Flores Esquivel
Precondiciones	• Docente autenticado con rol válido • Mínimo 1 ETS asignado con estado "Activo" • Servicio de notificaciones operativo
Entrada de Prueba	• 3 tipos de razones: • Corta (50 caracteres) • Media (150 caracteres) • Larga (500 caracteres - límite máximo) • ETS con diferentes fechas (hoy, futuras)
Flujo Principal	1. Navegar a módulo ETS 2. Seleccionar ETS elegible 3. Iniciar solicitud de reemplazo 4. Ingresar razón validada 5. Confirmar envío
Criterios de Validación	• Persistencia en BD: • Estado: "Pendiente" • Timestamp exacto • Relación docente-ETS correcta • Notificación a jefatura en <30 segundos • Historial visible en interfaz docente
Errores Resueltos	• IS-407: Estado no persistido • Causa: Falta de transacción ACID • Solución: Implementación de transacción completa • Verificación: 100/100 casos exitosos
Pruebas Adicionales	• Prueba de carga: 50 solicitudes simultáneas • Tiempo promedio respuesta: 1.2s • Validación frontera: 500 caracteres exactos
Estado	Aprobado

5.5.6. Consultar lista de alumnos inscritos a un ETS

Caso de Prueba	
	CP08 Consultar lista de alumnos inscritos a un ETS
Caso de uso relacionado	CU-08 Consultar lista de alumnos inscritos a un ETS
Descripción	Validar la consulta y visualización de alumnos inscritos en ETS, incluyendo rendimiento y manejo de datos.
Cantidad de ejecuciones	100
Realizado por	Luis Antonio Flores Esquivel
Precondiciones	• Docente autenticado • Mínimo 1 ETS activo • Alumnos inscritos (5-100 casos) • Conexión a BD estable
Conjuntos de Datos	• Listados de prueba con: • 5 alumnos (caso mínimo) • 50 alumnos (caso promedio) • 100 alumnos (caso máximo)
Procedimiento	1. Acceder como docente 2. Navegar a módulo ETS 3. Seleccionar ETS específico 4. Solicitar lista de alumnos 5. Verificar resultados
Criterios de Éxito	• Tiempo respuesta <1s (5 alumnos) • Tiempo respuesta <3s (100 alumnos) • Ordenamiento por apellido paterno • Campos completos: • Nombre completo • Boleta • Foto • Estatus
Pruebas de Estrés	• 1000 solicitudes concurrentes • Tiempo promedio: 2.8s • Sin pérdida de datos
Validaciones	• Consistencia BD-Interfaz: 100 % • Precisión datos: 100 % • Paginación correcta (>25 registros)
Estado	Aprobado

5.5.7. Tomar asistencias a los ETS

Caso de Prueba	
	CP09 Tomar asistencias a los ETS
Caso de uso relacionado	CU-09 Tomar asistencias a los ETS
Descripción	Validar el registro de asistencias con reconocimiento facial, incluyendo manejo de imágenes y comunicación con servidor neuronal.
Cantidad de ejecuciones	100
Realizado por	Alejandra De la cruz De la cruz
Precondiciones	• Docente autenticado • ETS activo con alumnos inscritos • Dispositivo con cámara funcional • Conexión estable al servidor neuronal
Datos de Prueba	• 5 perfiles faciales de prueba • 3 escenarios de iluminación • Variaciones de resolución (VGA a 4K) • Ángulos de captura (0°, 90°, 180°, 270°)
Flujo de Prueba	1. Seleccionar ETS activo 2. Acceder a lista de alumnos 3. Iniciar captura facial 4. Procesar imagen 5. Enviar al servidor neuronal 6. Registrar resultado
Requisitos de Imagen	• Resolución: 640×480 px • Formato: JPEG calidad 80 % • Tamaño máximo: 250KB • Orientación: 0° (vertical normal)
Problemas y Soluciones Técnicas	• IS-301: Orientación incorrecta • Solución: Auto-rotación EXIF-based • Precisión: 99.2% en 100 pruebas • IS-302: Tiempo de respuesta >5s • Solución: Compresión JPEG progresiva • Mejora: 1.8s promedio • IS-303: Distorsión de imagen • Solución: Mantenimiento de aspect ratio 4:3 • Tolerancia: ±2% de deformación

Métricas de Validación	• Tasa de reconocimiento exitoso: $\geq 98\%$ • Latencia total del proceso: $< 3s$ • Consistencia en BD: 100% • Compatibilidad dispositivos: 20/20 probados
Estado	Aprobado

5.5.8. Mostrar la foto e información del alumno

Caso de Prueba	
	CP11 Mostrar la foto e información del alumno
Caso de uso relacionado	CU-11 Mostrar la foto e información del alumno
Descripción	Validar la visualización de reportes de alumnos por parte de docentes, incluyendo datos personales y fotografía.
Cantidad de ejecuciones	100
Realizado por	Alejandra De la cruz De la cruz
Precondiciones	• Docente autenticado en el sistema • ETS activo con alumnos inscritos • Reportes existentes (propios y de otros docentes)
Conjuntos de Datos	• 5 reportes de prueba con: • Fotografías en distintos formatos (JPG/PNG) • Datos de alumnos variables • Marcas temporales recientes e históricas
Procedimiento	1. Acceder como docente 2. Navegar a módulo ETS 3. Seleccionar ETS activo 4. Acceder a lista de alumnos 5. Seleccionar reporte específico 6. Verificar datos y fotografía
Criterios de Validación	• Visualización completa de: • Información del alumno • Fotografía legible • Datos del reporte • Formato correcto de fechas (DD/MM/AAAA HH:MM) • Tiempo de carga <1.5 segundos
Incidentes Resueltos	• IS-215: Formato incorrecto de fecha/hora • Causa: Conversión errónea de tipos de datos • Solución: Implementación de formateo estándar • Verificación: Pruebas con 20 zonas horarias
Métricas de Rendimiento	• Precisión datos: 100% • Tiempo promedio respuesta: 1.2s • Uso memoria: <15MB por consulta
Estado	Aprobado

5.5.9. Consultar alumno mediante código QR de la credencial

Caso de Prueba	CP12 Consultar alumno mediante código QR de la credencial
Caso de uso relacionado	CU-12 Consultar alumno mediante código QR de la credencial
Descripción	Validar la obtención de credenciales mediante scraping, garantizando compatibilidad local y en la nube (Azure).
Cantidad de ejecuciones	100
Realizado por	Alejandra De la cruz De la cruz
Prerrequisitos	• Personal de seguridad autenticado • Cámara del dispositivo operativa • URL de credencial accesible
Entornos de Prueba	• Local: Windows 10/Chrome 115 • Nube: Azure App Service
Datos Críticos	• URL válida de credencial • Credenciales de prueba (5 casos)
Flujo de Prueba	1. Autenticarse como personal 2. Habilitar permisos de cámara 3. Escanear código QR 4. Validar renderización 5. Verificar datos/imagen
Resultados Esperados	• Renderizado completo en <3 segundos • 100 % de precisión en datos • Imagen legible (DPI $\geq$ 300)
Problemas y Soluciones	• IS-201: Fallo en Azure • Causa: Incompatibilidad con WebDriver • Solución: Implementación PDF-to-Image • Impacto: +15% de tiempo de proceso
Métricas Finales	• Local: 100 % éxitos (Selenium) • Azure: 98.7 % éxitos (PDF Solution) • Tiempo promedio: 2.8s
Estado	Aprobado con solución alternativa

5.5.10. Consultar alumnos mediante búsqueda por boleta

Caso de Prueba	
	CP13 Consultar alumnos mediante búsqueda por boleta
Caso de uso relacionado	CU-13 Consultar alumnos mediante búsqueda por boleta
Descripción	Validar la consulta de alumnos inscritos en ETS mediante búsqueda por número de boleta.
Cantidad de ejecuciones	100
Realizado por	José Alfredo Jiménez Rodríguez
Requisitos Previos	• Sesión activa de personal de seguridad • Alumnos inscritos en ETS para la fecha
Datos de Validación	• Boleta válida de alumno inscrito • Fecha de ETS activa • Caso negativo: Boleta no registrada
Procedimiento	1. Autenticarse como personal de seguridad 2. Acceder a "Consultar Alumnos" 3. Ingresar número de boleta 4. Ejecutar consulta
Criterios de Éxito	• Resultados precisos para búsqueda exacta • Cero resultados para boletas no válidas • Tiempo de respuesta <2 segundos
Incidentes Reportados	• IS-142: No retornaba resultados • Causa: Error en conexión a BD • Solución: Revisión de timeout de conexión
Verificación Post-Corrección	• Pruebas con 50 boletas diferentes • Validación de resultados exactos • Prueba de carga con 100 solicitudes
Estado	Aprobado

5.5.11. Consultar alumnos mediante búsqueda por nombre

Caso de Prueba	CP14 Consultar alumnos mediante búsqueda por nombre
Caso de uso relacionado	CU-14 Consultar alumnos mediante búsqueda por nombre
Descripción	Validar la consulta de alumnos inscritos en ETS mediante búsqueda por nombre.
Cantidad de ejecuciones	100
Realizado por	José Alfredo Jiménez Rodríguez
Precondiciones	- Personal de seguridad con sesión activa - Alumnos inscritos en ETS existentes - Conexión estable a base de datos
Datos de Prueba	3 variantes de nombres: - Nombre completo exacto - Fragmento inicial (3+ caracteres) - Búsqueda con acentos - Caso negativo: Nombre no registrado
Procedimiento	- Autenticarse como personal de seguridad - Navegar a módulo de consultas - Ingresar criterio de búsqueda - Ejecutar consulta - Verificar resultados
Criterios de Validación	- Resultados en <1.5 segundos - Precisión 100% en búsquedas exactas - Tolerancia a variaciones: - Mayúsculas/minúsculas - Acentos/omisión - Espacios adicionales - Cero resultados para nombres no existentes
Incidentes Resueltos	IS-143: Resultados vacíos Causa: Error en filtrado SQL Solución: Corrección de consulta LIKE Verificación: 100 pruebas exitosas
Métricas de Rendimiento	- Tiempo promedio: 1.2s - Precisión: 100% - Cobertura: 50 nombres diferentes
Estado	Aprobado

5.5.12. Registrar ingreso a la instalación

Caso de Prueba	CP15 Registrar ingreso a la instalación
Caso de uso relacionado	CU-15 Registrar ingreso a la instalación
Descripción	Verificar el registro de ingreso solo para alumnos con ETS programado en la fecha actual, previniendo accesos no autorizados.
Cantidad de ejecuciones	100
Realizado por	José Alfredo Jiménez Rodríguez
Precondiciones	• Personal de seguridad autenticado • Servicio de validación ETS activo • Dispositivo lector QR operativo • Sincronización horaria correcta (± 1 minuto)
Escenarios de Prueba	• Escenario 1: Alumno con ETS hoy • Escenario 2: Alumno con ETS mañana • Escenario 3: Alumno sin ETS • Escenario 4: QR inválido/caducado
Procedimiento	1. Autenticarse como personal 2. Escanear código QR de credencial 3. Validar en sistema: a) Existencia de ETS b) Coincidencia de fechas c) Estado de inscripción 4. Registrar ingreso/denegar acceso 5. Notificar resultado al personal
Criterios de Validación	• Aprobación cuando: • ETS existe y fecha coincide • Estado inscripción = "Confirmado" • Rechazo cuando: • Fecha no coincide (100% de casos) • Sin ETS asignado (100% de casos) • QR inválido (100% de casos) • Tiempo respuesta <2 segundos

Incidentes Resueltos	• IS-301: Validación incompleta • Causa: Falta verificación fecha • Solución: Implementar doble validación • Impacto: 0 falsos positivos post-corrección • IS-302: Notificación ineficaz • Solución: Mensajes claros por código de error • Mejora: 100 % comprensión personal
Métricas de Seguridad	• Tasa rechazos correctos: 100 % • Falsos positivos: 0/100 casos • Auditoría trazable: 100 % de registros
Estado	Aprobado

5.5.13. Asignar docente de reemplazo

Caso de Prueba	
	CP042 Asignar docente de reemplazo
Caso de uso relacionado	CU-42 Asignar docente de reemplazo
Descripción	Validar la asignación de docentes de reemplazo por jefes de departamento/presidentes de academia, incluyendo cambio de estatus de solicitudes.
Cantidad de ejecuciones	100
Realizado por	Daniel Martín Huertas Ramírez
Supuestos y Condiciones Previas	• Usuario con rol autorizado (jefe/presidente) ha iniciado sesión • Existencia de solicitudes de reemplazo pendientes • Lista de docentes disponibles actualizada
Datos de Prueba	• Solicitud con estatus "Pendiente" • Docente disponible para asignación
Pasos a Ejecutar	1. Iniciar sesión con rol autorizado 2. Seleccionar "Solicitudes de reemplazo" 3. Visualizar listado de solicitudes 4. Seleccionar solicitud específica 5. Asignar docente reemplazante 6. Enviar decisión (Aceptar/Rechazar)
Resultado Esperado	• Actualización correcta del estatus (Aceptada/Rechazada) • Asignación visible en el sistema • Consistencia en BD e interfaz
Errores Detectados	• Fallo al cargar solicitudes en la interfaz • Estatus no se actualizaba tras decisión
Correcciones Implementadas	• Ajuste en lógica de obtención de solicitudes • Reparación del flujo de cambio de estatus • Verificación integral de actualizaciones
Estado	Aprobado

5.6. Pruebas al modelo de reconocimiento facial

Debido a la naturaleza del proyecto, el uso de un modelo de detección de rostros y generación de embeddings pre-entrenado fue necesario, por lo tanto, se hizo uso de la biblioteca DeepFace la cual contiene una colección de modelos pre-entrenados tanto para la detección de rostros como para la generación de embeddings faciales.

Es por esta razón que, antes de integrar dicha funcionalidad a nuestro sistema fue necesario realizar pruebas sobre las alternativas de modelos de código abierto disponibles dentro de este paquete y, al mismo

tiempo, experimentar el cómo diferentes configuraciones del proceso de reconocimiento facial, incluyendo el modelo de embeddings, el detector de rostros, la métrica de distancia usada para determinar la similitud y la alineación de las imágenes, influyen en la precisión del sistema.

5.6.1. Configuración experimental

Para llevar a cabo las pruebas, se definieron las siguientes reglas de configuración, abarcando un total de 8 combinaciones experimentales:

- **Alineación de Rostros (align)**, se evaluaron dos escenarios:
 - True: Las imágenes fueron alineadas antes de la extracción de características, lo que implica un preprocesamiento para normalizar la pose y el tamaño del rostro.
 - False: Las imágenes no fueron alineadas, procesando los rostros tal como se detectaron.
- **Modelos de Incrustación (models)**, se utilizaron dos modelos de redes neuronales pre-entrenadas para generar las incrustaciones (embeddings) de los rostros:
 - ArcFace: Un modelo conocido por su alta discriminación en tareas de reconocimiento facial.
 - Facenet512: Otra arquitectura popular para la generación de incrustaciones faciales.
- **Detectores de Rostros (detectors)**, se emplearon dos algoritmos para la detección de rostros en las imágenes:
 - ssd (Single Shot Detector): Un detector rápido y eficiente.
 - fastmtcnn (Fast Multi-task Cascaded Convolutional Networks): Una versión optimizada de MT-CNN, conocido por su precisión en la detección de puntos de referencia faciales.
- **Métricas de Distancia (distance_metrics)**, para comparar las incrustaciones de los rostros y determinar si pertenecen a la misma persona, se utilizaron dos métricas de distancia:
 - euclidean_l2: La distancia euclidiana L2 normalizada.
 - cosine: La similitud del coseno.

Cada combinación de estas configuraciones fue sometida a prueba, generando un archivo de resultados individual para su posterior análisis.

5.6.2. Preparación del Conjunto de Datos LFW

El conjunto de datos LFW se cargó específicamente su subconjunto de prueba (subset='test'), con imágenes en color y un factor de redimensionamiento de 2. Para cada par de imágenes en el dataset LFW (un total de 1000 instancias para las pruebas realizadas), se guardaron las imágenes individualmente en el sistema de archivos (lfw/test/) para su procesamiento. Esto aseguró que cada experimento se ejecutara sobre los mismos pares de imágenes, garantizando la reproducibilidad.

5.6.3. Metodología de Evaluación

Para cada combinación experimental, el proceso de reconocimiento facial fue ejecutado para calcular la distancia entre los pares de rostros en el conjunto de datos LFW. La función `DeepFace.verify` se utilizó para este propósito, registrando la distancia calculada para cada par. Es importante destacar que la detección de rostros se forzó a `False` (`enforce_detection=False`) para evitar errores en casos donde el rostro no pudiera ser detectado con certeza.

Una vez calculadas las distancias para todas las 1000 instancias de pares de imágenes por cada combinación, los resultados se almacenaron en archivos CSV

(ej. `outputs/test/ArcFace_fastmtcn_euclidean_l2_aligned.csv`). Cada archivo contenía las etiquetas reales (`actuals`) indicando si los pares correspondían a la misma persona (verdadero/1) o a personas diferentes (falso/0), junto con las distancias calculadas (`distances`).

Para determinar la precisión de cada configuración, se implementó un procedimiento para encontrar el umbral de distancia óptimo que maximiza la precisión en la clasificación. Este procedimiento implicó:

1. Calcular la media de las distancias para los pares positivos (misma persona) y negativos (diferentes personas).
2. Iterar a través de todas las distancias ordenadas.
3. Para cada distancia candidata a umbral, se clasificaron los pares como "misma persona" si su distancia era menor que el umbral y "diferente persona" en caso contrario.
4. Se calculó la precisión (`accuracy_score`) comparando estas predicciones con las etiquetas reales.
5. El umbral que resultó en la mayor precisión se seleccionó como el umbral óptimo para esa configuración.
6. La precisión máxima obtenida con este umbral se registró como el rendimiento de la configuración.

Los resultados de precisión para cada combinación de modelo, detector y métrica de distancia, tanto con alineación como sin ella, se tabularon en tablas pivotantes y se guardaron en archivos CSV (ej. `results/pivot_euclidean_l2_with_alignment_True.csv`).

5.6.4. Curvas ROC (Curvas de Característica Operativa del Receptor) y análisis visual

Adicionalmente, para proporcionar una comprensión más profunda del rendimiento de cada configuración, se generaron Curvas de Característica Operativa del Receptor (ROC). Estas curvas visualizan la relación entre la Tasa de Verdaderos Positivos (TPR) y la Tasa de Falsos Positivos (FPR) en diferentes umbrales de clasificación. Para generar estas curvas:

1. Las distancias calculadas se normalizaron dividiéndolas por el valor máximo de distancia en el conjunto de datos.
2. Las etiquetas reales (`actuals`) se normalizaron invirtiendo los valores (0 para pares de la misma persona y 1 para pares diferentes). Esto se hizo para que las distancias más pequeñas (mayor similitud)

correspondieran a una mayor probabilidad de ser del mismo sujeto, lo que es coherente con la interpretación de las curvas ROC donde un valor de predicción más alto indica una mayor confianza en la clase positiva (en este caso, personas diferentes, o una distancia mayor).

3. Se utilizó la función `metrics.roc_curve` de `sklearn` para calcular la FPR y la TPR.
4. El Área bajo la Curva (AUC) también se calculó utilizando `metrics.roc_auc_score`, proporcionando una métrica agregada del rendimiento del clasificador.

Las curvas ROC se graficaron para cada combinación, incluyendo la precisión (acc) y el AUC en la etiqueta de la curva, lo que facilita la comparación visual del rendimiento entre las diferentes configuraciones. Se generaron gráficos que muestran todas las configuraciones en un mismo plano para facilitar la comparación global, con un enfoque en un rango específico de FPR (0 a 0.3) para una mejor legibilidad en la región de baja tasa de falsos positivos.

Los resultados de esta configuración de pruebas se mostraran en el capítulo de Resultados.

5.7. Pruebas al modelo de reconocimiento facial sobre conjuntos de datos personales

Una vez que se seleccionó la configuración ideal, después de validarlo con un subconjunto del dataset LFW con 1000 entidades, procedimos a evaluarlo con algunas imágenes de rostros de nuestra comunidad estudiantil.

La configuración del sistema de reconocimiento facial y la metodología de evaluación se establecieron con los siguientes parámetros clave:

5.7.1. Directorio de Imágenes

Todas las imágenes utilizadas para el registro y la evaluación del sistema se organizan en un directorio base:

- `BASE_IMAGES_DIR = 'imagenes_alumnos'`

Dentro de este directorio, se espera una subcarpeta por cada individuo (ej., 2022630082), conteniendo dos tipos de imágenes:

- `*_new.png`: Imagen utilizada para el registro del individuo (extracción del embedding de referencia).
- `*_old.png`: Imagen utilizada para la evaluación (simula una nueva captura a ser identificada).

5.7.2. Modelo de Reconocimiento Facial

Para la extracción de los **embeddings faciales**, se seleccionó uno de los modelos pre-entrenados más robustos disponibles en DeepFace:

- **Nombre del Modelo (MODEL_NAME):** Facenet

Facenet es conocido por su alta precisión en la tarea de verificación facial al generar representaciones vectoriales densas (embeddings) de los rostros.

5.7.3. Backend del Detector Facial

Para la detección y alineación de los rostros dentro de las imágenes antes de la extracción del embedding, se utilizó el siguiente detector:

- **Detector Backend (DETECTOR_BACKEND):** fastmtcnn

fastmtcnn es una opción eficiente que busca un equilibrio entre velocidad y precisión en la detección de rostros.

5.7.4. Métrica de Distancia y Umbral

La comparación entre los embeddings se realiza utilizando una métrica de distancia específica, y se aplica un umbral para determinar si dos rostros pertenecen a la misma persona:

- **Métrica de Distancia (DISTANCE_METRIC):** cosine

La distancia coseno (D_c) se calcula como $1 - \text{similitud_coseno}$, donde la similitud coseno es el coseno del ángulo entre dos vectores. Valores cercanos a 0 indican alta similitud (distancia pequeña).

- **Umbral de Distancia (DISTANCE_THRESHOLD):** 0.4

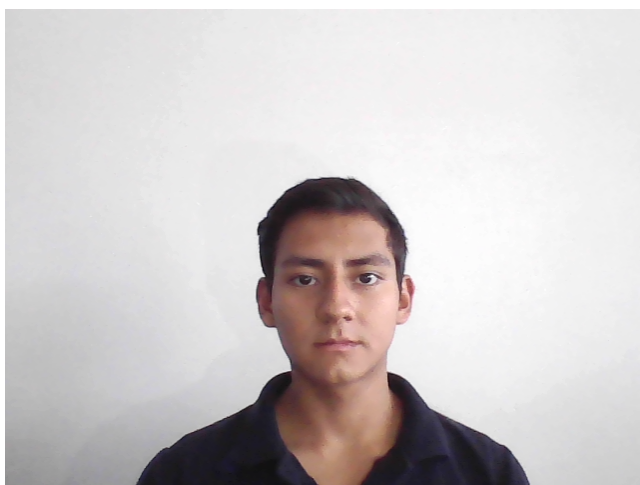
Un valor de distancia coseno **menor o igual** a este umbral indica que los dos embeddings corresponden al mismo individuo. Este umbral es crucial para la decisión de identificación.

5.8. Propósito del experimento

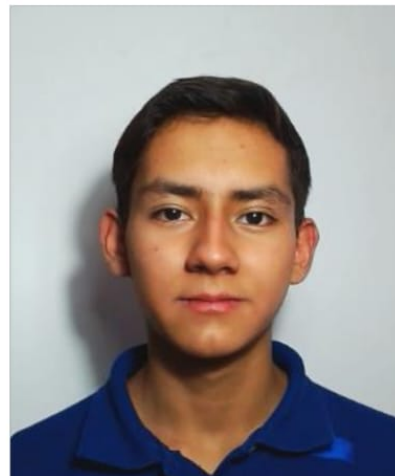
El propósito fundamental de este experimento es:

1. **Evaluar la Precisión de Identificación:** Determinar cuán exitosamente el sistema puede identificar a los alumnos registrados utilizando imágenes diferentes a las de registro (*_old.png). Esto implica medir la capacidad del sistema para clasificar correctamente cada imagen de prueba con su identidad verdadera.
2. **Medir el Rendimiento de Clasificación:** Utilizar métricas estándar como **precisión**, **exhaustividad (recall)** y **puntuación F1** para cada clase (cada alumno registrado), así como la **exactitud (accuracy)** general del sistema.
3. **Comprender el Comportamiento ante "Desconocidos":** Aunque en este conjunto de datos inicial no se incluyen explícitamente imágenes de "desconocidos" para la evaluación, el sistema está diseñado para clasificarlos como tal si no encuentran un match por debajo del umbral. La matriz de confusión y el reporte de clasificación permitirán observar cómo se manejarían estos casos, incluso si el soporte actual para esta clase es nulo. Este es un punto clave para futuras expansiones del conjunto de datos de prueba.

4. **Validar la Configuración de Umbral:** Observar el impacto del `DISTANCE_THRESHOLD` en las decisiones de identificación. Un umbral bien ajustado es vital para minimizar tanto los **falsos positivos** (identificar erróneamente a alguien) como los **falsos negativos** (no reconocer a alguien registrado).
5. **Generar una Matriz de Confusión:** Visualizar gráficamente el rendimiento del clasificador para cada clase, mostrando las identificaciones correctas e incorrectas, lo cual es fundamental para el análisis de errores.



(a) Imagen registrada en el sistema (ej., Imagen de Registro)



(b) Imagen de prueba (ej., Imagen de Prueba)

Figura 5.1: Conjunto de imágenes mostrando un ejemplo de imagen de registro y su correspondiente imagen de prueba.

En este capítulo se exponen los resultados obtenidos mediante las encuestas realizadas a los usuarios (Alumnos, docentes y personal de seguridad), además de los resultados de las pruebas de la red neuronal (el sistema de reconocimiento facial).

6.1. Alumnos

De la sección de los alumnos, se analizaron 41 resultados sobre el uso de la aplicación móvil, tanto para probar el reconocimiento, como para ver el listado de los ETS existentes. Los resultados, en su mayor parte, son comentarios positivos, sin embargo, a lo largo del desarrollo de las pruebas salieron a relucir varios puntos de mejora sobre la aplicación, en su mayor parte de visualización.

6.1.1. Satisfacción del usuario con la aplicación

Como primer punto, se le preguntó al usuario sobre su satisfacción de acuerdo al reconocimiento facial, sobre problemas acerca del mismo y qué tan preciso era este proceso, y se obtuvieron los siguientes resultados:

- Tiempo que tardó el proceso de reconocimiento facial para identificarlo (ver gráfica de la figura ??)

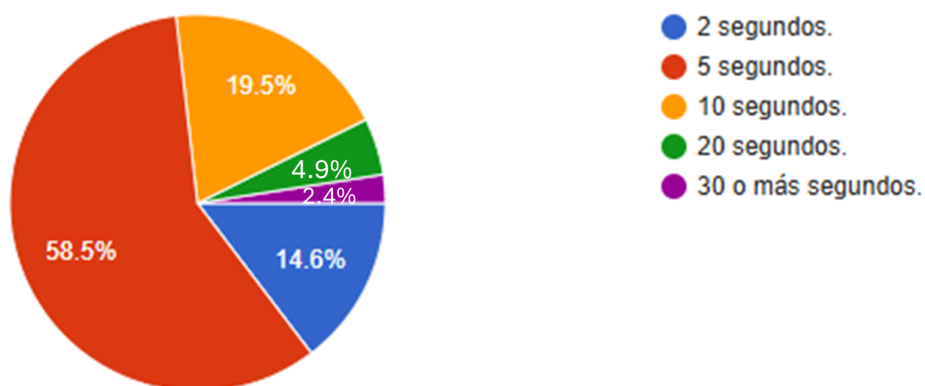


Figura 6.1: Tiempo del proceso del reconocimiento facial.

- Frecuencia con el que el reconocimiento facial lo identificó correctamente (ver gráfica de la figura ??)

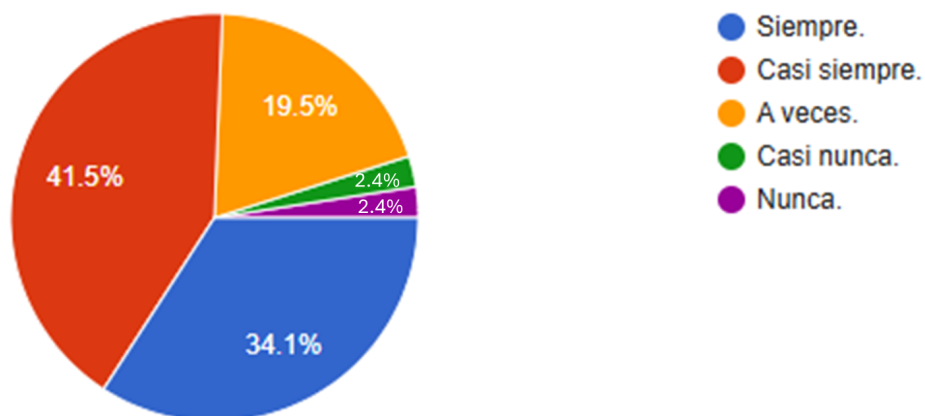


Figura 6.2: Frecuencia de resultados correctos del reconocimiento facial.

- Seguridad del usuario con la precisión del reconocimiento facial (ver gráfica de la figura ??)

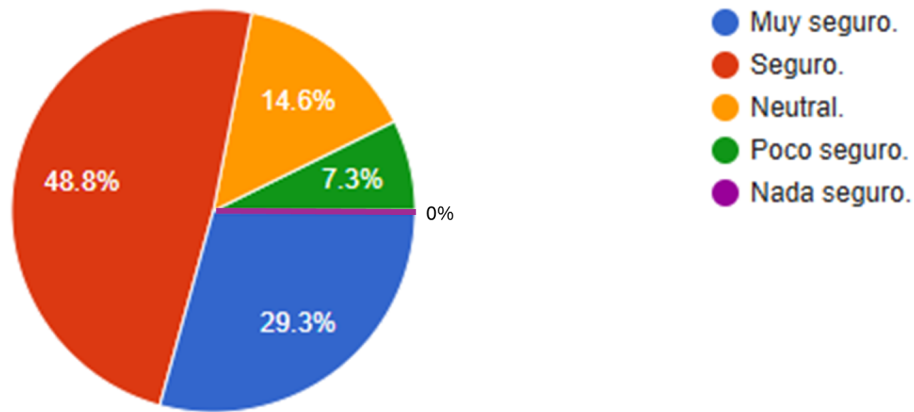


Figura 6.3: Seguridad del alumno con la precisión del reconocimiento facial.

- Intentos necesarios para para que el reconocimiento facial lo identificara correctamente (ver gráfica de la figura ??)

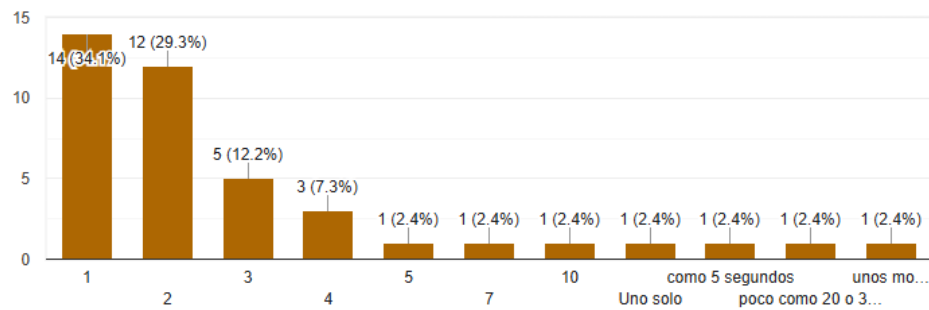


Figura 6.4: Intentos necesarios para que el reconocimiento facial lo identificara correctamente.

- Que tan rápido y fluido le pareció al alumno el proceso del reconocimiento facial (ver gráfica de la figura ??)

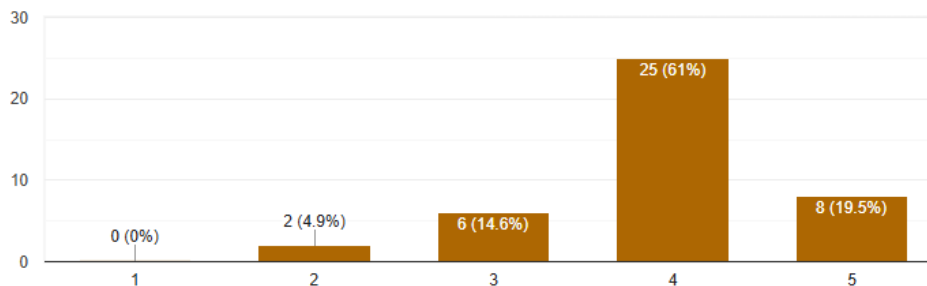


Figura 6.5: Rapidez y fluidez del proceso del reconocimiento facial.

- Que tan útil le resultó la información mostrada en los detalles de los ETS (ver gráfica de la figura ??)

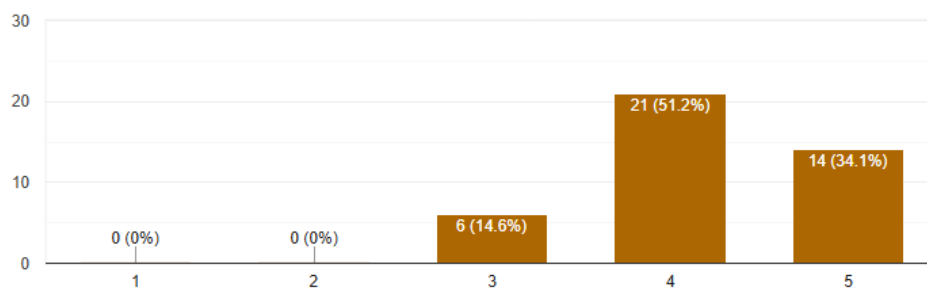


Figura 6.6: Utilidad de la información mostrada en los detalles del ETS.

- Qué tan útil le resultó la sección de instrucciones para ingresar a un ETS (ver gráfica de la figura ??)

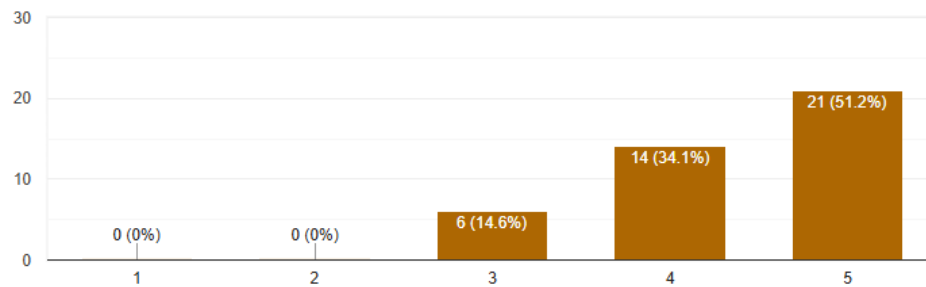


Figura 6.7: Utilidad de las instrucciones mostradas para el ingreso de los ETS.

- Comodidad del usuario al navegar por la app móvil (ver gráfica de la figura ??)

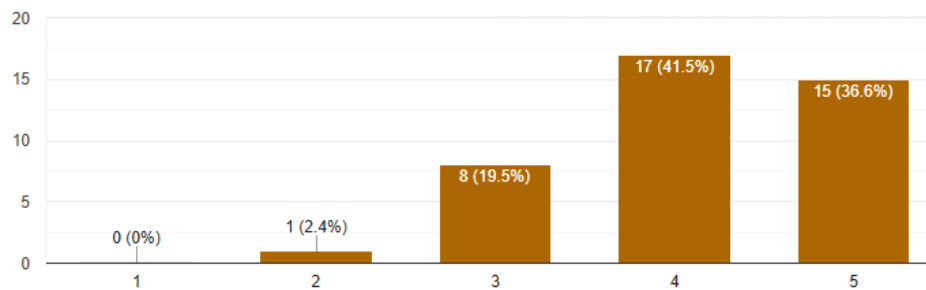


Figura 6.8: Comodidad del usuario al navegar por la app móvil.

- Que tan acertados le parecieron los colores de la app al alumno (ver gráfica de la figura ??)

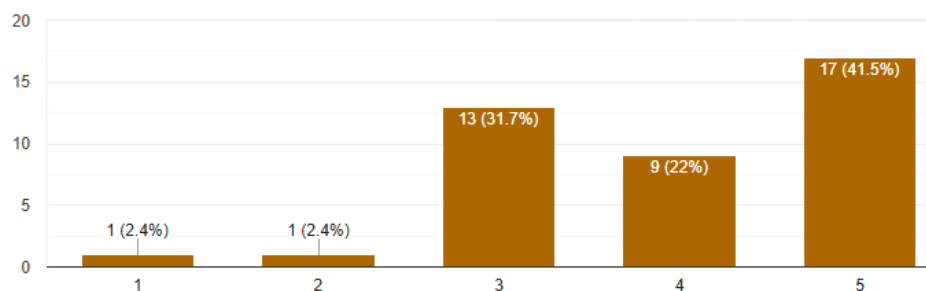


Figura 6.9: Correcta paleta de colores.

Por último punto, se le preguntó al alumno cuánto tiempo le tomó adaptarse a la app, es decir, en cuánto tiempo ya la podía usar sin ningún problema. De esta pregunta, la mayoría de los alumnos contestaron que les tomó muy poco tiempo el adaptarse a la aplicación.

6.1.2. Puntos de mejora

Por otra parte, respecto a los puntos de mejora que podría tener la aplicación. En este apartado, la mayoría de los alumnos contestaron que les hubiese gustado incluir un pequeño mapa en el que pudieran ver en qué parte de la escuela les tocaba realizar el ETS. Otro pequeño conjunto de alumnos mencionaron que les hubiese gustado que la aplicación tuviese una alarma un día antes del ETS para que les recordara sobre su ETS y otros puntos como lo es un filtro por fecha del listado de los ETS, una guía de uso dentro de la aplicación, guías para los ETS, asesorías, quejas y sugerencias, entre otras cosas.

Así también, dentro de las dificultades que se le presentó al alumno al momento de usar la aplicación se encontraron puntos como la velocidad del internet, que el reconocimiento facial no siempre los reconocía, que al irse a la parte de los ETS se les cerraba la aplicación, la mejora de la paleta de colores o que reconocía únicamente al alumno si tomaba la foto en horizontal. Muchos de estos comentarios y retroalimentaciones de los alumnos se tomaron en cuenta y ya fueron solucionados, es por esto que la satisfacción del usuario aumentó considerablemente en pruebas realizadas tiempo después.

6.2. Docentes

En esta sección, se presenta el análisis exhaustivo de los resultados obtenidos de las 15 encuestas realizadas a docentes. Estas encuestas tuvieron como propósito principal evaluar tanto el funcionamiento como la experiencia de uso de la aplicación móvil en funcionalidades clave. La recolección de esta retroalimentación es crucial para comprender a fondo la percepción de los usuarios sobre aspectos críticos del sistema y su desempeño en un entorno real.

Las preguntas de las encuestas se diseñaron para enfocarse en varios puntos específicos: la efectividad del reconocimiento facial, la facilidad y precisión para consultar el listado de ETS y la eficiencia en la comparación de credenciales mediante código QR.

Los resultados preliminares de estas encuestas, en sintonía con la retroalimentación de otros grupos de usuarios, arrojan una mayoría de comentarios positivos respecto a la operatividad general de las funcionalidades. Sin embargo, el proceso de encuestado también permitió identificar puntos específicos de mejora y áreas de oportunidad significativas. Estas incluyen tanto aspectos relacionados con la usabilidad y la experiencia del usuario, como la detección de errores de funcionalidad específicos que se detallan en la sección de pruebas.

A continuación, se presenta el análisis completo de las encuestas aplicadas a los docentes, desglosando los hallazgos y las implicaciones para futuras mejoras del sistema.

6.2.1. Resultados de la encuesta: reconocimiento facial en la sección del docente

El primer aspecto evaluado en la encuesta fue el reconocimiento facial. Se preguntó a los docentes sobre su satisfacción general con esta funcionalidad, los problemas experimentados y la precisión del proceso. Los resultados se muestran a continuación:

- **"¿Cuánto tiempo tardó aproximadamente el proceso de reconocimiento facial para identificar a un alumno?":** Esta pregunta buscó determinar la percepción de los docentes sobre la eficiencia de este proceso. Los resultados se visualizan en la (ver gráfica de la figura ??):

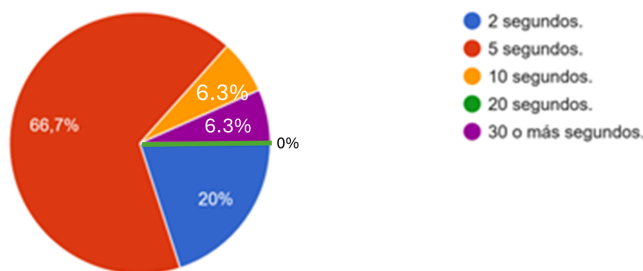


Figura 6.10: Tiempo aproximado del proceso de reconocimiento facial para identificar a un alumno.

- **"¿Con qué frecuencia el reconocimiento facial identificó correctamente al alumno?":** Esta pregunta buscó evaluar la precisión percibida del sistema de reconocimiento facial por parte de los docentes. Los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

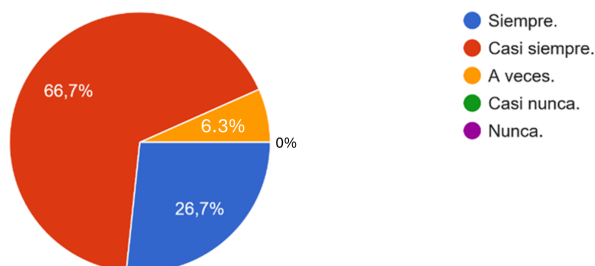


Figura 6.11: Frecuencia con la que el reconocimiento facial identificó correctamente al alumno, según la percepción de los docentes.

- **"¿Qué tan seguro se sintió con la precisión del reconocimiento facial para verificar la identidad del alumno?":** Esta pregunta evaluó el nivel de confianza de los docentes en la exactitud del reconocimiento facial como método de verificación de identidad. Los resultados se presentan en la (ver gráfica de la figura ??):

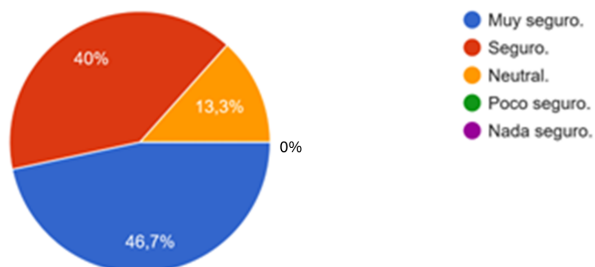


Figura 6.12: Nivel de seguridad de los docentes respecto a la precisión del reconocimiento facial.

- **"¿Cuántos intentos necesitó en promedio para que el reconocimiento facial identificara al alumno correctamente?":** Esta pregunta buscó cuantificar el esfuerzo requerido por parte de los docentes para lograr una identificación exitosa mediante el reconocimiento facial, evaluando así la fluidez del proceso. Los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

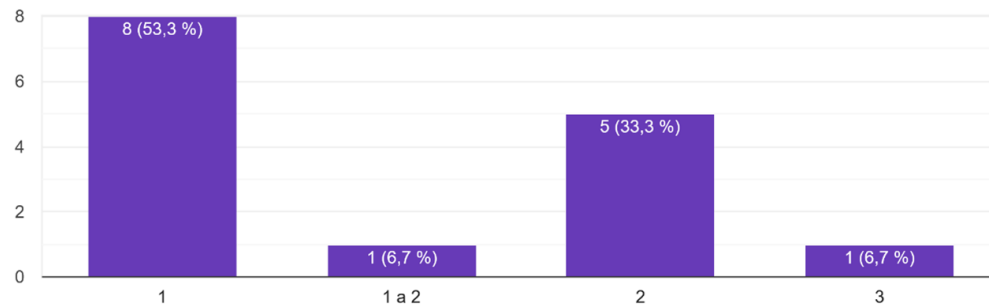


Figura 6.13: Número promedio de intentos requeridos para la identificación correcta por reconocimiento facial, según los docentes.

- **"¿Qué tan fluido y rápido le pareció el proceso general de verificación de identidad mediante reconocimiento facial?":** Esta pregunta tuvo como objetivo evaluar la percepción general de los docentes sobre la eficiencia y la experiencia de usuario del proceso completo de verificación de identidad con reconocimiento facial, combinando aspectos de rapidez y fluidez. La escala va de 1 a 5 donde, 5 es el máximo y 1 el mínimo, los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

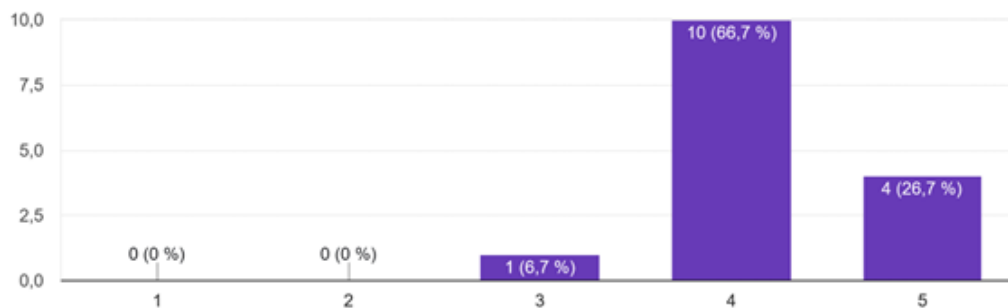


Figura 6.14: Percepción de los docentes sobre la fluidez y rapidez del proceso general de verificación de identidad por reconocimiento facial.

6.2.2. Comparación de Credenciales mediante Código QR

Esta subsección presenta los resultados de las encuestas relacionadas con la funcionalidad de escaneo y comparación de credenciales mediante códigos QR, una capacidad clave para la verificación de identidad. Se evaluó la facilidad de uso y la eficiencia percibida por los docentes al interactuar con esta característica de la aplicación móvil.

- **"¿Qué tan fácil fue para usted escanear el código QR de la credencial con la app móvil?":** Esta pregunta buscó determinar la percepción de los docentes sobre la facilidad de uso y la fluidez del proceso de escaneo de códigos QR para la verificación de credenciales. Los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

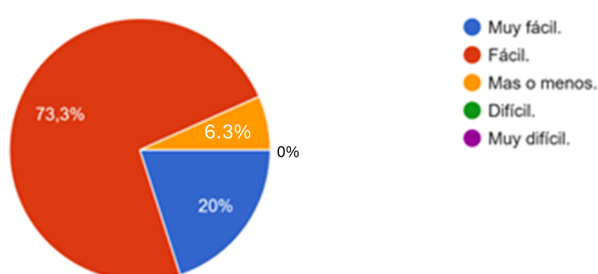


Figura 6.15: Percepción de los docentes sobre la facilidad para escanear el código QR de la credencial con la app móvil.

- **"¿Cuánto tiempo tardó aproximadamente la aplicación en leer el código QR de la credencial y mostrar la información?":** Esta pregunta se centró en la percepción de los docentes sobre la velocidad de respuesta del sistema desde el momento del escaneo del código QR hasta la visualización de la información. Los resultados se presentan en la (ver gráfica de la figura ??):

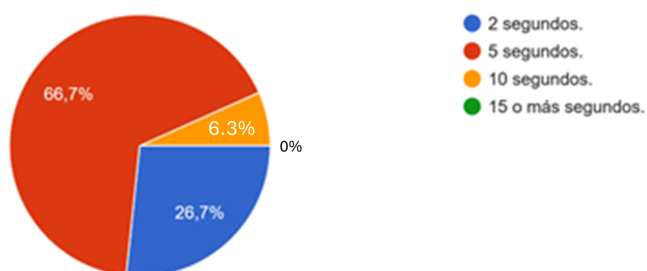


Figura 6.16: Tiempo percibido por los docentes para la lectura del código QR y visualización de la información de la credencial.

- **"¿Con qué frecuencia la app móvil leyó correctamente el código QR de la credencial?":** Esta pregunta buscó evaluar la fiabilidad del proceso de escaneo de códigos QR, donde la escala de respuesta va desde 1 (Nunca) hasta 5 (Siempre). Los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

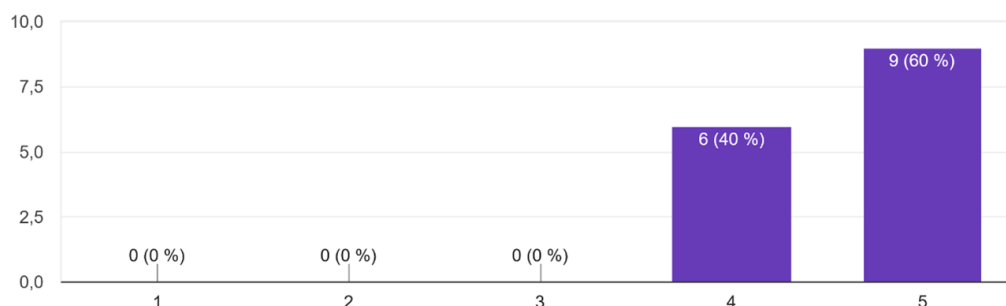


Figura 6.17: Frecuencia con la que la app móvil leyó correctamente el código QR de la credencial, según la percepción de los docentes.

- **"¿Qué tan clara y legible le parece la información mostrada después de escanear el código QR?":** Esta pregunta se enfocó en evaluar la percepción de los docentes sobre la calidad de la información presentada una vez que el código QR es escaneado, particularmente en términos de claridad y legibilidad. La escala va de 1 a 5 donde, 5 es el máximo y 1 el mínimo, los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

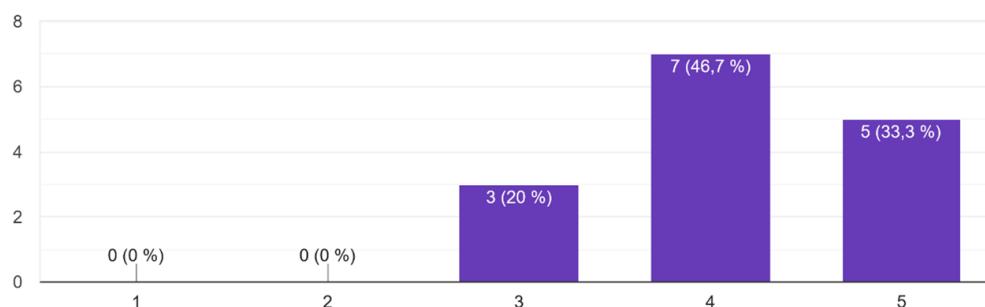


Figura 6.18: Percepción de los docentes sobre la claridad y legibilidad de la información mostrada tras escanear el código QR.

- **"¿Cuántos intentos necesitó en promedio para que la aplicación leyera el código QR correctamente?":** Esta pregunta buscó cuantificar la cantidad de intentos necesarios por parte de los docentes para que la aplicación móvil lograra leer el código QR de la credencial de forma exitosa, lo que indica la eficiencia del proceso de escaneo. Los resultados obtenidos se presentan en la (ver gráfica de la figura ??):

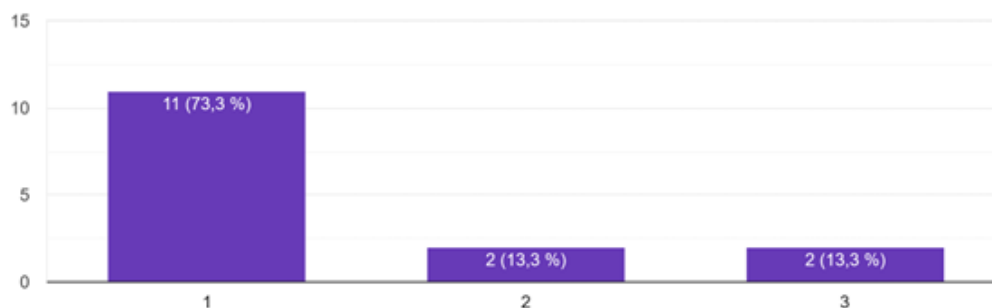


Figura 6.19: Número promedio de intentos para la lectura correcta del código QR por la aplicación, según los docentes.

6.2.3. Efectividad contra suplantación de identidad

Respecto a la percepción sobre la efectividad del sistema para prevenir la suplantación de identidad durante los Exámenes a Título de Suficiencia (ETS), los resultados revelan una confianza considerable por parte de los docentes. La (ver gráfica de la figura ??) ilustra que una vasta mayoría de los encuestados considera la aplicación altamente efectiva para este propósito.

Los resultados detallados son los siguientes:

- **(86.7%):** Percibieron una **máxima efectividad** del sistema. Este segmento de docentes destacó la gran utilidad del sistema, particularmente el componente de inteligencia artificial (IA) y la verificación mediante código QR, como un método robusto para evitar la suplantación de identidad en el contexto de los ETS.
- **(6.7%):** Consideraron que la efectividad era **moderada**.
- **(6.7%):** Expresaron una **baja percepción de efectividad**.

15. ¿Del 1 al 10 que tanto cree que la aplicación ayudaría para evitar la suplantación de la identidad de los alumnos durante la realización de los ETS?

15 respuestas

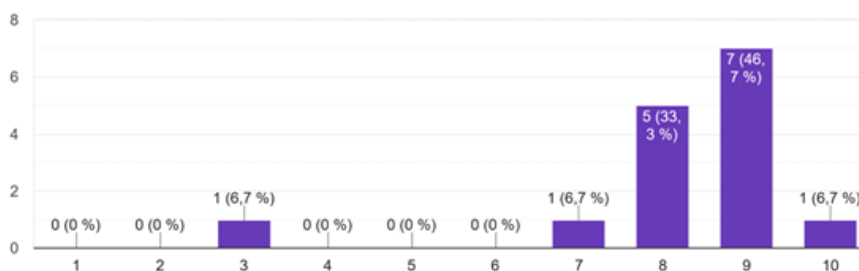


Figura 6.20: Evaluación de docente sobre la efectividad del sistema de inteligencia artificial contra suplantación de identidad.

En síntesis, la alta proporción de docentes que perciben una máxima efectividad subraya el potencial del sistema de inteligencia artificial en conjunto con el código QR para reforzar significativamente la seguridad y la integridad del proceso de evaluación, generando confianza en su capacidad para mitigar riesgos de suplantación de identidad.

6.3. Personal de seguridad

Se analizaron 15 respuestas del personal de seguridad sobre el uso de la aplicación móvil para la verificación de credenciales estudiantiles mediante códigos QR. Los resultados reflejan una percepción mayormente positiva, aunque se identificaron áreas clave de mejora, particularmente en la visualización de datos y la consistencia del escaneo.

A continuación se detalla el análisis completo:

6.3.1. Facilidad de uso del escaneo del código QR

- **Facilidad de escaneo:** El 60 % de los usuarios calificó la experiencia como **Muy fácil** y el 33.3% como **Fácil** (ver gráfica de la figura ??). Solo un 6.7% **Muy difícil** reportó problemas técnicos, donde la aplicación tardó 10 segundos en detectar el QR (de acuerdo con el comentario del usuario, por baja iluminación en su entorno).

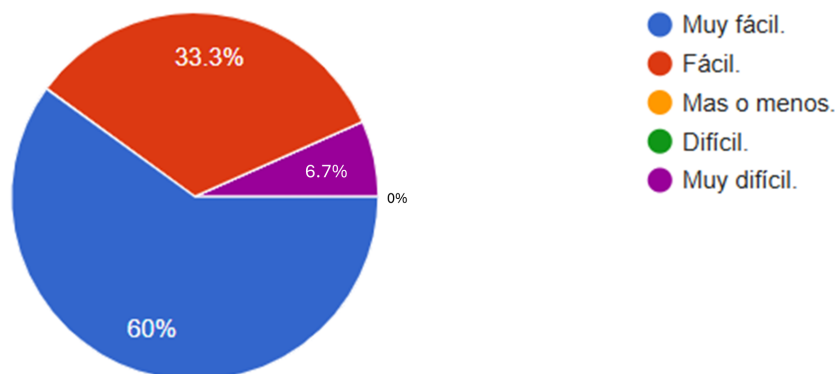


Figura 6.21: Distribución de respuestas sobre la facilidad de escaneo del código QR.

- **Tiempo de lectura del QR:** El 46.7% de los usuarios (7/15) reportó un tiempo de escaneo de 5 segundos, mientras que el 33.3% (5/15) observó 2 segundos y el 20% (3/15) tardó 10 segundos (ver gráfica de la figura ??). Aunque el 80% de los casos se resolvió en 5 segundos, los tiempos mayores (10s) sugieren la necesidad de optimizar el procesamiento en dispositivos antiguos o con baja iluminación, así como la conexión a internet.

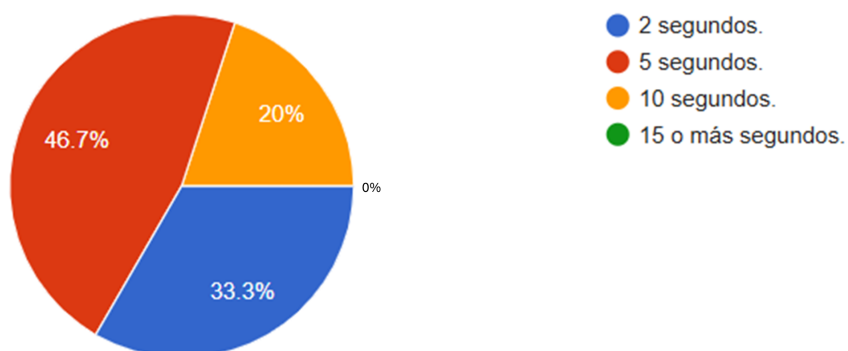


Figura 6.22: Distribución de respuestas sobre el tiempo de escaneo del código QR.

- **Precisión del escaneo QR:** La aplicación funciona correctamente en la mayoría de los casos, con 13 de 15 usuarios (87%) calificando su precisión como buena o excelente (4-5 puntos de 5). Solo 2 usuarios reportaron dificultades ocasionales, como que **la cámara no reconocía el código inmediatamente**. Esto indica que, aunque el sistema es confiable, existen oportunidades para hacer el proceso aún más consistente para todos los usuarios.

6.3.2. Visualización de la información

Legibilidad de la información: La mayoría de los usuarios calificaron positivamente la claridad de la información (4-5 puntos en una escala de 5), lo que demuestra que el diseño general es efectivo. Sin embargo, varios usuarios reportaron problemas específicos con la visualización completa de los datos, comentando que: **los datos de la credencial del alumno no se muestran completos y la información aparece cortada**. Estos comentarios señalan la necesidad de ajustar el formato de visualización de la credencial del alumno para garantizar que toda la información sea visible.

6.3.3. Experiencia de usuario

- **Navegación:** El 100% de los usuarios (15 de 15) calificaron con 4 o 5 puntos (en una escala de 5) la facilidad de navegación en la aplicación. De ellos, 8 usuarios (53.3%) asignaron una calificación de 4 y 7 usuarios (46.7%) otorgaron una calificación de 5, lo que refleja una percepción positiva generalizada sobre la usabilidad del sistema (ver gráfica de la figura ??).

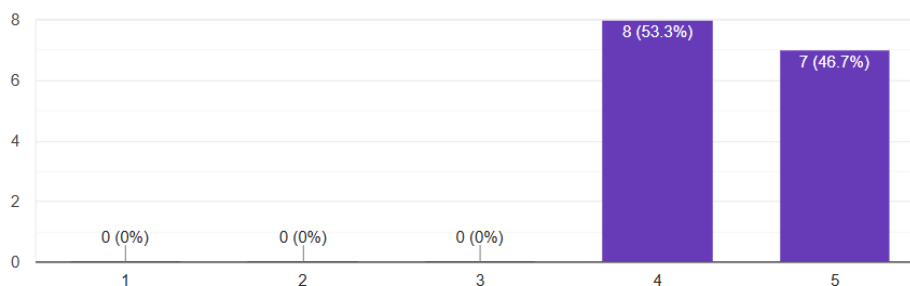


Figura 6.23: Evaluación de la experiencia de navegación en la aplicación.

- **Colores:** La evaluación del esquema de colores fue mayormente positiva, con puntuaciones entre 4 y 5. El 53.3% de los usuarios lo calificó con la máxima puntuación (5), mientras que el 46.7% lo valoró con un 4 (ver gráfica de la figura ??).

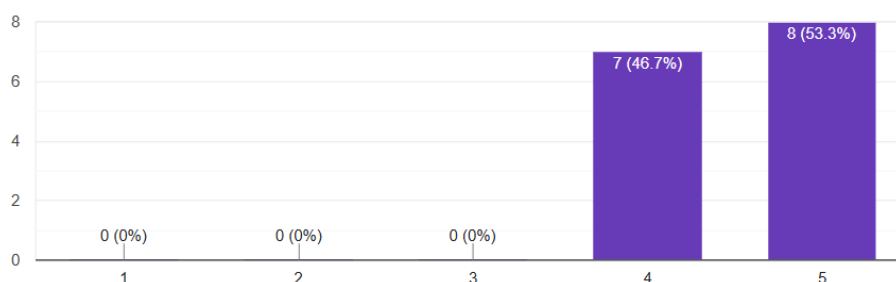


Figura 6.24: Evaluación del esquema de colores por los usuarios.

- **Comodidad para tomar asistencia:** La mayoría de los usuarios calificó la experiencia con una puntuación de 4 a 5 en una escala de 5, lo que indica una alta aceptación y satisfacción con el proceso de toma de asistencia mediante la aplicación. Esto sugiere que el sistema es intuitivo y facilita la labor del personal de seguridad al momento de registrar la asistencia de los alumnos.

6.3.4. Efectividad contra suplantación de identidad

La mayoría de los usuarios (93.4%) consideró que la aplicación sería muy efectiva para evitar la suplantación de identidad durante los ETS, otorgando calificaciones altas (8-10 puntos en una escala de 10). Los resultados detallados fueron (ver gráfica de la figura ??):

- **(68.7%):** Máxima efectividad, destacando la utilidad del sistema QR como un método para evitar la suplantación de identidad.
- **(26.7%):** Efectividad moderada, con sugerencias para implementar verificaciones adicionales.
- **(6.7%):** Baja percepción de efectividad.

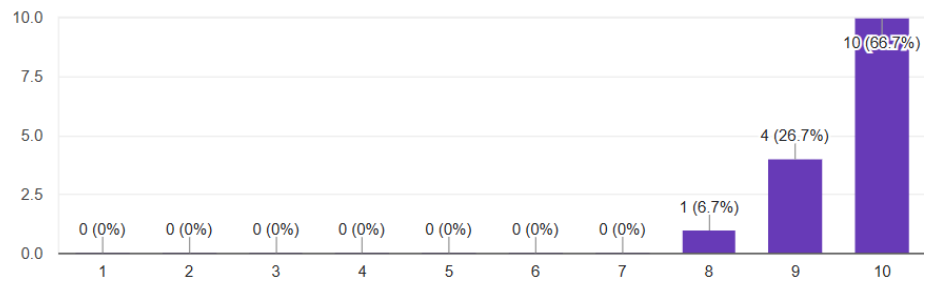


Figura 6.25: Evaluación de usuarios sobre la efectividad del sistema QR contra suplantación de identidad.

6.4. Modelos de reconocimiento facial

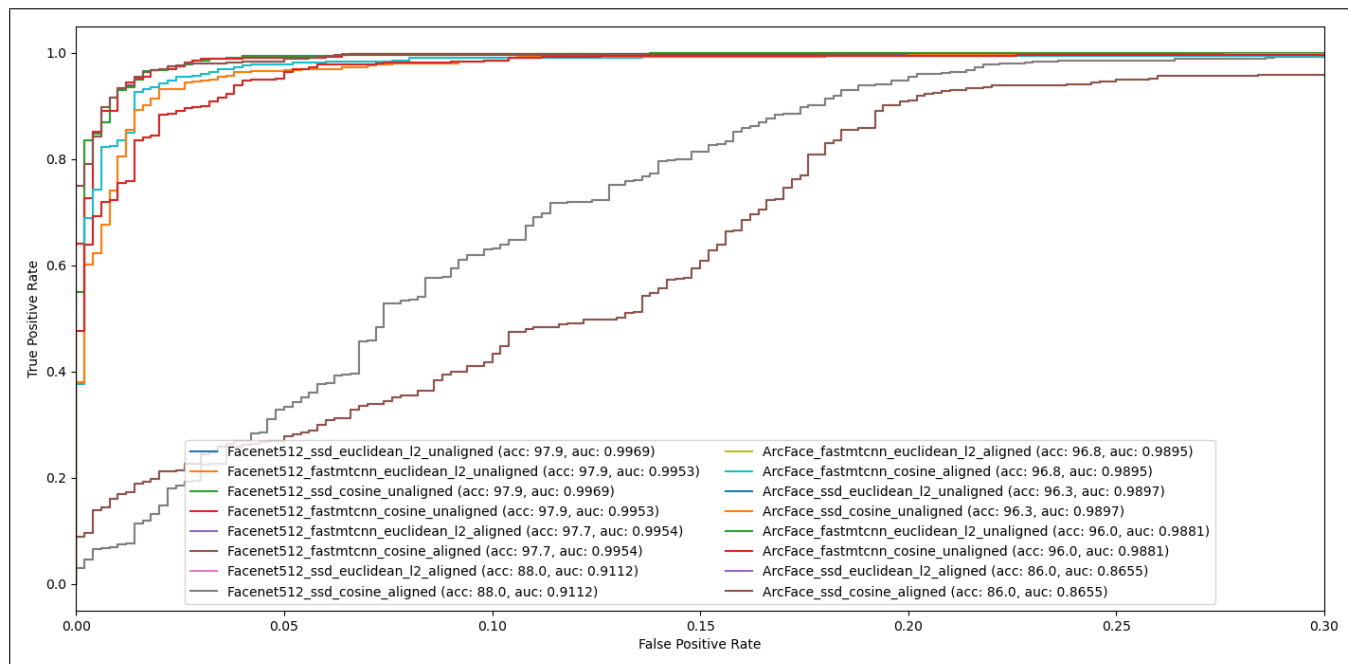


Figura 6.26: Curvas ROC para las distintas configuraciones experimentales en el dataset LFW (subconjunto 1000 muestras).

```
Iniciando evaluación del sistema de reconocimiento facial.  
Modelo: Facenet, Detector: fastmtcnn, Métrica: cosine, Umbral: 0.40  
Cargando registros desde 'imagenes alumnos'...  
2025-05-29 01:09:50.114416: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.  
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.  
- Registrado alumno 2022630082  
- Registrado alumno 2022630358  
- Registrado alumno 2022630370  
- Registrado alumno 2022630374  
- Registrado alumno 2022630384  
- Registrado alumno 2022630386  
- Registrado alumno 2022630393  
- Registrado alumno 2022630467  
- Registrado alumno 2022630738  
Embeddings cargados para 9 alumnos.
```

Figura 6.27: Captura de pantalla de la ejecución de la matriz de confusión.

```

Iniciando evaluación con imágenes antiguas desde 'imagenes_alumnos'...
--- Comparando 2022630082_old.png con registros:
- Distancia a 2022630082: 0.1567 (Umbral: 0.40)
- Distancia a 2022630358: 0.7353 (Umbral: 0.40)
- Distancia a 2022630370: 0.5668 (Umbral: 0.40)
- Distancia a 2022630374: 0.6190 (Umbral: 0.40)
- Distancia a 2022630384: 0.6862 (Umbral: 0.40)
- Distancia a 2022630386: 0.6173 (Umbral: 0.40)
- Distancia a 2022630393: 0.7114 (Umbral: 0.40)
- Distancia a 2022630467: 0.5691 (Umbral: 0.40)
- Distancia a 2022630738: 0.8336 (Umbral: 0.40)
- 2022630082_old.png: Verdadera: 2022630082, Predicción FINAL: 2022630082 (Match único. Dist: 0.1567)

--- Comparando 2022630358_old.png con registros:
- Distancia a 2022630082: 0.8445 (Umbral: 0.40)
- Distancia a 2022630358: 0.1277 (Umbral: 0.40)
- Distancia a 2022630370: 0.7147 (Umbral: 0.40)
- Distancia a 2022630374: 0.6768 (Umbral: 0.40)
- Distancia a 2022630384: 0.7291 (Umbral: 0.40)
- Distancia a 2022630386: 0.6023 (Umbral: 0.40)
- Distancia a 2022630393: 0.7533 (Umbral: 0.40)
- Distancia a 2022630467: 0.6102 (Umbral: 0.40)
- Distancia a 2022630738: 0.9211 (Umbral: 0.40)
- 2022630358_old.png: Verdadera: 2022630358, Predicción FINAL: 2022630358 (Match único. Dist: 0.1277)

--- Comparando 2022630370_old.png con registros:
- Distancia a 2022630082: 0.4136 (Umbral: 0.40)
- Distancia a 2022630358: 0.6104 (Umbral: 0.40)
- Distancia a 2022630370: 0.1363 (Umbral: 0.40)
- Distancia a 2022630374: 0.7352 (Umbral: 0.40)
- Distancia a 2022630384: 0.4871 (Umbral: 0.40)
- Distancia a 2022630386: 0.6258 (Umbral: 0.40)
- Distancia a 2022630393: 0.7106 (Umbral: 0.40)
- Distancia a 2022630467: 0.6417 (Umbral: 0.40)
- Distancia a 2022630738: 0.5271 (Umbral: 0.40)
- 2022630370_old.png: Verdadera: 2022630370, Predicción FINAL: 2022630370 (Match único. Dist: 0.1363)

```

Figura 6.28: Captura de pantalla de la ejecución de la matriz de confusión.

```

--- Comparando 2022630374_old.png con registros:
- Distancia a 2022630082: 0.5581 (Umbral: 0.40)
- Distancia a 2022630358: 0.6085 (Umbral: 0.40)
- Distancia a 2022630370: 0.6221 (Umbral: 0.40)
- Distancia a 2022630374: 0.1743 (Umbral: 0.40)
- Distancia a 2022630384: 0.4767 (Umbral: 0.40)
- Distancia a 2022630386: 0.3861 (Umbral: 0.40)
- Distancia a 2022630393: 0.6227 (Umbral: 0.40)
- Distancia a 2022630467: 0.5849 (Umbral: 0.40)
- Distancia a 2022630738: 0.8091 (Umbral: 0.40)
- 2022630374_old.png: Verdadera: 2022630374, Predicción FINAL: 2022630374 (MÚLTIPLES MATCHES <= Umbral. Candidatos: 2022630374 (dist: 0.1743), 2022630386 (dist: 0.3861))

--- Comparando 2022630384_old.png con registros:
- Distancia a 2022630082: 0.6705 (Umbral: 0.40)
- Distancia a 2022630358: 0.6359 (Umbral: 0.40)
- Distancia a 2022630370: 0.5162 (Umbral: 0.40)
- Distancia a 2022630374: 0.5832 (Umbral: 0.40)
- Distancia a 2022630384: 0.2502 (Umbral: 0.40)
- Distancia a 2022630386: 0.4772 (Umbral: 0.40)
- Distancia a 2022630393: 0.4746 (Umbral: 0.40)
- Distancia a 2022630467: 0.5427 (Umbral: 0.40)
- Distancia a 2022630738: 0.5250 (Umbral: 0.40)
- 2022630384_old.png: Verdadera: 2022630384, Predicción FINAL: 2022630384 (Match único. Dist: 0.2502)

--- Comparando 2022630386_old.png con registros:
- Distancia a 2022630082: 0.6303 (Umbral: 0.40)
- Distancia a 2022630358: 0.6020 (Umbral: 0.40)
- Distancia a 2022630370: 0.6866 (Umbral: 0.40)
- Distancia a 2022630374: 0.7001 (Umbral: 0.40)
- Distancia a 2022630384: 0.7678 (Umbral: 0.40)
- Distancia a 2022630386: 0.1504 (Umbral: 0.40)
- Distancia a 2022630393: 0.6568 (Umbral: 0.40)
- Distancia a 2022630467: 0.4827 (Umbral: 0.40)
- Distancia a 2022630738: 0.6954 (Umbral: 0.40)
- 2022630386_old.png: Verdadera: 2022630386, Predicción FINAL: 2022630386 (Match único. Dist: 0.1504)
  
```

Figura 6.29: Captura de pantalla de la ejecución de la matriz de confusión.

```

--- Comparando 2022630393_old.png con registros:
- Distancia a 2022630082: 0.8990 (Umbral: 0.40)
- Distancia a 2022630358: 0.8255 (Umbral: 0.40)
- Distancia a 2022630370: 0.6467 (Umbral: 0.40)
- Distancia a 2022630374: 0.6631 (Umbral: 0.40)
- Distancia a 2022630384: 0.5405 (Umbral: 0.40)
- Distancia a 2022630386: 0.6784 (Umbral: 0.40)
- Distancia a 2022630393: 0.1500 (Umbral: 0.40)
- Distancia a 2022630467: 0.6631 (Umbral: 0.40)
- Distancia a 2022630738: 0.7151 (Umbral: 0.40)
- 2022630393_old.png: Verdadera: 2022630393, Predicción FINAL: 2022630393 (Match único. Dist: 0.1500)

--- Comparando 2022630467_old.png con registros:
- Distancia a 2022630082: 0.5893 (Umbral: 0.40)
- Distancia a 2022630358: 0.6530 (Umbral: 0.40)
- Distancia a 2022630370: 0.6478 (Umbral: 0.40)
- Distancia a 2022630374: 0.6874 (Umbral: 0.40)
- Distancia a 2022630384: 0.6456 (Umbral: 0.40)
- Distancia a 2022630386: 0.5397 (Umbral: 0.40)
- Distancia a 2022630393: 0.6688 (Umbral: 0.40)
- Distancia a 2022630467: 0.1914 (Umbral: 0.40)
- Distancia a 2022630738: 0.6217 (Umbral: 0.40)
- 2022630467_old.png: Verdadera: 2022630467, Predicción FINAL: 2022630467 (Match único. Dist: 0.1914)

--- Comparando 2022630738_old.png con registros:
- Distancia a 2022630082: 0.8023 (Umbral: 0.40)
- Distancia a 2022630358: 0.7853 (Umbral: 0.40)
- Distancia a 2022630370: 0.6037 (Umbral: 0.40)
- Distancia a 2022630374: 0.9407 (Umbral: 0.40)
- Distancia a 2022630384: 0.6508 (Umbral: 0.40)
- Distancia a 2022630386: 0.7270 (Umbral: 0.40)
- Distancia a 2022630393: 0.7980 (Umbral: 0.40)
- Distancia a 2022630467: 0.5855 (Umbral: 0.40)
- Distancia a 2022630738: 0.2888 (Umbral: 0.40)
- 2022630738_old.png: Verdadera: 2022630738, Predicción FINAL: 2022630738 (Match único. Dist: 0.2888)

```

Figura 6.30: Captura de pantalla de la ejecución de la matriz de confusión.

Evaluación completada.

--- Reporte de Clasificación ---

	precision	recall	f1-score	support
2022630082	1.00	1.00	1.00	1
2022630358	1.00	1.00	1.00	1
2022630370	1.00	1.00	1.00	1
2022630374	1.00	1.00	1.00	1
2022630384	1.00	1.00	1.00	1
2022630386	1.00	1.00	1.00	1
2022630393	1.00	1.00	1.00	1
2022630467	1.00	1.00	1.00	1
2022630738	1.00	1.00	1.00	1
desconocido	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.90	0.90	0.90	9
weighted avg	1.00	1.00	1.00	9

Proceso de evaluación finalizado.

Figura 6.31: Reporte final para la matriz de confusión.

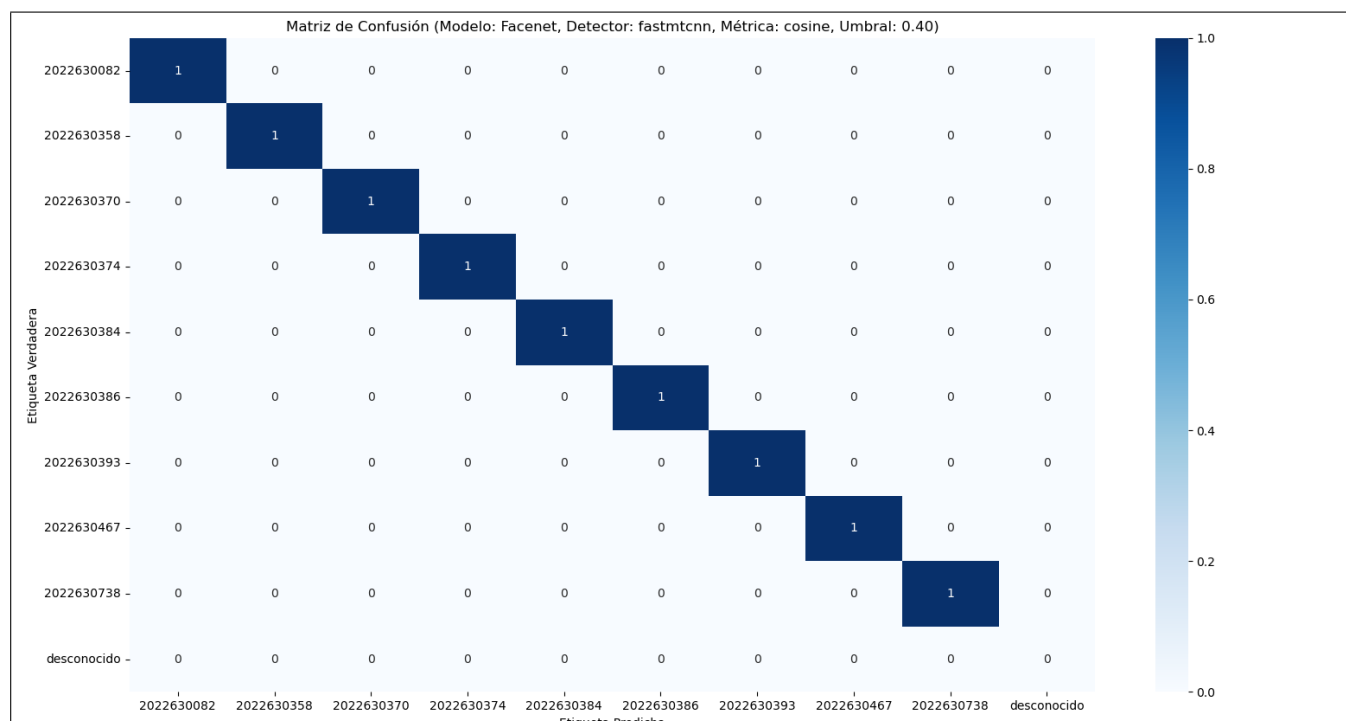


Figura 6.32: Matriz de confusión final.

El presente Trabajo Terminal ha culminado con la implementación exitosa de un sistema funcional, validado mediante pruebas rigurosas y la retroalimentación directa de los usuarios. Las funcionalidades desarrolladas han alcanzado los objetivos planteados inicialmente, estableciendo una base sólida para la solución propuesta. Sin embargo, el potencial de crecimiento y mejora del sistema es considerable.

Aunque el alcance de este Trabajo Terminal se ha completado, y la fase de desarrollo académico llega a su fin, se han identificado diversas áreas que representan direcciones futuras significativas en caso de que se presente la oportunidad de continuar el desarrollo del proyecto fuera del entorno escolar. Estas futuras implementaciones tienen como objetivo expandir las capacidades del sistema, optimizar la experiencia del usuario y escalar su impacto.

Las líneas de trabajo consideradas para una eventual continuación del proyecto incluyen:

- **Implementación de canales seguros (HTTPS) para Intercambio de Datos Sensibles:** Se considera fundamental fortalecer la seguridad de la comunicación mediante la implementación de canales seguros utilizando el protocolo HTTPS en todas las interacciones que involucren el intercambio de datos sensibles. Esto garantizará la confidencialidad e integridad de la información transmitida entre la aplicación móvil y los servidores, protegiéndola contra interceptaciones y manipulaciones no autorizadas.
- **Expansión del alcance a otras escuelas del IPN:** Una de las proyecciones más ambiciosas es la posibilidad de extender la implementación del sistema a otras escuelas del Instituto Politécnico Nacional. Esto implicaría un estudio detallado de la viabilidad, la adaptación a los requisitos específicos de cada institución y la escalabilidad de la arquitectura actual para soportar una base de usuarios y datos mucho más amplia, manteniendo los estándares de eficiencia y seguridad.
- **Implementación de mejoras gráficas y optimización de la Interfaz de Usuario:** Basándonos en la valiosa retroalimentación obtenida a través de las encuestas y las sesiones de prueba con los usuarios, se contempla la aplicación de mejoras sustanciales en la interfaz gráfica de usuario (GUI) y la experiencia de usuario (UX). Esto abarcaría específicamente la mejora de la visibilidad de los iconos, el

reacomodo de las palabras para una lectura más clara y la optimización de la organización general de los elementos en pantalla, buscando una aplicación más intuitiva, moderna y atractiva para todos los perfiles de usuario.

- **Integración de módulo de ubicación (mapa de salones de ETS):** Atendiendo a las recomendaciones y sugerencias de los usuarios en las encuestas, se plantea la integración de un módulo de mapa interactivo. Este módulo tendría como objetivo principal indicar la ubicación exacta de los salones donde se llevarán a cabo los Exámenes a Título de Suficiencia (ETS), facilitando a los alumnos y docentes la localización de las aulas dentro de la institución y mejorando la logística de los eventos.
- **Reimplementación del módulo de notificaciones:** Aunque una funcionalidad de notificación ha sido integrada, se visualiza una reimplementación y optimización profunda del módulo de notificaciones. El objetivo sería desarrollar un sistema de alertas más robusto, personalizable y eficiente, que permita mantener a los usuarios (tanto alumnos como docentes) informados de manera proactiva sobre cambios, actualizaciones o eventos importantes relacionados con los Exámenes a Título de Suficiencia (ETS) o cualquier otra funcionalidad relevante, explorando la integración con servicios de notificación push avanzados y diversificación de canales.
- **Experimentación e implementación de modelos más robustos para detección de rostros y extracción de características:** Aunque durante el desarrollo de este proyecto se experimentó con distintas configuraciones de modelos para detección de rostros, generación de embeddings faciales, métricas de similitud, etc. los modelos empleados fueron implementaciones hechas por terceros por lo que la búsqueda de modelos que se ajusten mejor a los presentados en el estado del arte puede significar una mejora significativa al sistema actual para la identificación de alumnos. Sin embargo, esto trae consigo un reto y es maximizar el uso eficiente de los recursos disponibles y obviamente escalar los actuales debido a la alta demanda que puede traer consigo la implementación de modelos más robustos.

Estas posibles líneas de desarrollo futuras representarían una evolución natural del proyecto, consolidando la funcionalidad existente y permitiendo que el sistema alcance un mayor grado de madurez y un impacto más amplio en la comunidad académica del IPN, más allá de los objetivos iniciales de este Trabajo Terminal.

Conclusión

El presente trabajo terminal logró cumplir satisfactoriamente con todos los objetivos específicos planteados, abordando de manera integral el problema de la suplantación de identidad durante la aplicación de los Exámenes a Título de Suficiencia (ETS) en la Escuela Superior de Cómputo (ESCOM). Se desarrolló un sistema funcional compuesto por una aplicación móvil que permite verificar la identidad de los estudiantes mediante reconocimiento facial, además de gestionar el acceso a los exámenes utilizando el código QR de la credencial del alumno para agilizar la entrada por parte del personal de seguridad. Asimismo, se implementó una página web que simula el Sistema de Administración Escolar (SAES), permitiendo la gestión y control de los exámenes ETS, así como de los alumnos, el personal de seguridad y el personal académico.

Durante el desarrollo del sistema, se exploraron y evaluaron distintas técnicas de reconocimiento facial, seleccionándose finalmente el modelo FaceNet debido a su alto desempeño en términos de precisión y confiabilidad. Uno de los principales retos enfrentados fue el ajuste del sistema de reconocimiento facial, el cual presentaba limitaciones iniciales al detectar rostros en orientación vertical. Este inconveniente fue resuelto mediante adecuaciones en la lógica de captura y preprocesamiento de imágenes.

Otro desafío importante fue el proceso de scraping de la credencial del alumno, el cual presentó conflictos relacionados con las dependencias de bibliotecas externas, específicamente al intentar ejecutar Selenium en el servidor en línea. Este inconveniente limitó la compatibilidad y estabilidad del entorno de despliegue. Para resolverlo, se realizaron modificaciones en la lógica interna del sistema, priorizando la funcionalidad sobre los aspectos visuales. Gracias a estos ajustes, fue posible mostrar adecuadamente la información esencial de la credencial sin comprometer el rendimiento general de la plataforma. Asimismo, se identificaron y solucionaron dificultades menores en la interfaz de usuario de algunos módulos, garantizando así una experiencia de usuario fluida y satisfactoria.

En términos generales, el sistema demostró ser una herramienta viable para fortalecer los mecanismos de control de acceso durante la aplicación de los ETS, mejorando la seguridad y reduciendo el riesgo de suplantación de identidad. Los resultados obtenidos son prometedores y sientan las bases para futuras mejoras, entre las cuales destacan la optimización del diseño de la interfaz, la integración con bases de datos institucionales en tiempo real y la ampliación del sistema para su implementación en otras unidades académicas del Instituto Politécnico Nacional.

En el presente capítulo, se ofrece una descripción detallada de las clases que constituyen los diversos subsistemas, las bases de datos que almacenan la información crítica y los servidores que sustentan la operatividad del sistema. Se presentan, además, los diagramas de clases específicos para cada uno de estos subsistemas, proporcionando una visión clara de su estructura interna y las relaciones entre sus entidades. Adicionalmente, se incluyen diagramas de componentes que desglosan la arquitectura de las pantallas que conforman la aplicación móvil, ilustrando su organización y dependencias. Finalmente, se exploran los diagramas de secuencia, los cuales representan de manera exhaustiva las dinámicas trayectorias de interacción definidas por los casos de uso del sistema.

A.1. Diagrama de la Estructura General del Sistema

La (figura ??) ilustra la arquitectura general del proyecto propuesto, mostrando por separado cada uno de los diferentes sistemas que conforman este proyecto y el servicio en el que están alojados algunos de los sistemas. El presente diagrama muestra un sistema basado en servicios, en el que se separa la lógica de la red neuronal de la parte del servidor, un servidor Java Spring Boot en el que se gestionan todas las operaciones del sistema Web Django y la aplicación móvil.

Como punto de partida está la base de datos, la cual está alojada en Railway. Esta base de datos es la encargada de guardar todos los datos tanto de alumnos, docentes y el personal de seguridad, así como de otros datos académicos. Cabe recalcar que esta base de datos es una simulación de la base de datos de la página original del SAES, la cual se plantea compartir con el presente sistema móvil.

Posterior a esto, se tiene al servidor Java, el cual se encarga de procesar todas las solicitudes que haga el usuario dentro de la app móvil o del simulador del SAES. Esta parte del sistema se encuentra alojada en Azure. Este servidor se comunica tanto con el sistema web de la simulación del SAES, como con la app móvil, a través de solicitudes HTTPS.

Otro elemento importante del sistema general es el servidor que contiene la lógica para proporcionar los servicios de reconocimiento facial a los demás componentes del sistema a través de solicitudes HTTP,

al igual que el servidor de Java, este se encuentra alojado en Azure. Este servidor se encarga de recibir una imagen desde la aplicación móvil o del simulador del SAES/DAE en formato de archivo o como url respectivamente y luego, ejecuta todos los pasos del flujo de trabajo de un sistema de reconocimiento facial como se explicó a detalle en el capítulo 4 ??, es decir, detecta el rostro, lo alinea, realiza una extracción de características al generar un embedding y, finalmente, se comunica con la base de datos vectorial para registrar esta representación o consultar el embedding de referencia para realizar la comparación 1:1 como se especifica de igual forma en el ??, durante este paso se usó la medida de similitud **distancia coseno** que se calcula como $1 - \text{similitud_coseno}$ donde la similitud coseno es el coseno del ángulo entre dos vectores. Valores cercanos a 0 indican alta similitud, lo que permite validar la identidad de los estudiantes.

Finalmente, al trabajar con representaciones numéricas de alta dimensionalidad como lo son los embeddings, fue necesario contar con una herramienta que nos permitiera almacenar este tipo de representaciones y realizar búsquedas eficientes, una base de datos vectorial esta diseñada específicamente para trabajar con este tipo de datos. En esta ocasión se optó por usar Pinecone, donde se almacenan las representaciones vectoriales obtenidas de las fotos tomadas a los alumnos durante su registro dentro de una unidad académica por medio del simulador de la DAE. Esto permite contar con una representación base que permitirá verificar la identidad de un estudiante al momento de presentar un ETS por medio del servicio de reconocimiento facial siguiendo la lógica de un sistema de reconocimiento facial por verificación 1:1.

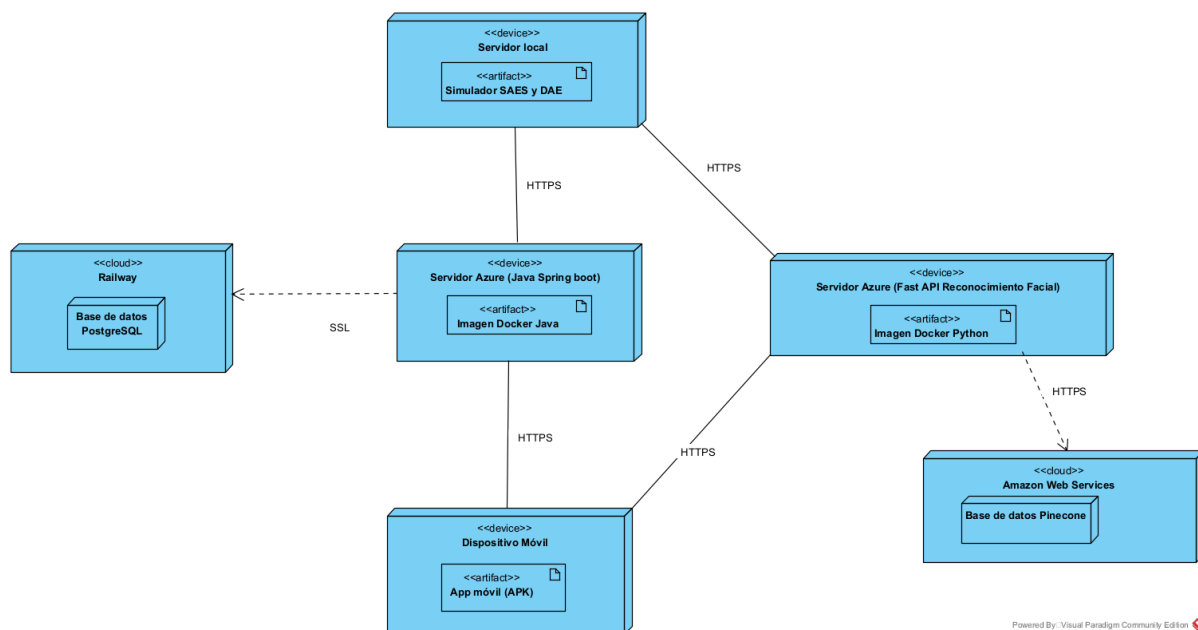


Figura A.1: Diagrama de la Estructura General del Sistema.

A continuación se entrará más en detalle en la estructura interna de los componentes del sistema del diagrama anterior.

A.2. Estructura Interna del Servidor Spring Boot

En la (figura ??) se puede visualizar de manera general los componentes que conforman al servidor Java Spring Boot y el cómo se comunican.

Como punto de partida se tiene el paquete de los controladores ('com.example.PruebaCRUD.Controllers'). Estas clases son las encargadas de manejar las solicitudes HTTP que le llegan al servidor, así como de devolver una respuesta a dicha solicitud [?]. En otras palabras, estas clases son las encargadas de pasar la información entre la vista y el modelo. Para este caso en particular, estas clases se comunican con las clases de servicio, las cuales se encuentran en el paquete 'com.example.PruebaCRUD.Services' y hacen uso de las clases que se encuentran en el paquete 'com.example.PruebaCRUD.DTO'.

Los DTO (Data Transfer Object), son clases que facilitan la transferencia de datos a través de la red o entre diferentes capas dentro del sistema [?]. Como se puede ver dentro del diagrama, estos DTO son usados por la mayoría de las clases: las clases Controller, las cuales ya fueron descritas, así como las clases Service y Repository, de las cuales se explicarán a continuación.

Las clases de tipo Service son clases que se encargan de gestionar toda la lógica de negocio del sistema [?]. Funciona como intermediario entre los controladores y los repositorios.

Por otra parte, se encuentran los archivos de tipo Repository. Estos archivos son interfaces que permiten el acceso a la base de datos y el realizar operaciones CRUD [?]. Al extender de 'JpaRepository', estas interfaces heredan métodos para manipular entidades, eliminando la necesidad de implementar muchas de estas operaciones de manera manual.

Finalmente, se muestran a las clases de tipo Entity. Estas clases son parte de la capa de persistencia, y permiten mapear una clase Java, a una tabla de una base de datos [?]. De esta manera, las variables dentro de una clase, funcionarán como columnas dentro de una base de datos, haciendo que los datos que manda el usuario, se puedan guardar, actualizar, eliminar o consultar.

Servidor Java Spring-Boot

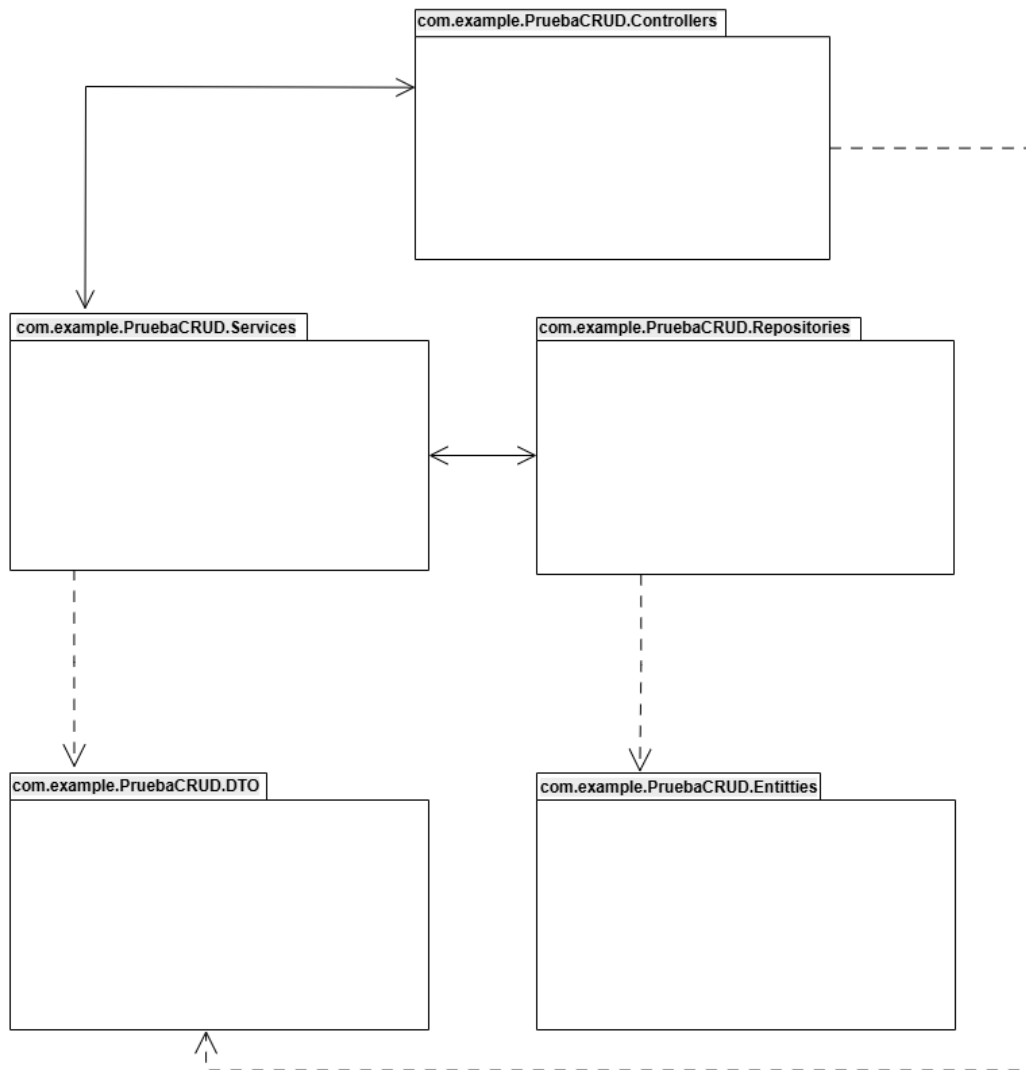


Figura A.2: Diagrama de la Estructura General del Servidor Java.

A continuación, se presentan los diagramas de cada clase dentro de los paquetes explicados anteriormente.

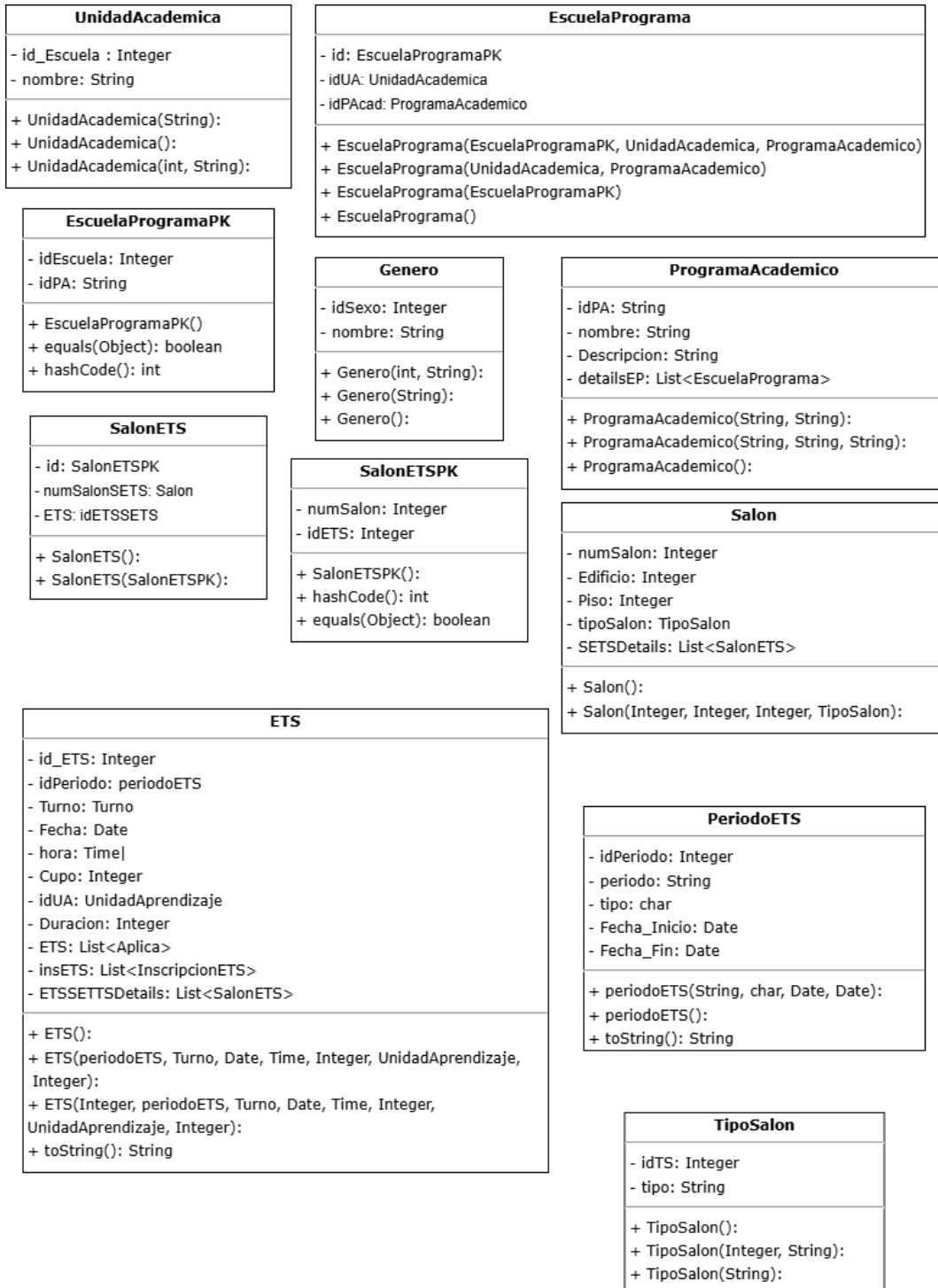


Figura A.4: Diagrama de clases de las entidades del servidor parte 1.

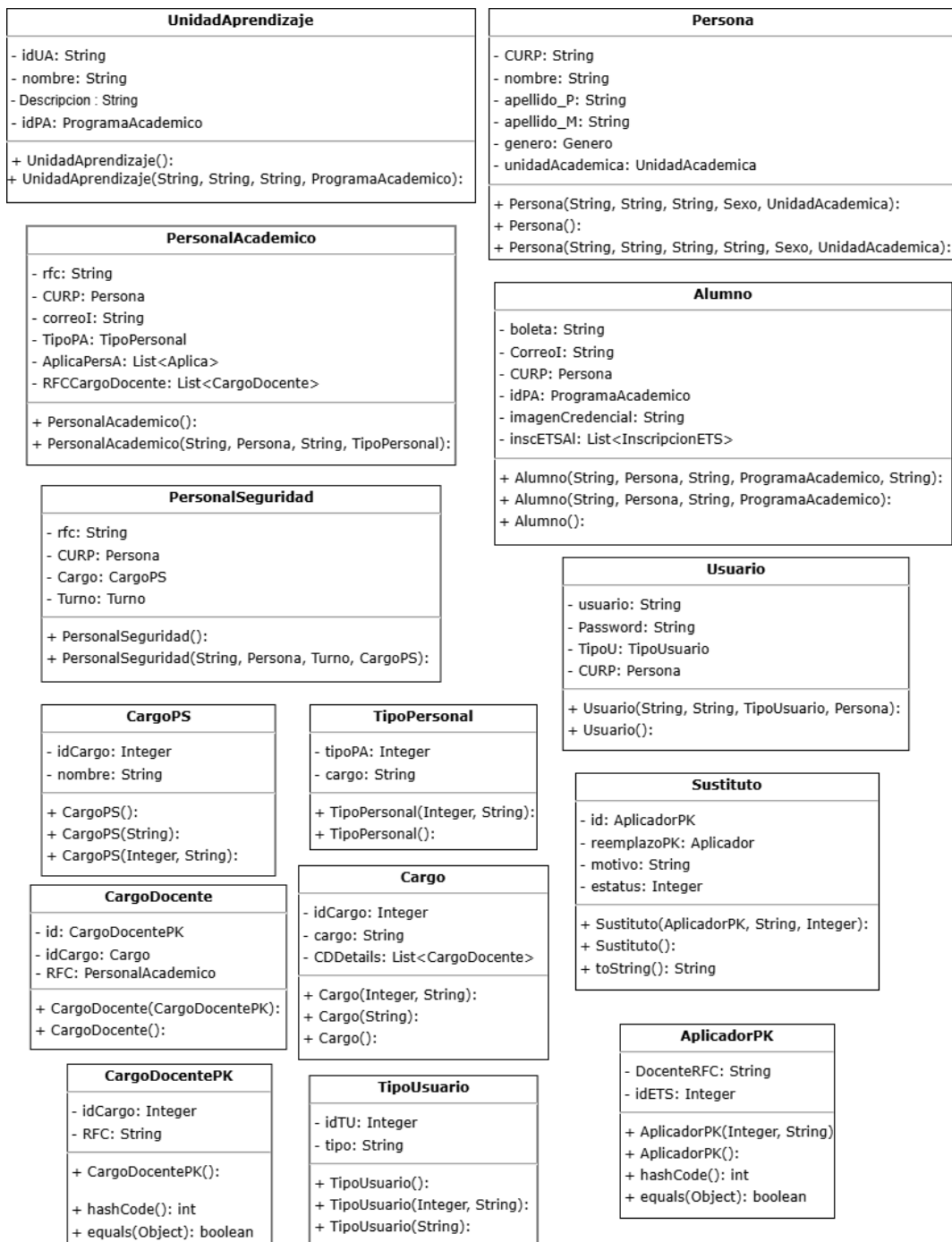


Figura A.5: Diagrama de clases de las entidades del servidor parte 2.

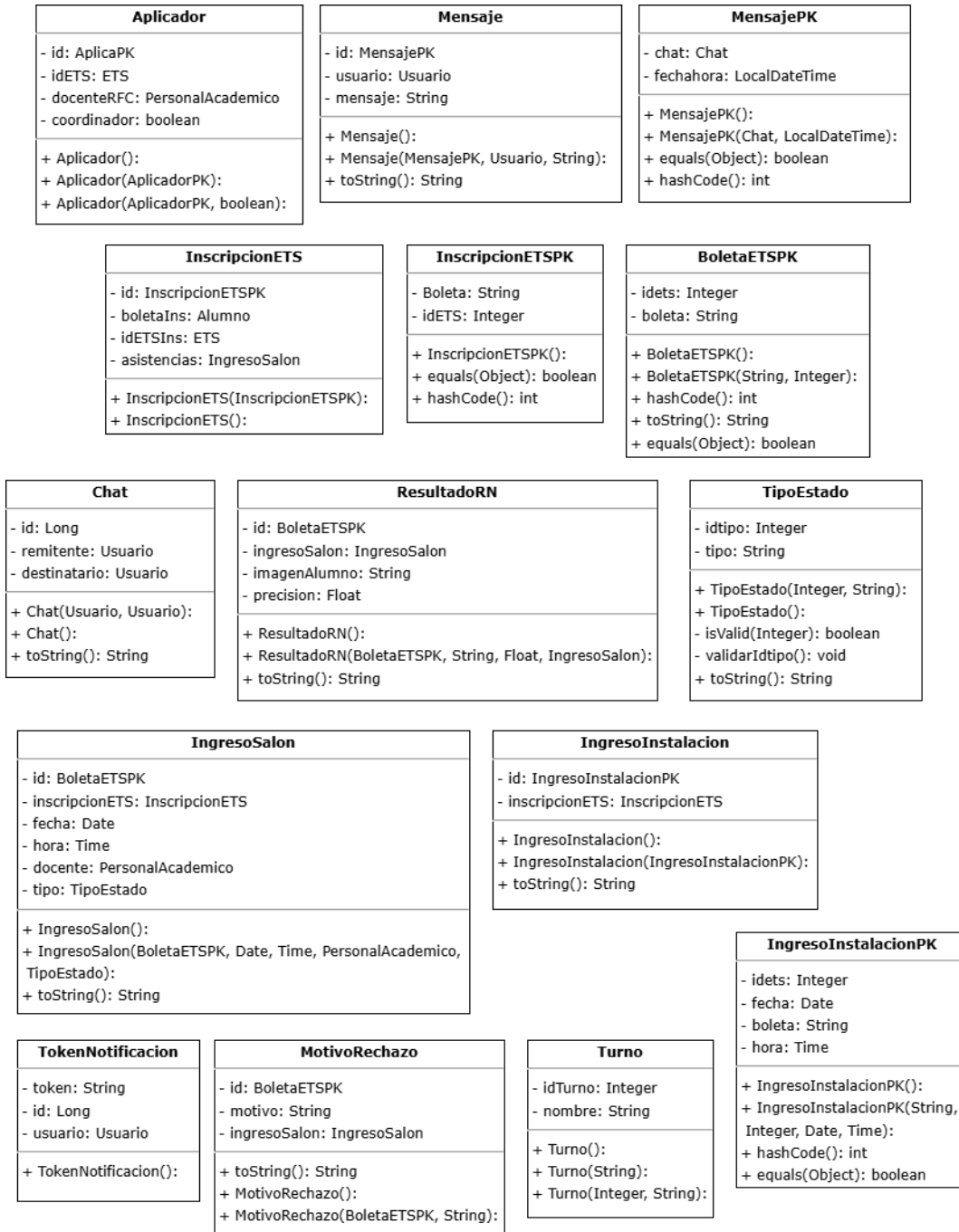


Figura A.6: Diagrama de clases de las entidades del servidor parte 3.

A.2.2. Controladores

En las (figuras ??, ??, ??, ?? y ??), se presentan los diagramas de las clases de los controladores junto con sus respectivas propiedades.

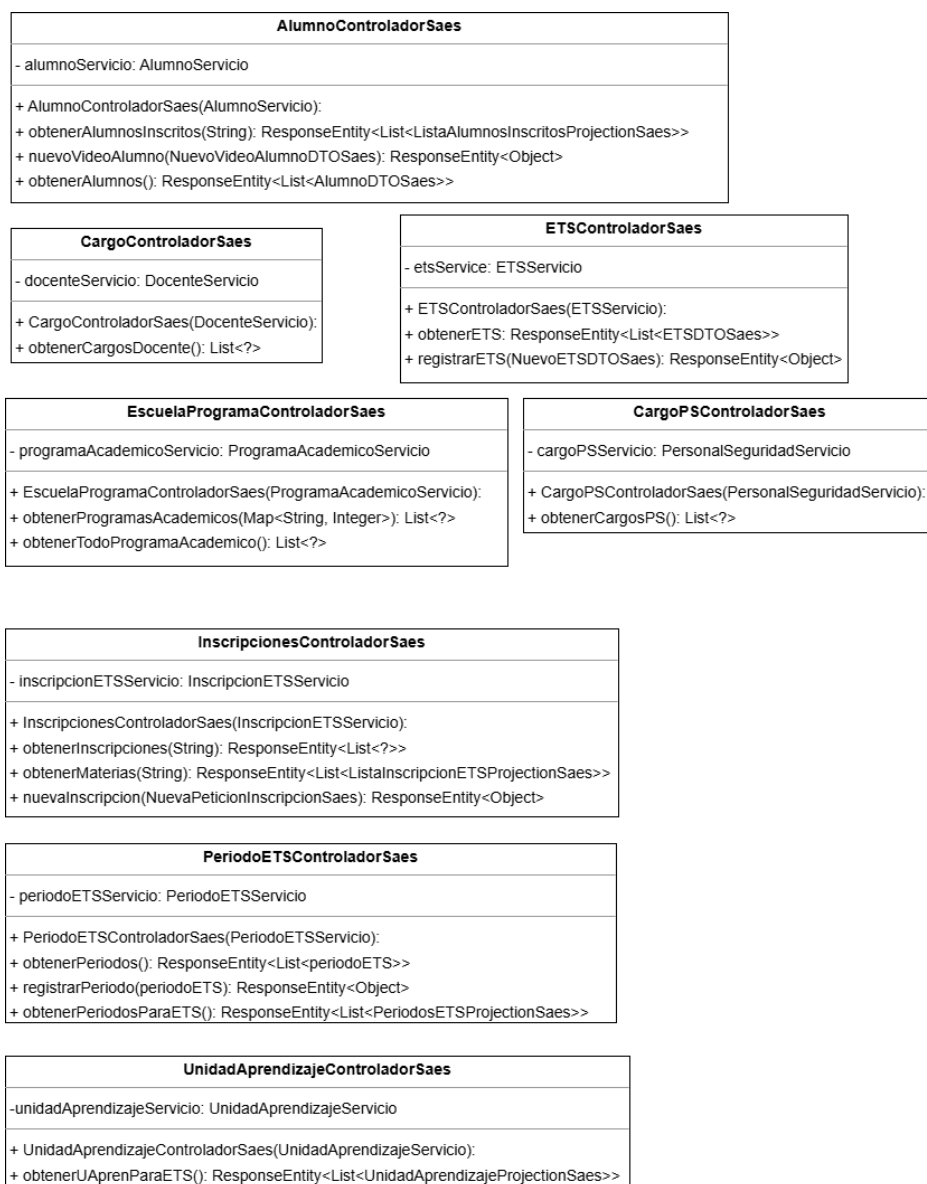


Figura A.7: Diagrama de clases de los controladores del servidor parte 1.

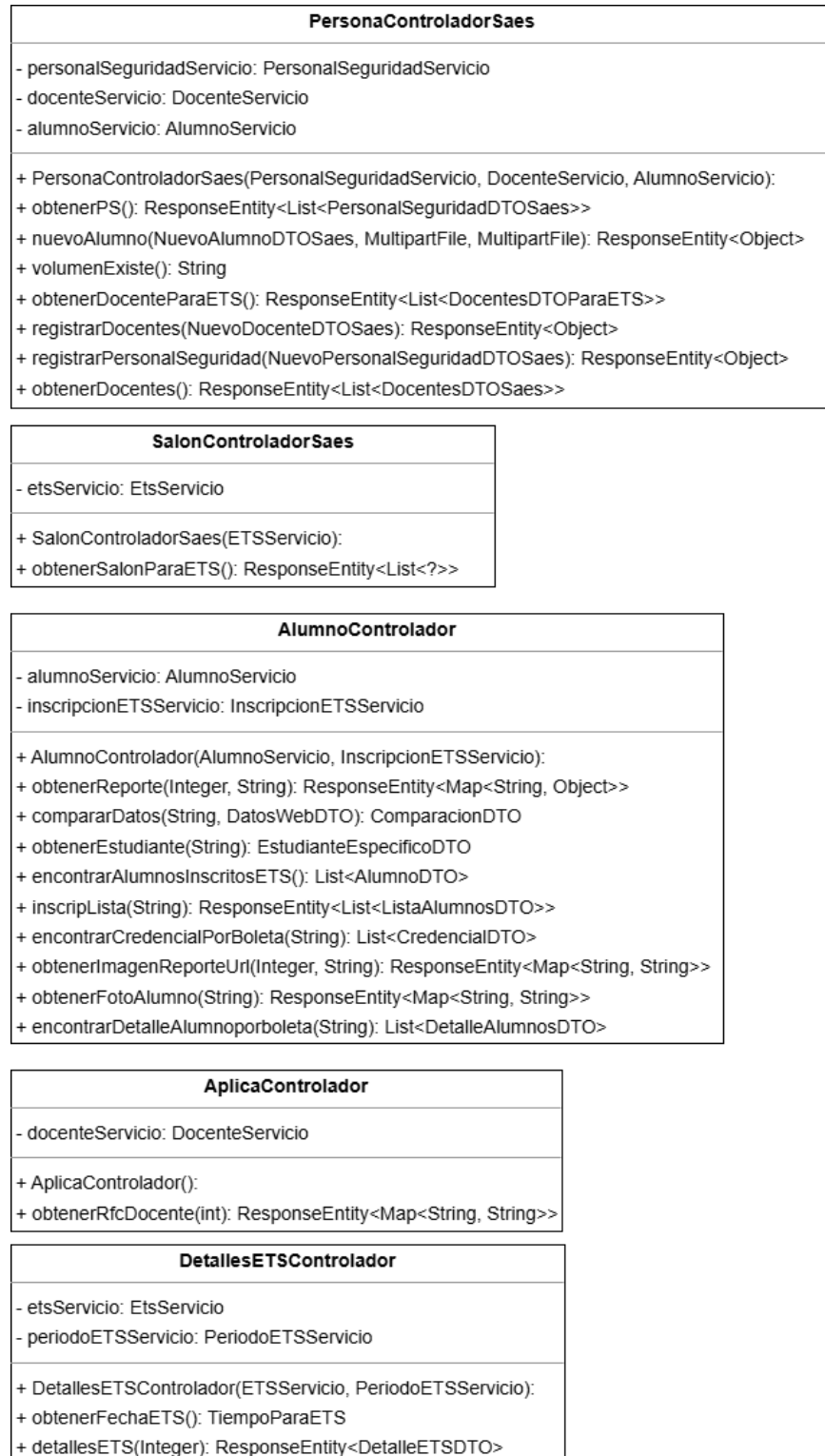


Figura A.8: Diagrama de clases de los controladores del servidor parte 2.

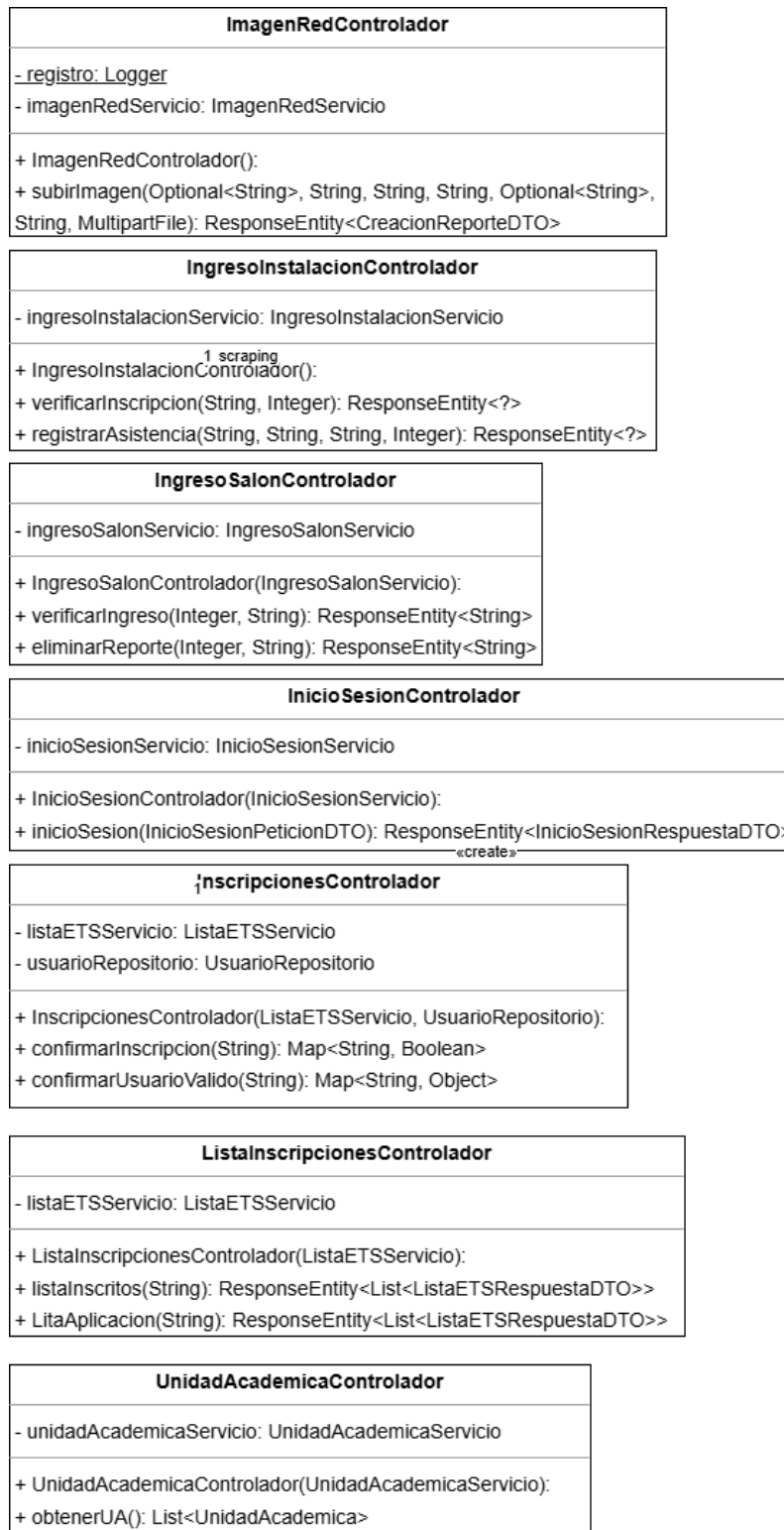


Figura A.9: Diagrama de clases de los controladores del servidor parte 3.

MensajeControlador
- mensajeServicio: MensajeServicio - usuarioRepositorio: UsuarioRepositorio
+ MensajeControlador(MensajeServicio, UsuarioRepositorio): + enviarMensaje(MensajeRecibidoDTO): ResponseEntity<Map<String, Object>> + obtenerUsuariosParaChatear(String): List<ListadoUsuariosDTO> + obtenerChat(String): ResponseEntity<Object> + obtenerHistorial(String, String): ResponseEntity<List<MensajeDTO>>

PersonaControlador
- personaServicio: PersonaServicio
+ PersonaControlador(PersonaServicio): + obtenerNombre(String): ResponseEntity<List<DatosPersonaDTO>> + registrarPersona(Persona): ResponseEntity<Object> + obtenerPersona(): List<PersonaDTO> + eliminarPersona(String): ResponseEntity<Object>

PersonalAcademicoControlador
- docenteServicio: DocenteServicio
+ PersonalAcademicoControlador(DocenteServicio): + obtenerTodosDocentes(): ResponseEntity<List<DocenteDTO>>

SustitutoControlador
- sustitutoServicio: SustitutoServicio
+ SustitutoControlador(SustitutoServicio): + crearSolicitudReemplazo(SolicitudReemplazoDTO): ResponseEntity<SolicitudReemplazoDTO> + obtenerTodasSolicitudes(): ResponseEntity<List<SolicitudReemplazoDTO>> + obtenerSolicitudesPendientes(): ResponseEntity<List<SolicitudReemplazoDTO>> + aprobarReemplazo(Integer, String, String): ResponseEntity<Map<String, Object>> + rechazarReemplazo(Integer, String, String): ResponseEntity<Map<String, Object>> + handleRuntimeException(RuntimeException): ResponseEntity<Map<String, Object>> + verificarSolicitudPendiente(Integer, String): ResponseEntity<VerificacionSolicitudResponseDTO>

TokenNotificationControlador
- tokenNotificationServicio: TokenNotificationServicio
+ TokenNotificationControlador(TokenNotificationServicio): + registrarToken(String, String): TokenRespuestaDTO

Figura A.10: Diagrama de clases de los controladores del servidor parte 4.

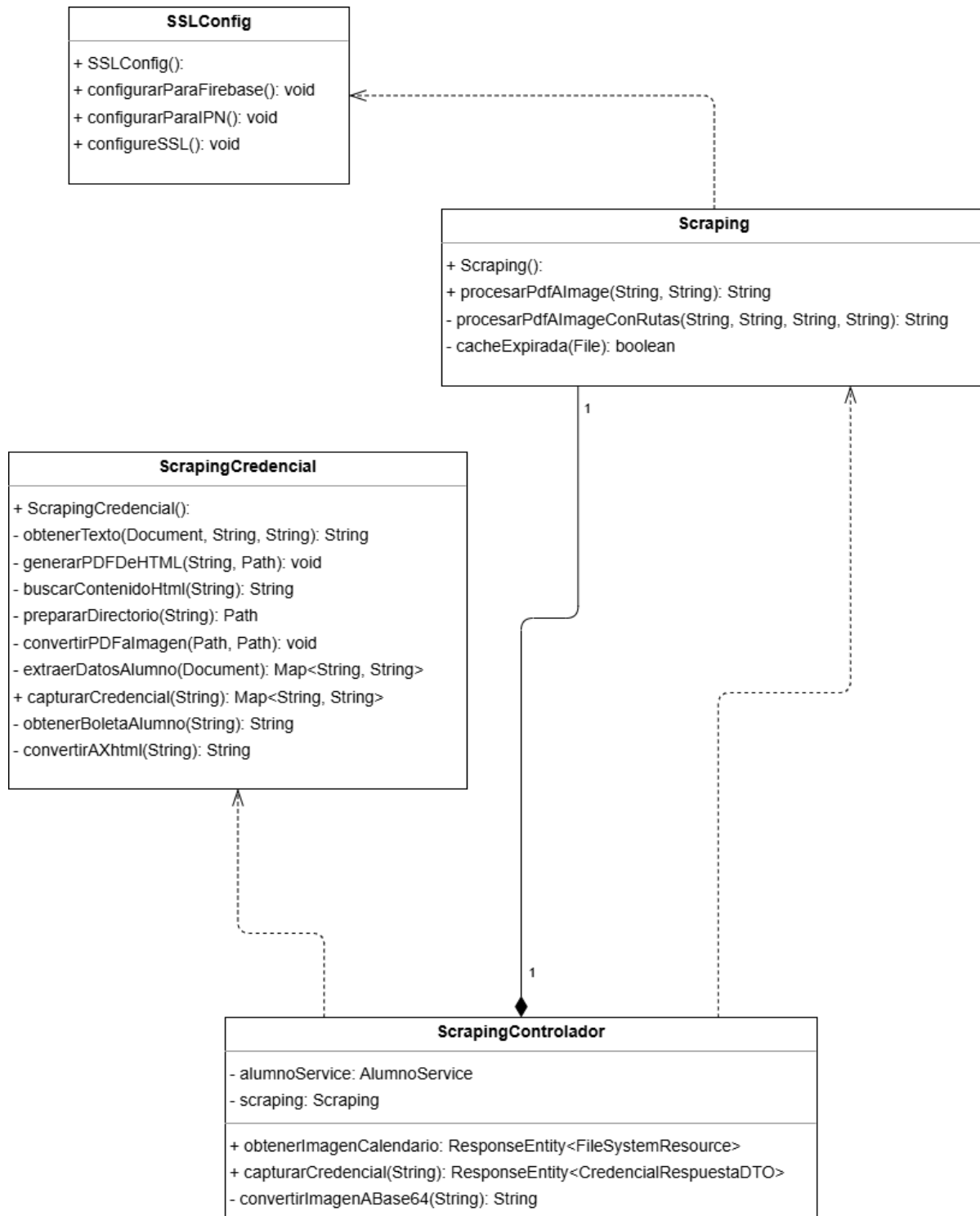


Figura A.11: Diagrama de clases de los controladores del servidor parte 5.

A.2.3. Servicios

A continuación , en las (figuras ??, ??, ??, ??, ??, ?? y ??), se mostrarán los diagramas de los servicios que conforman al servidor, es decir, los que contienen la lógica del negocio de todo nuestro sistema. A su vez, se mostrarán las propiedades de cada uno de estas clases, es decir, las variables y los métodos que conforman a cada clase.



Figura A.12: Diagrama de clase AlumnoServicio de los servicios del servidor parte 1.

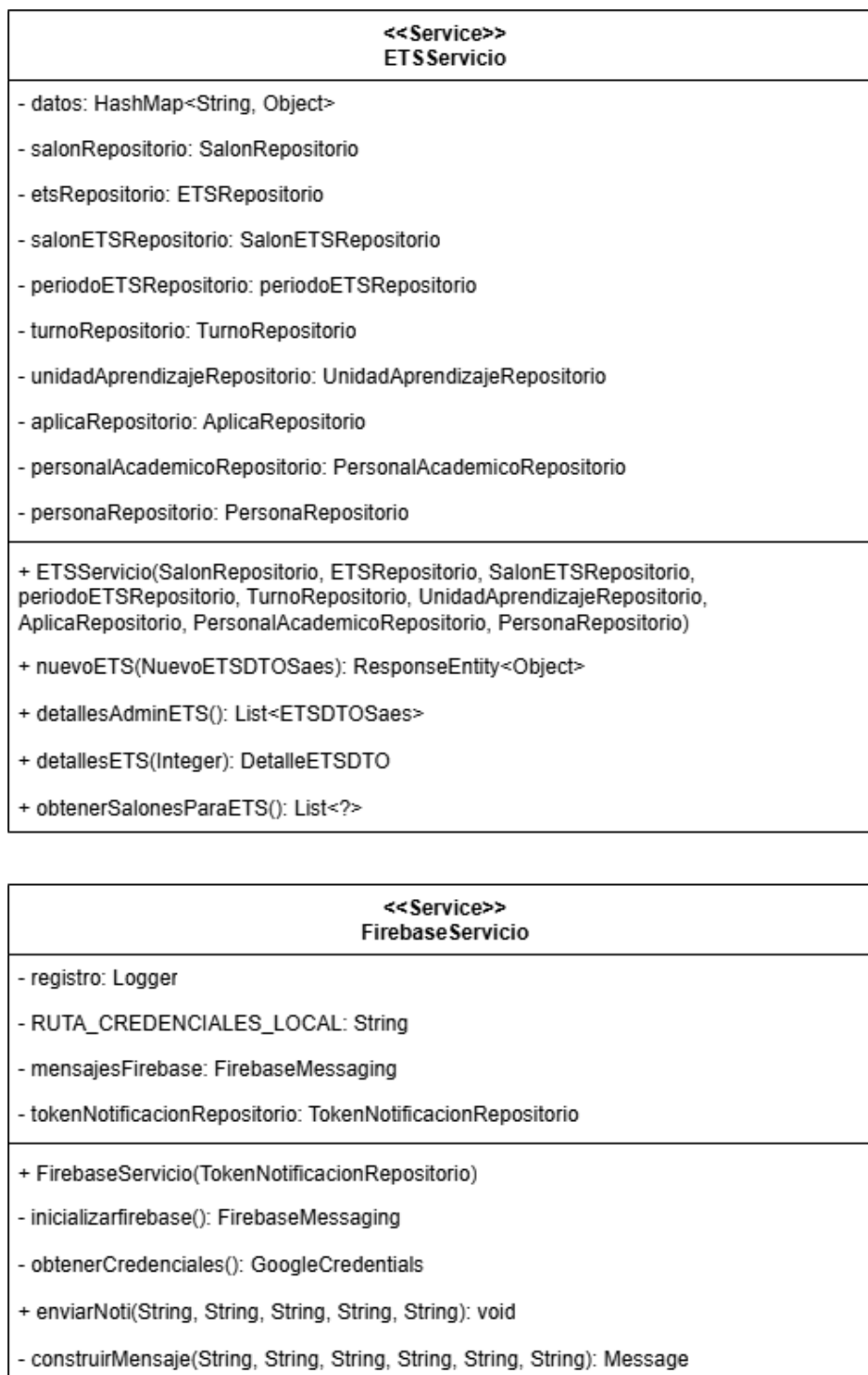


Figura A.13: Diagrama de clases de los servicios del servidor parte 2.



Figura A.14: Diagrama de clases de los servicios del servidor parte 3.

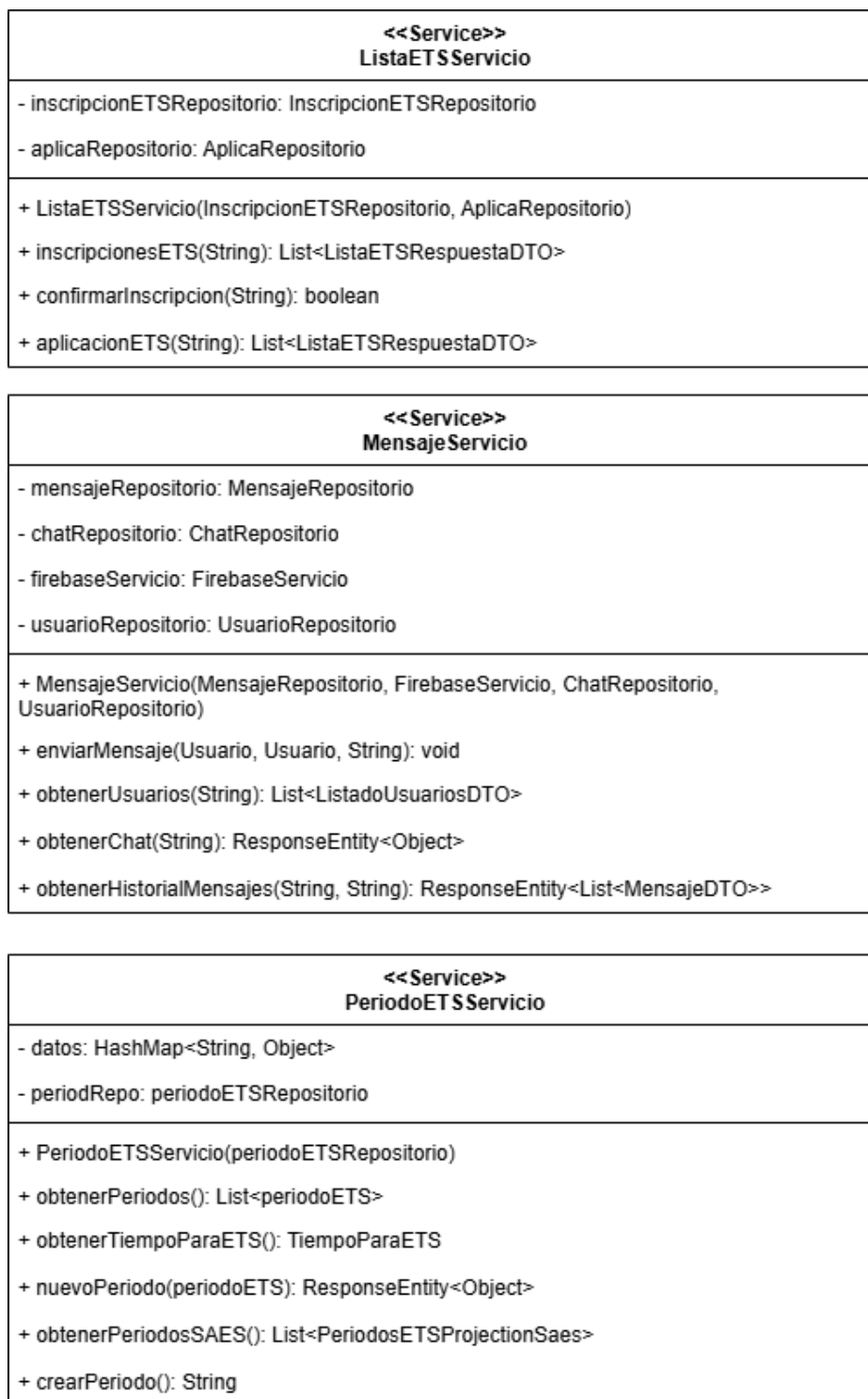


Figura A.15: Diagrama de clases de los servicios del servidor parte 4.

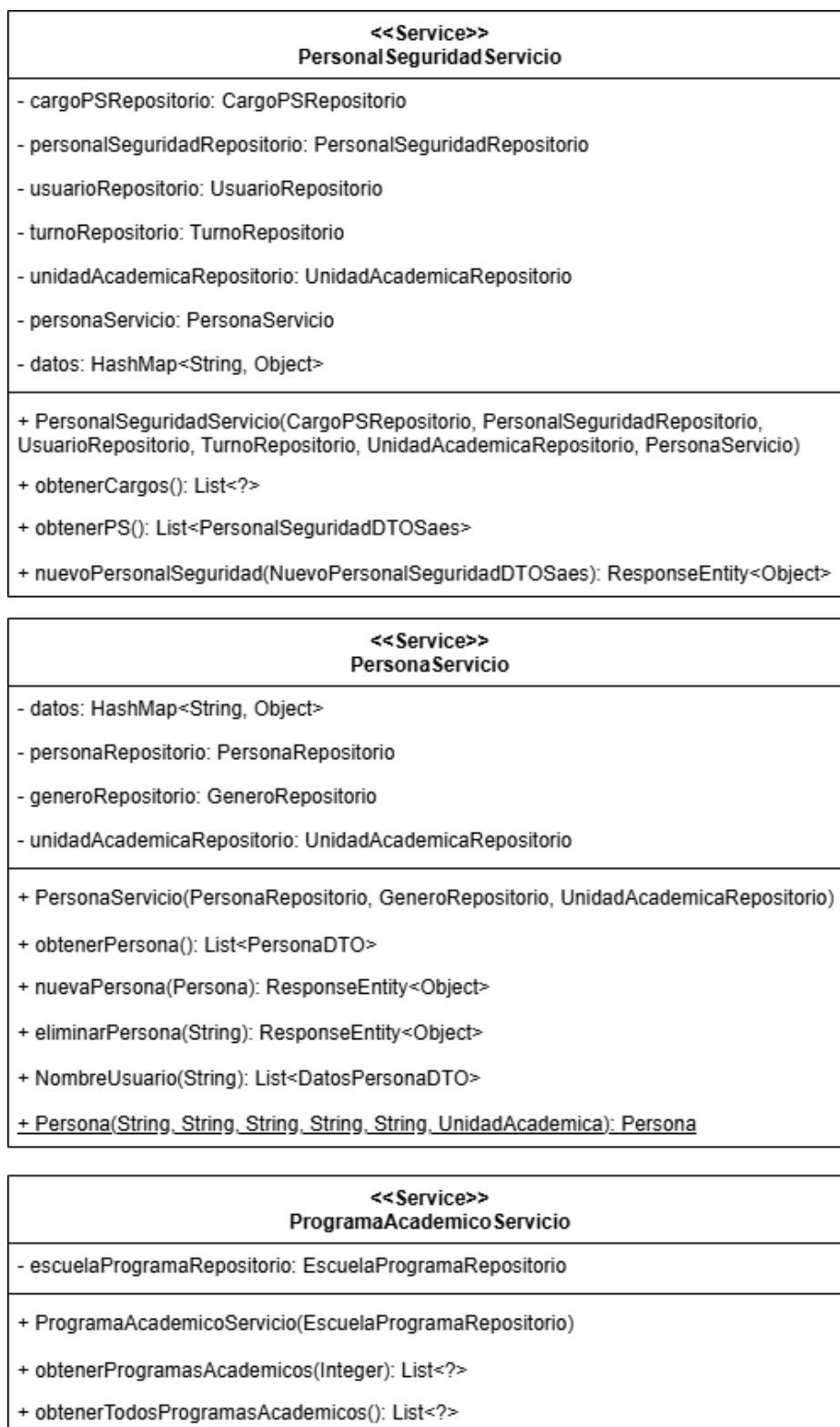


Figura A.16: Diagrama de clases de los servicios del servidor parte 5.

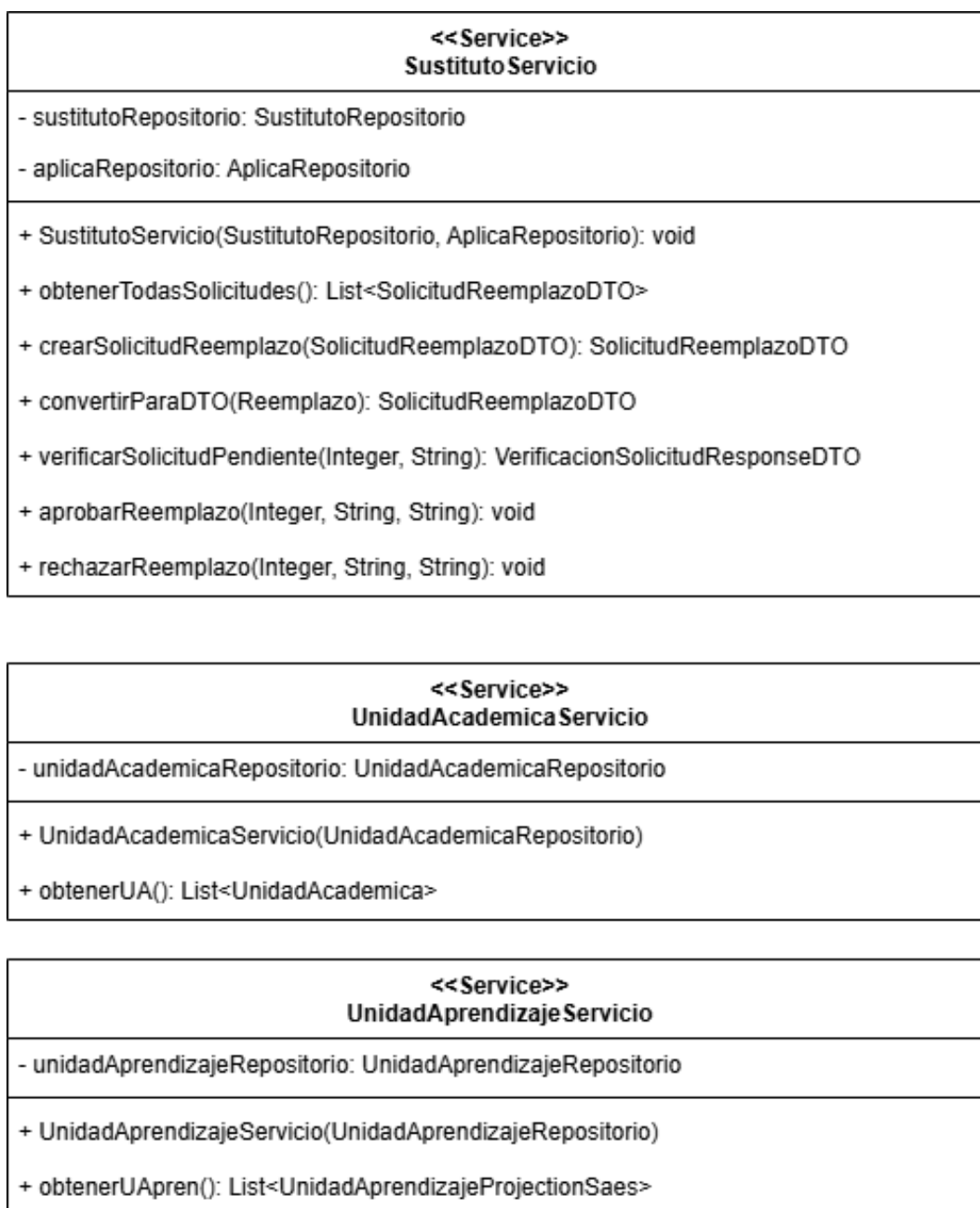


Figura A.17: Diagrama de clases de los servicios del servidor parte 6.

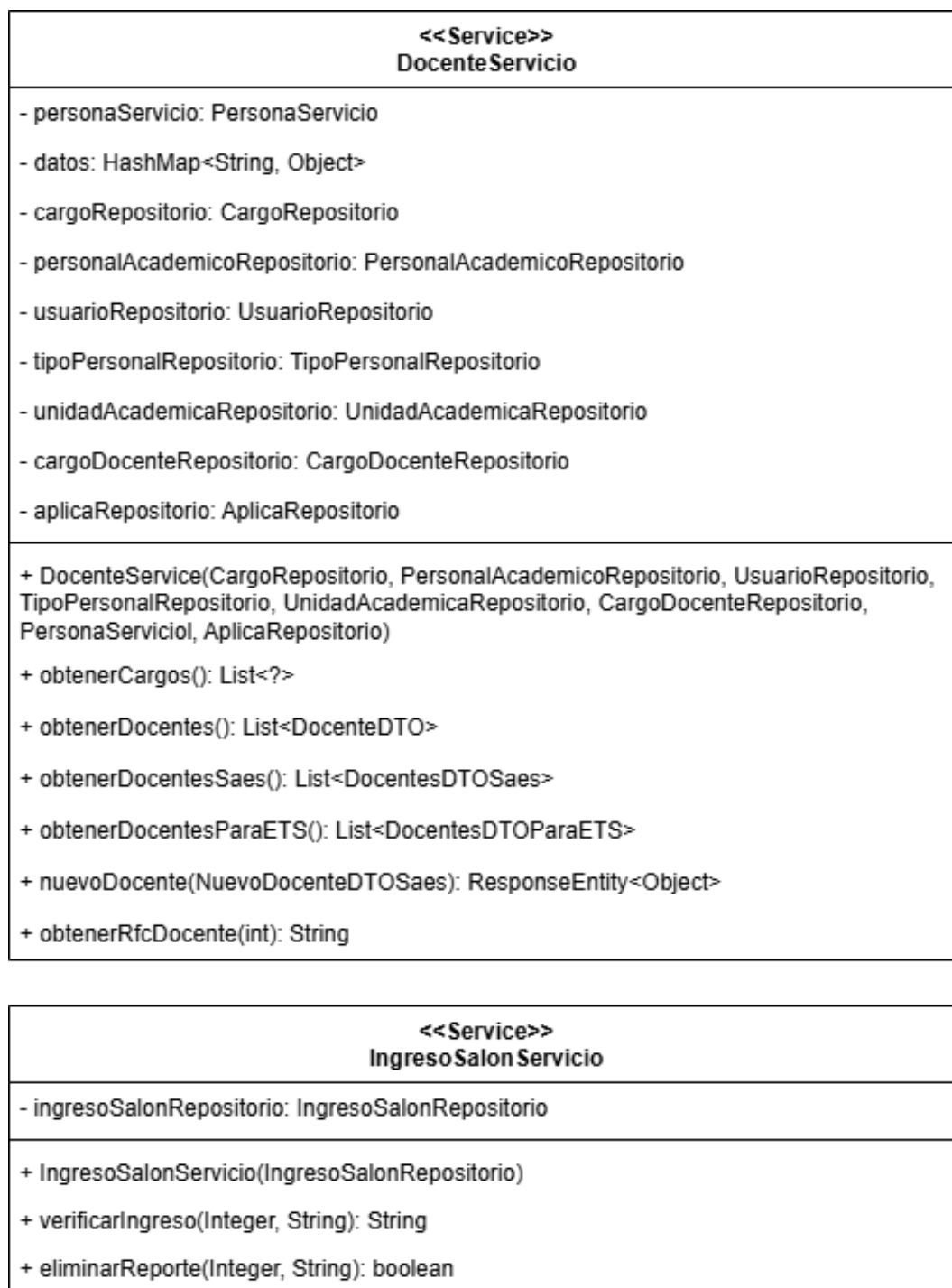


Figura A.18: Diagrama de clases de los servicios del servidor parte 7.

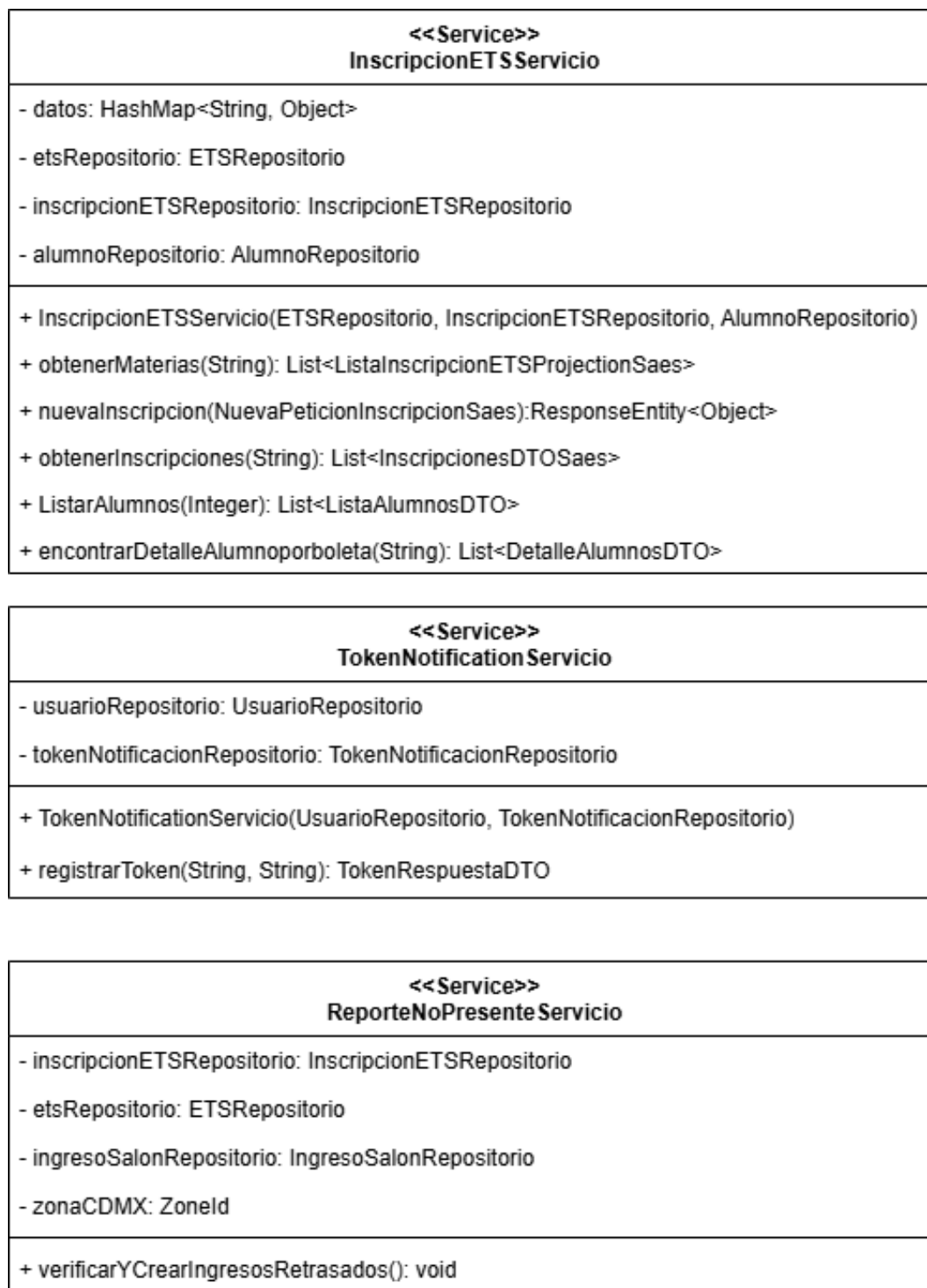


Figura A.19: Diagrama de clases de los servicios del servidor parte 8.

A.2.4. Repositorios

Ahora, en las (figuras ?? y ??), se mostrará más a detalle las propiedades y los métodos de cada una de las clases de tipo repositorio que se encuentran dentro del servidor.

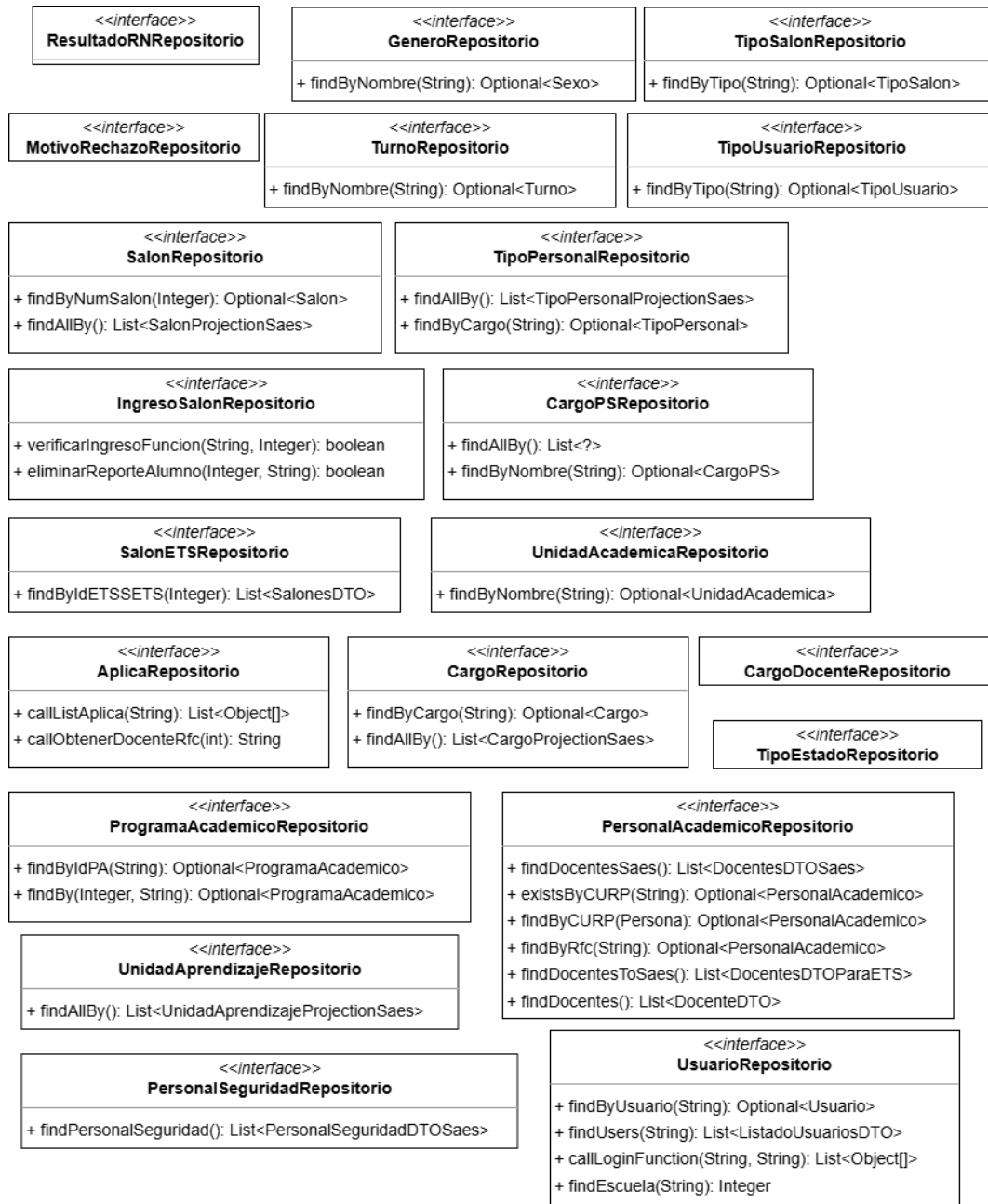


Figura A.20: Diagrama de clases de los repositorios del servidor parte 1.



Figura A.21: Diagrama de clases de los repositorios del servidor parte 2.

A.2.5. Objetos de Transferencia de datos (DTO)

Por última parte, en las (figuras ??, ??, ??, ??, ?? y ??) , tenemos a las clases DTO. Estas clases, como se mencionó anteriormente, son utilizadas por las clases de tipo repositorio, servicio y controlador, por lo que son una pieza clave dentro del proyecto.

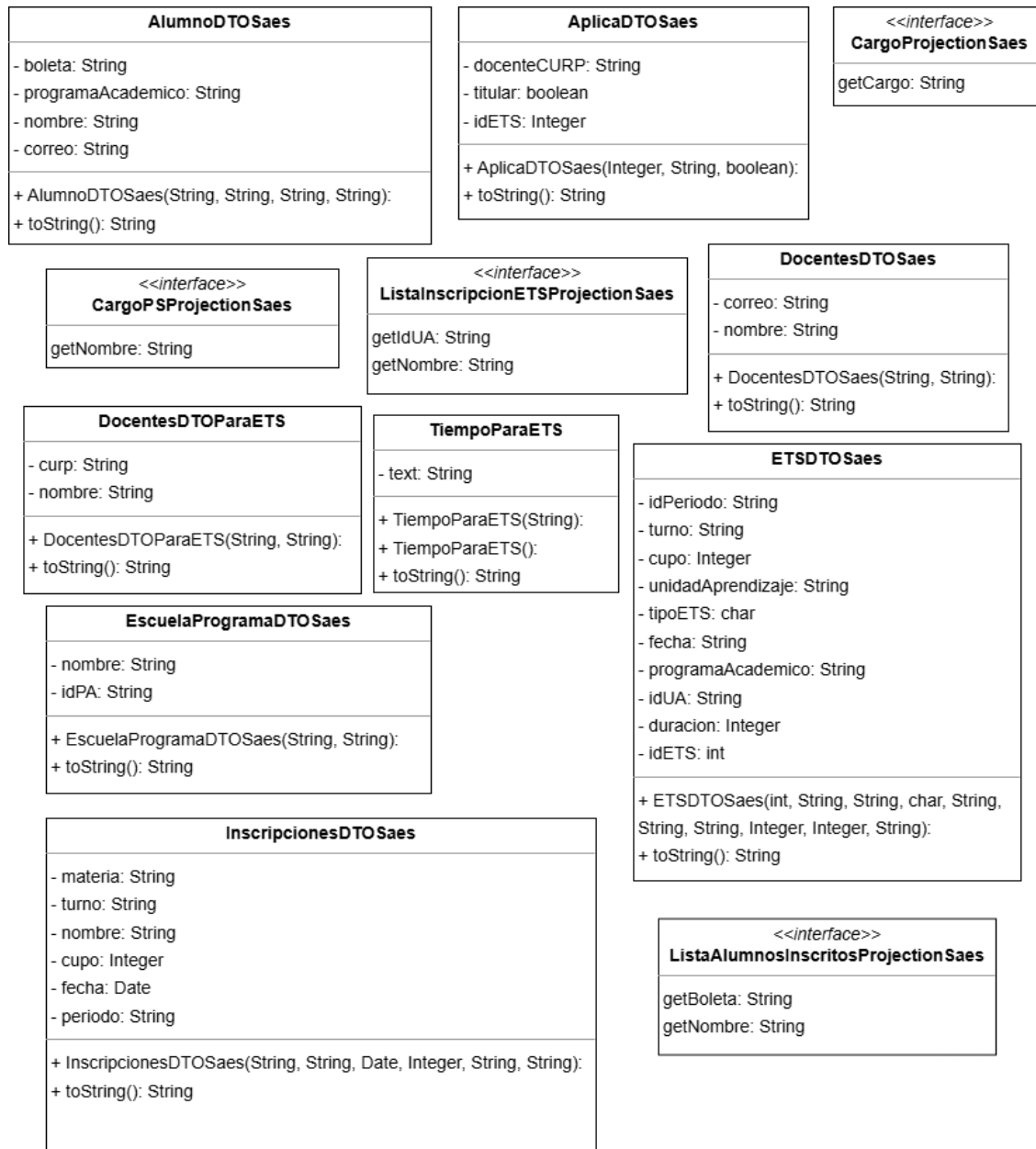


Figura A.22: Diagrama de clases de los DTO del servidor parte 1.

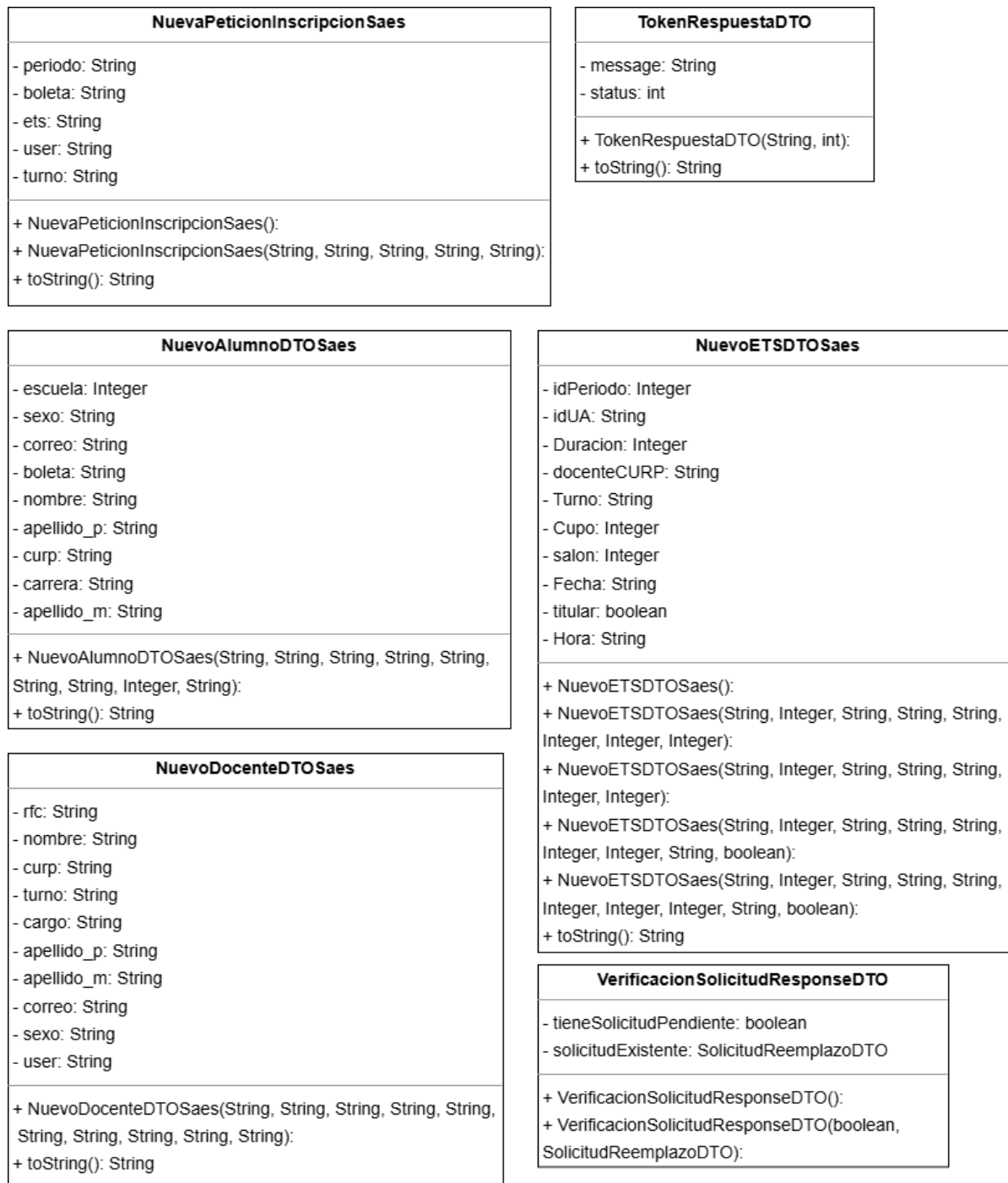


Figura A.23: Diagrama de clases de los DTO del servidor parte 2.

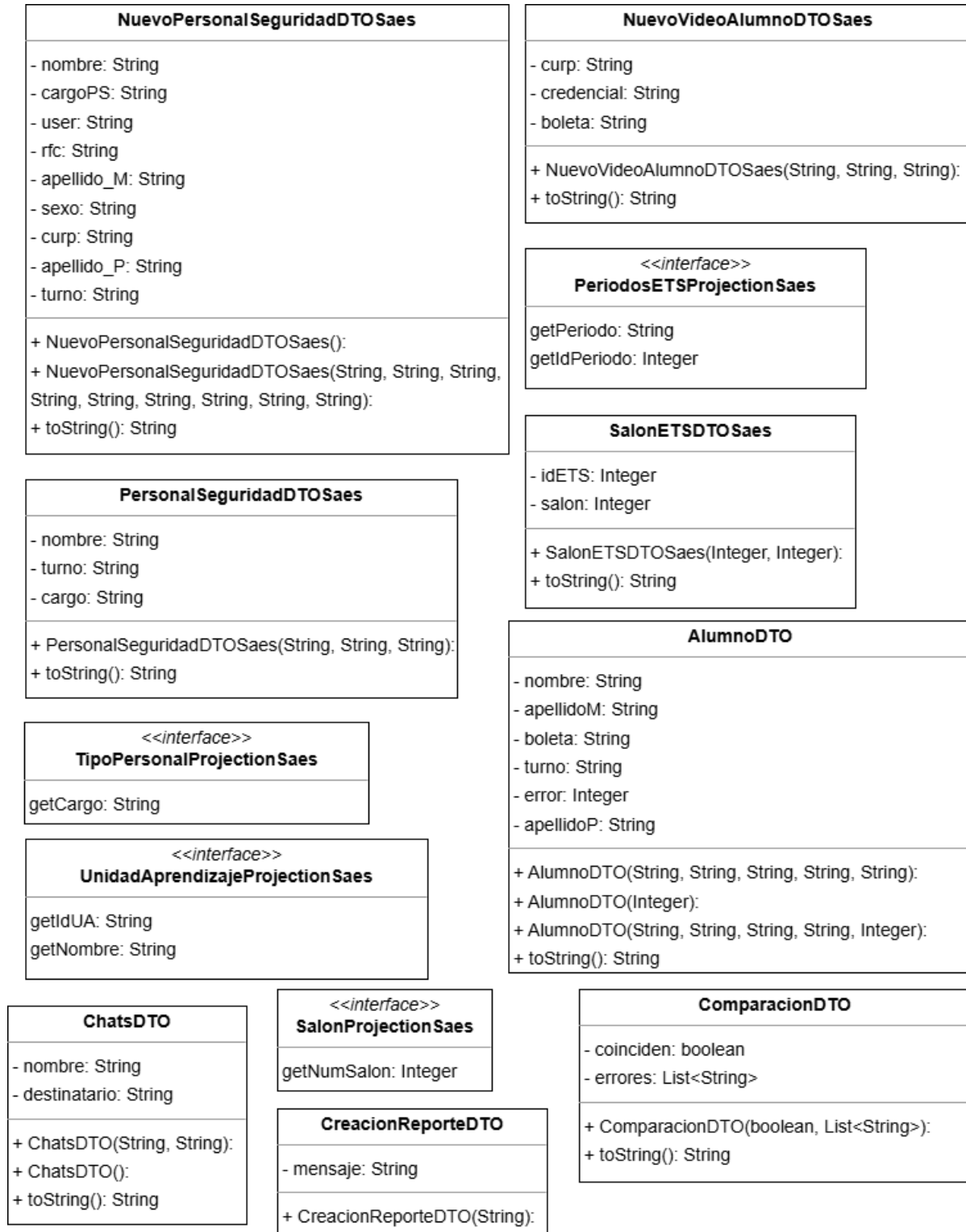


Figura A.24: Diagrama de clases de los DTO del servidor parte 3.

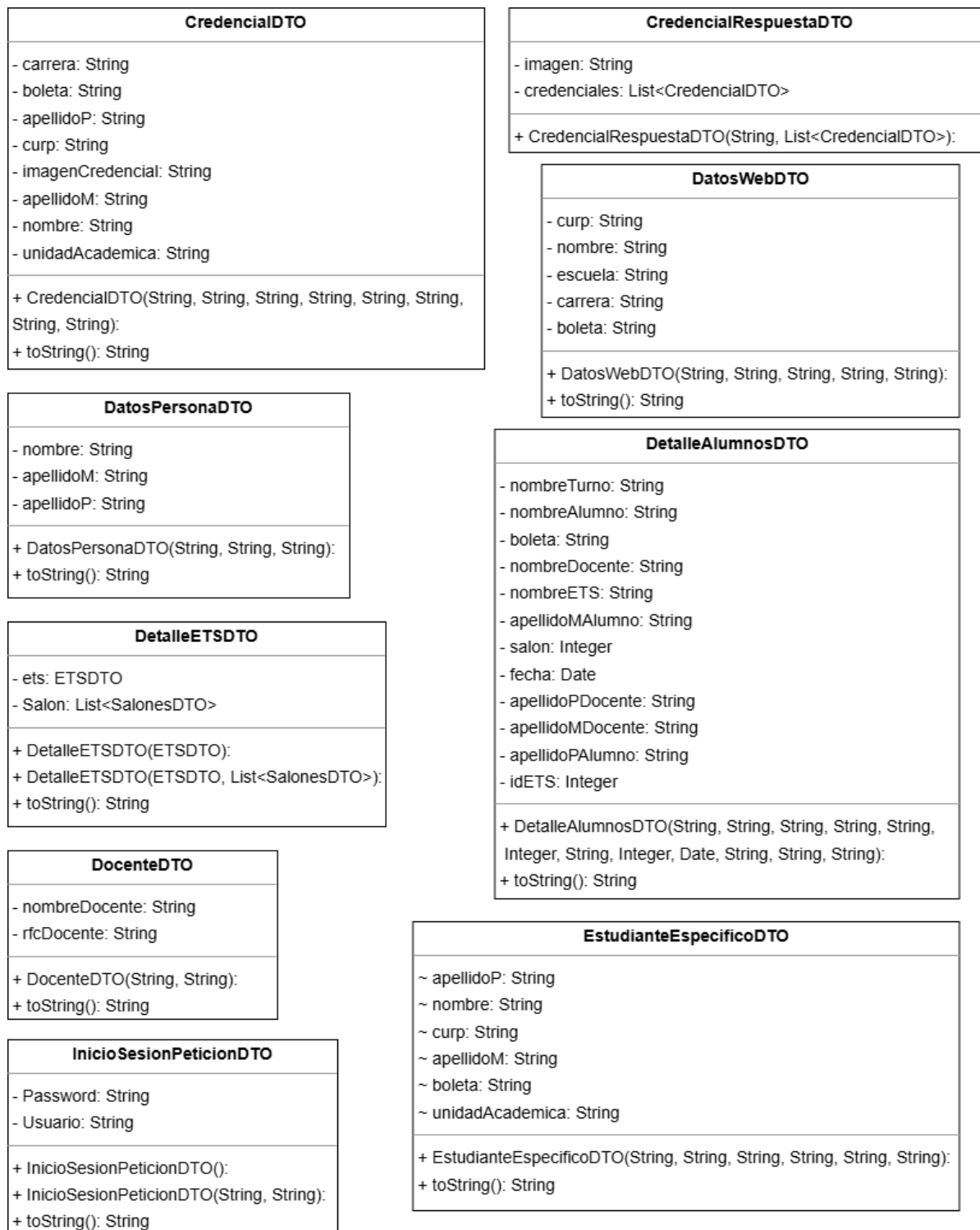


Figura A.25: Diagrama de clases de los DTO del servidor parte 4.

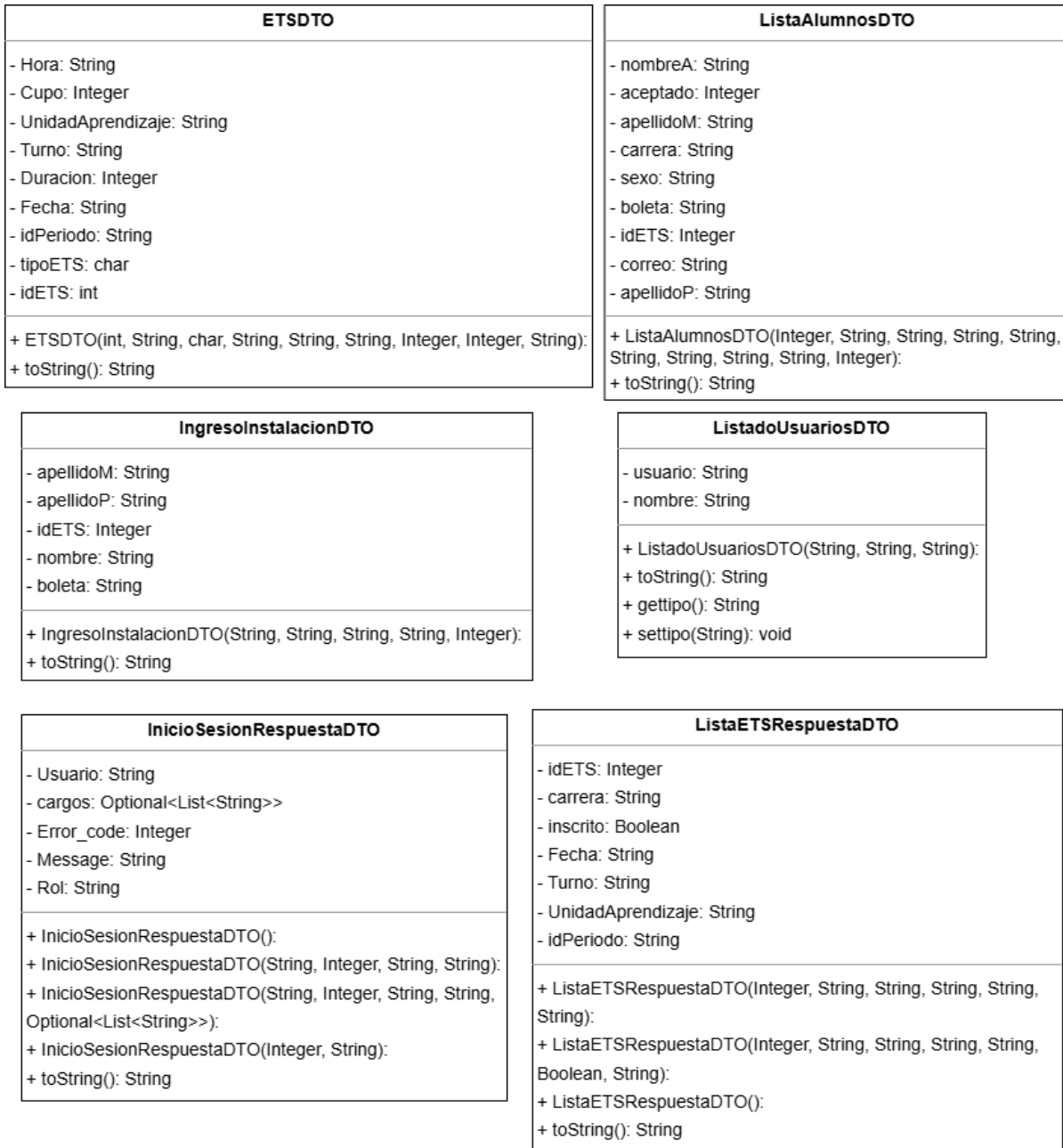


Figura A.26: Diagrama de clases de los DTO del servidor parte 5.

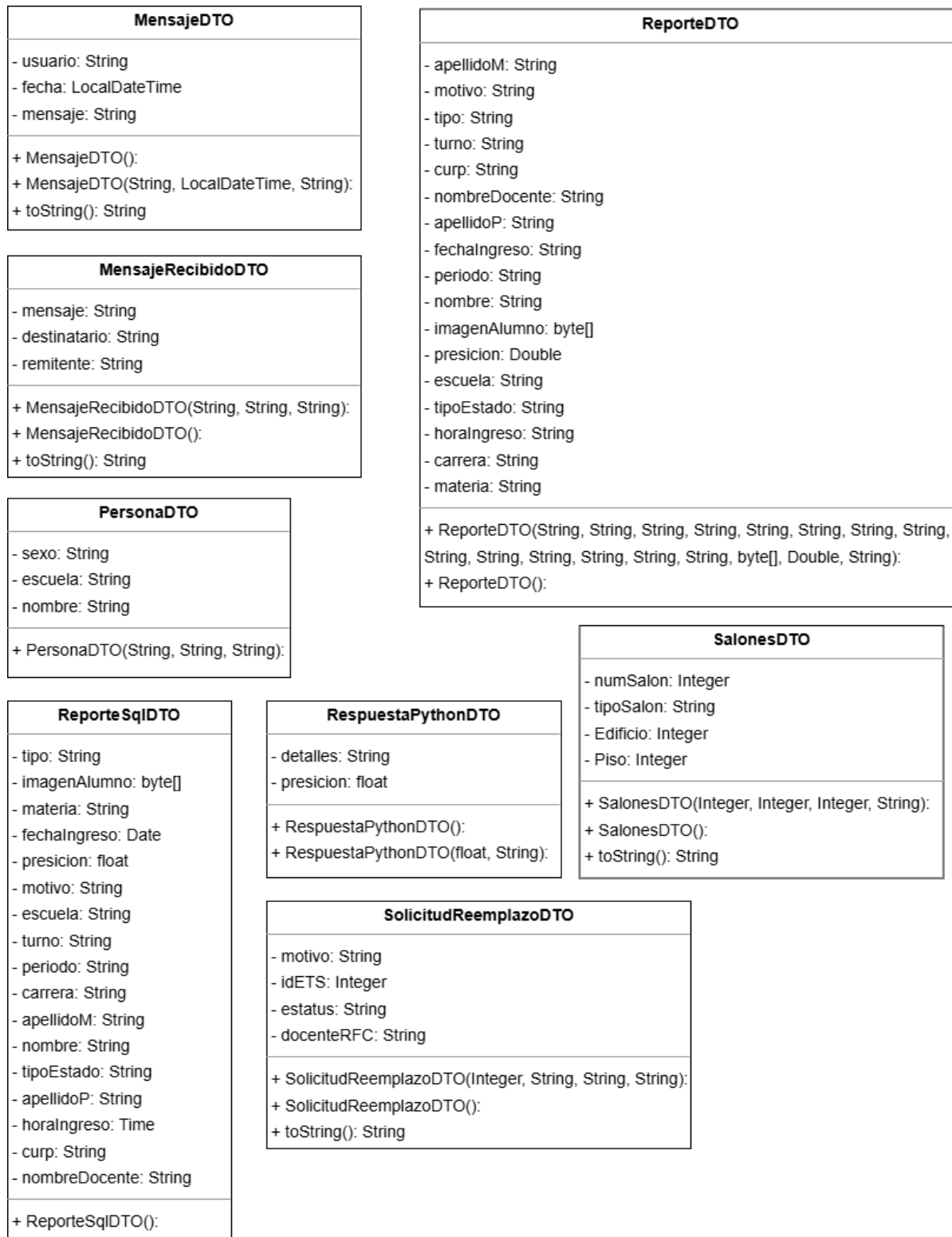


Figura A.27: Diagrama de clases de los DTO del servidor parte 6.

A.3. Estructura Interna Detallada de la Aplicación Móvil

La (figura ??) profundiza en la arquitectura interna de la aplicación móvil, presentando los paquetes y componentes clave que la conforman.

El punto de entrada principal de la aplicación es la actividad 'MainActivity.kt', la cual orquesta el flujo general de la aplicación.

La interfaz de usuario de la aplicación se construye dentro del paquete 'Pantallas'. Este paquete contiene las implementaciones de las pantallas de la aplicación utilizando Jetpack Compose. Cada pantalla se define mediante funciones Composable, que describen la estructura y el comportamiento de la interfaz de usuario de manera declarativa.

La lógica y la gestión del estado de las 'Pantallas' se encuentran en el paquete de la aplicación móvil llamado 'com.example.prueba3.Views'. Dentro de este paquete, los ViewModels actúan como intermediarios, proporcionando los datos necesarios a las Composables y manejando la lógica de las interacciones del usuario. Existe una relación directa entre las 'Pantallas' y los 'Views', donde cada pantalla suele estar asociada a un ViewModel específico.

El modelo de datos de la aplicación se define principalmente en el paquete 'com.example.prueba3.Clases'. Este paquete contiene las clases que representan los datos utilizados por la aplicación. En su mayoría, estas clases son Data Classes, diseñadas para almacenar y transportar datos de manera concisa y eficiente.

La comunicación con los servicios backend se gestiona a través del paquete 'Retrofit'. Este paquete contiene las interfaces de Retrofit que definen los endpoints de la API de los servidores remotos. Estas interfaces especifican las operaciones HTTP (GET, POST y DELETE) necesarias para interactuar con el Servidor Java Spring-Boot y el Servidor Red Neuronal. Retrofit se encarga de generar el código necesario para realizar estas solicitudes y procesar las respuestas del servidor.

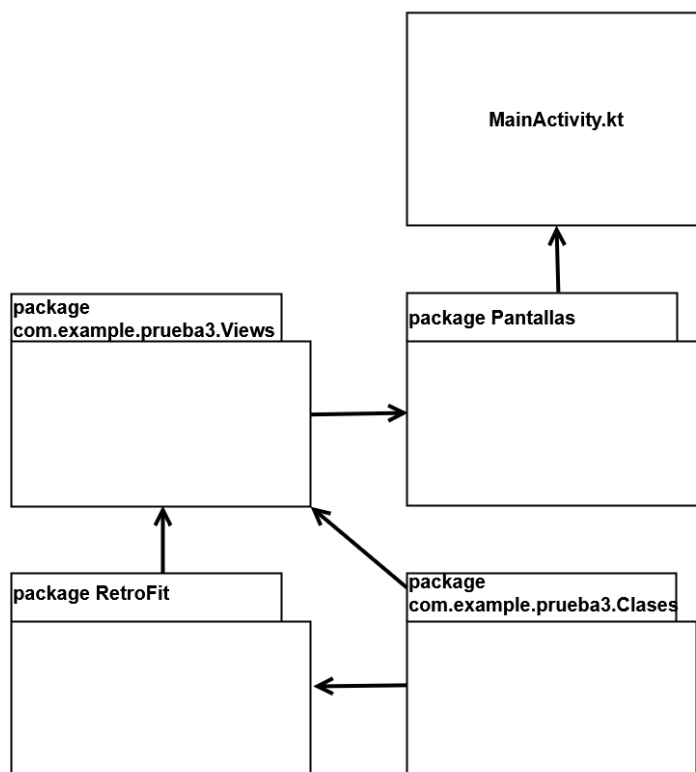


Figura A.28: Estructura Interna Detallada de la Aplicación Móvil por Paquetes.

A continuación, se presentarán las clases contenidas en los paquetes 'com.example.prueba3.Views' (ViewModels), 'com.example.prueba3.Clases' (Data Classes) y las interfaces definidas en el paquete 'Retrofit', detallando su estructura y funcionalidad sección por sección. Adicionalmente, se incluirán diagramas de componentes específicos para cada una de las 'Pantallas' de la aplicación móvil, ilustrando la composición de sus elementos y sus dependencias internas.

A.3.1. Diagramas de Componentes de package Pantallas

Para una comprensión más profunda de la arquitectura de la interfaz de usuario de la aplicación móvil, a continuación se presentan los diagramas de componentes para cada una de las pantallas principales. Estos diagramas ilustran la composición interna de cada pantalla, mostrando los elementos de la interfaz de usuario (Composables), los datos que manejan y las posibles interacciones o dependencias con otros componentes. Cada diagrama proporciona una vista modular de la pantalla, facilitando la visualización de cómo se construyen las distintas secciones de la interfaz de usuario y cómo se relacionan entre sí. La (figura ??) muestra las pantallas (archivos .kt) presentes en package.Pantallas.

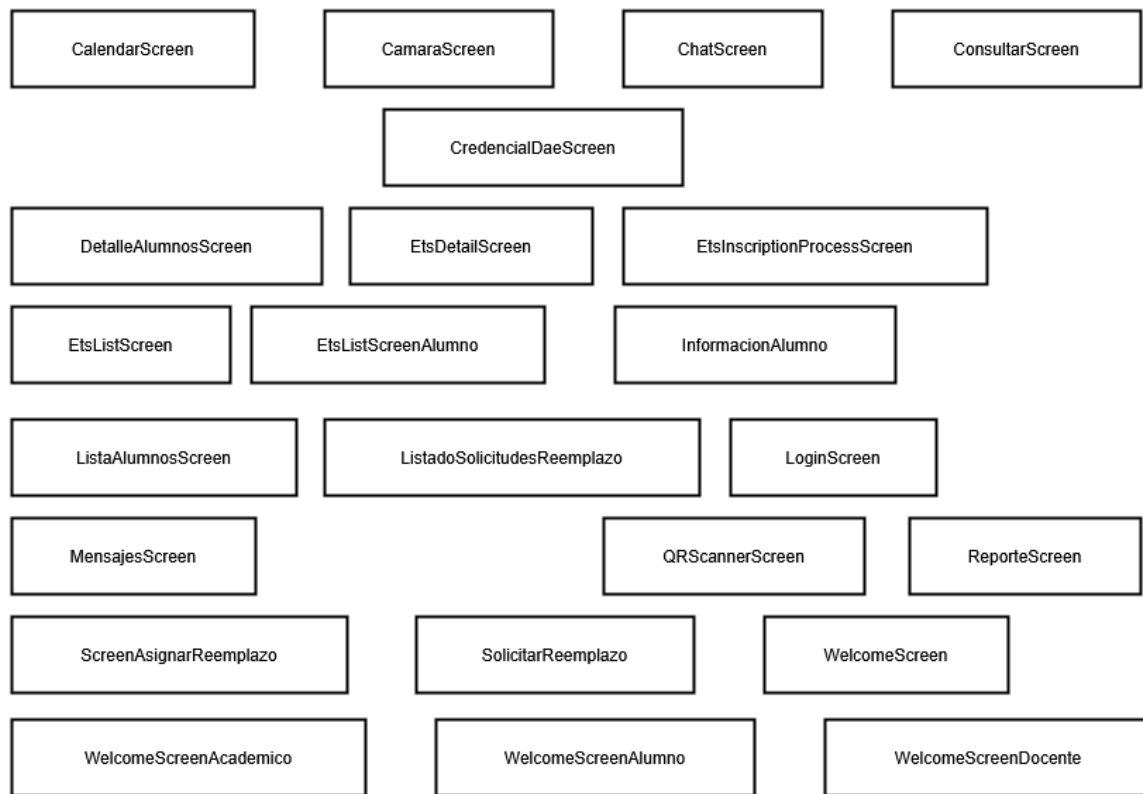


Figura A.29: Pantallas presentes en package.Pantallas (archivos .kt).

A.3.2. Diagrama de Componentes de CalendarScreen.kt

La (figura ??) muestra el diagrama de componentes para CalendarScreen.kt que es un composable que contiene a la IU02 Pantalla Consultar calendario escolar, que muestra el calendario escolar y permite al usuario ver cuantos días faltan para el periodo de ETS próximo si esta disponible.

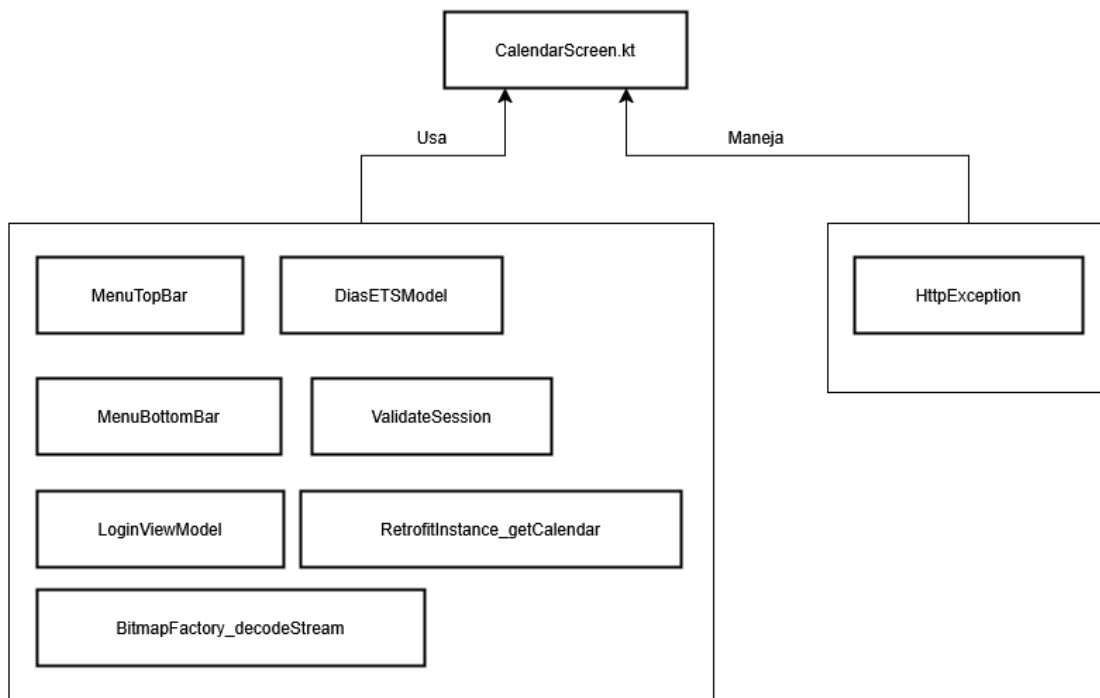


Figura A.30: Diagrama de componentes para CalendarScreen.kt .

A.3.3. Diagrama de Componentes de CamaraScreen.kt

La (figura ??) muestra el diagrama de componentes para CamaraScreen.kt que es un composable que contiene a la IU17 Pantalla de Reconocimiento facial y la IU19 Pantalla de Reconocimiento facial alumno, que muestran la cámara usada para obtener la foto que se usara para el proceso de reconocimiento facial.

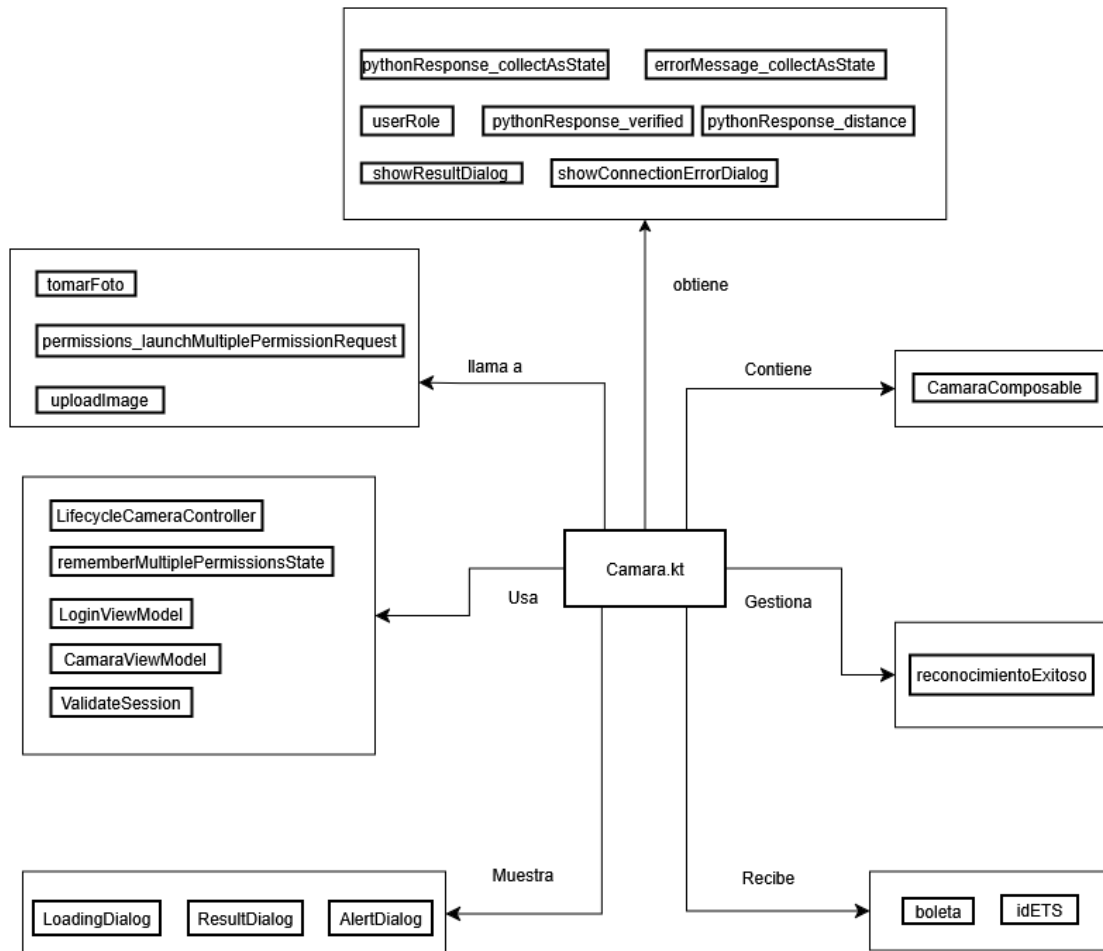


Figura A.31: Diagrama de componentes para CamaraScreen.kt .

A.3.4. Diagrama de Componentes de ChatScreen.kt

La (figura ??) muestra el diagrama de componentes para ChatScreen.kt que es un composable que contiene una pantalla que muestra el chat de mensajes que pueden usar los usuarios.

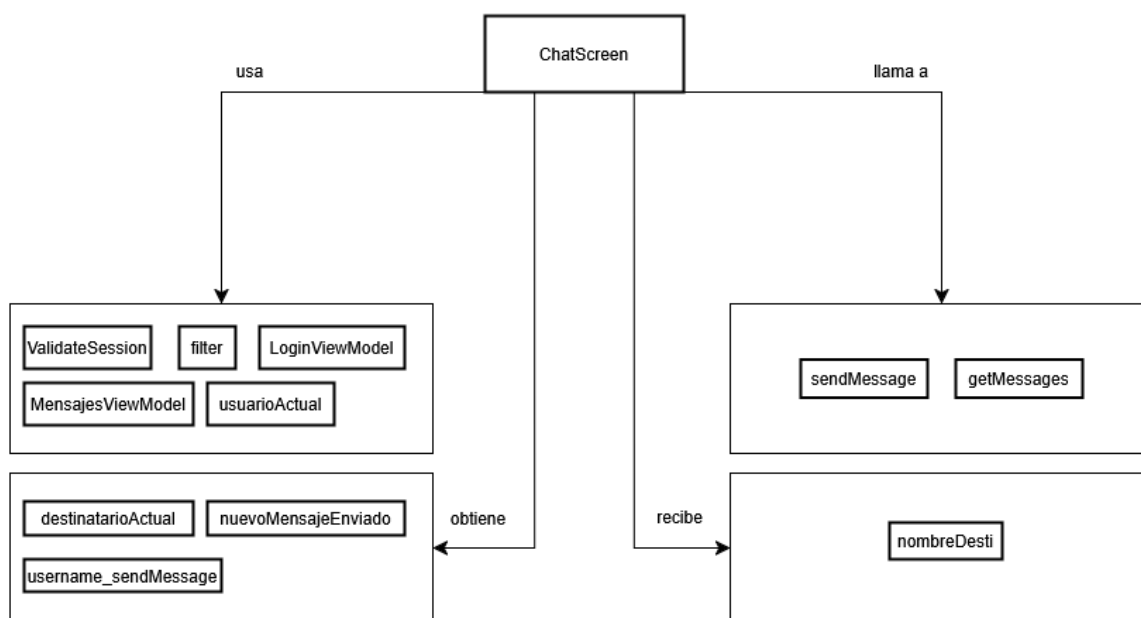


Figura A.32: Diagrama de componentes para ChatScreen.kt .

A.3.5. Diagrama de Componentes de ConsultarScreen.kt

La (figura ??) muestra el diagrama de componentes para ConsultarScreen.kt que es un composible que contiene a la  IU12 Pantalla Buscar alumno, que muestra la lista de los alumnos inscritos a un ETS para el personal de seguridad y le permite filtrarlos por nombre o boleta.

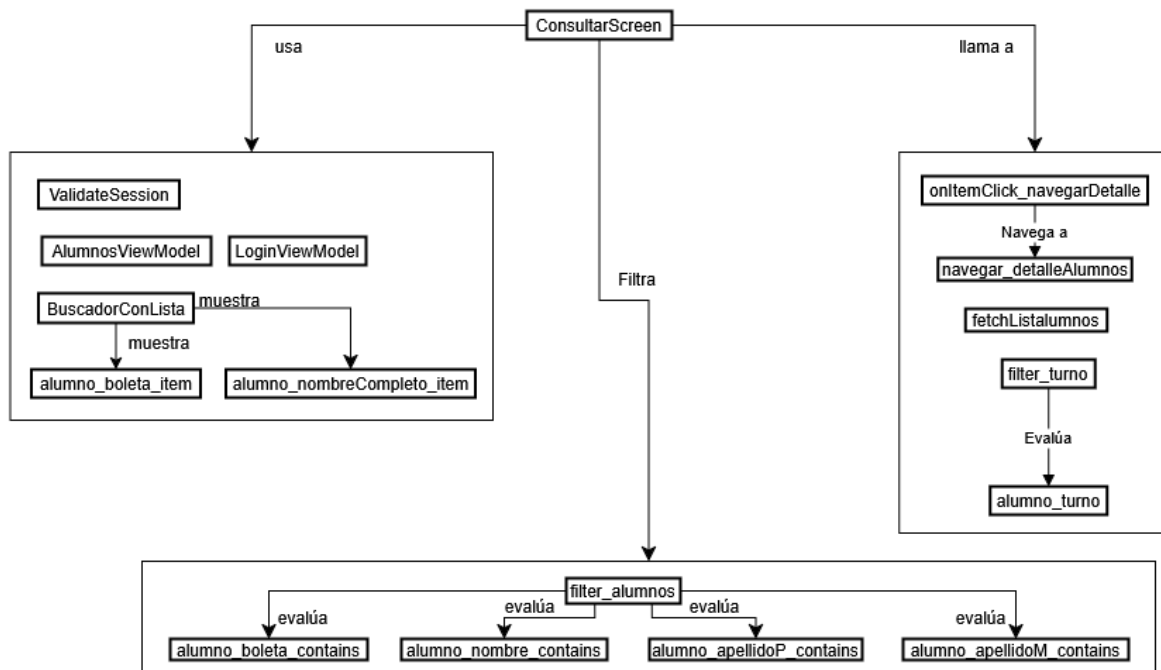


Figura A.33: Diagrama de componentes para ConsultarScreen.kt .

A.3.6. Diagrama de Componentes de CredencialDaeScreen.kt

La (figura ??) muestra el diagrama de componentes para CredencialDaeScreen.kt que es un composable que contiene a la IU11 Pantalla Credencial del alumno, que muestra la comparación de la credencial de la DAE con la credencial creada por el sistema.

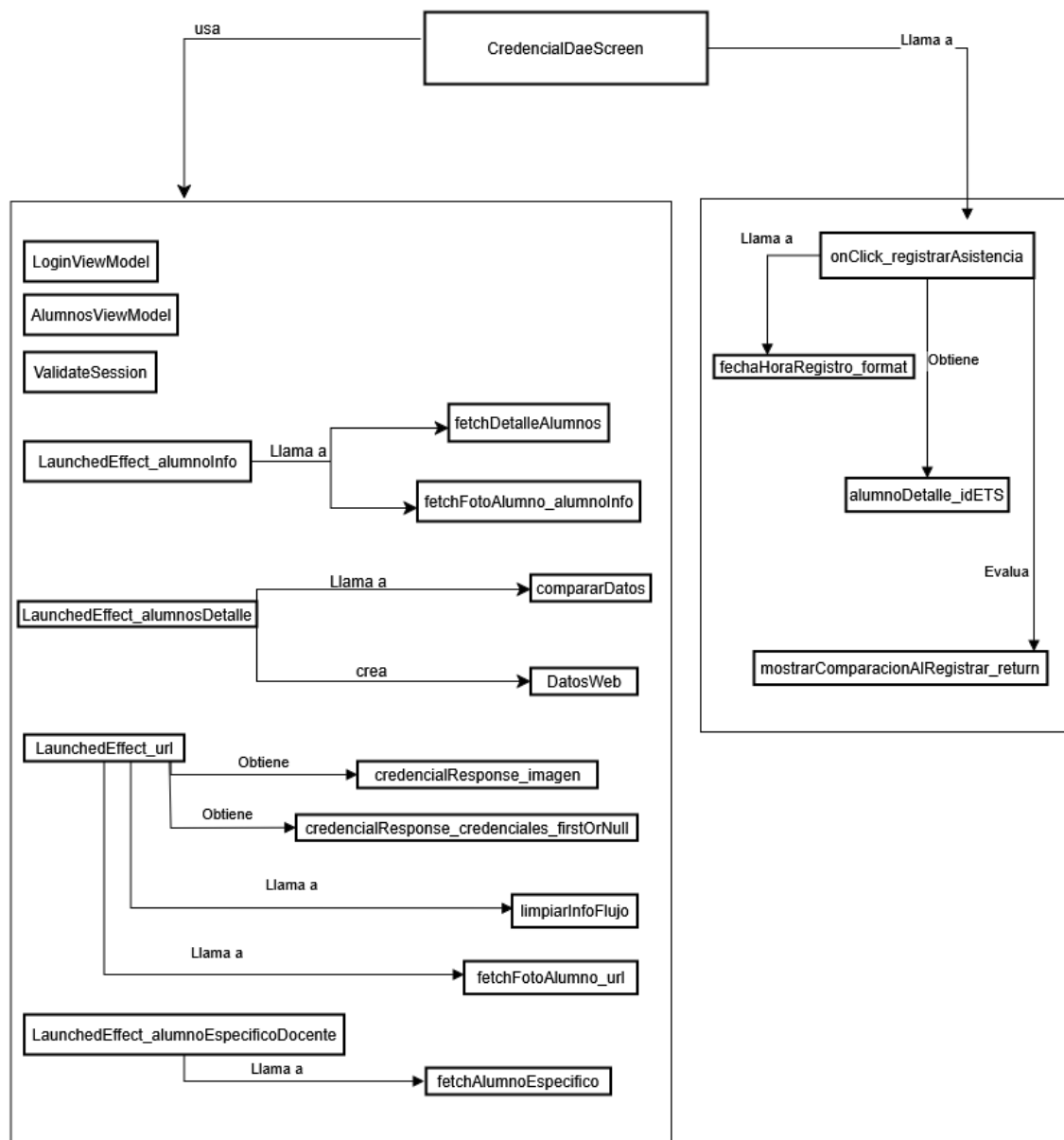


Figura A.34: Diagrama de componentes para CredencialDaeScreen.kt .

A.3.7. Diagrama de Componentes de DetallesAlumnosScreen.kt

La (figura ??) muestra el diagrama de componentes para DetallesAlumnosScreen.kt que es un composable que contiene a la IU11 Pantalla Credencial del alumno, que en este caso muestra la información relevante del alumno.

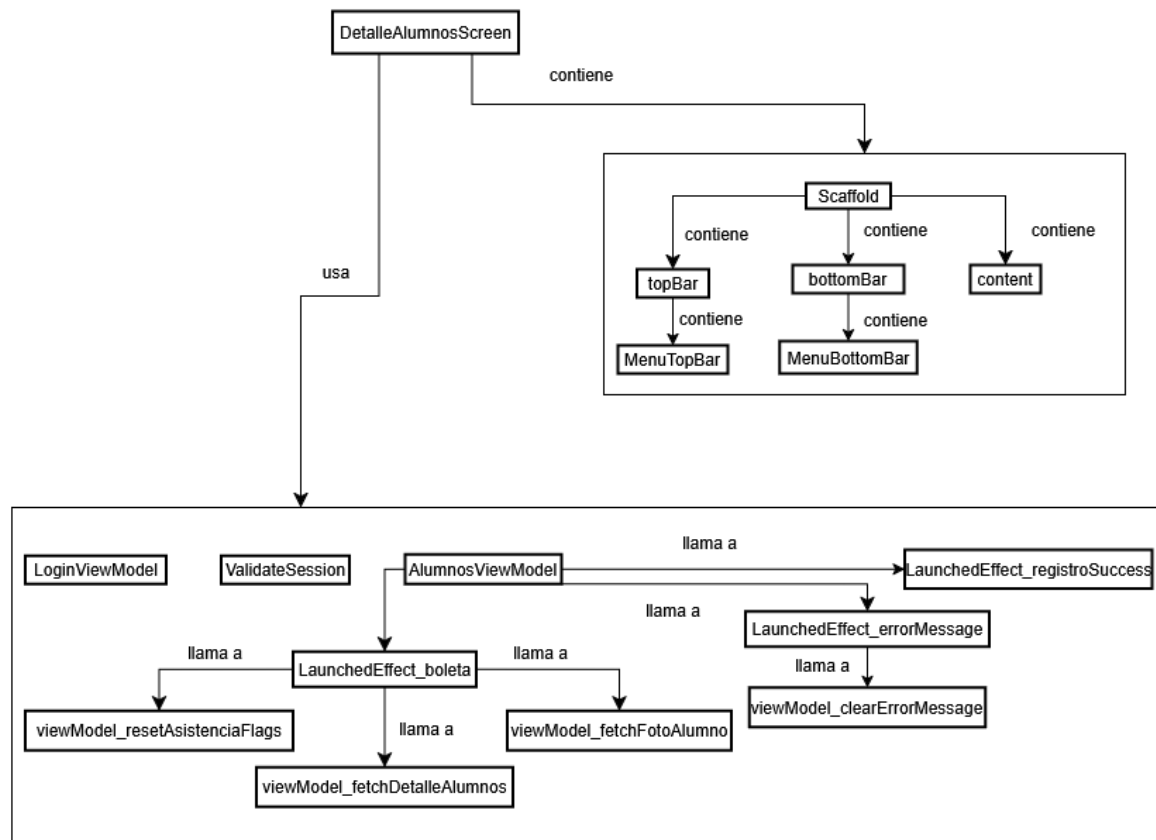


Figura A.35: Diagrama de componentes para DetallesAlumnosScreen.kt .

A.3.8. Diagrama de Componentes de EtsDetailScreen.kt

La (figura ??) muestra el diagrama de componentes para EtsDetailScreen.kt que es un composable que contiene a la IU06 Pantalla Información de ETS y IU16 Pantalla Información de ETS del alumno, que muestra la información del ETS elegido por el usuario.

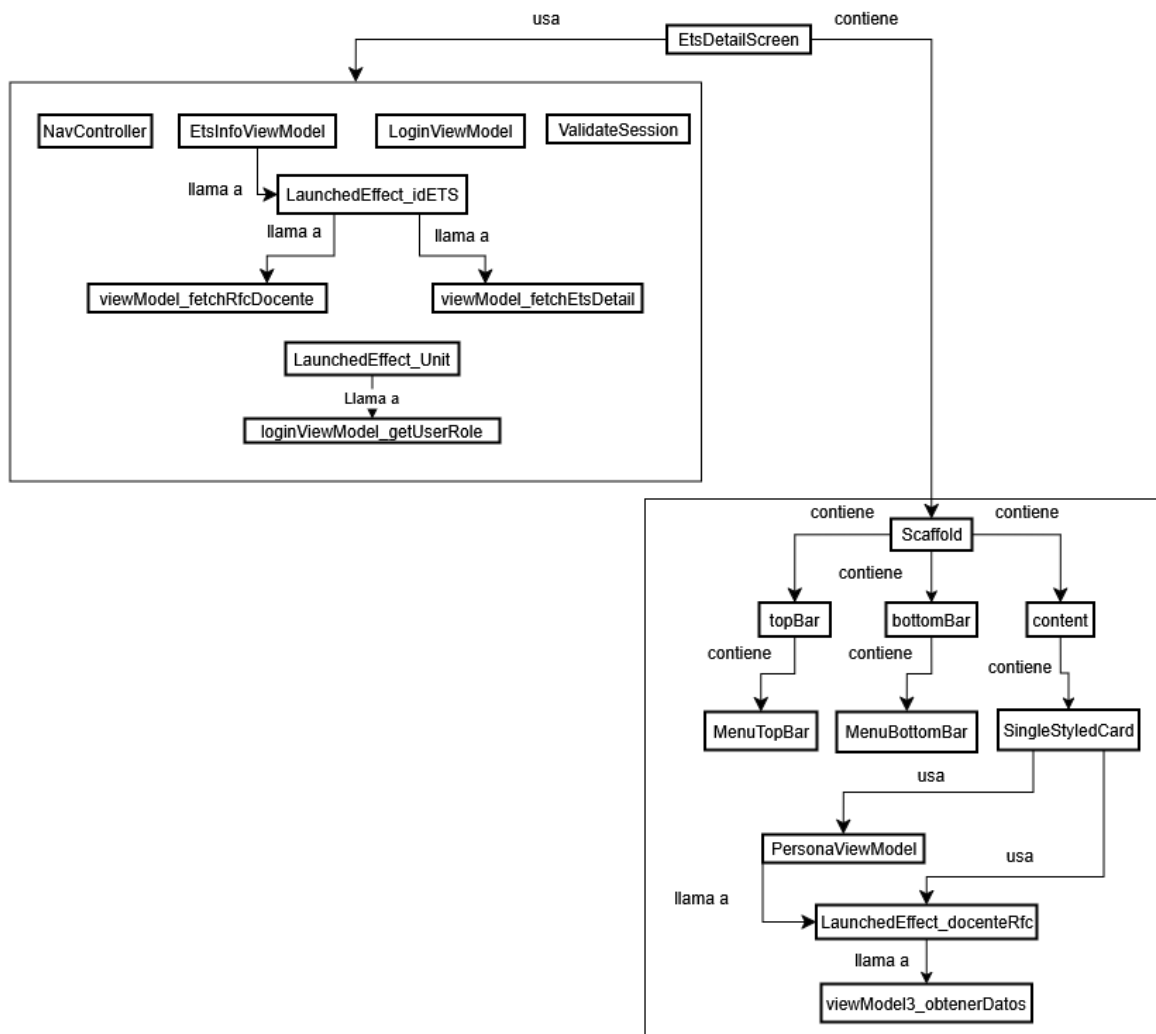


Figura A.36: Diagrama de componentes para EtsDetailScreen.kt .

A.3.9. Diagrama de Componentes de EtsInscriptionProcessScreen.kt

La (figura ??) muestra el diagrama de componentes para EtsInscriptionProcessScreen.kt que es un composable que contiene a la IU18 Pantalla de Detalles del proceso de ETS, que muestra la información necesaria para inscribirse a un ETS.

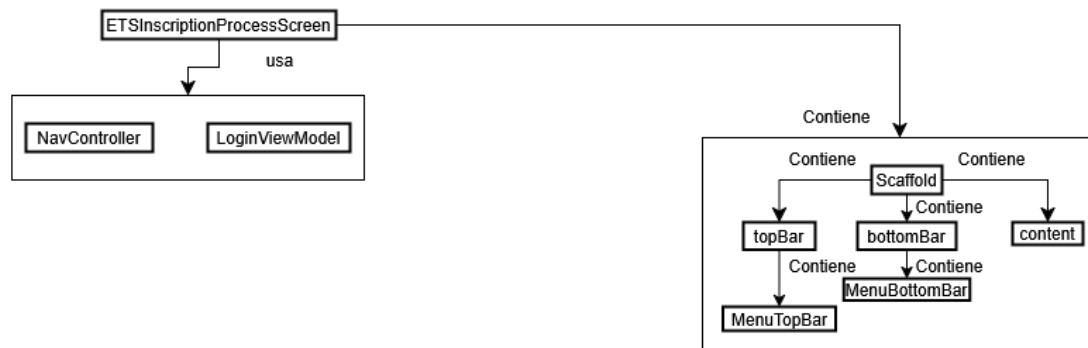


Figura A.37: Diagrama de componentes para EtsInscriptionProcessScreen.kt .

A.3.10. Diagrama de Componentes de EtsListScreen.kt

La (figura ??) muestra el diagrama de componentes para EtsListScreen.kt que es un composable que contiene a la  IU05 Pantalla Consultar ETS, que muestra la lista de ETS del docente.

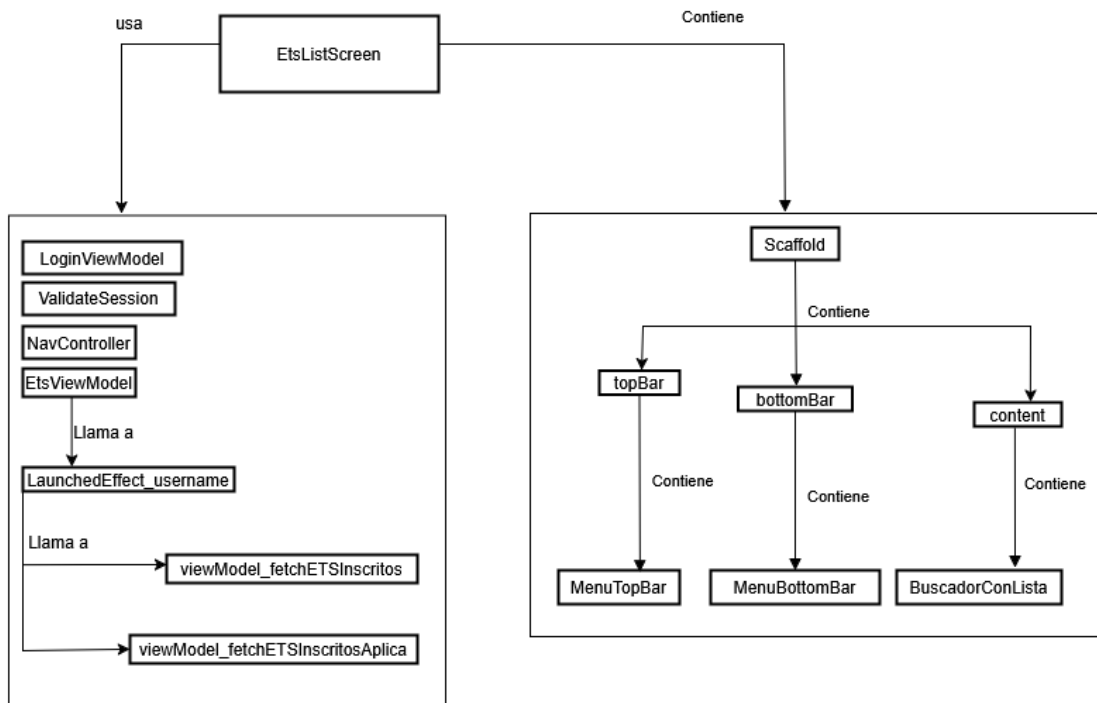


Figura A.38: Diagrama de componentes para EtsListScreen.kt .

A.3.11. Diagrama de Componentes de EtsListScreenAlumno.kt

La (figura ??) muestra el diagrama de componentes para EtsListScreenAlumno.kt que es un composable que contiene a la  IU05 Pantalla Consultar ETS, que muestra la lista de ETS del alumno.

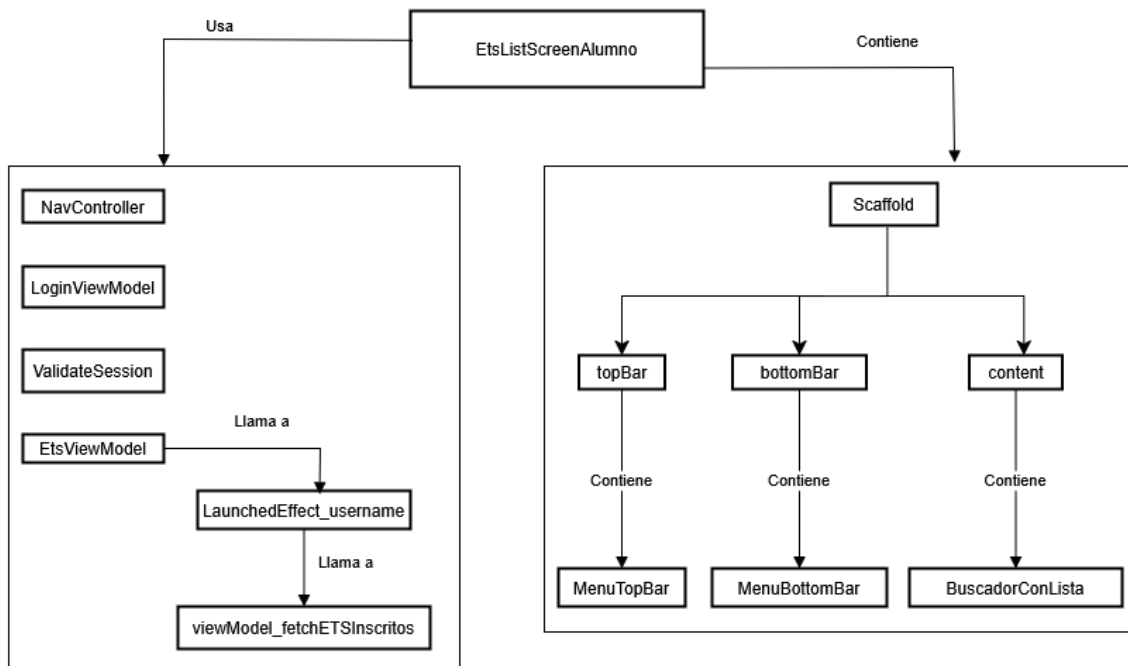


Figura A.39: Diagrama de componentes para EtsListScreenAlumno.kt .

A.3.12. Diagrama de Componentes de InformacionAlumno.kt

La (figura ??) muestra el diagrama de componentes para InformacionAlumno.kt que es un composable que contiene a la IUE07 Creación del reporte, que se utiliza para la creación de reportes y a su vez muestra la información esencial del alumno seleccionado.

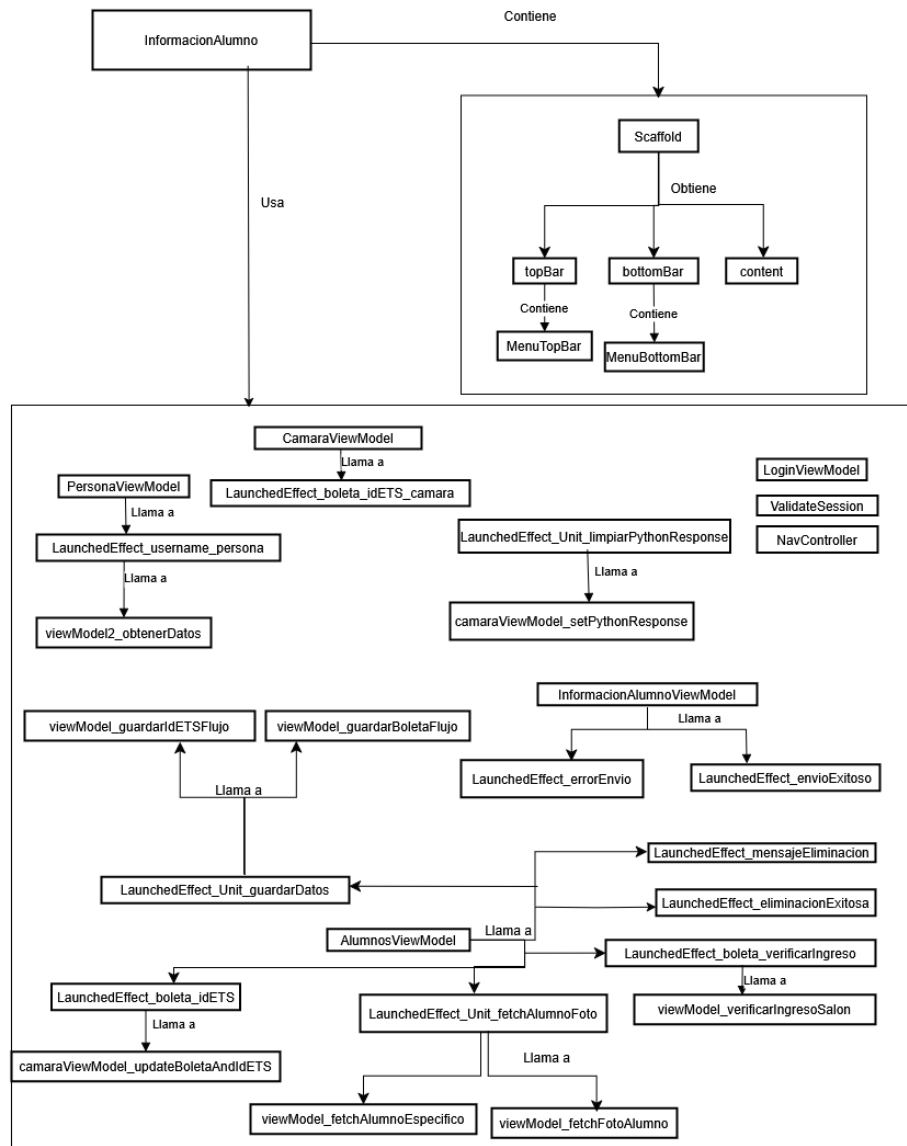


Figura A.40: Diagrama de componentes para InformacionAlumno.kt .

A.3.13. Diagrama de Componentes de ListaAlumnosScreen.kt

La (figura ??) muestra el diagrama de componentes para ListaAlumnosScreen.kt que es un composable que contiene a la  IU08 Pantalla Lista de asistencia de ETS, que es la que le muestra la lista de alumnos inscritos a un ETS al docente, junto con su estado de asistencia con un icono.

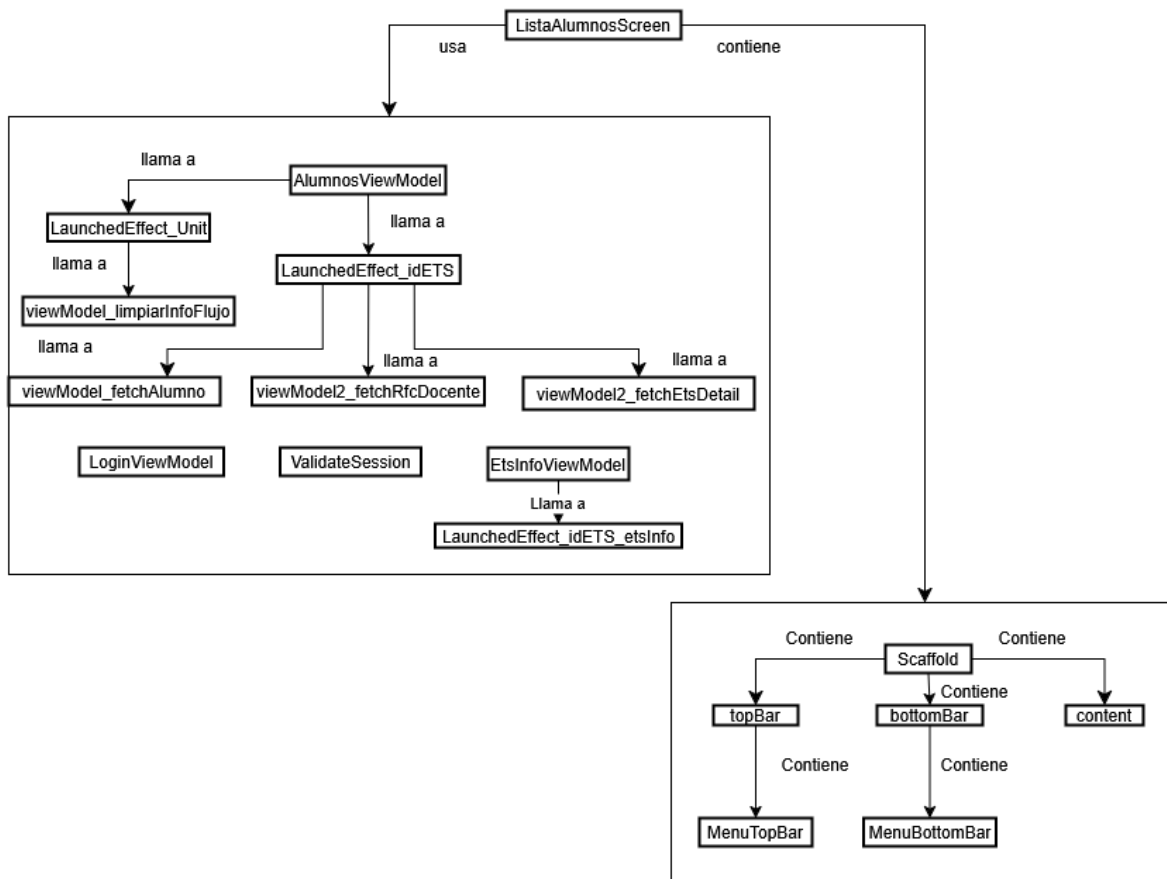


Figura A.41: Diagrama de componentes para ListaAlumnosScreen.kt .

A.3.14. Diagrama de Componentes de ListadoSolicitudesReemplazo.kt

La (figura ??) muestra el diagrama de componentes para ListadoSolicitudesReemplazo.kt que es un composable que contiene una pantalla dedica a la visualización de las solicitudes de reemplazo y sus estados de aceptación o denegación.

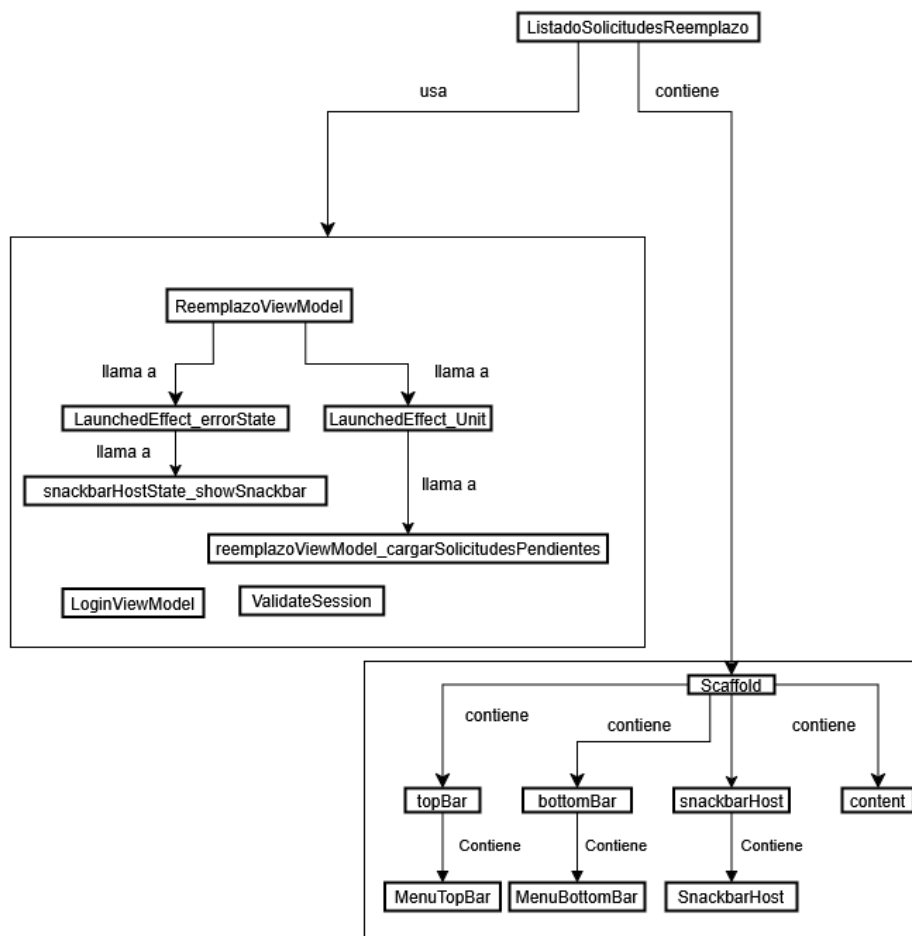


Figura A.42: Diagrama de componentes para ListadoSolicitudesReemplazo.kt .

A.3.15. Diagrama de Componentes de LoginScreen.kt

La (figura ??) muestra el diagrama de componentes para LoginScreen.kt que es un composable que contiene a la  IU01 Pantalla Iniciar sesión del sistema móvil, que es el login de la aplicación móvil.

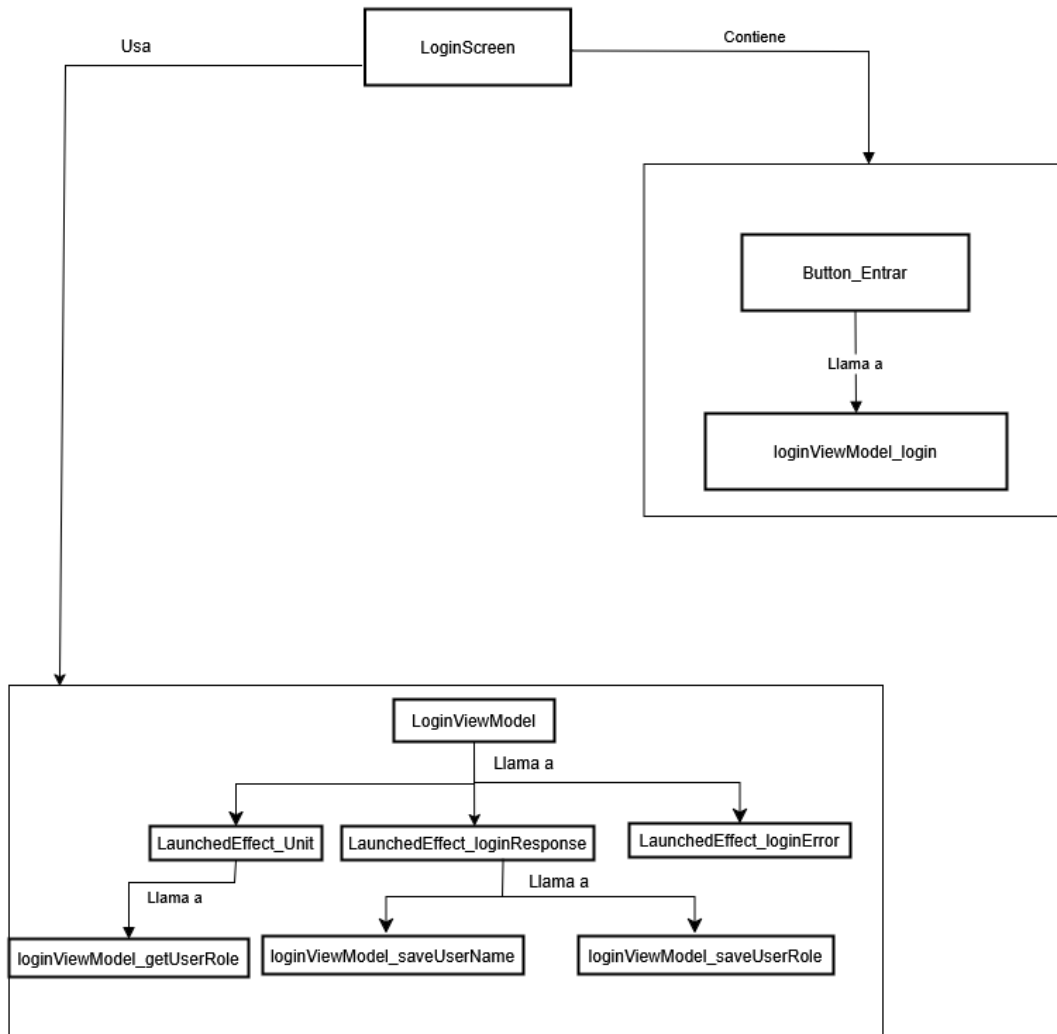


Figura A.43: Diagrama de componentes para LoginScreen.kt .

A.3.16. Diagrama de Componentes de MensajesScreen.kt

La (figura ??) muestra el diagrama de componentes para MensajesScreen.kt que es un composable que es la pantalla donde se ven los mensaje específicos mandados a otro usuario.

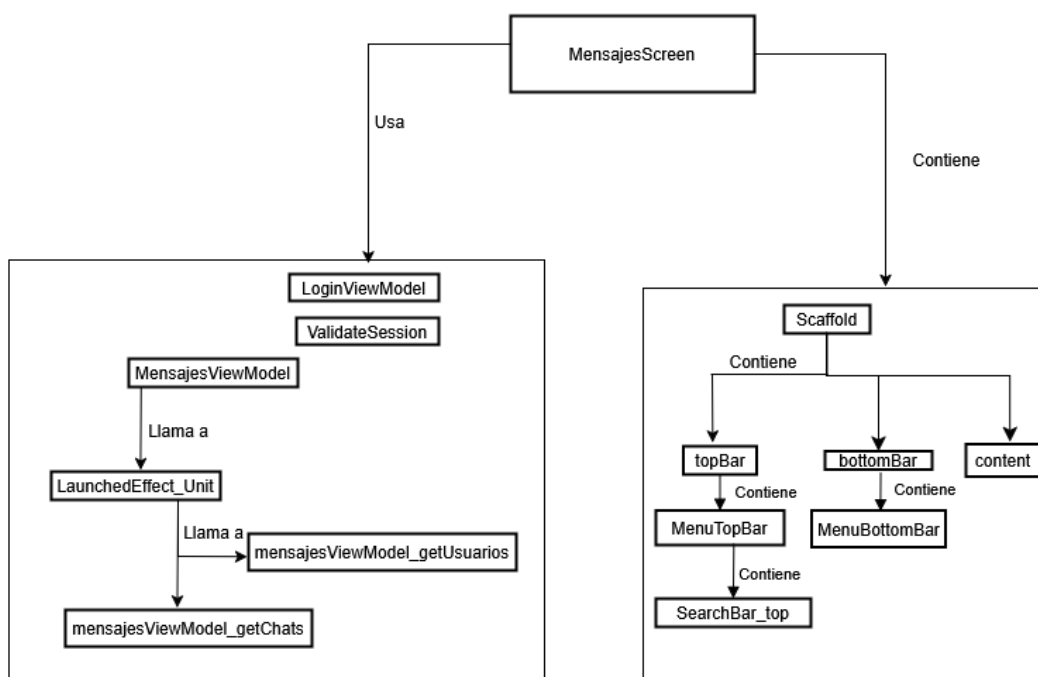


Figura A.44: Diagrama de componentes para MensajesScreen.kt .

A.3.17. Diagrama de Componentes de QRScannerScreen.kt

La (figura ??) muestra el diagrama de componentes para QRScannerScreen.kt que es un composable que contiene a la IU10 Pantalla Código QR, que muestra la cámara para poder escanear el código QR de la credencial del alumno.

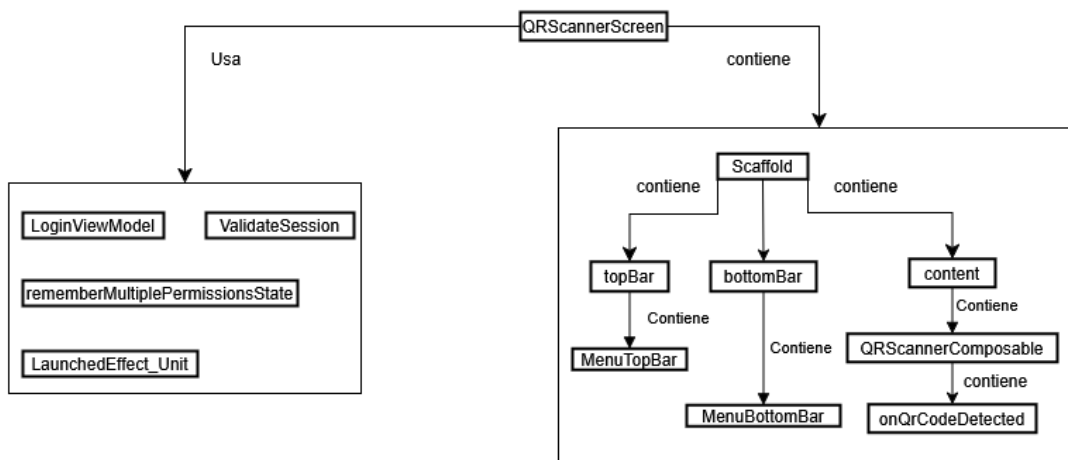


Figura A.45: Diagrama de componentes para QRScannerScreen.kt .

A.3.18. Diagrama de Componentes de ReporteScreen.kt

La (figura ??) muestra el diagrama de componentes para ReporteScreen.kt que es un composable que contiene a la IU20 Pantalla Reporte del Alumno, que es donde el docente puede revisar el reporte de la asistencia del alumno.

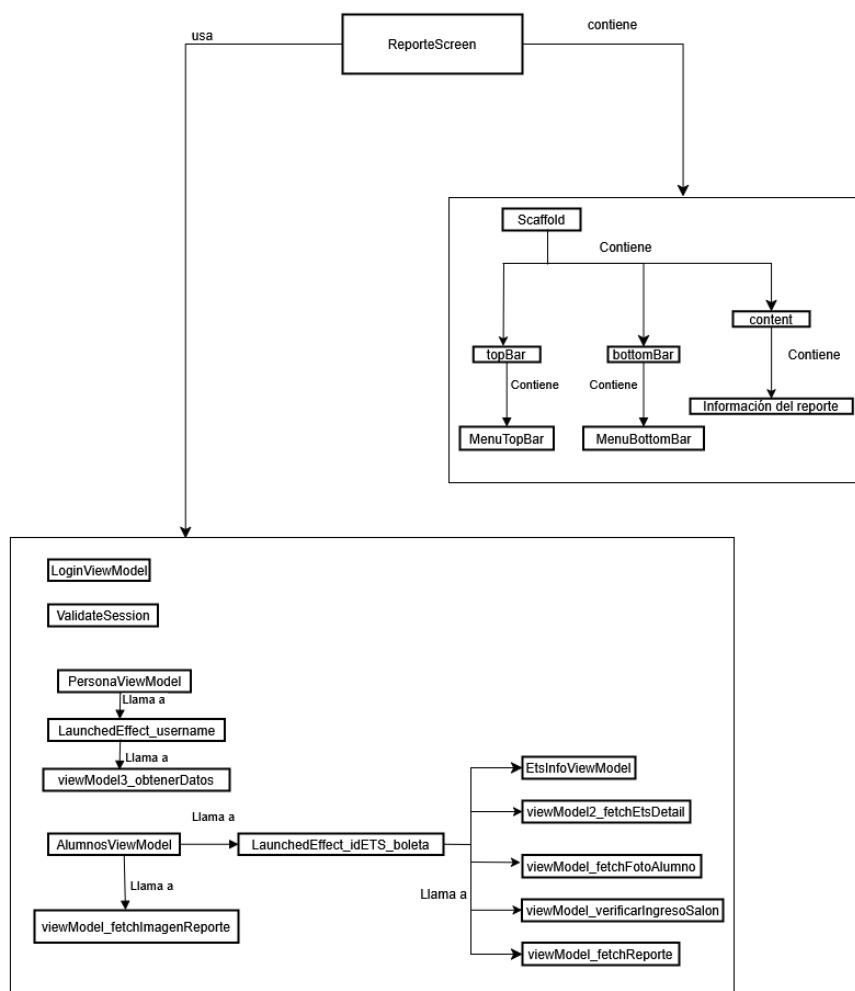


Figura A.46: Diagrama de componentes para ReporteScreen.kt .

A.3.19. Diagrama de Componentes de ScreenAsignarreemplazo.kt

La (figura ??) muestra el diagrama de componentes para ScreenAsignarreemplazo.kt que es un composa-
ble que contiene a la pantalla IU09 Asignar docente de reemplazo, que es donde el jefe de departamento
y/o el presidente de academia pueden responder a las solicitudes de reemplazo.

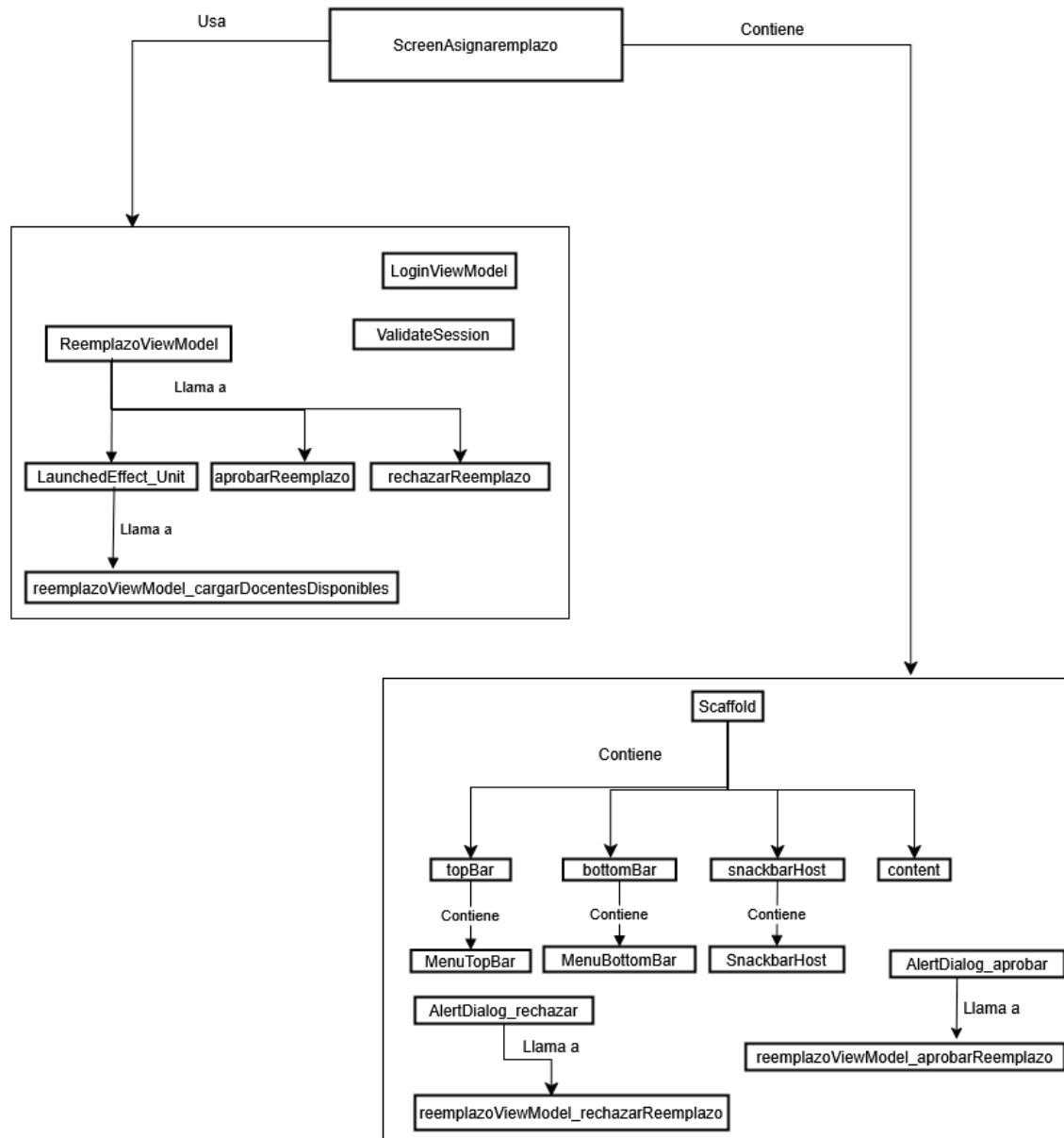


Figura A.47: Diagrama de componentes para ScreenAsignarreemplazo.kt .

A.3.20. Diagrama de Componentes de SolicitarReemplazo.kt

La (figura ??) muestra el diagrama de componentes para SolicitarReemplazo.kt que es un composable que contiene a la IU07 Pantalla de Solicitar reemplazo, que es donde el docente puede pedir un reemplazo como aplacador del ETS.

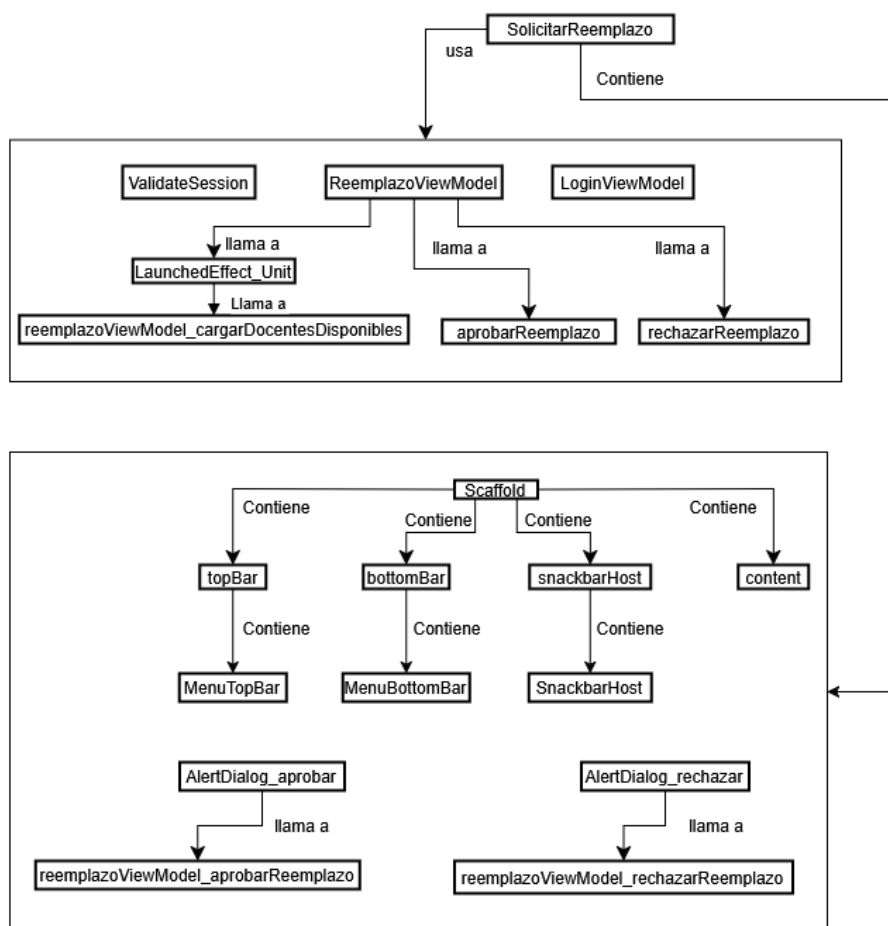


Figura A.48: Diagrama de componentes para SolicitarReemplazo.kt .

A.3.21. Diagrama de Componentes de WelcomeScreen.kt

La (figura ??) muestra el diagrama de componentes para WelcomeScreen.kt que es un composible que contiene a la  IUE02 Pantalla de saludo del personal de seguridad, que es la pantalla de inicio del personal de seguridad (donde pude seleccionar la operación que quiere realizar y ver un saludo con su nombre).

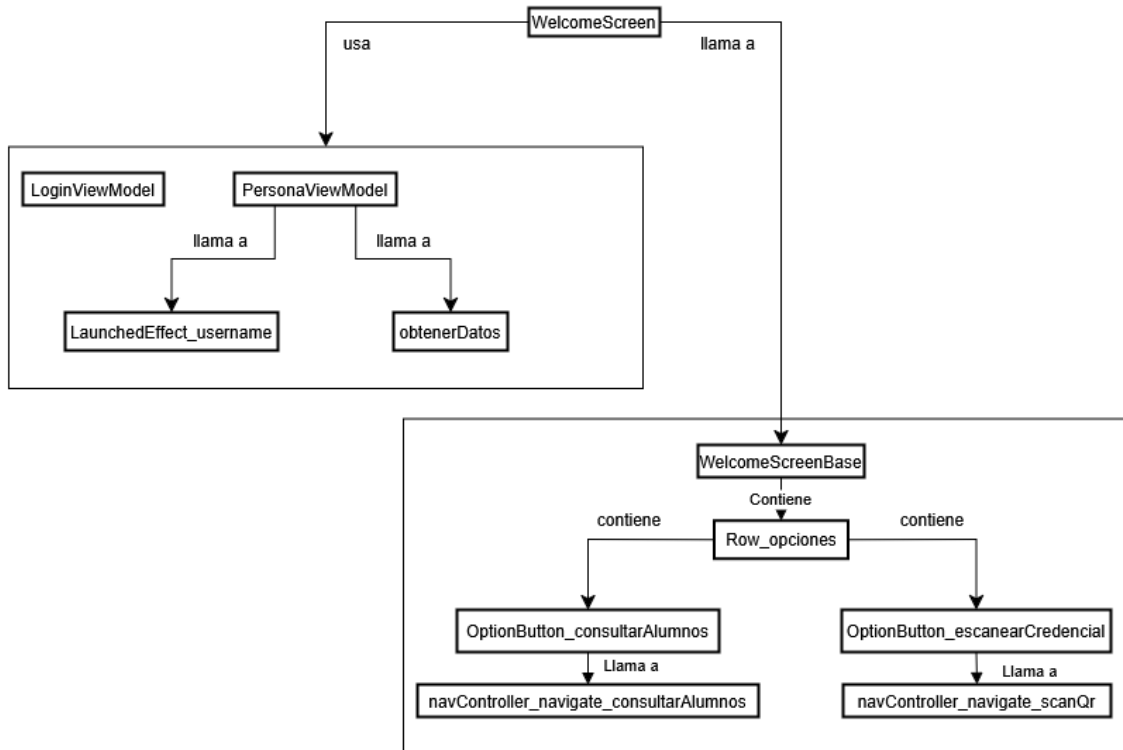


Figura A.49: Diagrama de componentes para WelcomeScreen.kt .

A.3.22. Diagrama de Componentes de WelcomeScreenAcademico.kt

La (figura ??) muestra el diagrama de componentes para WelcomeScreenAcademico.kt que es un composable que contiene a la pantalla IUE06 saludo del presidente de academia y el jefe de departamento, que es la pantalla de inicio del jefe de departamento y/o el presidente de academia (donde puede seleccionar la operación que quiere realizar y ver un saludo con su nombre).

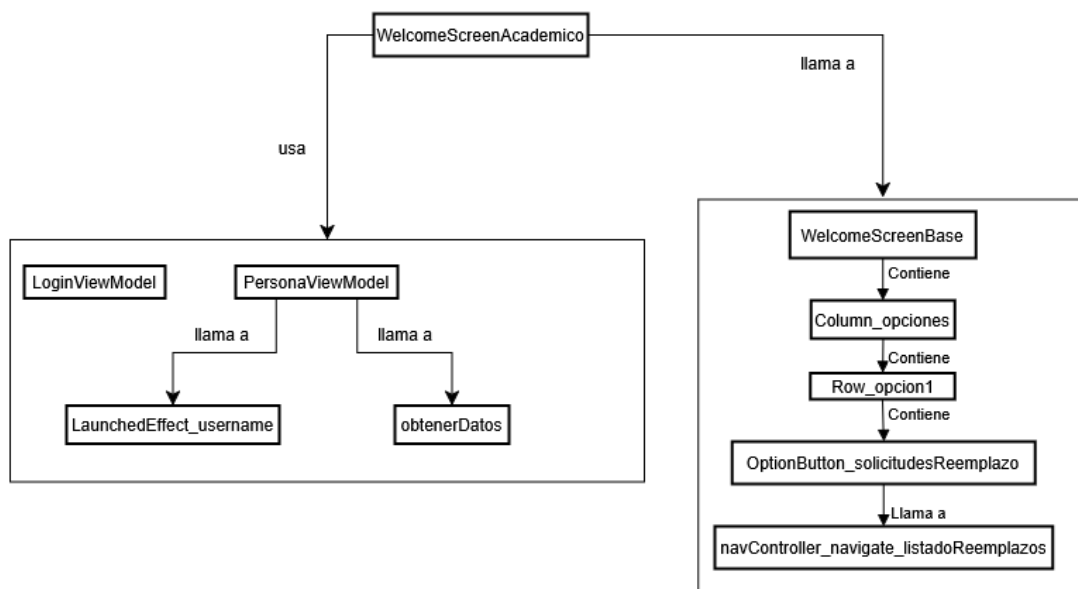


Figura A.50: Diagrama de componentes para WelcomeScreenAcademico.kt .

A.3.23. Diagrama de Componentes de WelcomeScreenAlumno.kt

La (figura ??) muestra el diagrama de componentes para WelcomeScreenAlumno.kt que es un composable que contiene a la pantalla  IUE03 Pantalla saludo del alumno, que es la pantalla de inicio del Alumno (donde puede seleccionar la operación que quiere realizar y ver un saludo con su nombre).

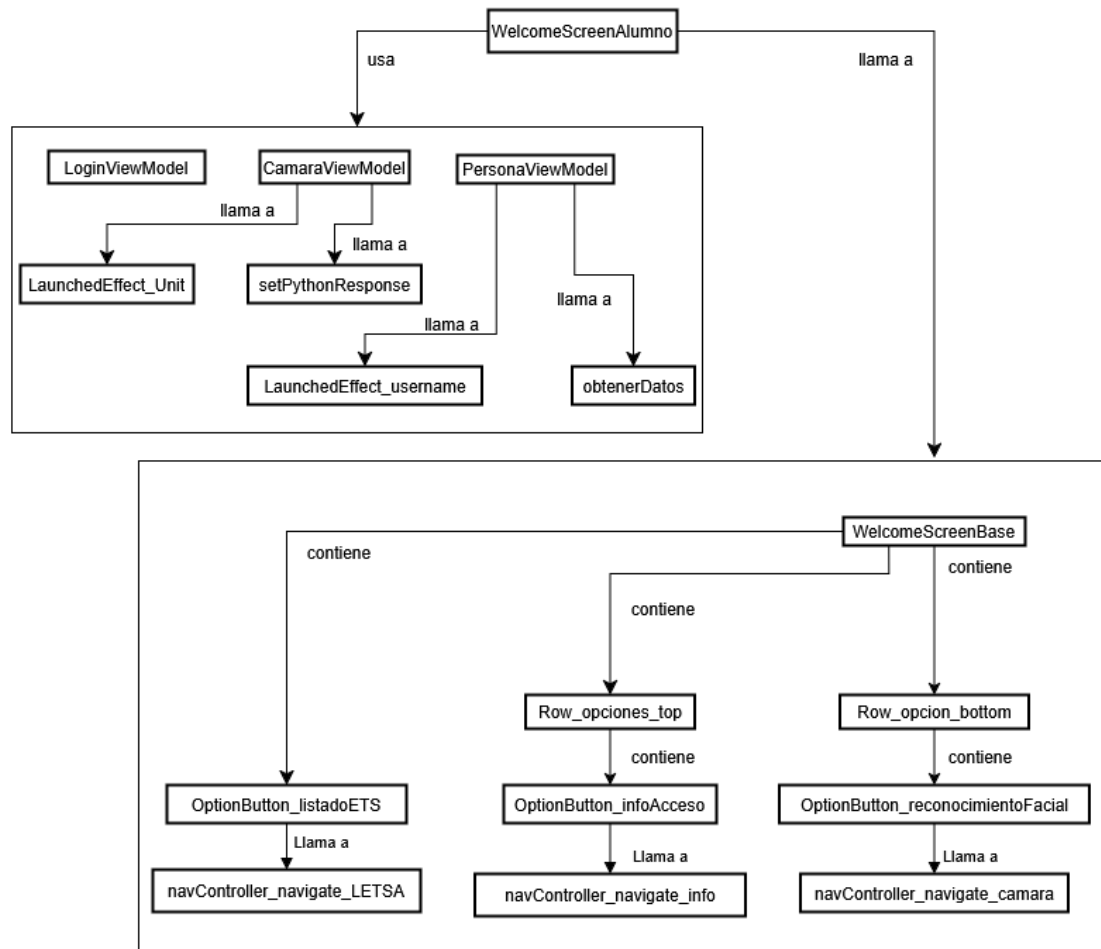


Figura A.51: Diagrama de componentes para WelcomeScreenAlumno.kt .

A.3.24. Diagrama de Componentes de WelcomeScreenDocente.kt

La (figura ??) muestra el diagrama de componentes para WelcomeScreenDocente.kt que es un componente que contiene a la pantalla IUE01 Pantalla saludo de docente, que es la pantalla de inicio del Docente (donde puede seleccionar la operación que quiere realizar y ver un saludo con su nombre).

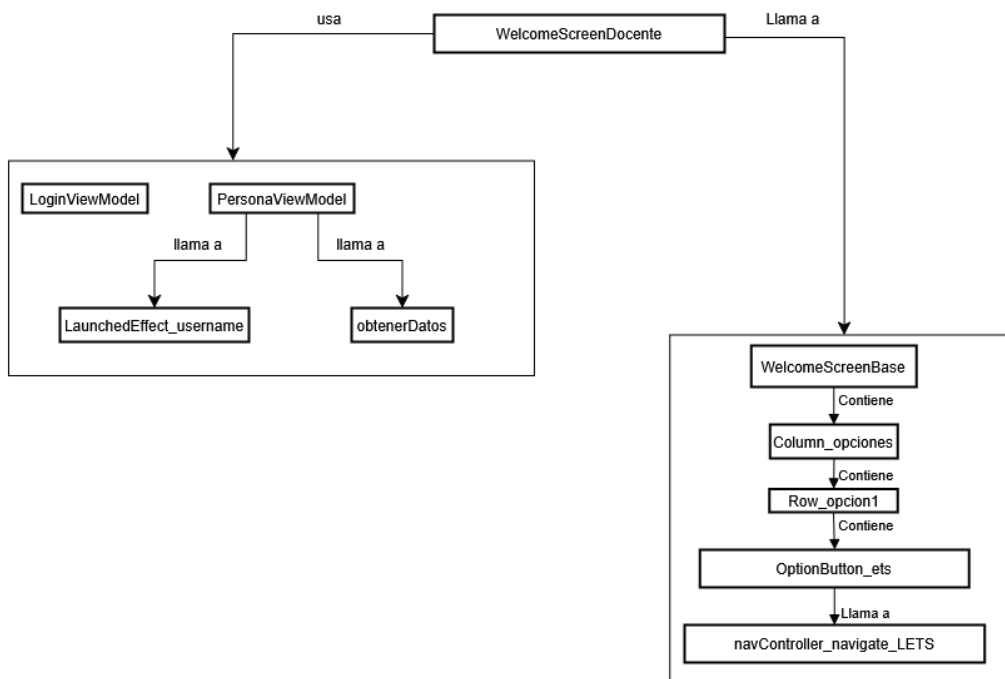


Figura A.52: Diagrama de componentes para WelcomeScreenDocente.kt .

A.3.25. Diagramas de clases de package com.example.prueba3.Views

La arquitectura de nuestra aplicación móvil se organiza en paquetes bien definidos para una clara separación de responsabilidades. Dentro de este esquema, el paquete 'com.example.prueba3.Views', que contiene nuestros ViewModels, se erige como un componente central y fundamental.

Los ViewModels son elementos clave en la construcción de interfaces de usuario robustas y reactivas. Su función primordial es actuar como intermediarios estratégicos entre las 'Pantallas' (implementadas como Composables en el paquete 'Pantallas'), y la lógica de negocio o las fuentes de datos. Un ViewModel se encarga de exponer los datos necesarios para su 'Pantalla' asociada de una manera que sea óptima para la interfaz de usuario, garantizando que el estado de la UI persista a través de cambios de configuración del dispositivo, como rotaciones o reconstrucciones de la actividad.

A continuación se presentara las clases que conforman a package.example.prueba3.Views:

A.3.26. Diagrama de clases de package.example.prueba3.Views parte 1

La (figura ??) muestra la primera parte del diagrama de clases de package.example.prueba3.Views.



Figura A.53: Diagrama de clases para package.example.prueba3.Views parte 1.

A.3.27. Diagrama de clases de package.example.prueba3.Views parte 2

La (figura ??) muestra la segunda parte del diagrama de clases de package.example.prueba3.Views.

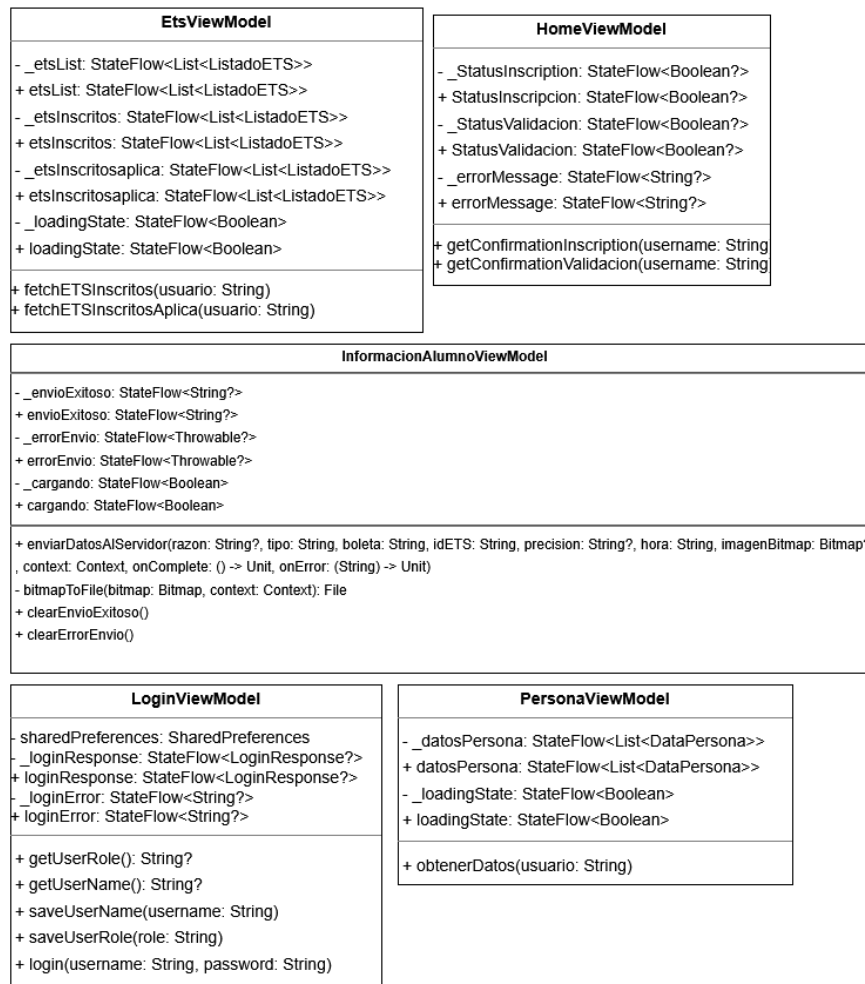


Figura A.54: Diagrama de clases para package.example.prueba3.Views parte 2.

A.3.28. Diagrama de clases de package.example.prueba3.Views parte 3

La (figura ??) muestra la tercer parte del diagrama de clases de package.example.prueba3.Views.

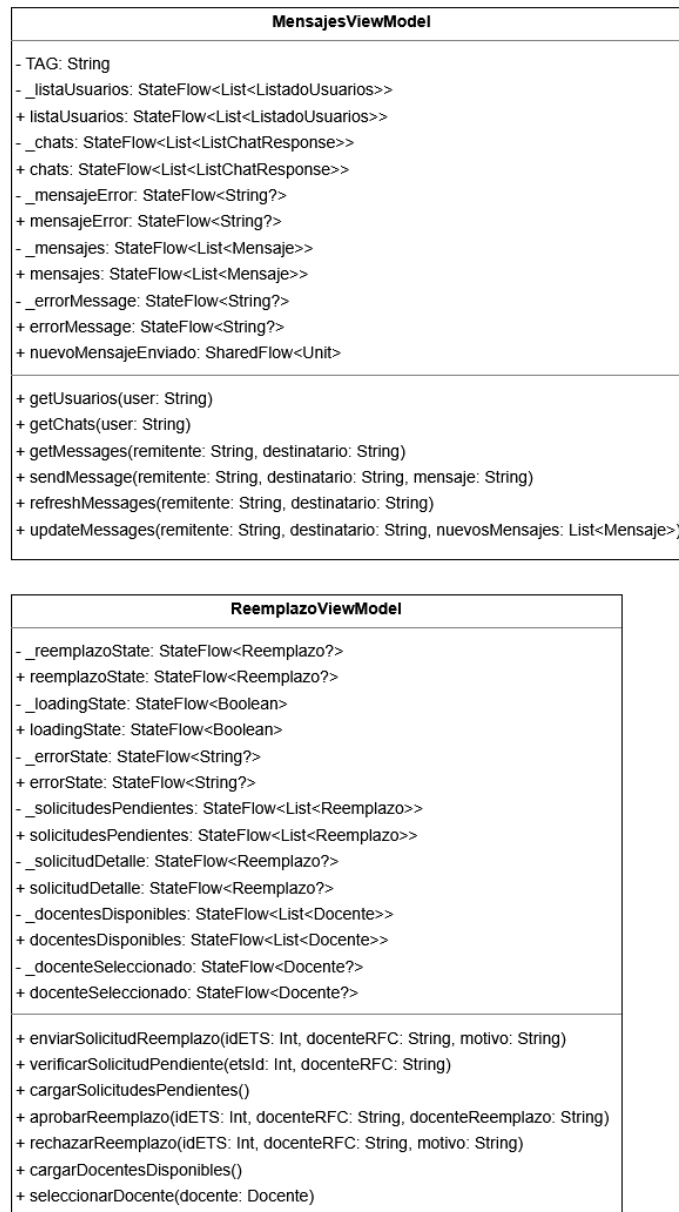


Figura A.55: Diagrama de clases para package.example.prueba3.Views parte 3.

A.4. Clases e Interfaces de package Retrofit

En esta sección, se presenta una visualización de las clases e interfaces que componen el paquete 'Retrofit', el cual es esencial para la comunicación de la aplicación móvil con los servidores backend. Tal como se muestra en la (figura ??) (en la sección de Estructura Interna Detallada de la Aplicación Móvil), este paquete encapsula toda la lógica de interacción con las APIs remotas.

El enfoque principal de esta sección es mostrar las clases e interfaces que definen cómo la aplicación se comunica con el Servidor Java Spring-Boot y el Servidor Red Neuronal. Estas imágenes proporcionan una representación visual directa del código, facilitando la comprensión de la estructura y las relaciones entre los diferentes componentes del paquete 'Retrofit'.

A continuación, se mostrarán las imágenes de las clases e interfaces que conforman el paquete 'Retrofit'. Cada imagen representa una parte del código, mostrando las definiciones de las interfaces, los métodos que mapean a las operaciones HTTP (GET, POST, DELETE, etc.) y las clases de datos utilizadas para las solicitudes y respuestas. Estas imágenes permiten una inspección visual del código, complementando la descripción general de la arquitectura de la aplicación móvil.

A.4.1. Diagrama de clases de package Retrofit parte 1

La (figura ??) muestra la primer parte del diagrama de clases de package Retrofit.

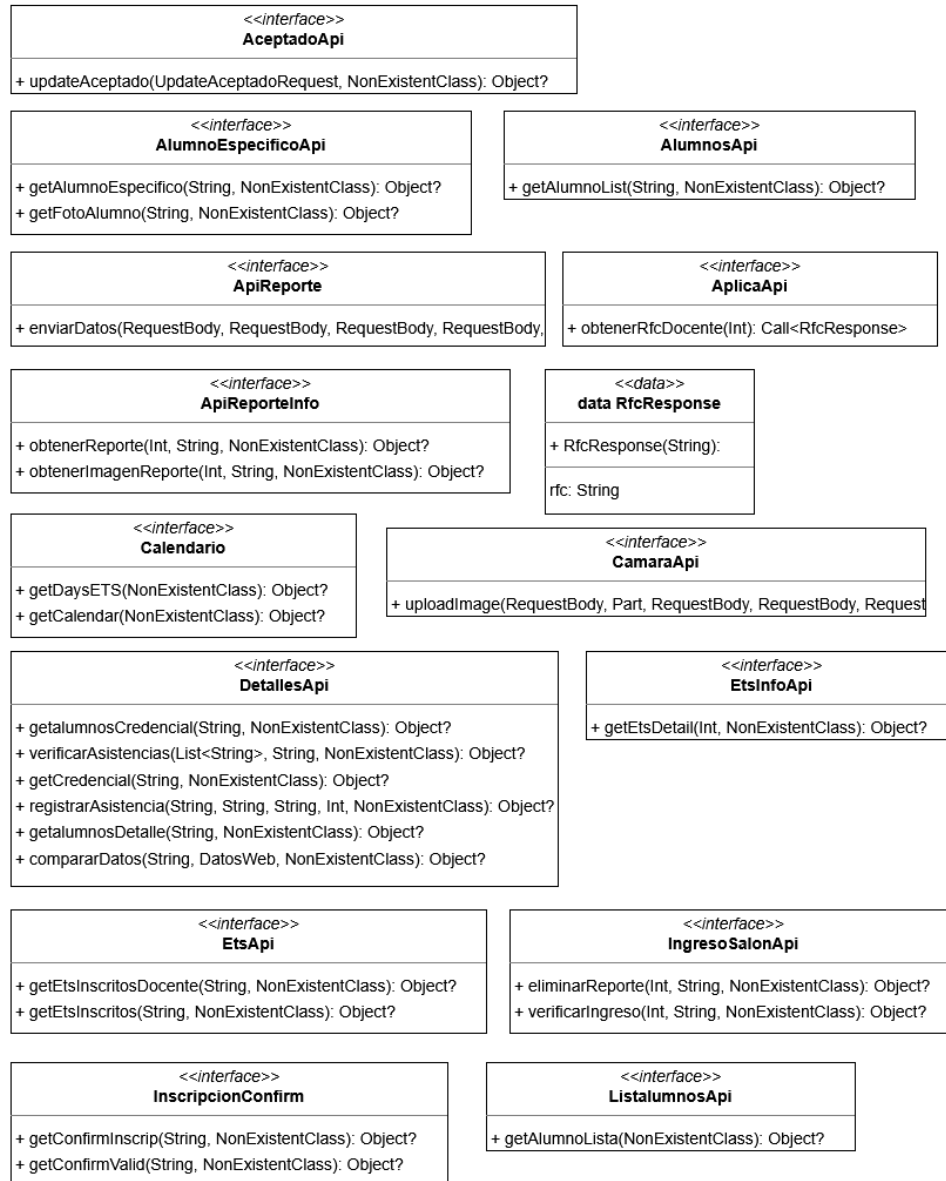


Figura A.56: Diagrama de clases para package Retrofit parte 1.

A.4.2. Diagrama de clases de package Retrofit parte 2

La (figura ??) muestra la segunda parte del diagrama de clases de package Retrofit.

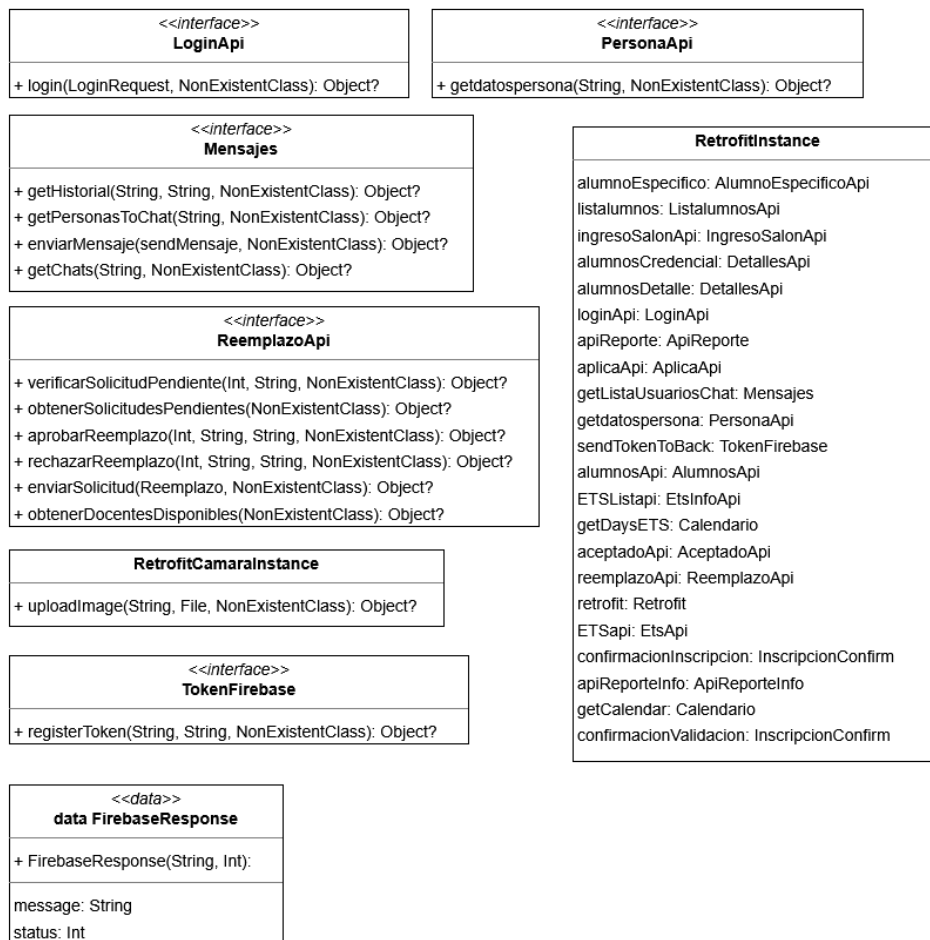


Figura A.57: Diagrama de clases para package Retrofit parte 2.

A.5. Clases de `com.example.prueba3.Clases`

Esta sección se dedica a la presentación de las Clases que constituyen el modelo de datos de la aplicación, ubicadas principalmente en el paquete '`com.example.prueba3.Clases`'. Como se indicó en la (figura ??) (en la sección de Estructura Interna Detallada de la Aplicación Móvil), estas clases son fundamentales para la manipulación y el transporte de información dentro de la aplicación.

La mayoría de estas clases son Data Classes, un tipo de clase concisa en Kotlin diseñada específicamente para contener datos. Su función principal es servir como contenedores de información, estructurando los datos que se obtienen de los servicios remotos (a través de '`Retrofit`'), los que se manipulan dentro de la lógica de negocio, y los que finalmente se presentan en la interfaz de usuario a través de los '`Views`' (`ViewModels`). Estas clases aseguran la consistencia y la tipificación estricta de los datos a lo largo de todo el flujo de la aplicación.

A continuación, se mostrarán las imágenes de las Data Classes que conforman el paquete llamado '`com.example.prueba3.Clases`'. Cada imagen representará la estructura de una clase de datos, incluyendo sus propiedades (atributos) y tipos de datos, proporcionando una visión clara de cómo se modela la información esencial que fluye entre los componentes de la aplicación.

A.5.1. Diagrama de clases de com.example.prueba3.Clases parte 1

La (figura ??) muestra la primera parte del diagrama de clases de com.example.prueba3.Clases.

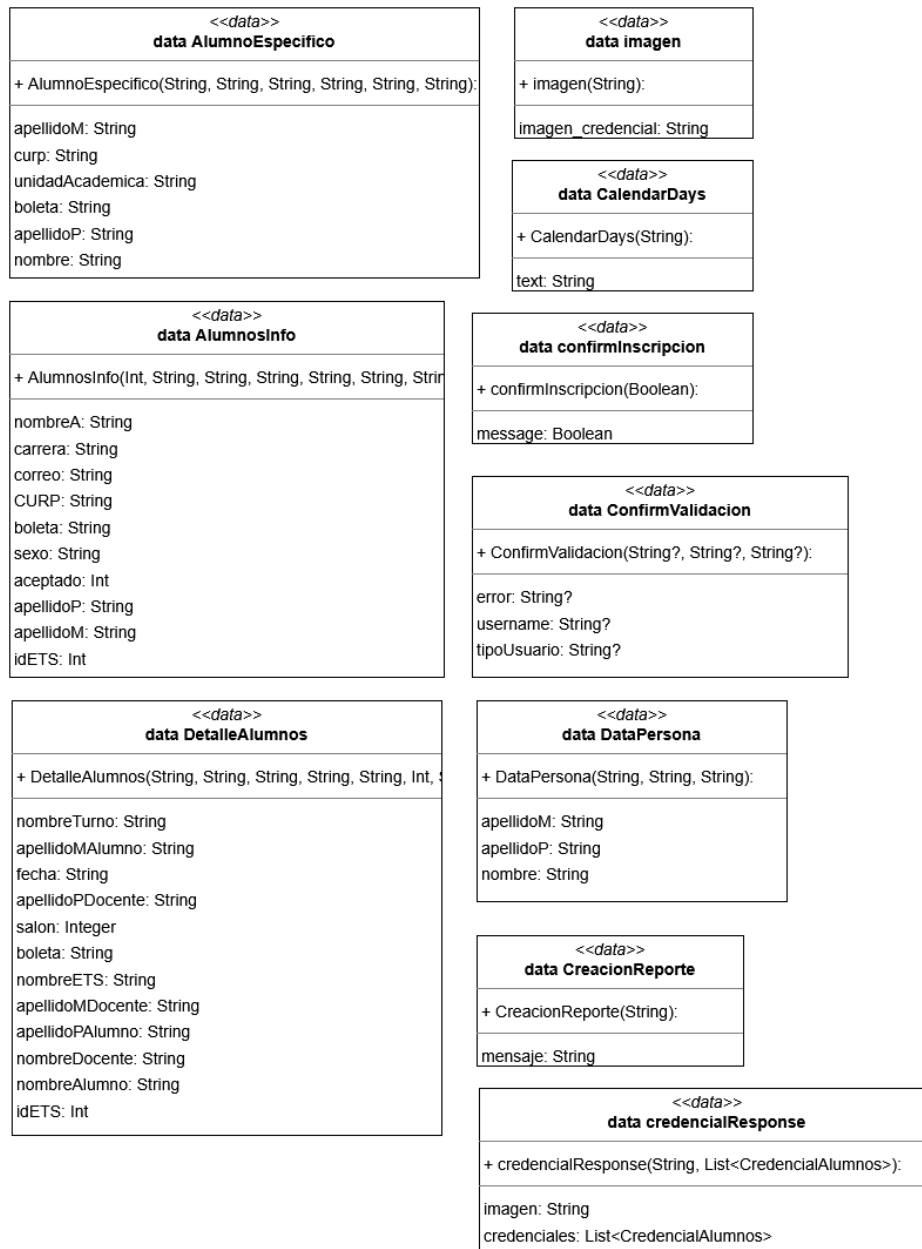


Figura A.58: Diagrama de clases para com.example.prueba3.Clases parte 1.

A.5.2. Diagrama de clases de com.example.prueba3.Clases parte 2

La (figura ??) muestra la segunda parte del diagrama de clases de com.example.prueba3.Clases.

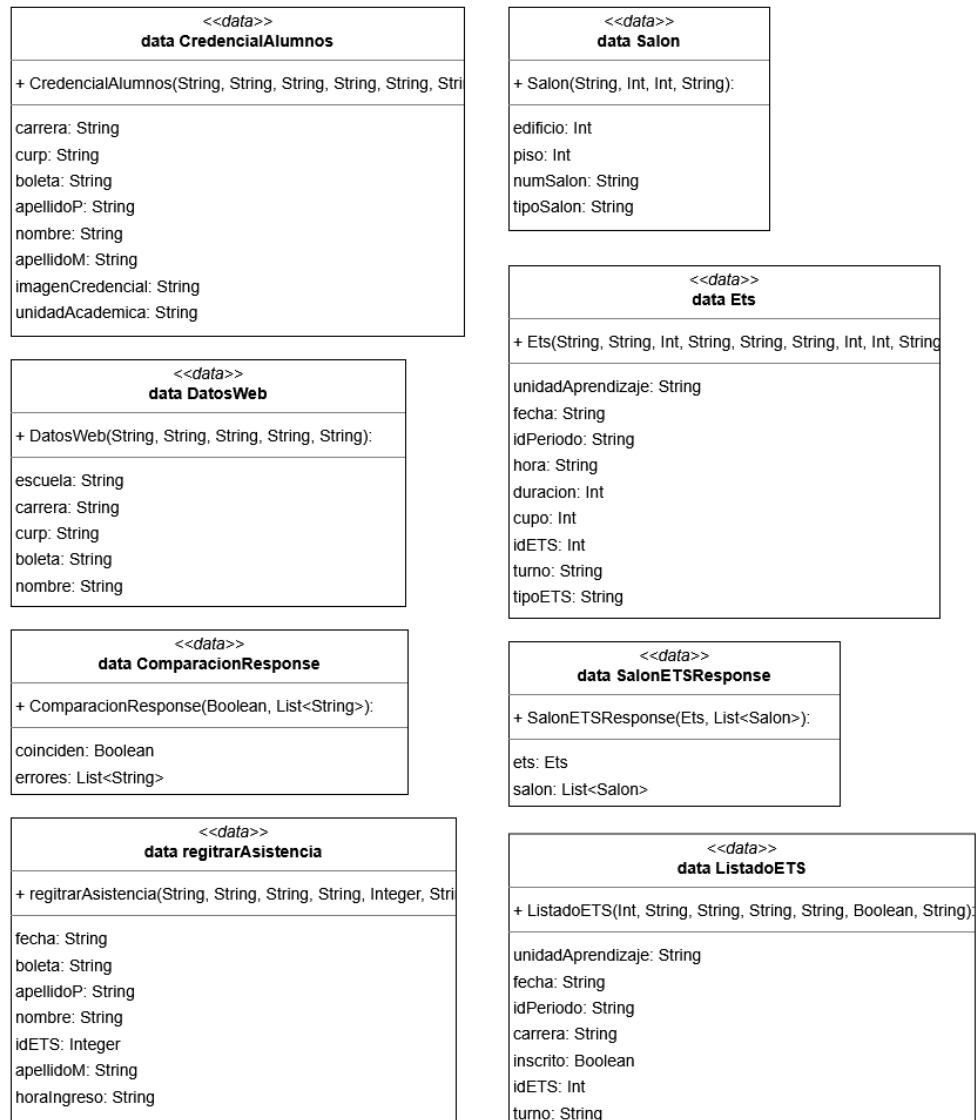


Figura A.59: Diagrama de clases para com.example.prueba3.Clases parte 2.

A.5.4. Diagrama de clases de com.example.prueba3.Clases parte 4

La (figura ??) muestra la cuarta parte del diagrama de clases de com.example.prueba3.Clases.

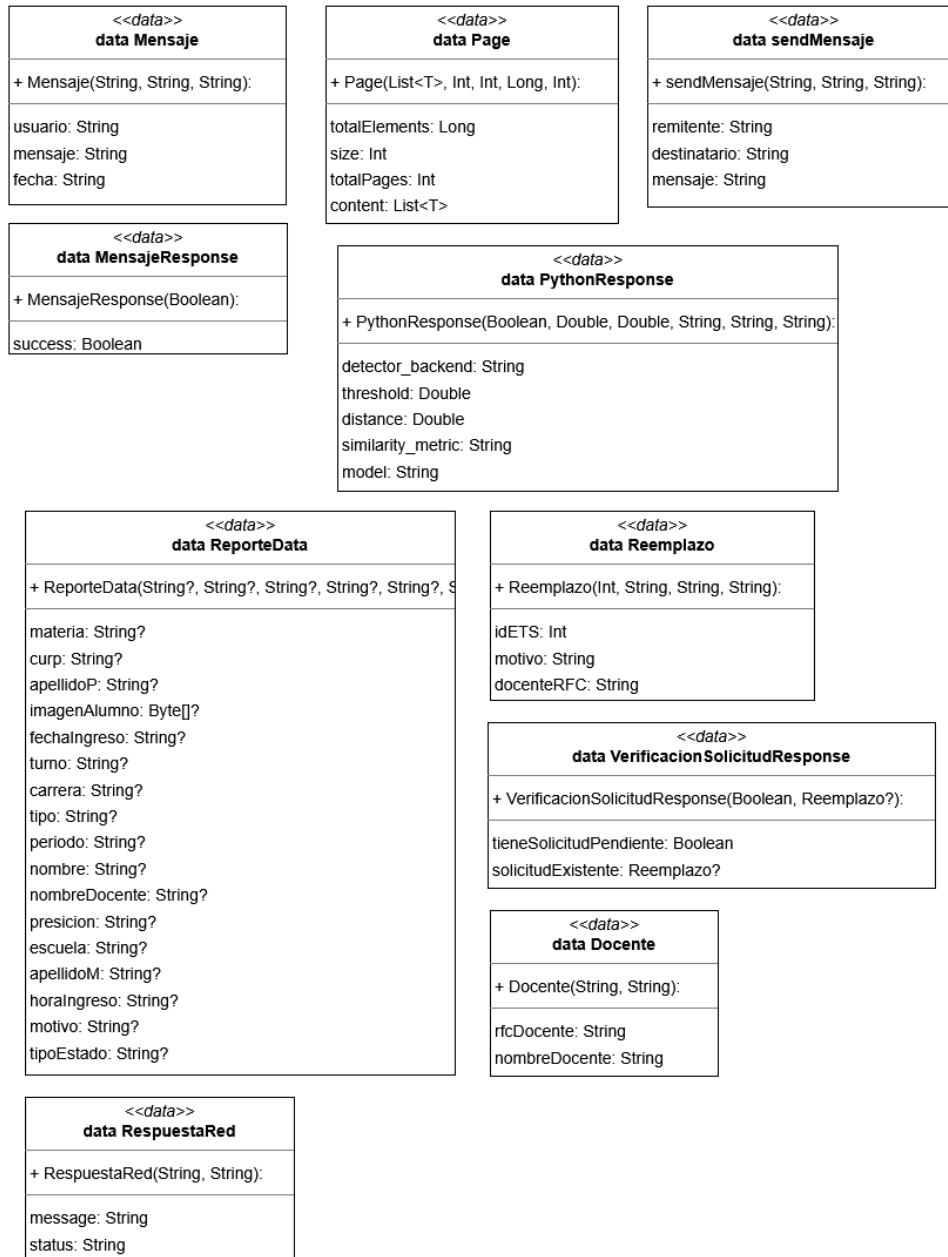


Figura A.61: Diagrama de clases para com.example.prueba3.Clases parte 4.

A.5.5. Diagrama de clases de com.example.prueba3.Clases parte 5

La (figura ??) muestra la quinta parte del diagrama de clases de com.example.prueba3.Clases.



Figura A.62: Diagrama de clases para com.example.prueba3.Clases parte 5.

A.6. Estructura Interna del Servidor Fast API

En la (figura ??) se puede visualizar de manera general la forma en la que se encuentra estructurada la API de Fast API que proporciona los servicios de reconocimiento facial.

Esta API fue desarrollada siguiendo una arquitectura en capas, incluyendo elementos esenciales como los controladores, en este caso especificados como rutas de acceso, servicios que contienen la lógica del negocio, repositorios que sirven como abstracción para acceder a los datos de la base de datos vectorial, schemas que permiten validar el formato de las respuestas recibidas por el servicio y otros elementos esenciales como paquetes con funciones de utilidad y archivos de configuración.

El flujo dentro de esta API inicia con el archivo de configuración que establece algunas constantes que se utilizarán a lo largo de los distintos métodos, aquí encontramos variables de configuración general como el Host y el puerto del servidor, otros para inicializar los clientes de dependencias externas como Pinecone y finalmente variables para configurar el sistema de reconocimiento facial.

Una vez que se inicializaron estas constantes se procede a la capa de los controladores o rutas en este caso, la cual permite manejar las solicitudes HTTP de clientes como la aplicación móvil o el backend de la simulación del SAES-DAE. Cuando la solicitud se procesa y verifica que tenga el formato apropiado, se manda a llamar al servicio correspondiente.

La siguiente capa de la aplicación es la capa de servicios que contiene la parte fuerte de la lógica de negocio, por ejemplo, el servicio para registrar un alumno usa funciones de utilidad para la generación de embeddings faciales a través de la biblioteca Deepface, comunicarse con el repositorio para registrar dicha representación en la base de datos vectorial o, de ya encontrarse registrada, simplemente actualizarla. Aquí también entran en juego los llamados Schemas, que permiten construir respuestas estructuradas a los controladores, los cuales posteriormente validarán para determinar si la respuesta que tienen que mandar hacia el cliente tiene el formato correcto/esperado por este.

Después, se encuentra la capa de los repositorios, en esta ocasión, dado el dominio del problema solo se cuenta con una clase llamada StudentRepository la cual contiene interfaces que permiten el acceso a la base de datos vectorial y realizar operaciones de inserción y actualización en este caso.

La base de datos que se utiliza para esta aplicación es una base de datos vectorial que como ya se comentó anteriormente, permite realizar operaciones eficientes sobre el tipo de datos que se trabaja en un sistema de reconocimiento facial, es decir, esta optimizada para el uso de vectores multidimensionales.

Finalmente, el proceso del sistema de reconocimiento facial se encuentra abstraído en los archivos de utilidad, específicamente en Verification, que hace uso de la biblioteca DeepFace para los primeros pasos y más determinantes de detección, alineación y representación de los rostros. La lógica para la verificación de rostros se encuentra implementada en este mismo archivo y sigue la lógica del cálculo de la distancia coseno y verificación con un valor de umbral preestablecido.

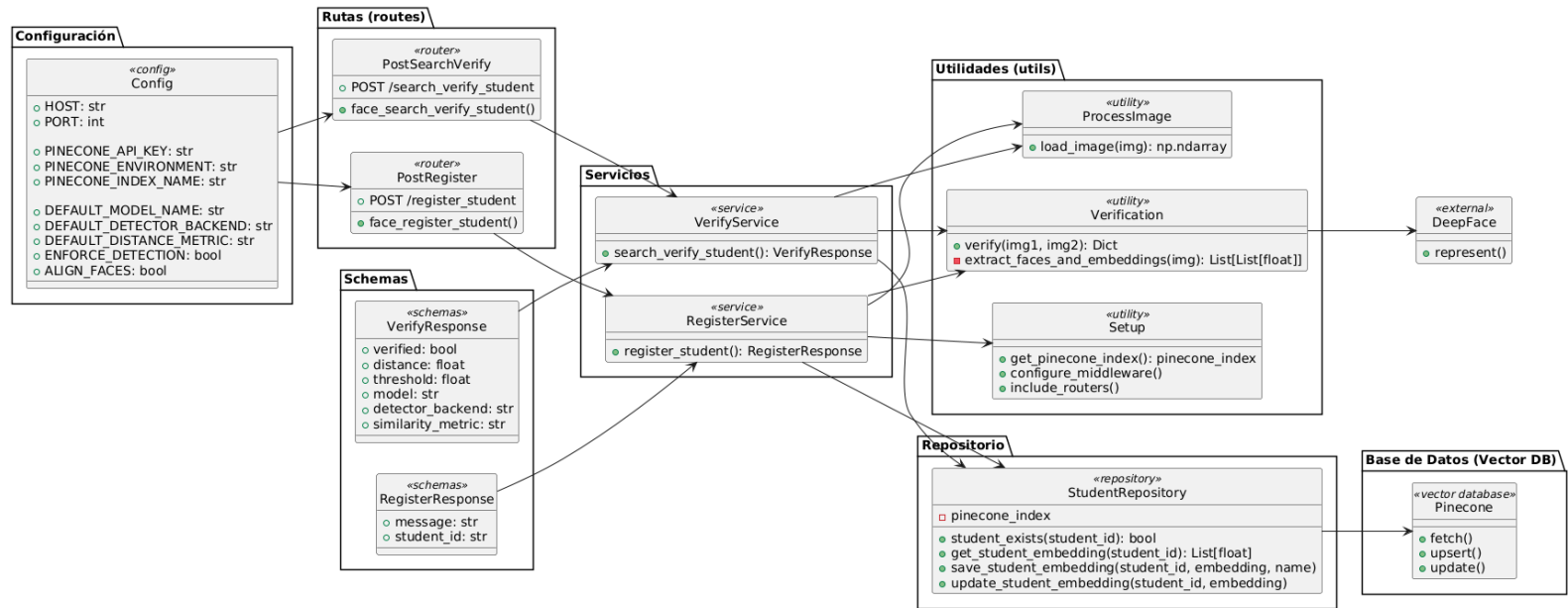


Figura A.63: Diagrama de la Estructura General del Servidor Fast API.

A.7. Diagramas de secuencia

A continuación, se presentan los diagramas de secuencia identificados para para la propuesta de solución presentada en este documento. En ellos se busca ilustrar el funcionamiento dinámico del sistema y modelar las interacciones entre los distintos componentes y actores dependiendo de los casos de uso. Cada diagrama representa el flujo de mensajes e información entre los actores, la aplicación, los servidores y la base de datos.

A.7.1. SE-01 Iniciar sesión del sistema móvil

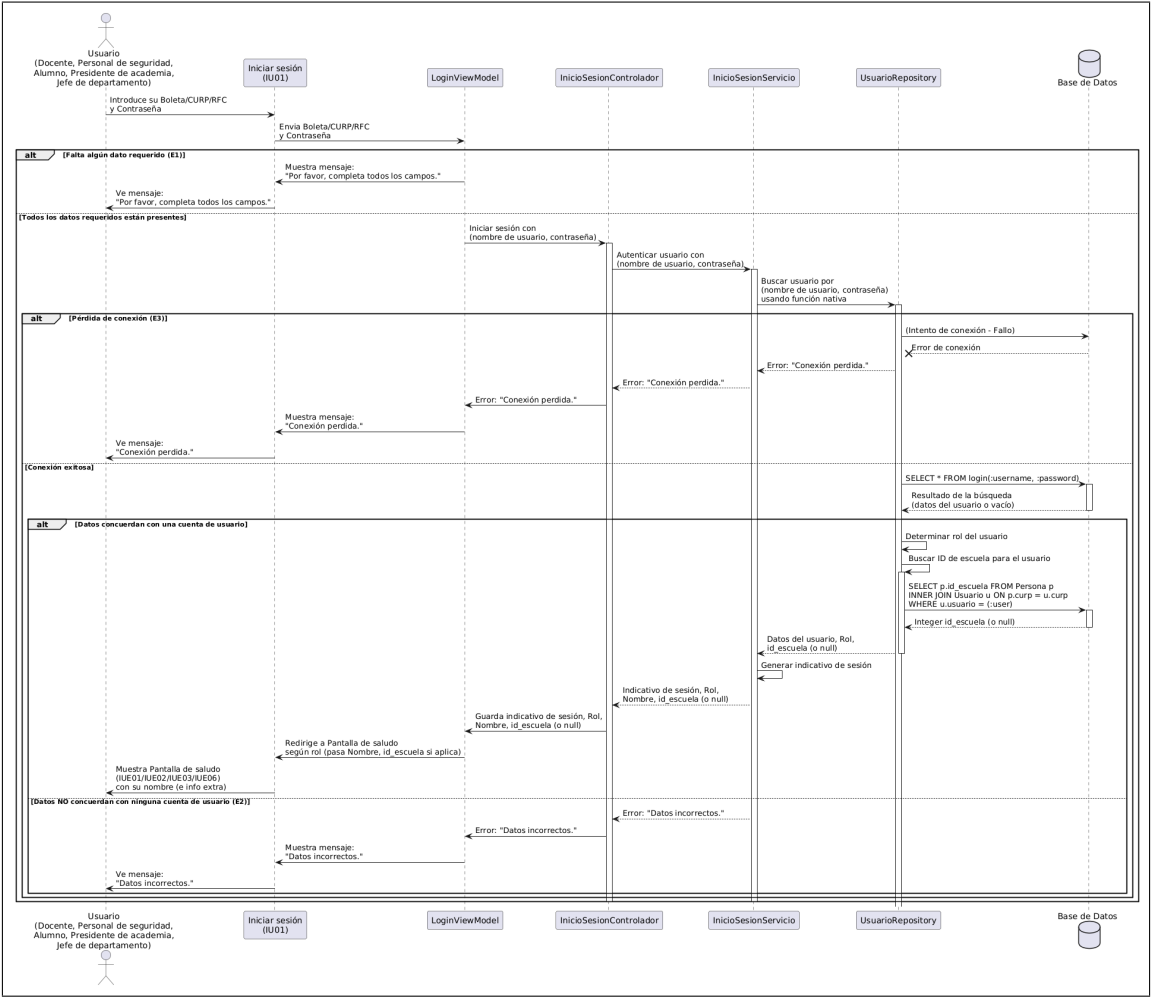


Figura A.64: Diagrama de secuencia del caso de uso número 01 (Iniciar sesión del sistema móvil).

En este diagrama de secuencia (figura ??) de secuencia se describe el proceso planeado para el caso de uso CU-01 Iniciar sesión del sistema móvil, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.2. SE-02 Consultar calendario escolar

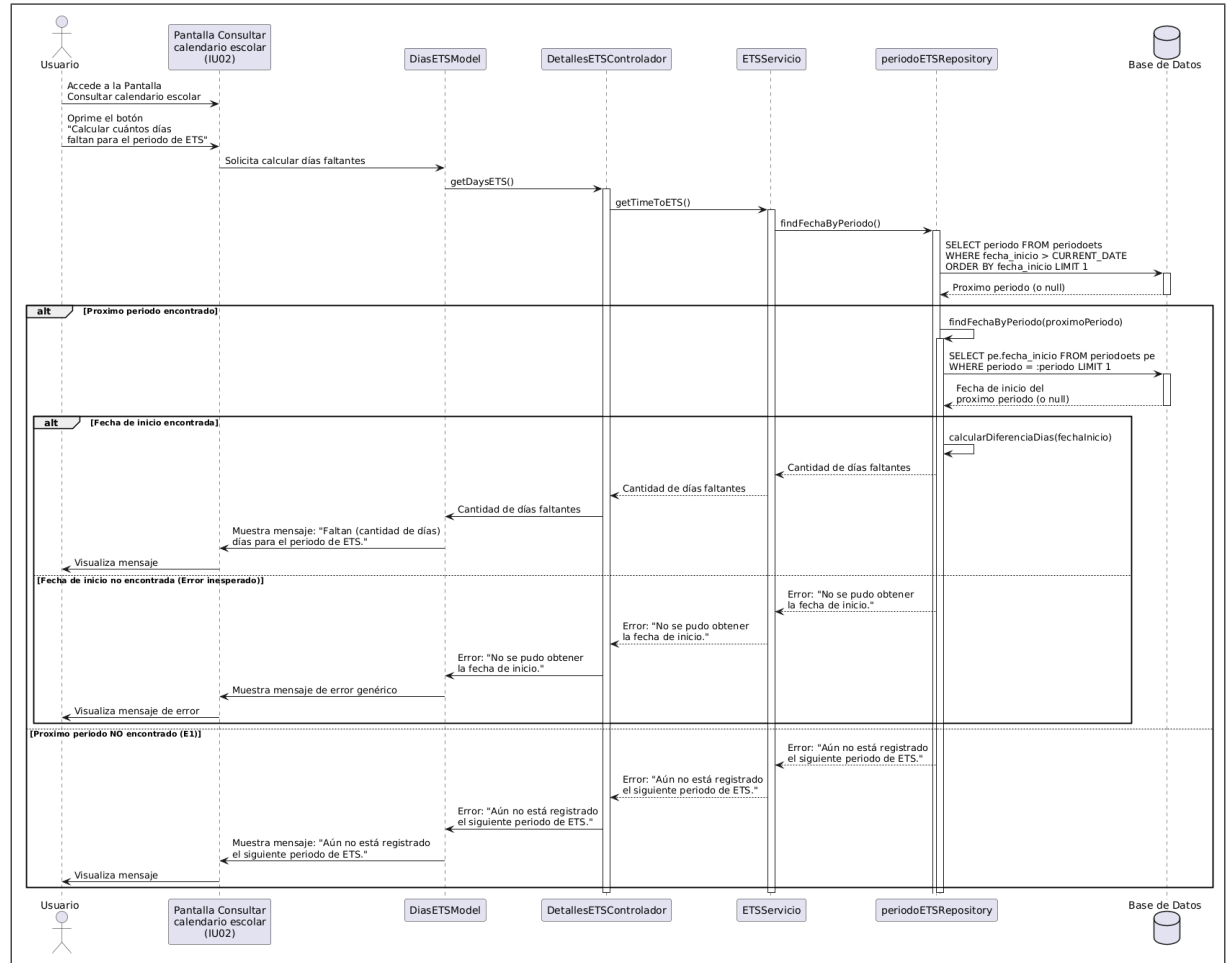


Figura A.65: Diagrama de secuencia del caso de uso número 02 (Consultar calendario escolar).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-02 Consultar calendario escolar, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.3. SE-03 Consultar notificaciones

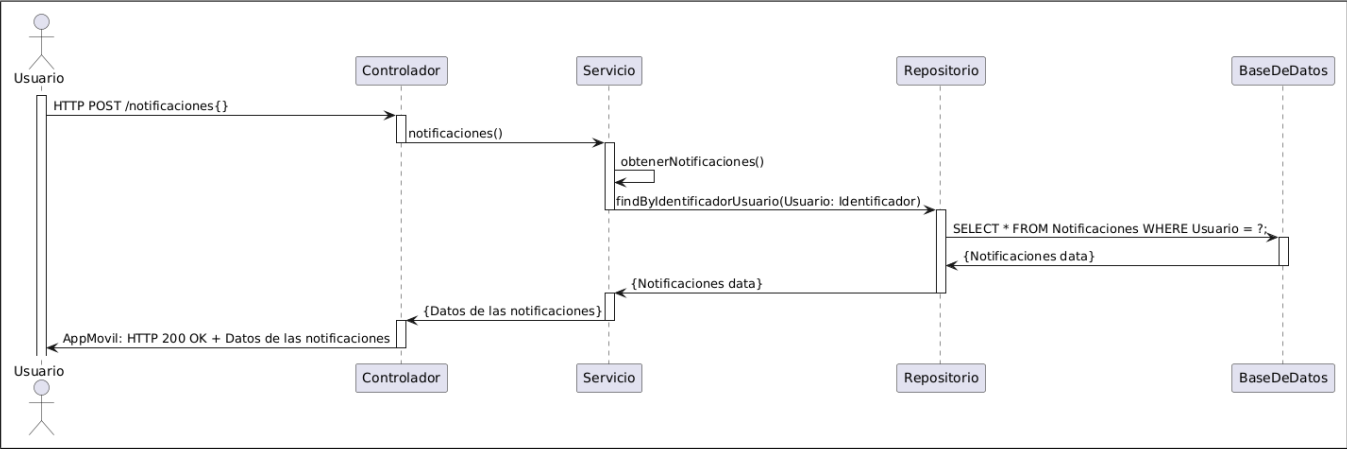


Figura A.66: Diagrama de secuencia del caso de uso número 03 (Consultar notificaciones).

En este diagrama de secuencia (figura ?? se describe el proceso planeado para el caso de uso CU-03 Consultar notificaciones, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.4. SE-04 Consultar periodos de ETS asignados al docente

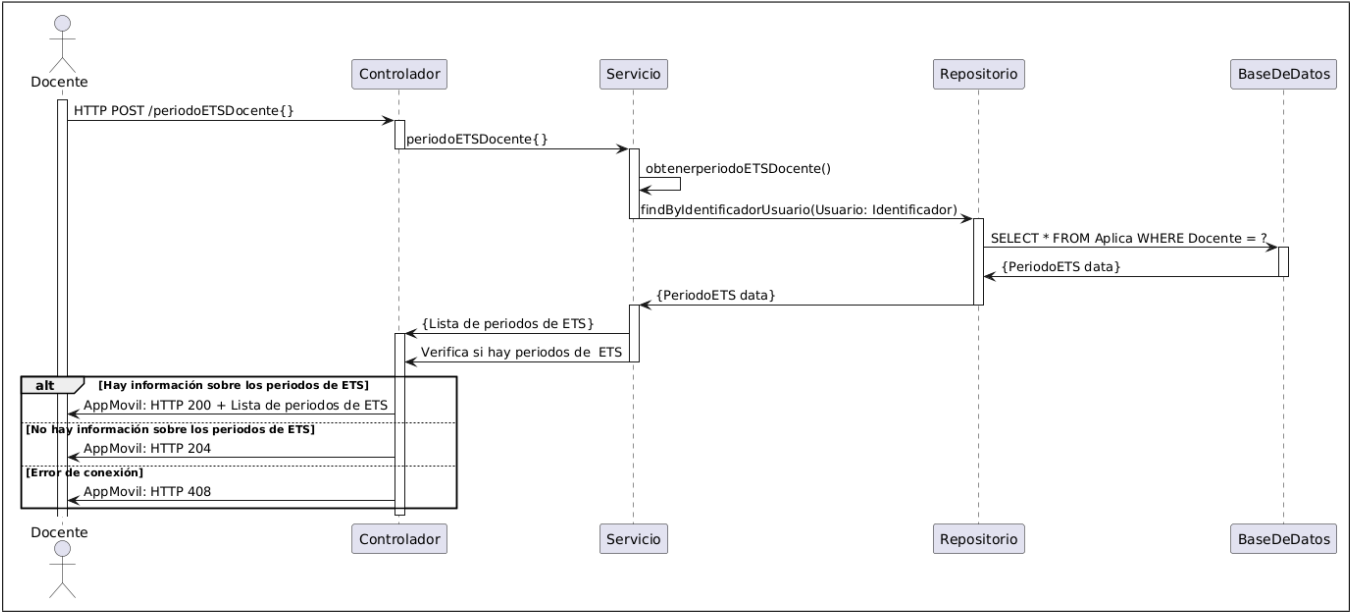


Figura A.67: Diagrama de secuencia del caso de uso número 04 (Consultar periodos de ETS asignados al docente).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-04 Consultar periodos de ETS asignados al docente, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.5. SE-05 Consultar ETS asignados

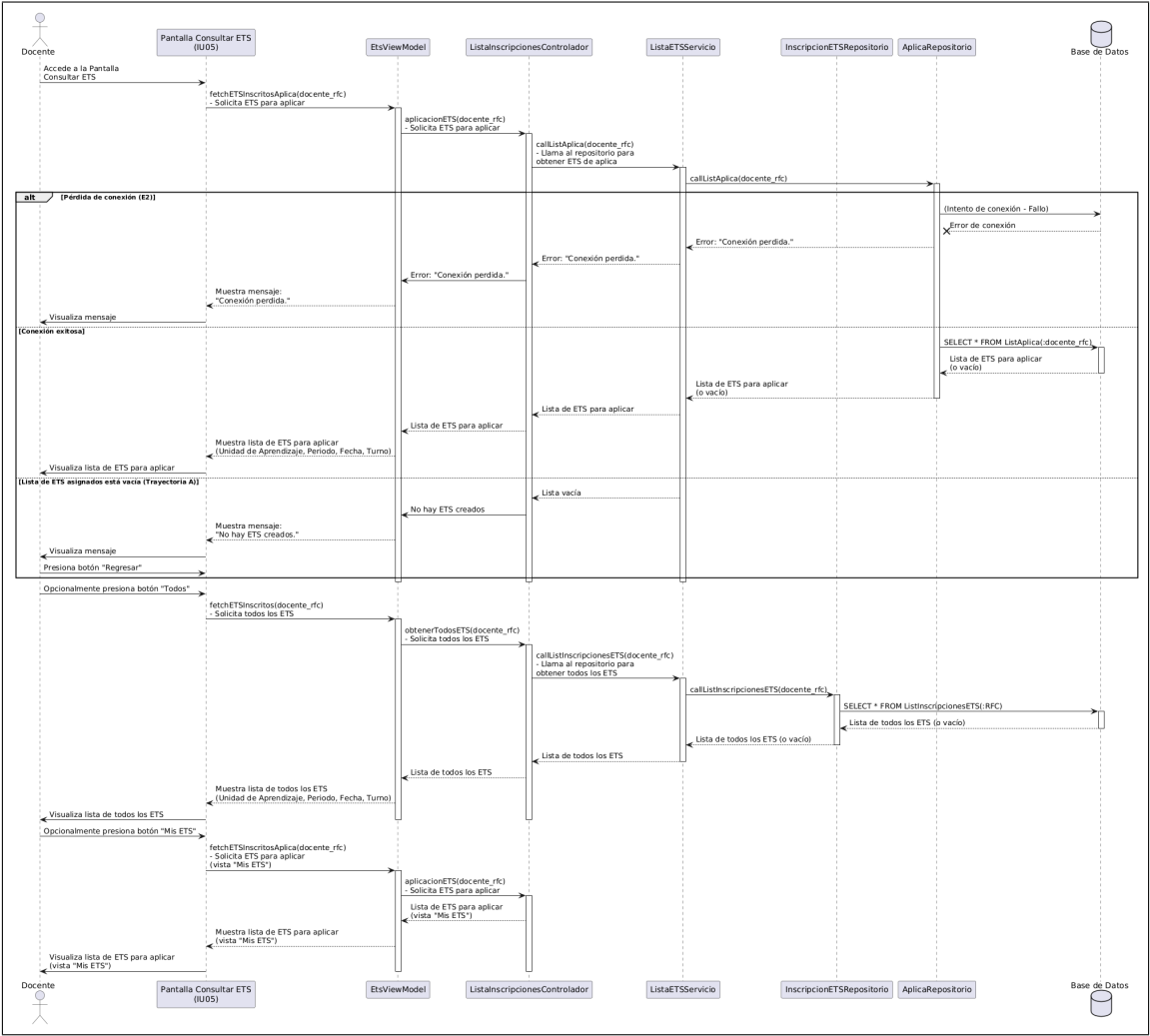


Figura A.68: Diagrama de secuencia del caso de uso número 05 (Consultar ETS asignados).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-05 Consultar ETS asignados, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.6. SE-06 Mostrar información de los ETS asignados

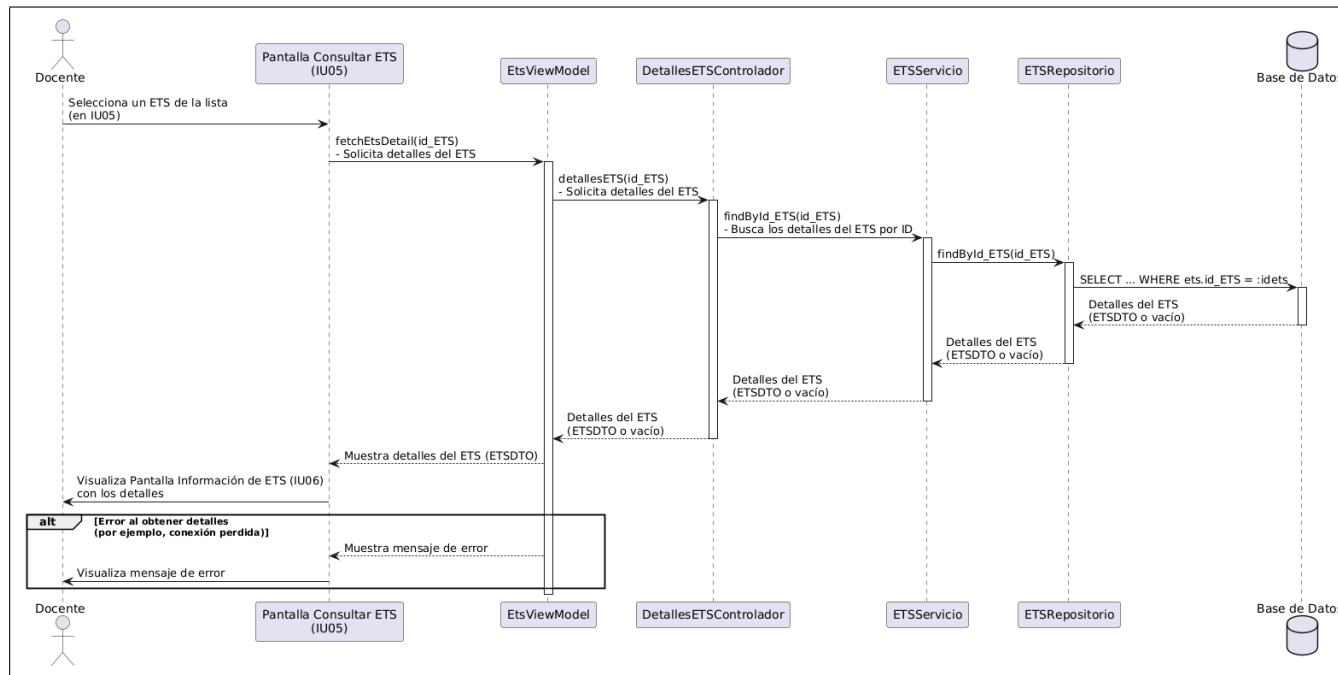


Figura A.69: Diagrama de secuencia del caso de uso número 06 (Mostrar información de los ETS asignados).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-06 Mostrar información de los ETS asignados, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.7. SE-07 Solicitar remplazo

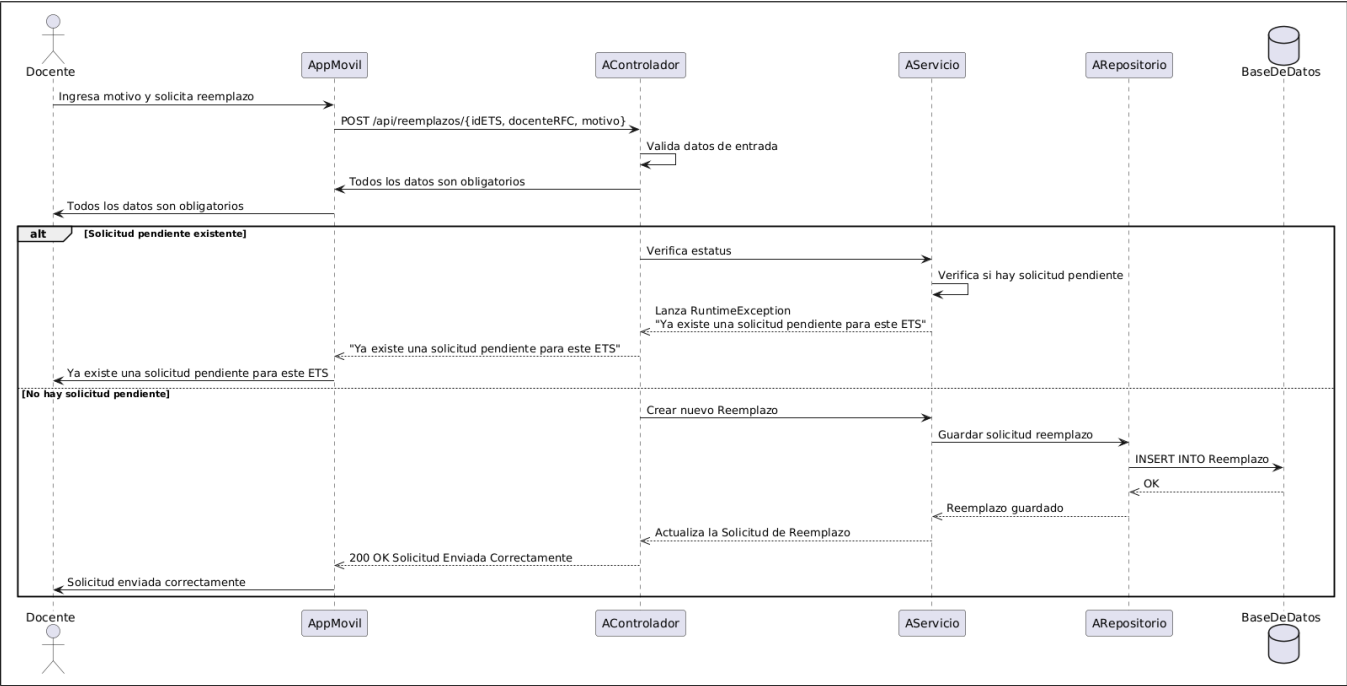


Figura A.70: Diagrama de secuencia del caso de uso número 07 (Solicitar remplazo).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-07 Solicitar remplazo, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.8. SE-08 Consultar lista de alumnos inscritos a un ETS

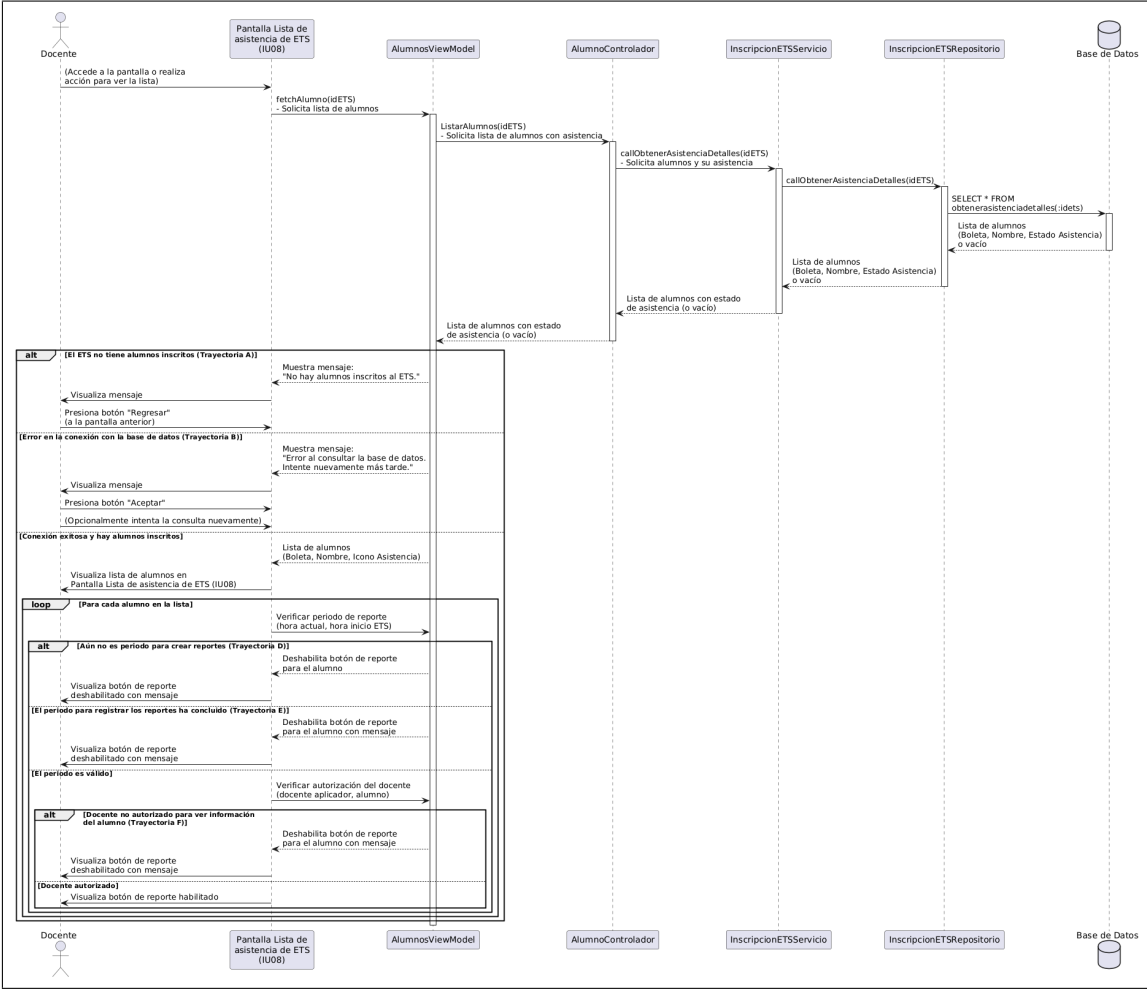
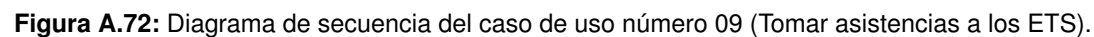


Figura A.71: Diagrama de secuencia del caso de uso número 08 (Consultar lista de alumnos inscritos a un ETS).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-08 Consultar lista de alumnos inscritos a un ETS, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

189



A.7.10. SE-10 Consultar lista de asistencia de alumnos inscritos a los ETS

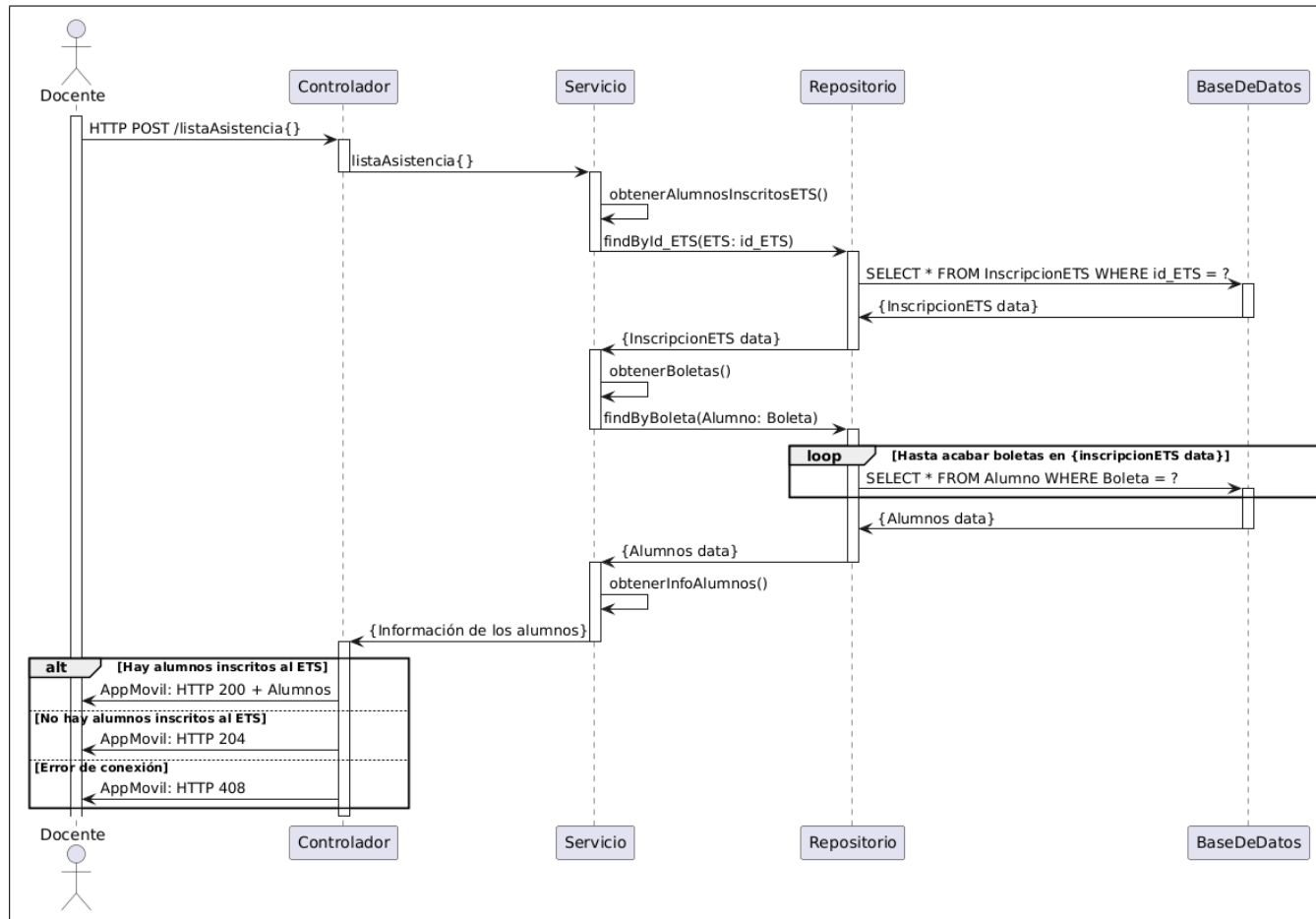


Figura A.73: Diagrama de secuencia del caso de uso número 10 (Consultar lista de asistencia de alumnos inscritos a los ETS).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-10 Consultar lista de asistencia de alumnos inscritos a los ETS, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.11. SE-11 Mostrar la foto e información del alumno

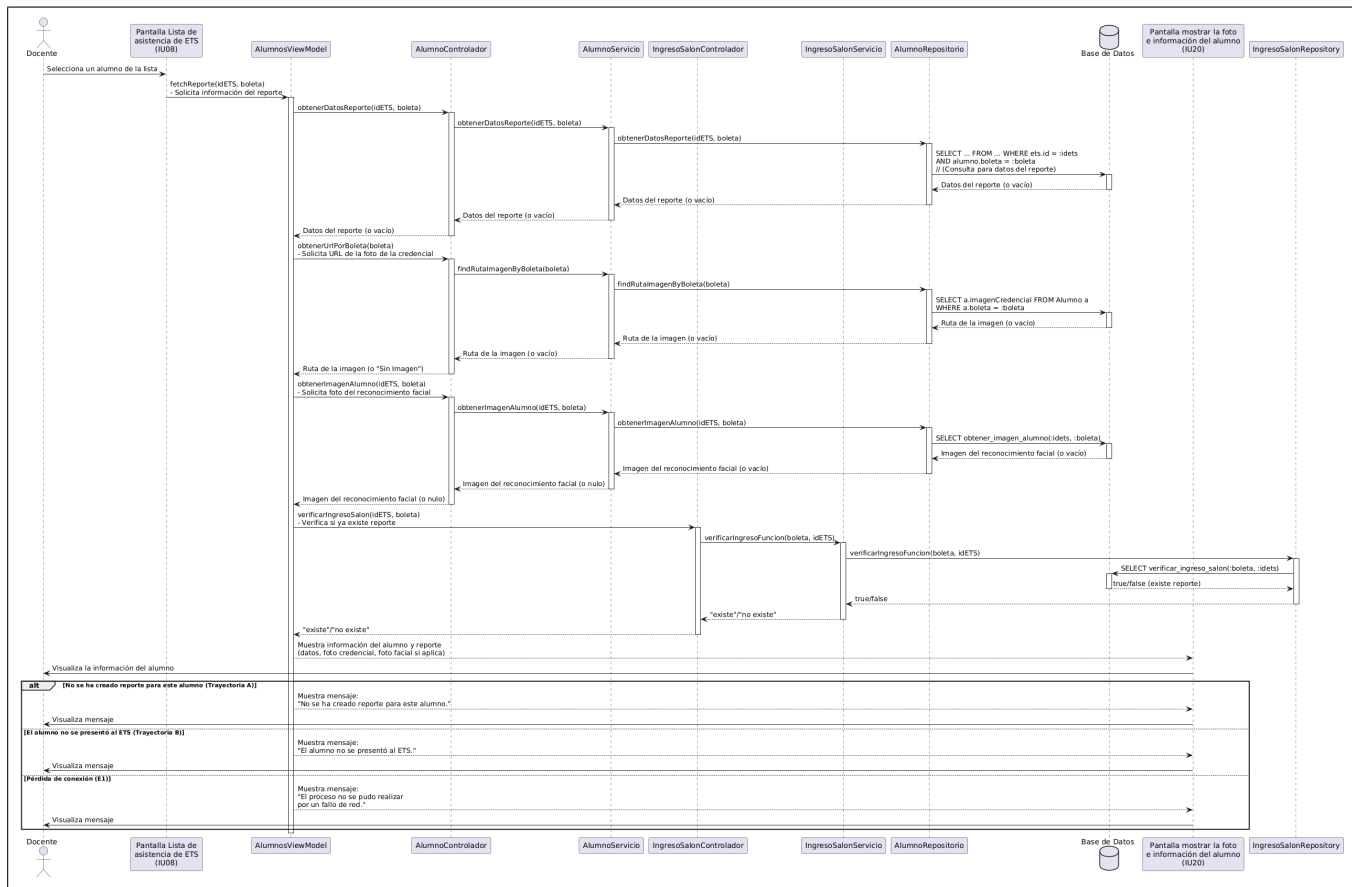


Figura A.74: Diagrama de secuencia del caso de uso número 11 (Mostrar la foto e información del alumno).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-11 Mostrar la foto e información del alumno, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.12. SE-12 Consultar alumno mediante código QR de la credencial

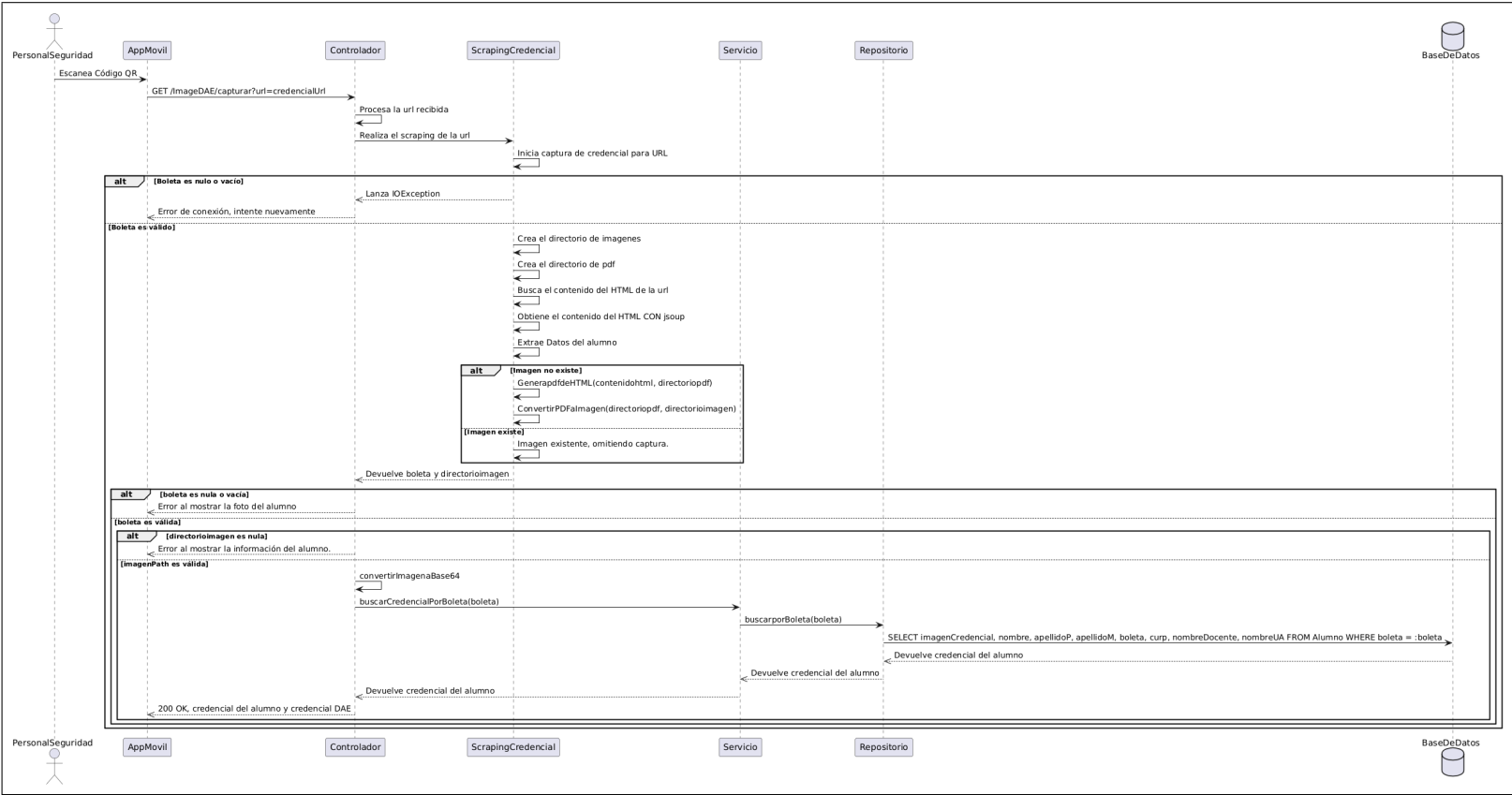


Figura A.75: Diagrama de secuencia del caso de uso número 12 (Consultar alumno mediante código QR de la credencial).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-12 Consultar alumno mediante código QR de la credencial, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.13. SE-13 Buscar alumno por boleta

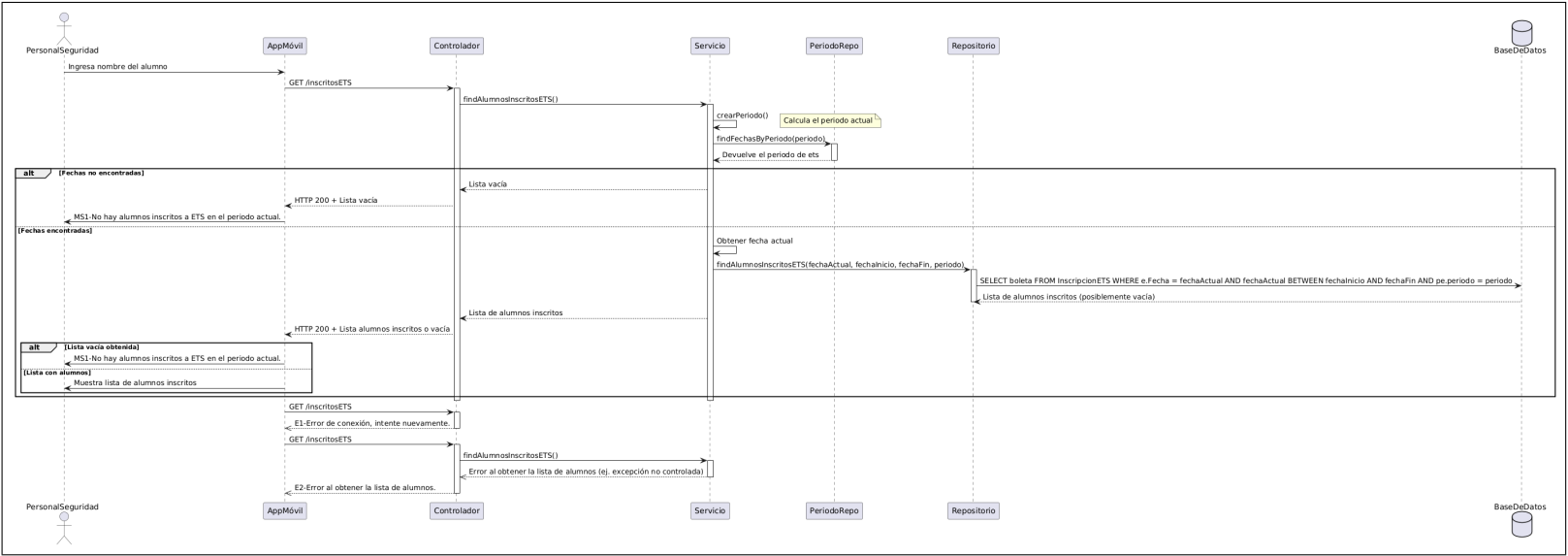


Figura A.76: Diagrama de secuencia del caso de uso número 13 (Buscar alumno por boleta).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-13 Buscar alumno por boleta, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.14. SE-14 Buscar alumno por nombre

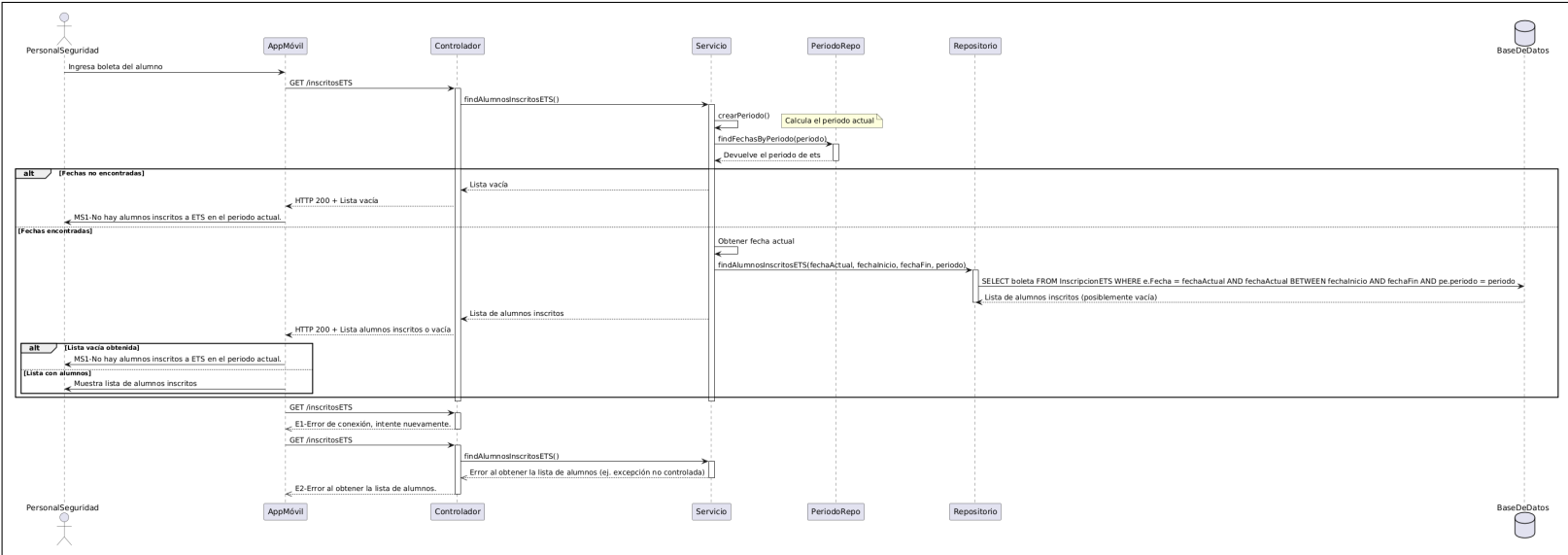


Figura A.77: Diagrama de secuencia del caso de uso número 14 (Buscar alumno por nombre).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-14 Buscar alumno por nombre, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.15. SE-15 Registrar asistencia

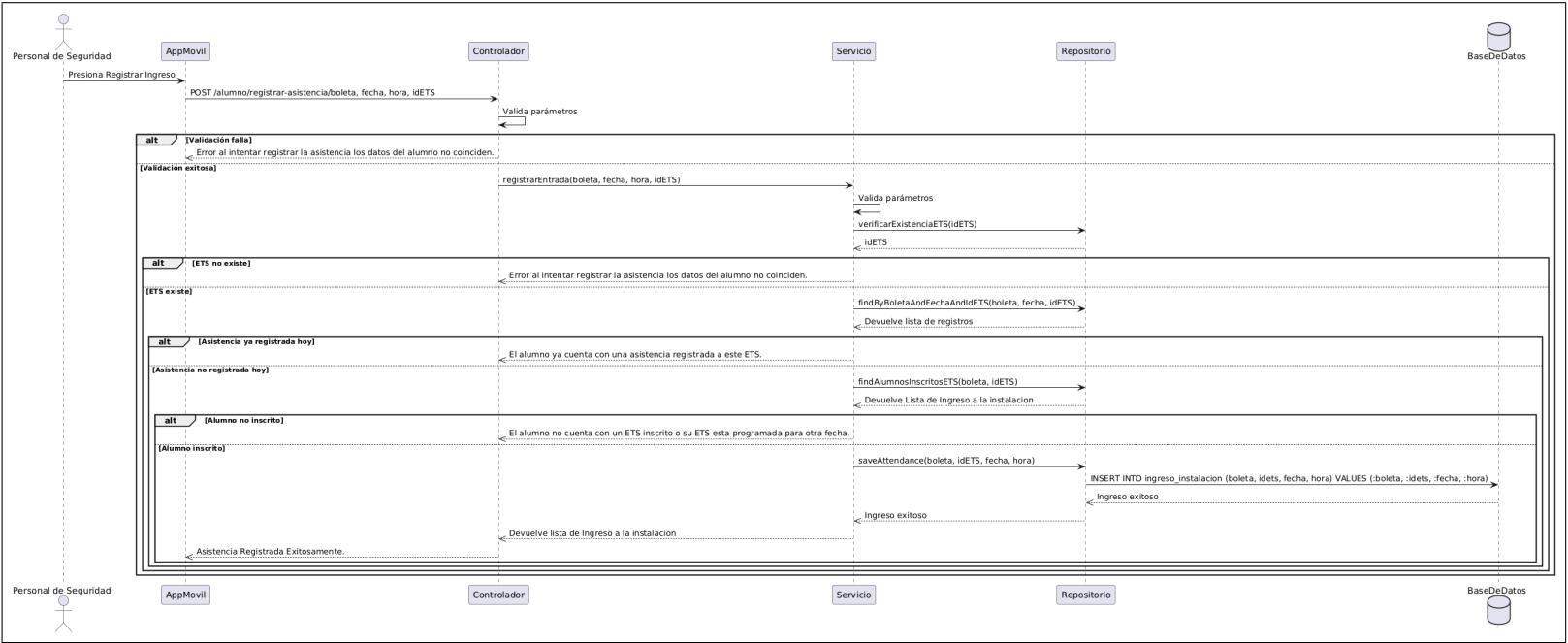


Figura A.78: Diagrama de secuencia del caso de uso número 15 (Registrar asistencia).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-15 Registrar asistencia, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.16. SE-16 Consultar periodos de ETS inscritos del alumno

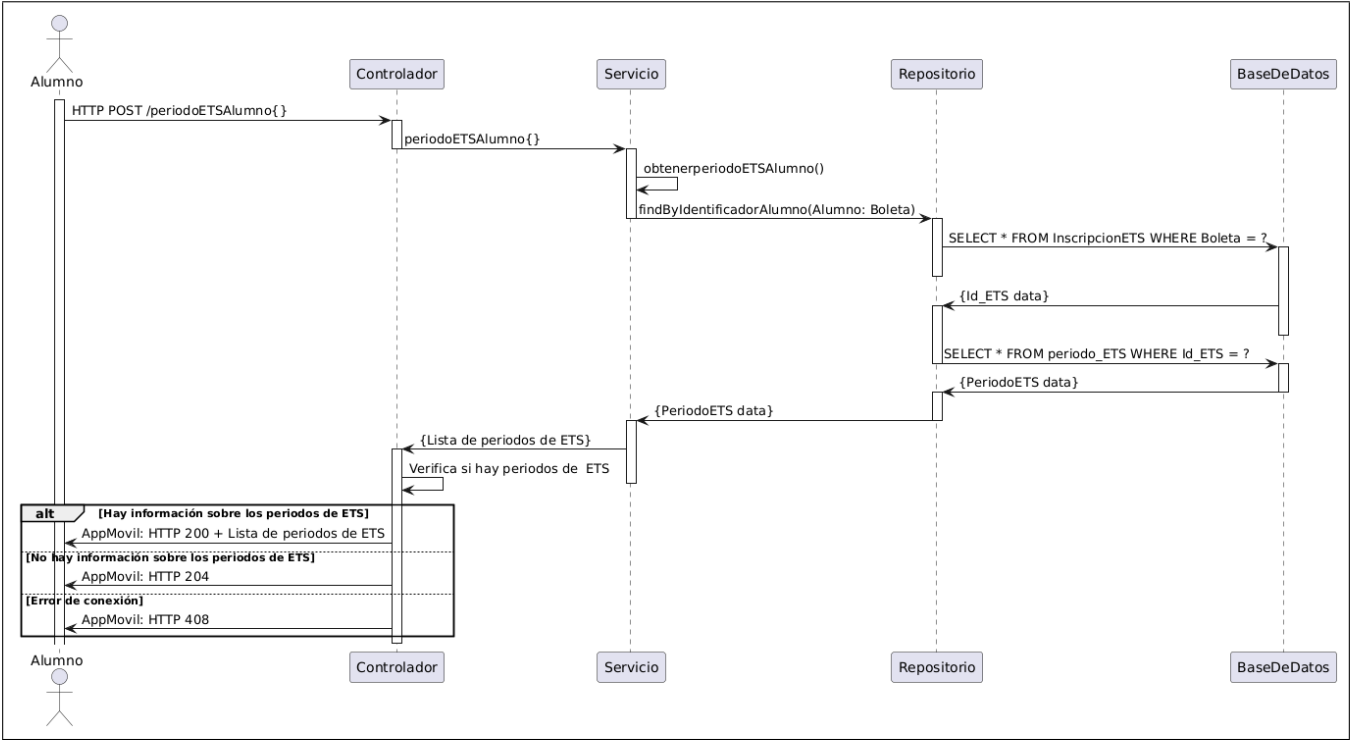


Figura A.79: Diagrama de secuencia del caso de uso número 16 (Consultar periodos de ETS inscritos del alumno).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-16 Consultar periodos de ETS inscritos del alumno, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.17. SE-17 Consultar ETS inscritos

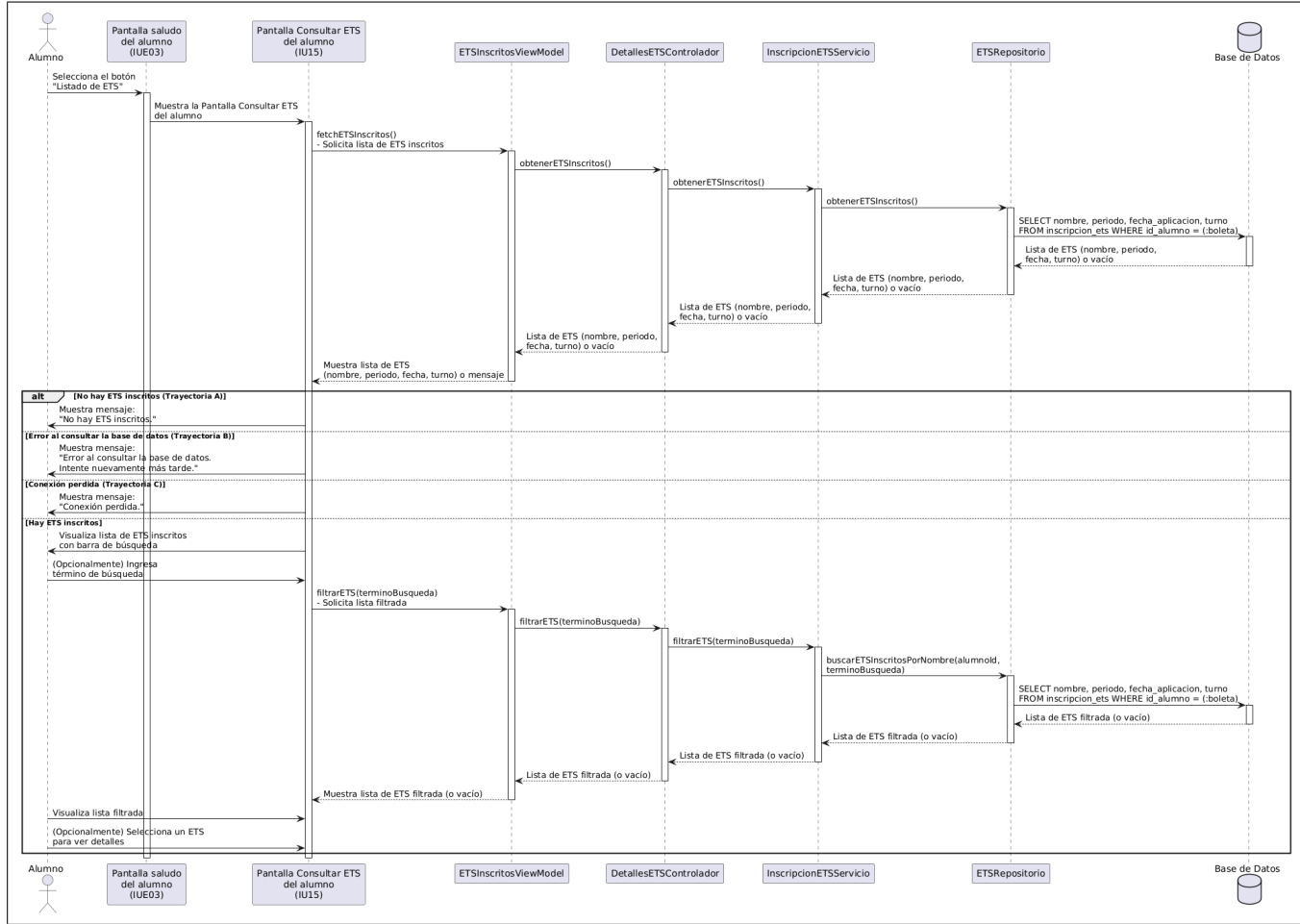


Figura A.80: Diagrama de secuencia del caso de uso número 07 (Consultar ETS inscritos).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-17 Consultar ETS inscritos, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.18. SE-18 Mostrar información del los ETS inscritos

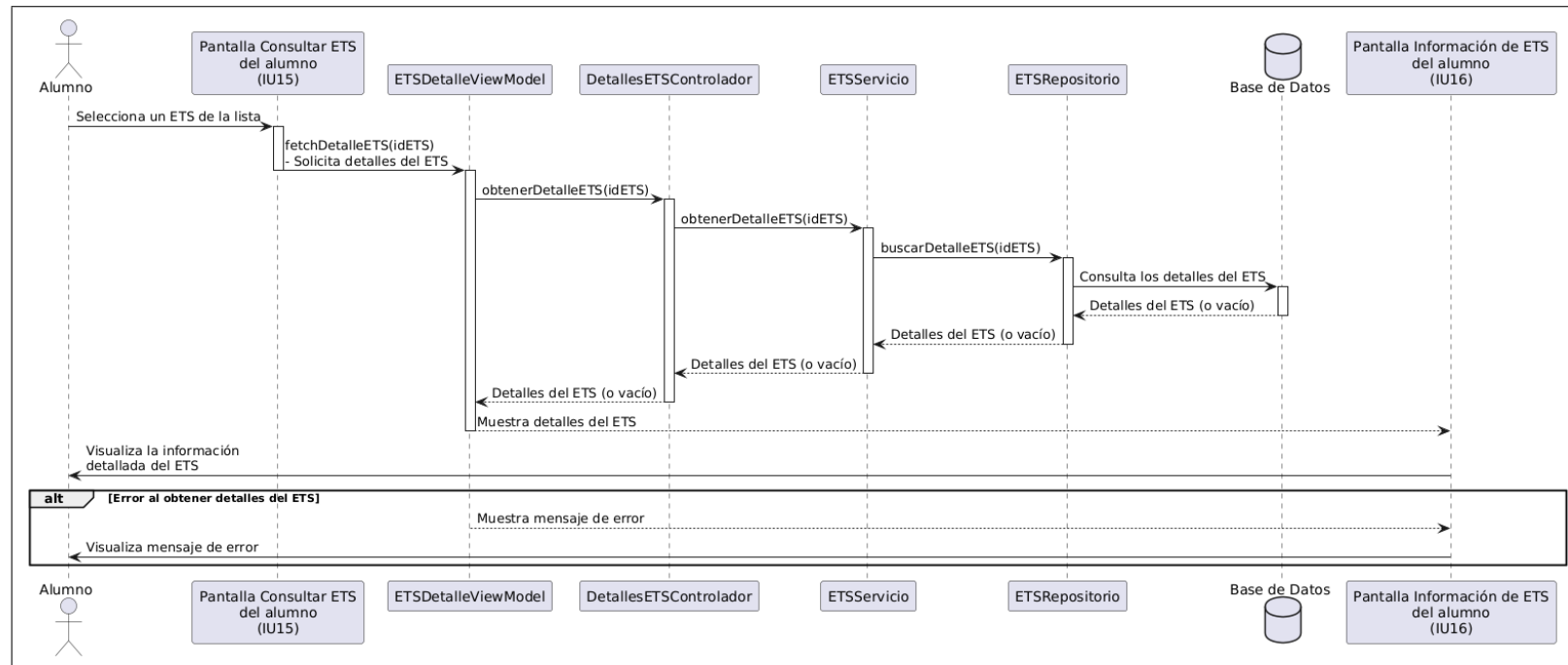


Figura A.81: Diagrama de secuencia del caso de uso número 18 (Mostrar información del los ETS inscritos).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-18 Mostrar información del los ETS inscritos, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.19. SE-19 Probar reconocimiento facial

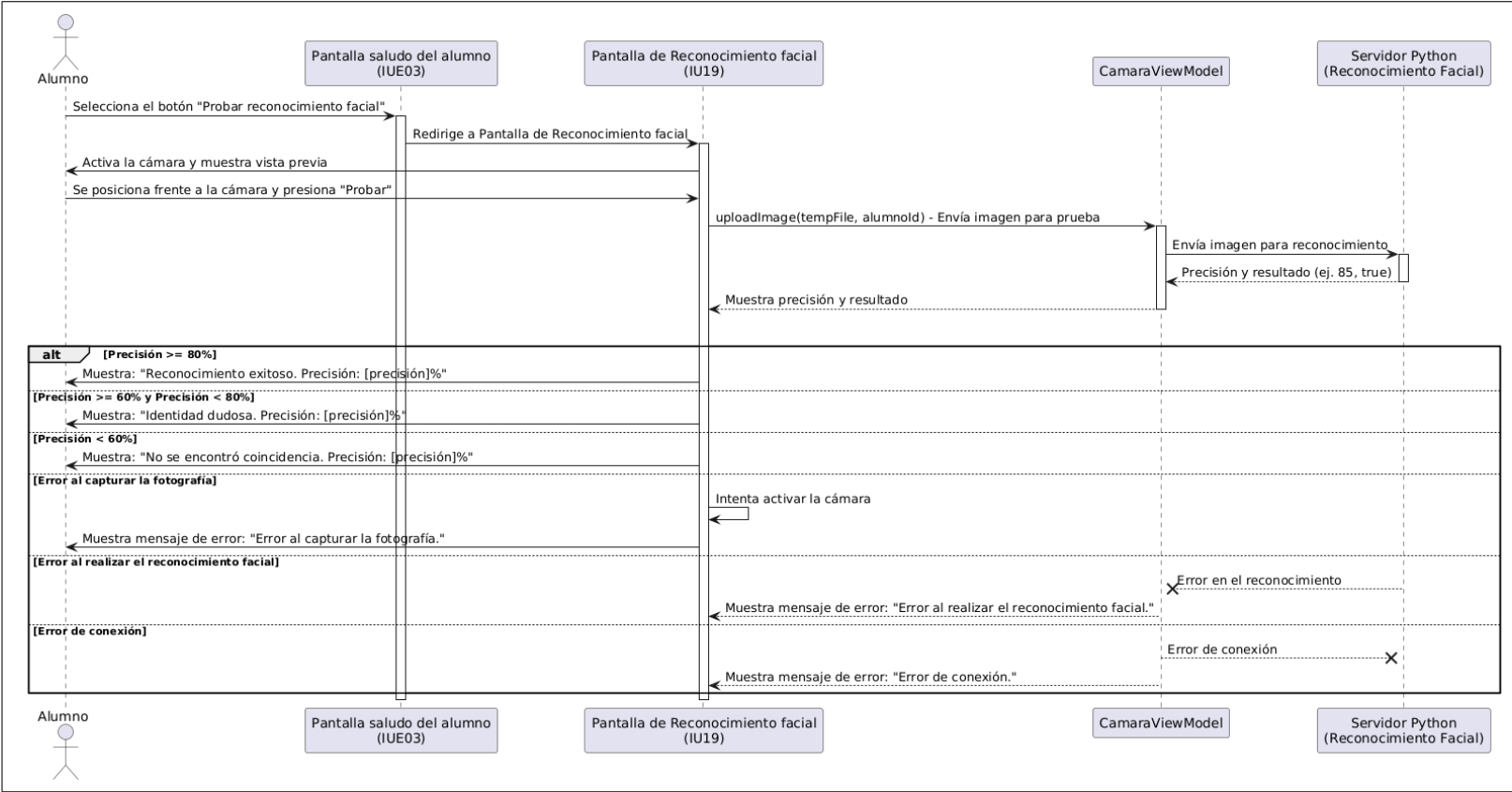


Figura A.82: Diagrama de secuencia del caso de uso número 19 (Probar reconocimiento facial).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-19 Probar reconocimiento facial, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.20. SE-20 Revisar información de acceso a los ETS

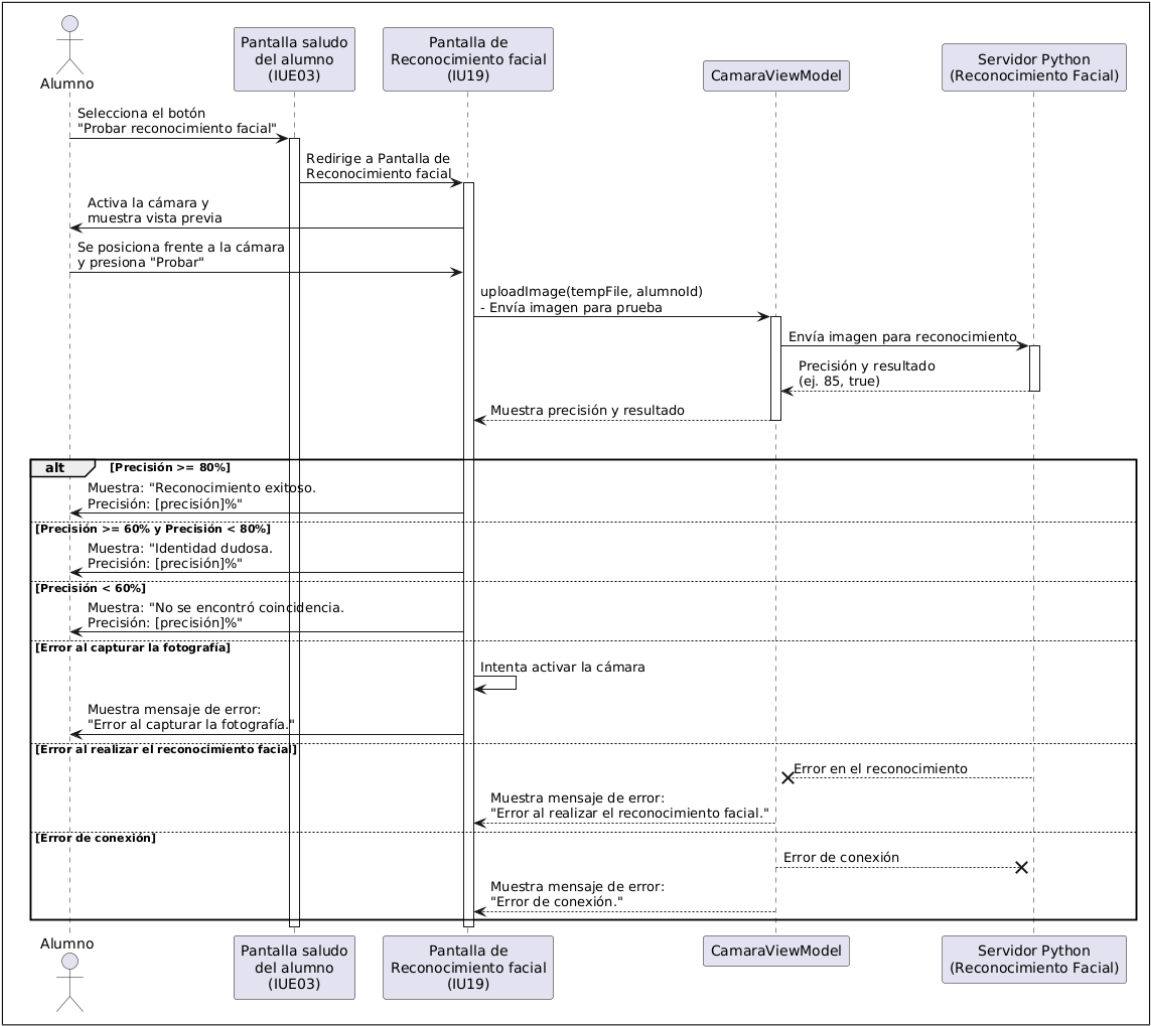


Figura A.83: Diagrama de secuencia del caso de uso número 20 (Revisar información de acceso a los ETS).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-20 Revisar información de acceso a los ETS, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.21. SE-21 Asignar docente de remplazo

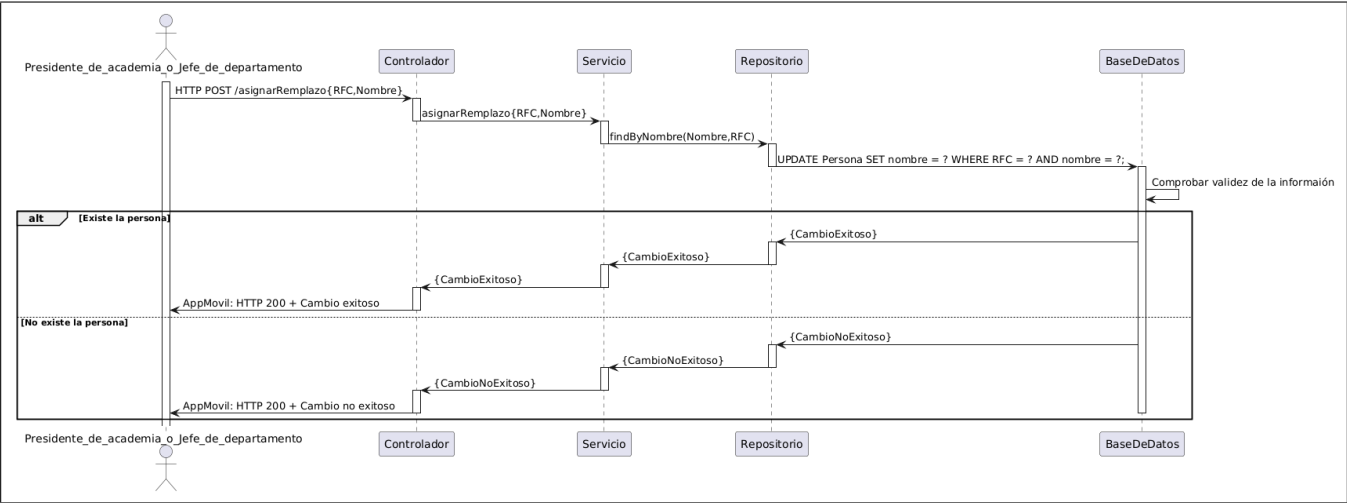


Figura A.84: Diagrama de secuencia del caso de uso número 42 (Asignar docente de remplazo).

En este diagrama de secuencia (figura ??) se describe el proceso planeado para el caso de uso CU-42 Asignar docente de remplazo, mostrando las interacciones que tendrá con la vista, el controlador, el servicio, el repositorio y la base de datos.

A.7.22. SE-22 Registrar estudiante (Sistema de reconocimiento facial)

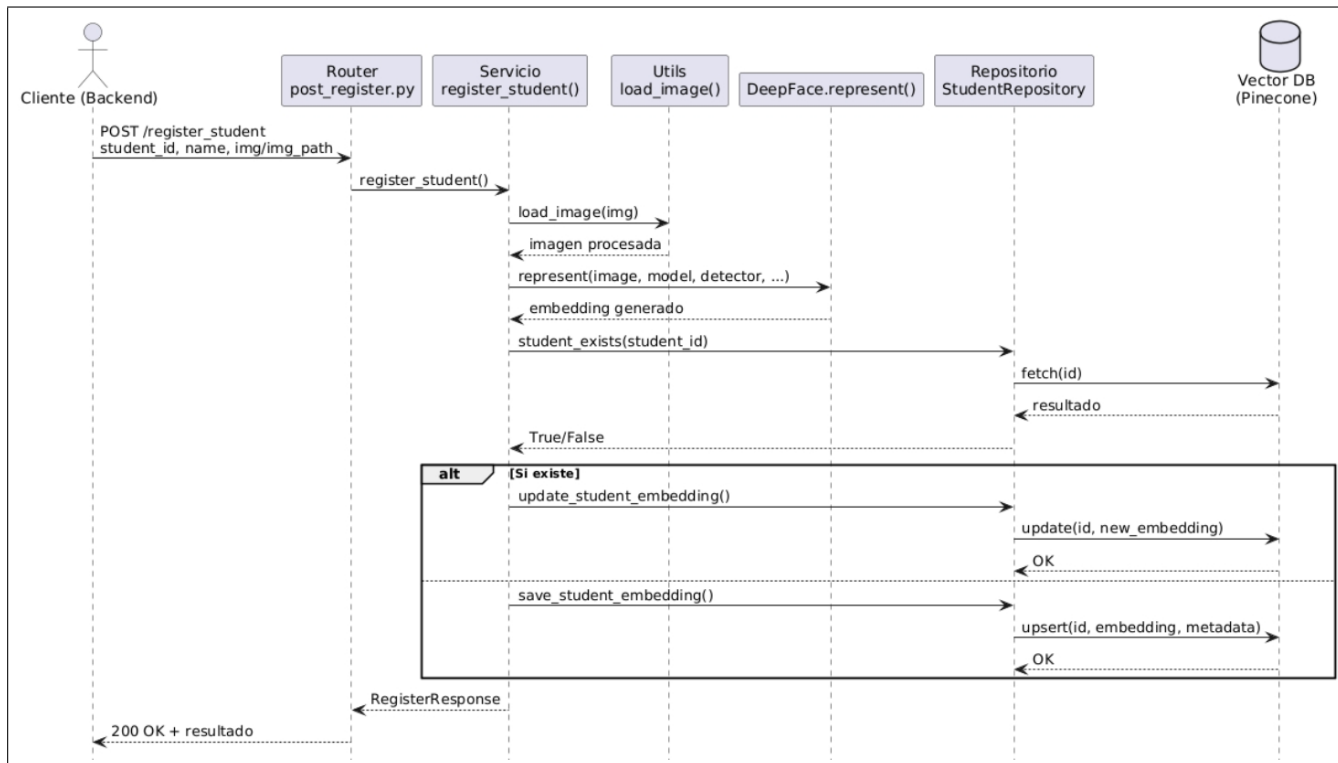


Figura A.85: Diagrama de secuencia del endpoint register student).

En este diagrama de secuencia (figura ??) se describe el proceso que sigue la lógica implementada para el sistema de reconocimiento facial cuando se necesita registrar a un estudiante, por si sola, la secuencia de pasos era muy grande para incluirla en los diagramas de secuencia que llaman a este endpoint, por lo que aquí se detalla mejor su funcionamiento.

A.7.23. SE-23 Buscar y verificar identidad de estudiante (Sistema de reconocimiento facial)

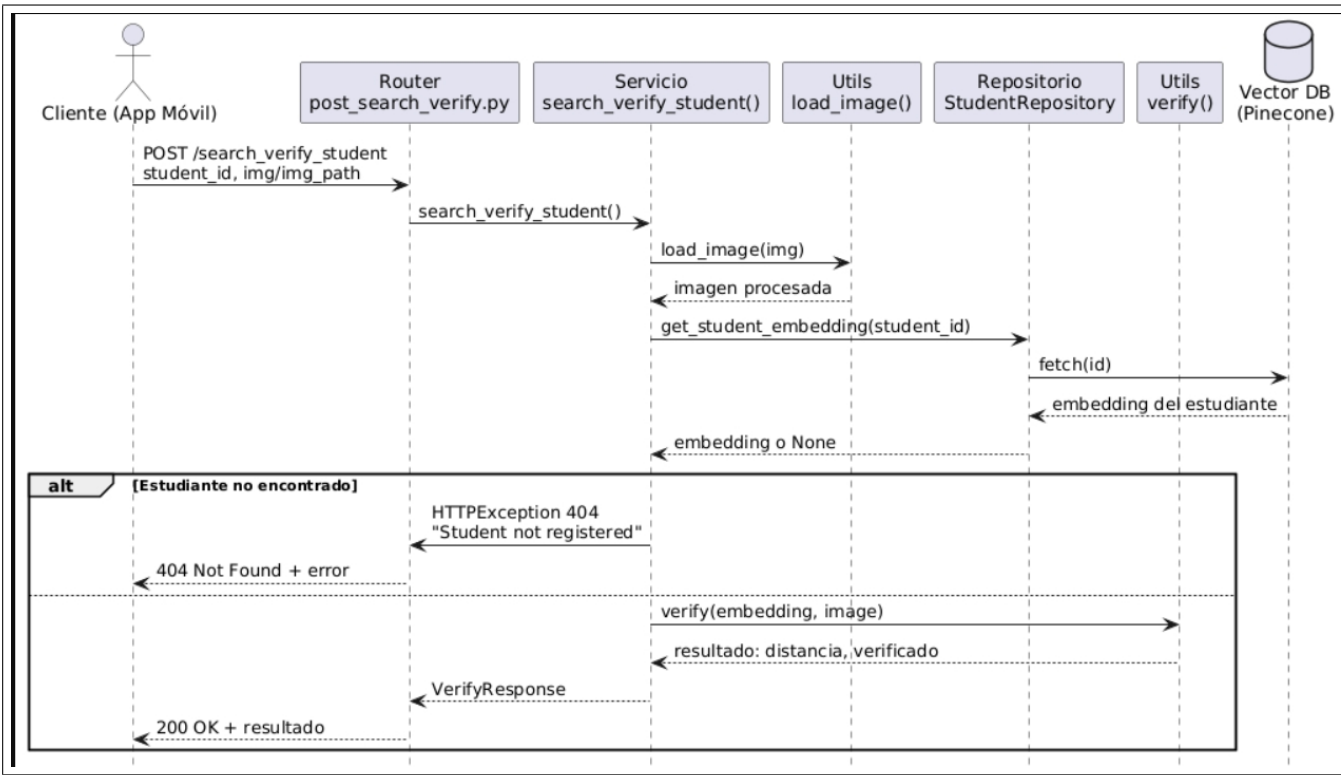


Figura A.86: Diagrama de secuencia del endpoint search and verify.

En este diagrama de secuencia (figura ??) se describe el proceso que sigue la lógica implementada para el sistema de reconocimiento facial cuando se necesita consultar a un estudiante dentro de la base de datos vectorial y verificar su identidad, por si sola, la secuencia de pasos era muy grande para incluirla en los diagramas de secuencia que llaman a este endpoint, por lo que aquí se detalla mejor su funcionamiento.